

Zemax 编程语言 (ZPL) 应用指南

Open Source Photonics

osphotonics.wordpress.com

**Sponsored by
*www.612photonics.com***

目录

前言

第一章 ZEMAX 光学设计软件和 ZEMAX 编程语言(ZPL)简介

第一节 ZEMAX 光学设计软件简介

第二节 ZEMAX 编程语言(ZEMAX Programming Language, ZPL)简介

第二章 ZEMAX 编程语言基本知识

第一节 ZPL 程序的基本结构

第二节 ZPL 中的常量和变量

第三节 ZPL 中的函数

第四节 ZPL 中的关键词

第五节 ZPL 中的程序流控制

第六节 ZPL 中的子程序

第七节 ZPL 中的输入输出和文件操作

第三章 ZPL 指令详解

第一节 ZPL 中的数值运算函数

第二节 ZPL 中的字符串函数

第三节 ZEMAX 运行环境和系统参数的设置与读取

第四节 镜头参数的设置和读取

第五节 评价函数(merit function)

第六节 求解(solve)

第七节 优化(optimization)

第八节 光线追迹

第九节 系统分析

第十节 非顺序元件

第十一节 多组态(multi-configuration)

第十二节 屏幕显示

第十三节 文件操作

第十四节 ZBF 文件

第四章 ZPL 应用实例

第一节 顺序光学系统

4.1-1 光线追迹参数设定

4.1-2 近焦平面光斑

4.1-3 几何光束及高斯光束比较

4.1-4 不同玻璃材料传输特性比较

4.1-5 从数据库中读取折射率及透光率

第二节 非顺序光学系统

4.2-1 导光管

4.2-2 余弦四次方规律

4.2-3 重点采样

4.2-4 相干条纹

4.2-5 积分球效率

4.2-6 利用体探测器生成 3D 光强分布图

前言

现代计算机技术的发展，在很大程度上改变了光学设计的方法和效率。以前只有极少数专家才能进行的繁杂的光学设计，现在借助功能强大的设计软件，只要经过适当的培训，大多数专业技术人员都能够完成。可以毫不夸张地说，专业光学设计软件使今天的光学工程师们如虎添翼，极大地拓展了其设计领域。在目前市面上出现的几种光学设计软件中，ZEMAX 以其功能强大、运行速度快、界面简单、易于掌握而为广大光学工程师所青睐。然而，尽管 ZEMAX 已经具有十分强大的功能，但在某些情况下，设计人员还需要对其功能作进一步扩展，以满足自己的特殊需求。为此，ZEMAX 专门提供了一种编程语言，称为 ZEMAX Programming Language (ZPL)。只有掌握了这个工具，才能从真正意义上让 ZEMAX 软件物尽其用，大幅度地提高设计灵活性和工作效率。遗憾的是，有关这方面的资料在市面上很少见，大多数人都是靠慢慢地琢磨 ZEMAX 用户手册来逐渐熟悉和领会 ZEMAX 的编程技巧，这无疑是一个漫长痛苦的过程。尽管 ZEMAX 用户手册中对 ZPL 的各条指令都有详细介绍，但其组织方式却并不利于读者学习，而且缺乏足够的例子帮助读者理解。很多人因缺乏合适的参考资料而放弃了对 ZPL 这一先进工具的学习和使用，殊为遗憾。本书从方便学习者的角度，对 ZEMAX 程序设计进行了系统的介绍，并把在光学设计中常用到的 ZPL 关键词和函数按其应用，进行了归纳和整理，同时提供了大量的实例，帮助读者理解与掌握。全书共分为四章，第一章对 ZEMAX 软件及其所提供的程序设计语言 ZPL 进行了背景介绍，第二章对 ZPL 的基本结构和使用进行了系统的介绍，第三章对 ZPL 的常用指令加以归类，并逐条进行了解析，第四章通过大量实例，对 ZPL 的具体运用进行了示范。书中融入了作者多年的学习心得，希望能帮助读者少走弯路，尽快掌握 ZPL 这一利器。对于已经熟悉 ZEMAX 程序设计的读者，本书也不失为一本案头必备的工具书，供编程时查阅。

第一章 ZEMAX 光学设计软件和 ZEMAX 编程语言(ZPL)简介

第一节 ZEMAX 光学设计软件简介

ZEMAX 是由美国公司 ZEMAX Development Corporation 开发的光学设计软件，可用于设计和分析各种光学系统。对于镜头设计、照明设计、激光光束传播、杂散光分析、自由光学设计等各种应用而言，ZEMAX 是一个十分有效的工具。

ZEMAX 有两种不同的版本：ZEMAX -SE (标准版本) 和 ZEMAX -EE (工程版本)，其中工程版本包含标准版本的各种功能，并同时包含许多更高级的功能。

ZEMAX 所采用的主要方法是光线追迹 (Ray Tracing)，包括顺序光线追迹(Sequential Ray Tracing)和非顺序光线追迹(Non-Sequential Ray Tracing)。

在顺序光线追迹模式中，ZEMAX 将光学系统定义为由许多表面组成，并假定光线从物表面出发，顺序经过光学系统的不同表面，最后到达像表面。图 1.1-1 给出顺序光线追迹的一个例子。

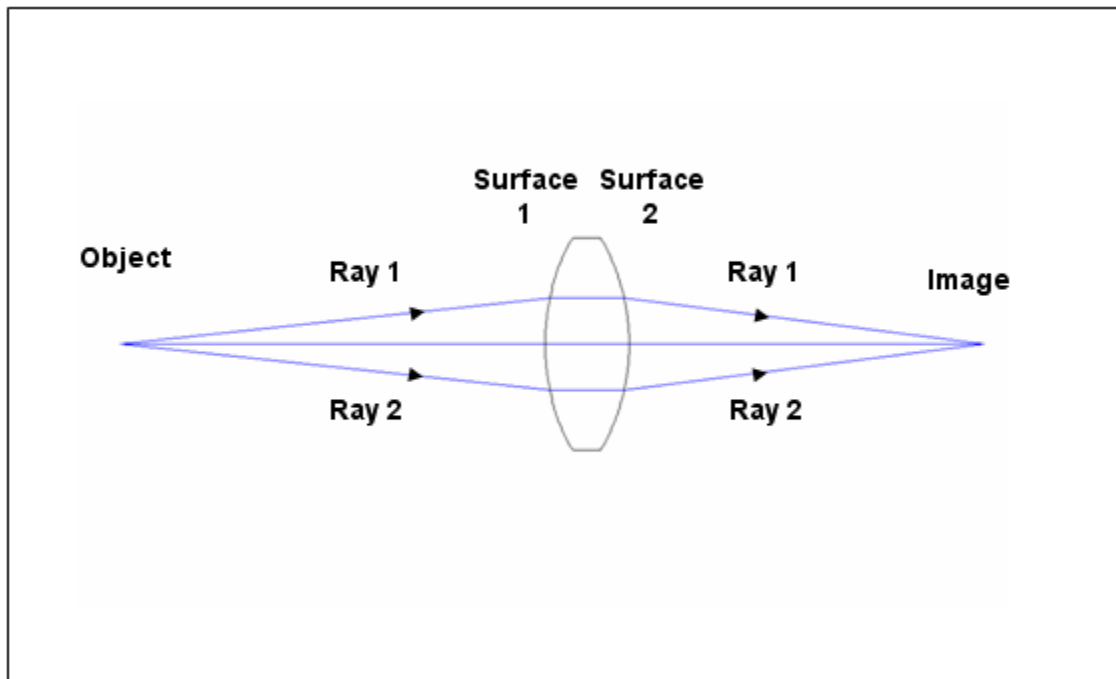


图 1.1-1 顺序光线追迹

我们可以看到，光线 1 从物表面出发，依次经过表面 1 和表面 2，最后到达像表面。光线 2 经过的路径和光线 1 不同，但是，光线 2 经过各个表面的顺序，和光线 1 是完全一致的。换句话说，不同的光线在光学系统中的行为是完全可以预测的，因为它们经过不同的

表面都有固定的顺序。ZEMAX 通过顺序追迹每一条光线，可以确定整个光学系统的性能。

但是，在有些情况下，不同的光线在光学系统中经过各个表面的顺序并不完全相同，这时，就需要用到非顺序光线追迹方法。在非顺序光线追迹模式下，ZEMAX 将光学系统定义为由许多元件(或固体模块)组成，每个元件称为一个物体。这样的光学系统称为非顺序光学系统。非顺序光学系统中常有的物体有透镜、导光管、棱镜、透镜阵列、光源、光探测器、光全内反射元件、半透半反元件等。

图 1.1-2 给出一个非顺序光线追迹的例子。

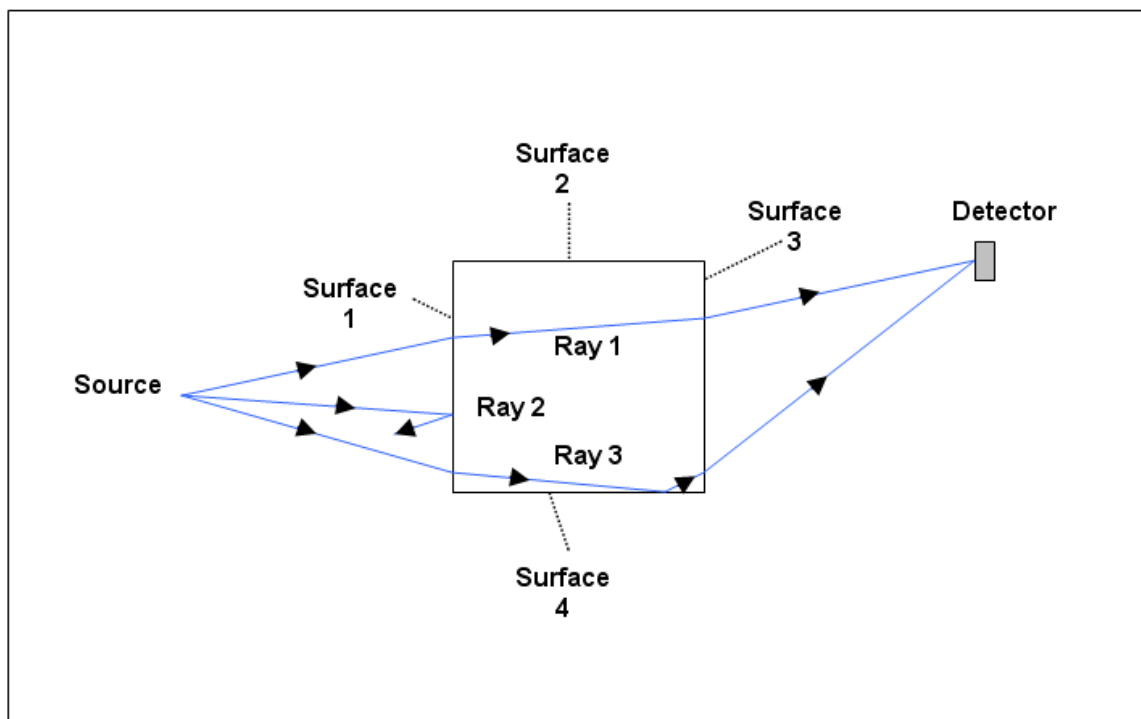


图 1.1-2 非顺序光线追迹

从图中可以看到，光线 1 从光源出发，经过表面 1，然后经过表面 3，最后到达探测器。光线 2 也从光源出发，但被表面 1 反射后，未能到达探测器。光线 3 从光源出发，依次经过表面 1、表面 4 和表面 3，最后到达探测器。很显然，不同的光线经过了不同的路径，它们经过光学系统中各个表面的顺序也不尽相同，只能由各个表面的几何形状和基本的物理规律来确定。在这种情况下，ZEMAX 需要追迹每一条具体的光线，根据每一条光线在传播过程中的具体情况来计算整个光学系统的性能。

如果要在 ZEMAX 中建立一个完整的顺序光学系统，至少需要提供以下数据：

- * 表面数目
- * 各表面相关数据
- * 系统孔径
- * 工作波长
- * 视场点

如果要建立一个完全的非顺序系统，至少需要提供以下数据：

- * 各物体(包括光源和探测器)的相关参数和空间位置
- * 工作波长

除了完全的顺序系统和完全的非顺序系统以外，ZEMAX 也提供一种混合系统模式。在这种模式下，ZEMAX 将系统的一部分按照顺序系统处理，而将系统的其余部分按照非顺序系统处理。

ZEMAX 的用户界面由不同的窗口组成。当运行 ZEMAX 时，将会看到一个缺省窗口，称为主窗口，如图 1.1-3 所示。在主窗口中，我们可以看到主菜单条、按钮条、镜头数据编辑器(Lens Data Editor, LDE)，以及状态条。

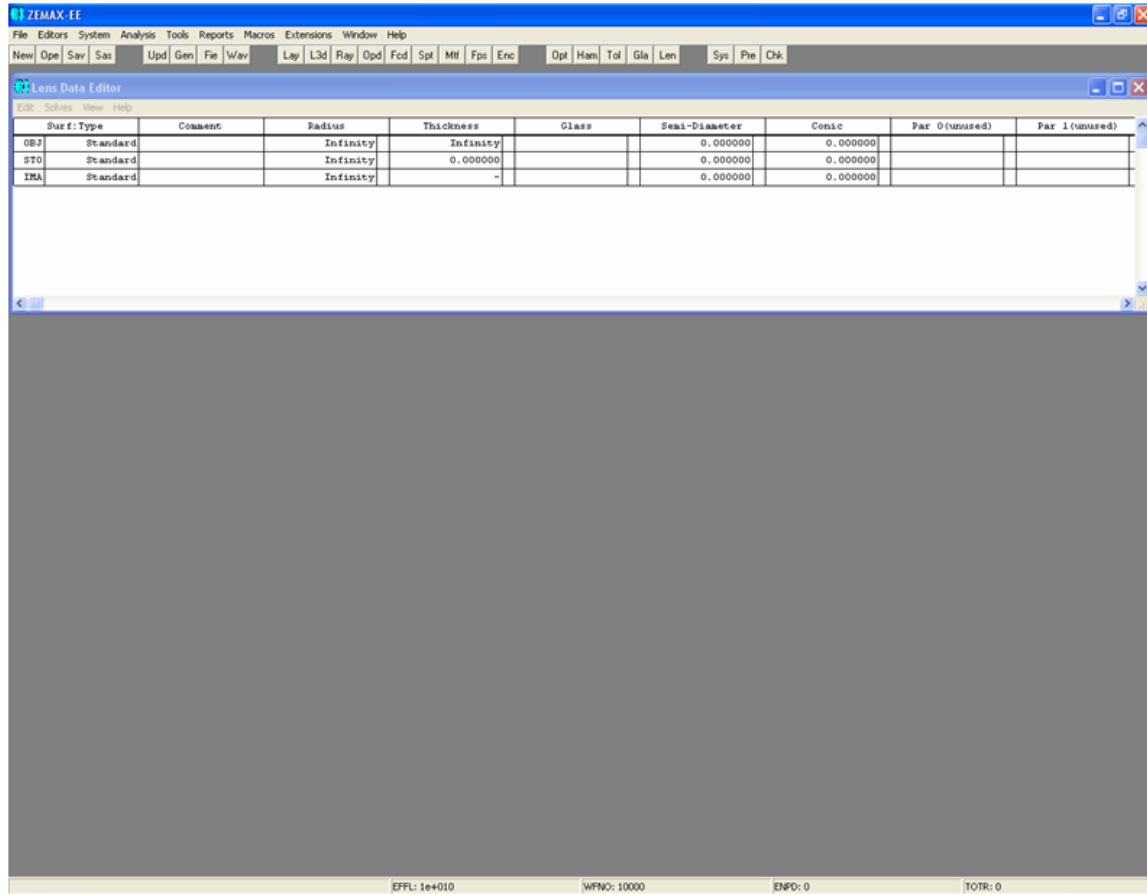


图 1.1-3 ZEMAX 主窗口

除主窗口外，ZEMAX 还有另外四种不同的窗口：

编辑器，用于定义和修改表面及其它数据；

图形窗口，用于显示图形数据；

文本窗口，用于显示文本数据；

对话框，用于编辑和确认有关其它窗口或整个光学系统的数据，报告出错信息等。

在图 1.1-3 所示的主窗口中，我们已经看到了镜头数据编辑器(LDE)。图 1.1-4 ~ 图 1.1-9 给出了 ZEMAX 常用到的不同编辑器和窗口。

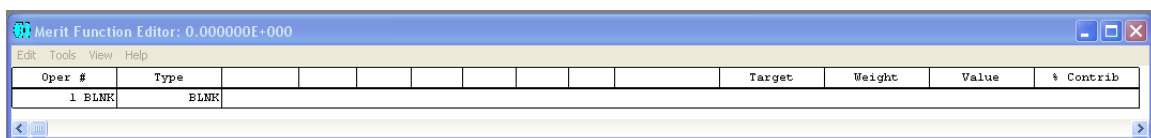


图 1.1-4 评价函数编辑器 (Merit Function Editor)

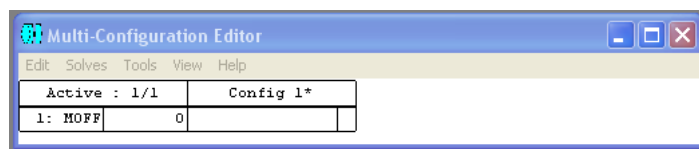


图 1.1-5 多组态编辑器(Multi-Configuration Editor)

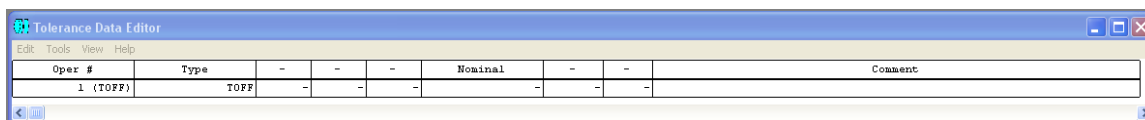


图 1.1-6 公差数据编辑器(Tolerance Data Editor)

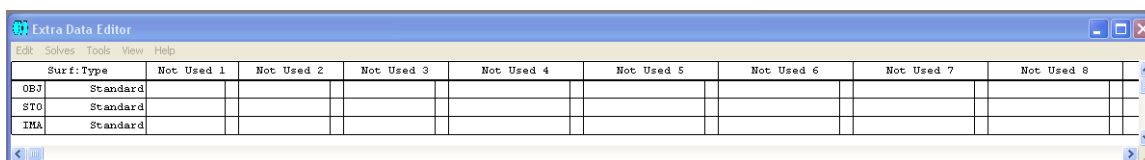


图 1.1-7 附加数据编辑器(Extra Data Editor)

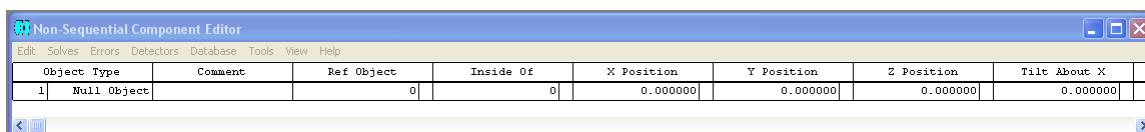


图 1.1-8 非顺序元件编辑器(Non-Sequential Component Editor)

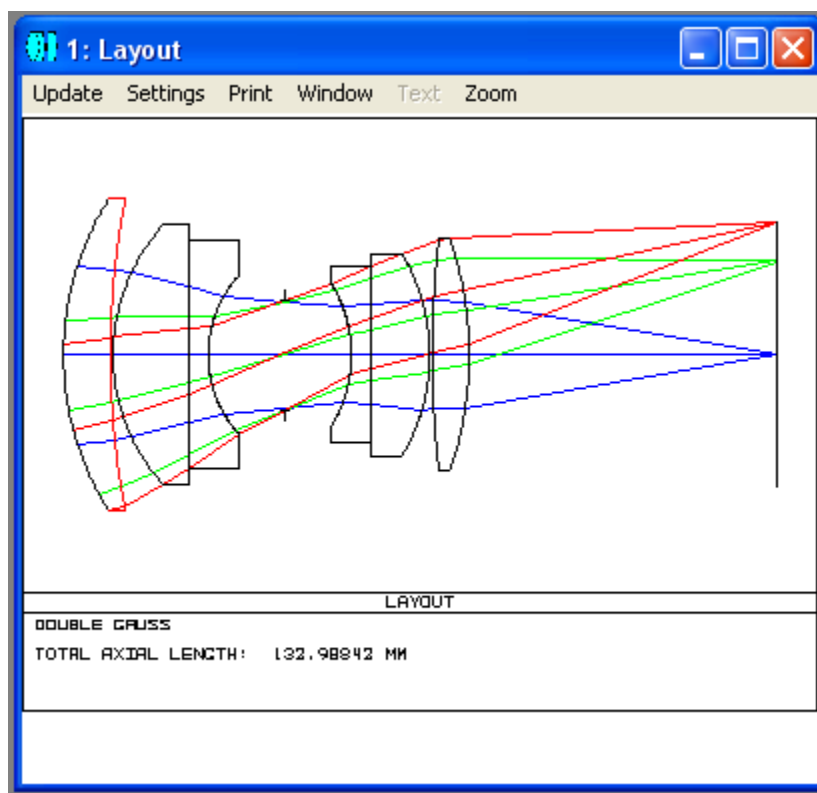


图 1.1-9 图形窗口(Graphic Window)

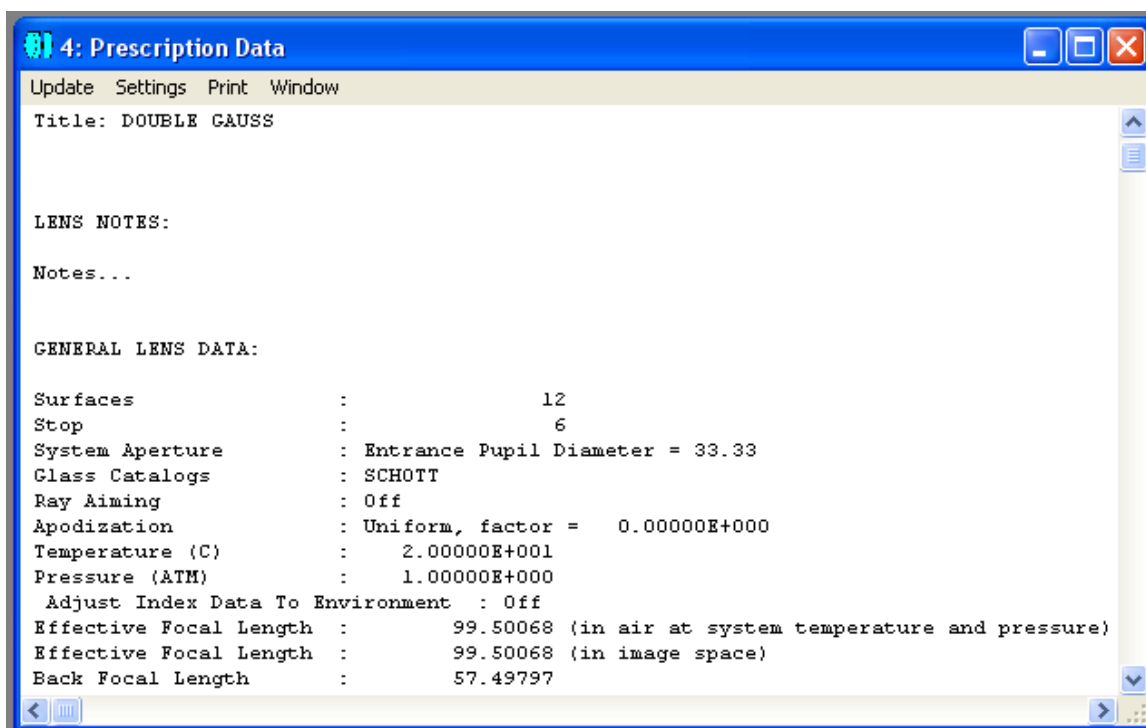


图 1.1-10 文本窗口(Text Window)

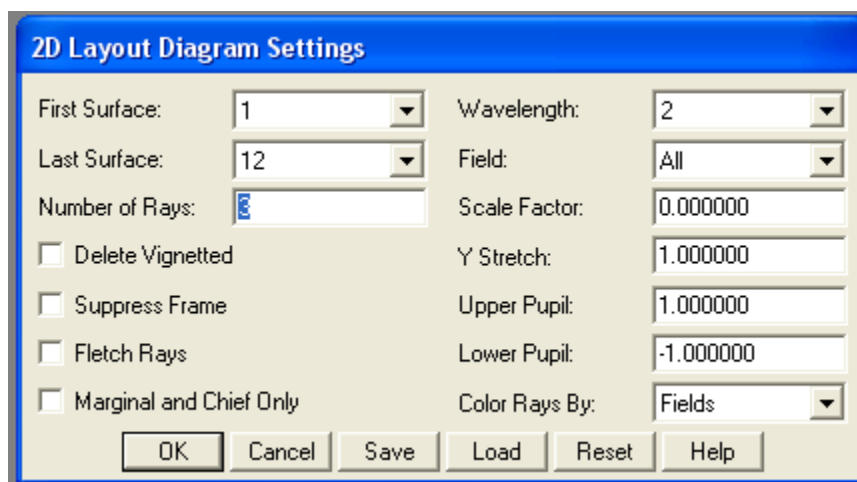


图 1.1-11 对话框(Dialog Box)

总的来说，ZEMAX 具有十分强大的光学设计功能。它可以准确地计算光路、折射和反射、相位和光程差、光学成像和畸变、偏振、镀膜的透射和吸收、散射等。但是，尽管具有强大的功能，ZEMAX 并不能帮你掌握基本的光学设计理论。一个好的光学设计师，应该将 ZEMAX 作为自己进行光学设计的一个有效工具，而不能完全依赖于这个工具。在必要的时候，光学设计师应该知道如何对这个工具进行扩展，使其更得心应手。

第二节 ZEMAX 编程语言(ZEMAX Programming Language, ZPL)简介

如上一节所述, 尽管 ZEMAX 已经具有十分强大的功能, 但是, 在有些情况下, 用户还需要对其功能作进一步扩展, 以满足自己特殊的设计需要。为此, ZEMAX 专门提供了一种编程语言, 称为 ZEMAX Programming Language (ZPL)。ZPL 是专门为 ZEMAX 设计的一种宏语言。它和大家熟悉的 BASIC 编程语言很相似, 区别在于并不是所有的 BASIC 语言的结构和关键词 ZPL 都能支持, 但另一方面, ZPL 又增加了许多和光学设计相关的函数、关键词及其它功能。

ZPL 程序可用任何 ASCII 文本编辑器(例如 Windows 下的 Notepad)进行编写。可以给 ZPL 程序取任何有效的文件名, 文件名可包含字母和数字, 字母不分大小写, 不能包含一些特殊字符如 ~ () = + - * / ! > < ^ & | # 等。程序的扩展名必须为 ZPL, 编写好的程序必须放在指定的路径下, 如果未指定路径, 缺省路径为 “\MACROS”。如果想要改变指定路径, 可通过 ZEMAX 主菜单下的 File → Preferences → Directories 进行修改, 如图 1.2-1 所示:

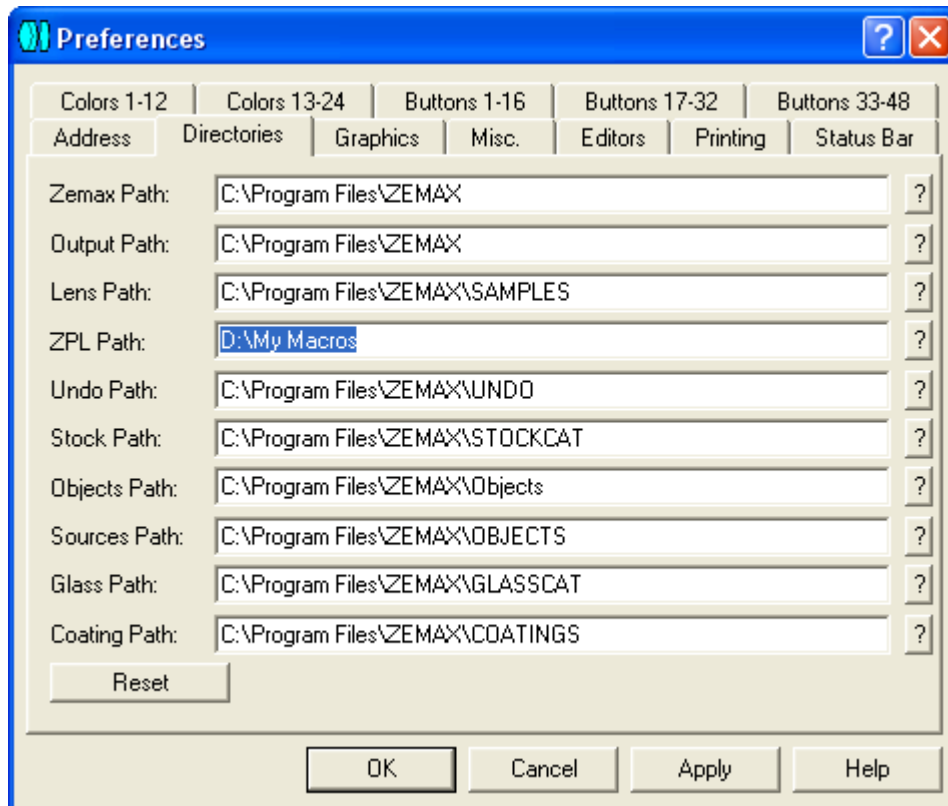


图 1.2-1 ZPL 路径设置

在每次修改路径后, 都应该通过 ZEMAX 主菜单下的 Macros → Refresh Macro List 对宏列表进行更新, 以保证在主菜单下能看到正确路径下的 ZPL 文件。

另外，也可以在设置好的路径下另外创建新的文件夹，并将 ZPL 程序放置其中，而不需要再修改对路径的设置。这样既方便操作，又利于管理。例如，在上图所示的“D:\My Macros”路径下，可以创建两个不同的文件夹 Project 1 和 Project 2，每个文件夹下可以放置不同的 ZPL 程序，如图 1.2-2 所示。

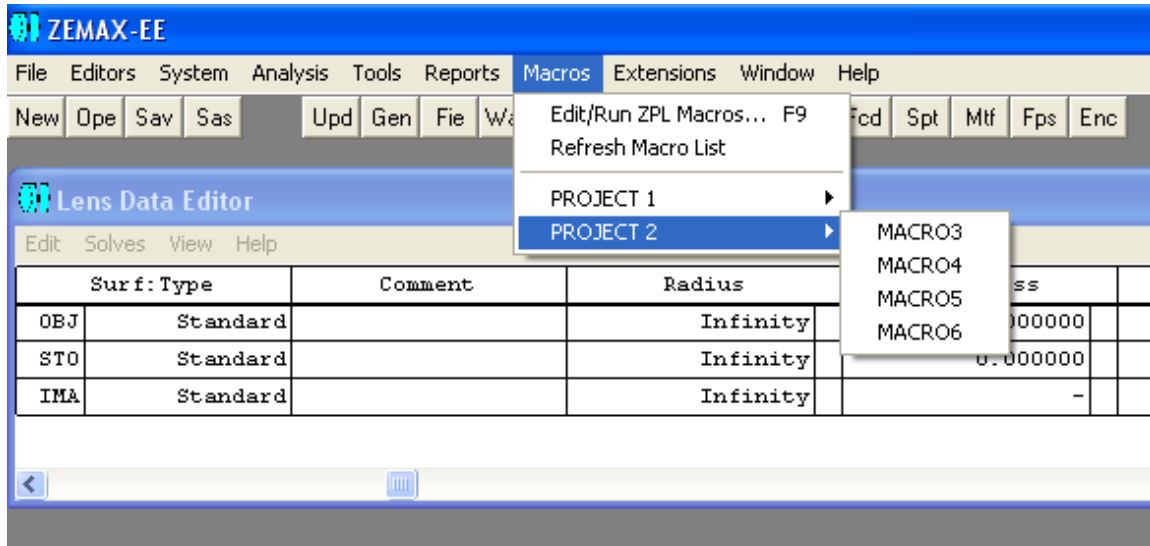


图 1.2-2 通过文件夹对 ZPL 进行管理

当然，在每次创建新的文件夹和编写新的 ZPL 程序后，都应该更新主菜单下的宏列表。

要运行 ZPL 程序，可以通过主菜单下的 Macros → Edit/ Run ZPL Macros 来进行。这时，在主窗口中将会出现 ZPL 控制对话框，如图 1.2-3 所示。

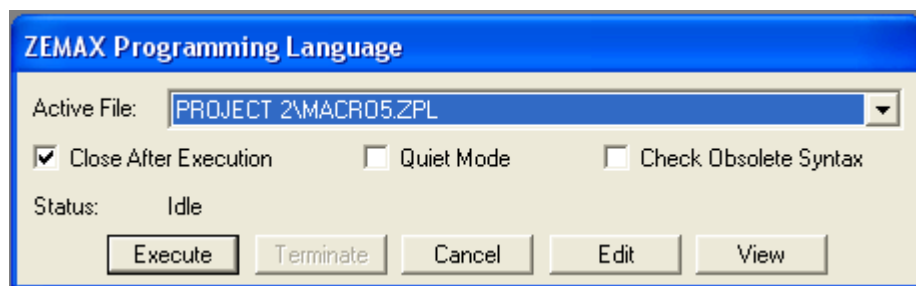


图 1.2-3 ZPL 控制对话框

在对话框中，有以下选项：

Active File: 通过下拉菜单选择当前路径下的所需执行的 ZPL 程序；

Close After Execution: 如果选择，在 ZPL 程序运行完后将自动关闭对话框；

Quiet Mode: 如果选择，将不会显示缺省的文本信息窗口，这对只关心图形输出的程序很有用；

Status: 在 ZPL 程序运行过程中, ZEMAX 将在此区域显示当前所运行的指令的所在的行数, 状态信息每隔 0.25 秒更新一次;

Execute: 运行 Active File 中所选择的 ZPL 程序;

Terminate: 中止正在运行的 ZPL 程序;

Cancel: 中止正在运行的 ZPL 程序; 如果没有程序正在运行, 则关闭 ZPL 对话框;

Edit: 激活 Windows 下的 Notepad, 对当前 ZPL 程序进行编辑;

View: 打开一个文本窗口, 显示当前 ZPL 程序, 但不能进行编辑。

除了通过 ZPL 控制对话框来运行 ZPL 程序外, 还可以直接点击主菜单中 Macros 下的程序名来运行, 如图 1.2-4 所示。

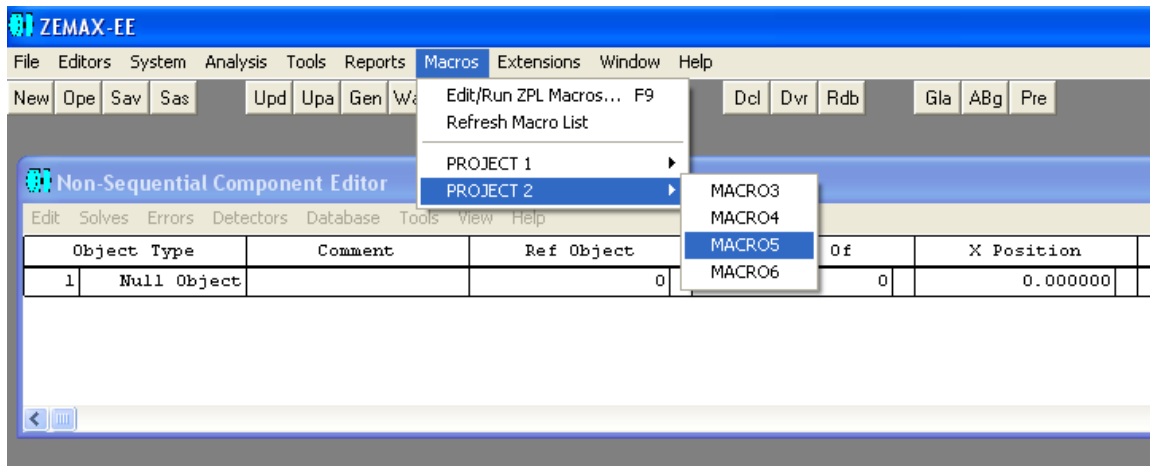


图 1.2-4 直接运行 ZPL 程序

另外, ZEMAX 提供了许多快捷按钮, 可以将一些常用的 ZPL 程序和这些按钮链接起来, 这样, 只要直接点击快捷按钮, 就可以运行相应的 ZPL 程序。快捷按钮的链接可以通过主菜单下的 File → Preferences 来实现。在 Preferences 对话框中, 可以选择将某一个按钮和一个程序链接起来, 如图 1.2-5 所示。

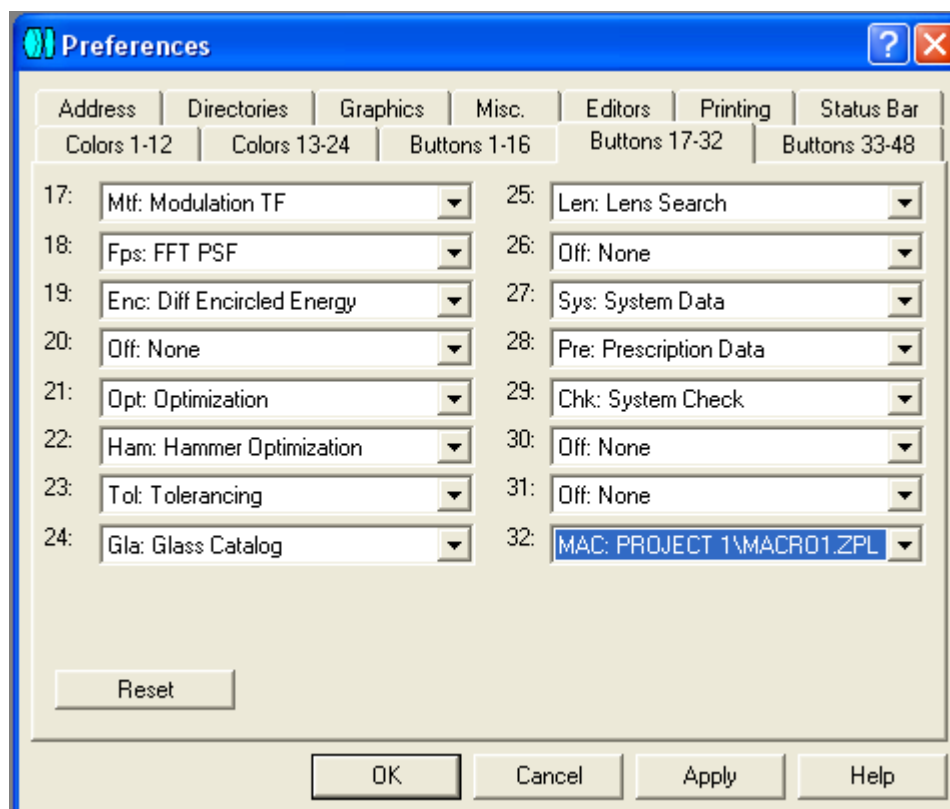


图 1.2-5 快捷按钮设置

第二章 ZEMAX 编程语言基本知识

和学习其它编程语言一样，学习 ZPL 的最佳途径就是通过实际应用。在本章中，我们将通过大量的例子，将 ZPL 的基本知识和编程技巧介绍给大家。

第一节 ZPL 程序的基本结构

首先，让我们来看一个具体的 ZPL 程序，如例 2.1-1 所示。

例 2.1-1: ZPL 程序基本结构

```
1 ! ex20101
2 ! This program introduces the basic structure of ZPL
3
4 ! The following 3 lines show 3 different ways of Comment
5 REM This is the first way
6 ! This is the second way
7 # This is the third way
8
9 ! The following 3 lines show some Assignments
10 x = 3
11 y = SINE(x)      # built in function
12 newString$ = "This is a string!"    # string assignment
13
14 ! The following line shows the PRINT Keyword
15 PRINT "Hello Optics!"
16
```

从例子中可以看到，程序是一个文本文件，由一系列命令行组成。命令行的内容可以是注释、赋值语句或关键词，当然也可以是空行。我们给每一行的前面加了一个行号，但这是为了方便解读，在实际的 ZPL 程序中不存在这些行号。

在 ZPL 中，可以有三种方式对程序进行注释，如第 5、6、7 行所示。第一种方式是以关键词 **REM** 开头，表明这一行是注释行，注释行不参加程序的运行。第二种方式是以符号“**!**”开头，也表明这一行是注释行。还有一种方式就是在命令行的任何位置插入符号“**#**”，表明本行中此符号后面的内容为注释，不参加程序的运行。

ZPL 中赋值语句的基本格式为：

variable = (expression)

其中，变量 **variable** 可以是以字母开头的任何字母和数字的组合，包括下划线“**_**”，但不能含有特殊字符如“**~()=+-* /!><^&|#**”等，也不能包含空格。ZPL 中变量名字母不分大小写。另外，变量名的长度不能超过 28 个字符。**x**、**y1**、**variable_z**、**myVariable**

等都是有效的变量名，但要注意的是，变量名不能与 ZPL 保留的关键词和函数名相同。作为一种好的编程习惯，选用变量名时既要简洁，同时又要便于理解，特别是当有多人阅读同一程序时。

ZPL 中有三种变量：数值变量、数组变量和字符串变量。我们将在下一节对这些不同的变量作进一步的讨论。

在赋值语句中，赋值符号“=”右边的表达式 (expression) 可以是常量、其它已经事先赋值的变量、或者包括不同常量、变量及函数等的复杂运算公式。当赋值语句运行时，先对表达式进行计算，然后再将表达式的结果赋给“=”左边的变量。

关键词用于完成特定的任务，如程序中的 PRINT 就可以将需要的结果在屏幕上显示出来。

第二节 ZPL 中的常量和变量

在 ZPL 中，常量是一些事先确定的数值量或字符串，如 0、-5、3.1416、“Here is my string”等，而变量则分为数值变量、数组变量和字符串变量，用于存储不同类型的数据。

1. 数值变量

数值变量用于存储数值量，即便其所存储的数值量并不事先知道。如果不特别指明，ZEMAX 会给每一个数值变量分配 64 比特空间，数值量以双精度数的形式存储。另外，ZPL 中也支持 32 比特的整形变量(其中包含符号位)。和很多其它高级语言不同，在 ZPL 中数值变量不需要预先声明。

2. 数值量的运算

ZPL 中可以对数值量进行最基本的四则运算，分别为：加(+)、减(-)、乘(*)、除(/)。如果需要进行更为复杂的运算如乘方、开方等，则需要通过 ZPL 内部定义的不同数值函数来实现。我们将在后面对此作进一步的介绍。

3. 数组变量

数组变量用于存储单维或多维数组。与普通数值变量不同的是，数组变量需要在使用前预先声明。数组声明的方法是：

```
DECLARE name, type, num_dimensions, dimension1 [, dimension 2 [, dimension 3 [, dimension 4] etc...]]
```

其中，数组名 *name* 可以是如前所述的任何有效的变量名，类型 *type* 必须是双精度型 (DOUBLE) 或整数型 (INTEGER)，维数 *num_dimensions* 必须是 1 至 4 之间的整数，包含 1 和 4，而各维长度 *dimension1*、*dimension2* 等也应为整数，定义了数组每一维的大小。需要指出的是，数组变量的下标索引从 1 开始，因此，一个长度为 10 的数组，其下标索引应该是从 1 到 10。例外是 ZPL 自己提供的 4 个一维数组 VEC1 ~ VEC4，其下标索引可以从 0 开始，见本小节最后的说明。

数组变量可以在程序的任何地方进行声明，只要在数组变量被使用之前就行。如果数组变量中存储的是数值量，则可对其进行如前所述的运算操作。

在数组变量使用完毕后，对于占内存较大的数组变量，如果在后面的程序运行中不需要继续使用，可以让 ZEMAX 释放内存，方法是：

RELEASE name

当然，即使在程序中不主动释放内存，在整个程序结束后，程序中用到的各个变量所占内存也都会被释放掉。

对数组变量进行赋值的方法是：

name (index1, index2, ...) = value

通过这种形式，可以将数值 *value* 赋给数组变量 *name (index1, index2, ...)*。在对数组赋值以后，可以在任何时候将储存在数组变量中的值取出赋给某一变量，方法如下：

newValue = name (index1, index2, ...)

下面我们给出一个有关数组变量的例子。

例 2.2-1: 数组变量的赋值与操作

```
1 ! ex20201
2 ! This example shows declaration and operations of Array Variables
3
4 DECLARE A, INTEGER, 2, 3, 3
5 A(1,1) = 8
6 A(1,2) = 1
7 A(1,3) = 6
8 A(2,1) = 3
9 A(2,2) = 5
10 A(2,3) = 7
11 A(3,1) = 4
12 A(3,2) = 9
13 A(3,3) = 2
14
15 x = A(1,1) + A(1,2) + A(1,3)
16 y = A(1,1) + A(2,1) + A(3,1)
17 z = A(1,1) + A(2,2) + A(3,3)
18
19 PRINT
20 PRINT "x = ", x, " y = ", y, " z = ", z
21
22 RELEASE A
```

在这个例子中，我们定义了一个整数型的 3 x 3 魔方矩阵。这个矩阵是一个 2 维数组，其每一行各元素的和、每一列各元素的和及两条对角线各元素的和均相等。

将此程序存为 EX20201.ZPL，按照第一章第二节所介绍的方法，点击 ZEMAX 主菜单中 Macros 下的程序名 EX20201，则可得如下运行结果：

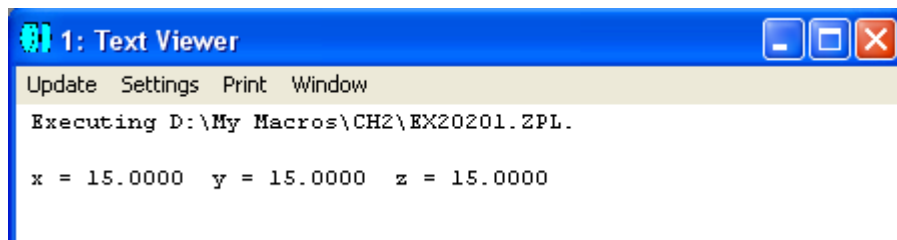


图 2.2-1 程序 ex20201.ZPL 运行结果

另外，为方便起见，ZPL 还定义了 4 个一维数组(矢量) VEC1、VEC2、VEC3 和 VEC4，用于存储双精度浮点数。每个数组的缺省长度为 1000。这几个数组可以直接使用而不需再行定义。需要指出的是，这 4 个一维数组的下标可以从 0 开始，如 VEC1(0)，因此缺省长度为 1000 实际上可以有 1001 个数组元素，即 VEC1(0) ~ VEC1(1000)。如果需要修改数组的长度，可通过关键词 SETVECSIZE 实现。我们将在下一章中结合一些实例对缺省数组的使用再作进一步的介绍。

4. 数值量的逻辑运算

逻辑运算用于构造复杂的命令，其最终结果为 0 或 1。大多数逻辑运算的形式为 (左表达式)(运算符)(右表达式)，唯一的例外是逻辑非运算 “!”，其运算形式为 !(右表达式)。

下表所列为 ZPL 支持的数值量逻辑运算符：

表 2.2-1 ZPL 所支持的数值量的逻辑运算

运算符	说明
&	与，当左右两个表达式均非 0 时，返回值 1
	或，当至少一个表达式非 0 时，返回值 1
^	异或，当只有一个表达式非 0 时，返回值 1
!	非，当右表达式非 0 时，返回值 0，否则返回值 1
= =	相等，当左右两个表达式相等时，返回值 1
>	大于，当左表达式大于右表达式时，返回值 1
<	小于，当左表达式小于右表达式时，返回值 1
>=	大于或等于，当左表达式大于或等于右表达式时，返回值 1
<=	小于或等于，当左表达式小于或等于右表达式时，返回值 1
!=	不等于，当左右表达式不相等时，返回值 1

5. 字符串变量及运算

ZPL 也支持字符串变量，并可对字符串进行运算。和普通数值变量一样，字符串变量也不需要事先声明。字符串变量以符号 “\$” 结尾，作为其特殊标志符。对字符串变量赋值的方法如下：

```
myString$ = “Here is my string”
```

其中 myString\$ 为字符串变量，双引号 “ ” 中的内容为字符串常量，但不包括双引号本身。

ZEMAX 还定义了许多有关字符串的函数。我们将在后面的章节对其进行详细介绍。

通过运算符 + 可将不同的字符串(包括字符串常量和字符串变量)相联在一起，方法如下：

```
total$ = "A$ is " + A$ + " and B$ is " + B$
```

通过关键词 **PRINT** 可以将字符串的内容在文本窗口中显示出来。需要注意的是，**PRINT** 只支持单一的字符串变量，所以不能在语句中进行字符串运算及调用字符串函数。如果要在同一行中显示不同的字符串，可以通过符号 "," 来实现，方法如下：

```
PRINT A$, B$, C$
```

另外，如果 **PRINT** 语句的最后一个字符为逗号 ",", 则打印结束后不换行，下一条打印语句在当前位置继续往下打印。

PRINT 还常常和另一个关键词 **REWIND** 一起使用。**REWIND** 的作用是删除 **PRINT** 语句所打印的最后一行信息，并将指针定位到前一行的末尾。这样可以通过覆盖的方式显示信息，在需要计数器时常常用到。

6. 字符串的逻辑运算

字符串的逻辑运算与数值量的逻辑运算很相似，主要的区别在于前者所比较的是字符串而不是数字。**ZPL** 所支持的字符串逻辑运算符如下表所示：

表 2.2-2 ZPL 所支持的字符串的逻辑运算

运算符	说明
\$=	相等，当左右两个字符串相等时，返回值 1
\$>	大于，当左字符串大于右字符串时，返回值 1
\$<	小于，当左字符串小于右字符串时，返回值 1
\$>=	大于或等于，当左字符串大于或等于右字符串时，返回值 1
\$<=	小于或等于，当左字符串小于或等于右字符串时，返回值 1
\$!=	不等于，当左右字符串不相等时，返回值 1

第三节 ZPL 中的函数

ZPL 中定义了许多数值函数，可用于计算不同的数值量。这些函数可能不包含任何参数，也可能包含一个或多个参数。不论有无参数，函数名后面都要求跟一对圆括号()，如果有参数，则置于圆括号内。所有的函数都只有一个返回值。

例 2.3-1 给出一个正弦函数 SINE(x) 的用法。

```
1 ! ex20301
2 ! This program shows how to use ZPL functions
3
4 pi = 3.1416    # define a constant
5
6 x = 45         # angle in degree
7
8 y = SINE(x*pi/180)
9
10 PRINT "SINE(",x," degree) = ", y
```

ZPL 中也定义了许多字符串函数，其返回值为字符串。我们将在下一章中对字符串函数作进一步介绍。

第四节 ZPL 中的关键词

关键词是 ZPL 的一个重要组成部分，用于控制程序流、输出结果，以及执行一些重要任务如修改镜头参数及进行光线追迹等。在本章前面的几个程序例子中用到的 DECLARE、RELEASE 及 PRINT 等都是常用的 ZPL 关键词。

关键词的基本用法为：

KEYWORD argument1, argument2, argument3...

有些关键词不带参数，而另一些有多个参数。参数可能为数值表达式、字符串常量或字符串变量。还有一些关键词接受数值与字符串的混合变量。

我们将在本章后面部分及第三章中对不同的关键词作进一步介绍。

作为参考，现将 ZPL 中涉及到的关键词列表于下：

APMN, APMX, APTP, APXD, APYD	EXPORTBMP
ATYP, AVAL	EXPORTCAD
BEEP	EXPORTJPG
CALLMACRO	EXPORTWMF
CALLSETDBL	FINDFILE
CALLSETSTR	FLDX, FLDY, FWGT, FVDX, FVDY,
COAT	FVCX, FVCY, FVAN
CLOSE	FOR, NEXT
CLOSEWINDOW	FORMAT
COLOR	FTYP
COMMAND	GCRS
COMMENT	GDATE
CONI	GETEXTRADATA
CONVERTFILEFORMAT	GETGLASSDATA
COPYFILE	GETLSF
CURV	GETMTF
DECLARE	GETPSF
DEFAULTMERIT	GETSYSTEMDATA
DELETE	GETTEXTFILE
DELETECONFIG	GETVARDATA
DELETEFILE	GETZERNIKE
DELETEMCO	GLAS
DELETEMFO	GLASSTEMPLATE
DELETEOBJECT	GLENSNAME
DELETETOL	GLOBALTOLOCAL
EDVA	GOSUB, SUB, RETURN, and END
END	GOTO

GRAPHICS	PLOT
GTEXT	PLOT2D
GTEXTCENT	POLDEFINE
GTITLE	POLTRACE
HAMMER	POP
IF-THEN-ELSE-ENDIF	PRINT
IMA	PRINTFILE
IMAGECOMBINE	PRINTWINDOW
IMAGEEXTRACT	PWAV
IMASHOW	QUICKFOCUS
IMASUM	RADI
IMPORTEXTRADATA	RANDOMIZE
INPUT	RAYTRACE
INSERT	RAYTRACEX
INSERTCONFIG	READ
INSERTMCO	READNEXT
INSERTMFO	READSKIP
INSERTOBJECT	READSTRING
INSERTTOL	RELEASE
LABEL	RELOADOBJECTS
LINE	REM, !, #
LOADARCHIVE	REMOVEVARIABLES
LOADCATALOG	RENAMEFILE
LOADDETECTOR	RETURN
LOADLENS	REWIND
LOADMERIT	SAVEARCHIVE
LOADTOLERANCE	SAVEDETECTOR
LOCALTOGLOBAL	SAVELENS
LOCKWINDOW	SAVEMERIT
MAKEFACETLIST	SAVETOLERANCE
MAKEFOLDER	SAVEWINDOW
MODIFYSETTINGS	SCATTER
NEXT	SDIA
NSLT	SETAIM
NSTR	SETAIMDATA
NUMFIELD	SETAPODIZATION
NUMWAVE	SETCONFIG
OPEN	SETDETECTOR
OPENANALYSISWINDOW	SETMCOPERAND
OPTIMIZE	SETNSCPARAMETER
OPTRETURN	SETNSCPOSITION
OUTPUT	SETNSCPROPERTY
PARM	SETOPERAND
PARAXIAL	SETSTDD
PAUSE	SETSURFACEPROPERTY, SURP
PIXEL	SETSYSTEMPROPERTY, SYSP

SETTEXTSIZE
SETTITLE
SETTOL
SETUNITS
SETVAR
SETVECSIZE
SETVIG
SHOWBITMAP
SHOWFILE
SOLVEBEFORESTOP
SOLVERETURN
SOLVETYPE
STOPSURF
SUB
SURFTYPE
TELECENTRIC
TESTPLATEFIT
THIC
TIMER
TOLERANCE
UNLOCKWINDOW
UPDATE
VEC1, VEC2, VEC3, VEC4
WAVL, WWGT
XDIFFIA
ZBF2MAT
ZBFCLR
ZBFMULT
ZBFPROPERTIES
ZBFREAD
ZBFRESAMPLE
ZBFSHOW
ZBFSUM
ZBFTILT
ZBFWRITE
ZRD2MAT
ZRDAPPEND
ZRDFILTER
ZRDPLAYBACK
ZRDSAVERAYS
ZRDSUM

第五节 ZPL 中的程序流控制

程序流控制是计算机语言中的一个核心部分。ZPL 提供了以下这些进行程序流控制的关键词：

```
FOR-NEXT  
IF-THEN-ELSE-ENDIF  
LABEL  
GOTO  
PAUSE  
GOSUB-SUB-RETURN-END
```

在此小节中，我们将对 FOR-NEXT、IF-THEN-ELSE-ENDIF、LABEL、GOTO 及 PAUSE 进行介绍，在第六节中，将对 GOSUB-SUB-RETURN-END 继续进行介绍。

FOR-NEXT 总是成对出现，用于定义一段需要重复执行多次的程序，其用法如下：

```
FOR variable, start_value, stop_value, increment  
(commands)  
NEXT
```

其中，FOR 标明需要重复执行的程序段的开始，在此语句中，需要定义一个变量 *variable* 作为计数器(此变量不一定为整数)，还需要定义计数器的起始值 *start_value*、中止值 *stop_value* 及增量 *increment*。NEXT 标明需要重复执行的程序段的结束。在 FOR-NEXT 之间的语句，则是需要重复执行的程序段，其重复执行的次数由 FOR 语句中定义的参数来控制。FOR-NEXT 关键词可以多层套用。

在 FOR 语句的第一个变量 *variable* 后面，除了跟逗号“,”之外，也可以改成等号“=”，即 *FOR variable = start_value, stop_value, increment*，有许多编程者偏爱这种形式，认为这样程序更加清晰。另外，在 NEXT 后面可以加上其它字符而不影响程序的运行，因此有很多编程者喜欢在 NEXT 后加上相应的循环变量名，特别是在多重循环套用的时候，这使程序结构更加清晰。需要指出的是，NEXT 后面的字符不具有任何实质意义，在使用时要注意别引起混淆。

当程序运行到 FOR 语句时，此语句中的起始值、中止值及增量将被存储起来。在随后的程序运行过程中，直至整个程序段重复运行结束，中止值及增量将不再改变。所以在程序段中其它地方不能再对其进行赋值。

例 2.5-1 给出一个 FOR-NEXT 应用的例子。

```

1 ! ex20501
2 ! This program shows how to use FOR/NEXT key words
3
4 start_value = 0
5 stop_value = 5
6 increment = 1
7
8 PRINT
9 FOR i, start_value, stop_value, increment
10   start_value = 8 # try to modify loop constant
11   stop_value = 60 # try to modify loop constant
12   increment = 2 # try to modify loop constant
13   PRINT "i = ", i
14 NEXT
15

```

注意我们在重复执行的程序体中试图对 start_value、stop_value 和 increment 重新进行赋值，但程序的运行结果并未受其影响，如图 2.5-1 所示。

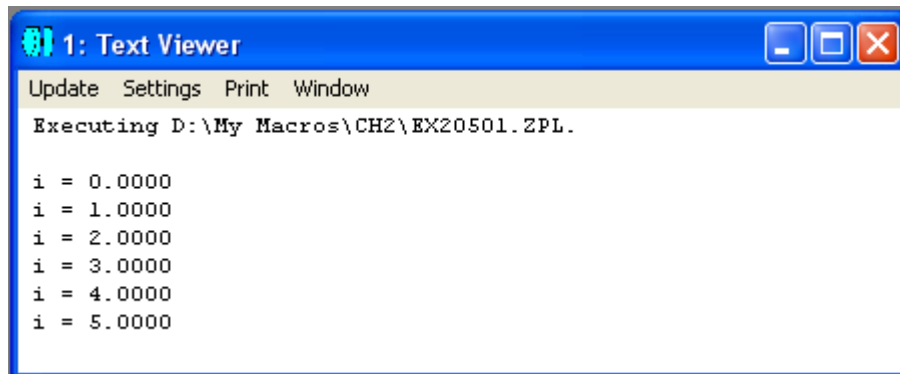


图 2.5-1 程序 ex20501.ZPL 运行结果

IF-THEN-ELSE-ENDIF 为条件执行语句，其用法为：

```

IF (expression)
  (commands)
ELSE
  (commands)
ENDIF

```

或

```

IF (expression) THEN (command)

```

当逻辑表达式 expression 结果为真时，执行 IF 后面的 commands 命令语句，否则执行 ELSE 后面的 commands 命令语句。注意此处的括号 () 可以省略。一般情况

下，条件表达式中 IF 和 ENDIF 总是成对出现，而 ELSE 则为可选项。IF-ENDIF 关键词可以多层套用。

IF (expression) THEN (command) 为条件表达式的简略形式，用于只有单一命令语句需要执行的情况。在简略形式中，不需要 ENDIF，也不支持 ELSE 选项。

例 2.5-2 给出一个 IF-THEN-ELSE-ENDIF 条件语句应用的例子。程序中还用到了一个随机函数 RAND(x)，用于产生一个 0 和 x 之间的随机浮点数。

```

1 ! ex20502
2 ! This program shows how to use IF-ELSE-ENDIF key words
3
4 theta0 = 45
5 theta = RAND(90)
6
7 PRINT
8 IF (theta > theta0)
9     PRINT theta, " is larger than 45 degree"
10 ELSE
11     IF (theta == theta0) THEN PRINT "theta equals to 45 degree"
12     IF (theta < theta0) THEN PRINT theta, " is smaller than 45 degree"
13 ENDIF
14

```

在某些情况下，需要让程序转向指定的位置去继续执行，这时就可以用到关键词 GOTO。关键词 GOTO 总是与另一关键词 LABEL 配合使用，其用法为：

```

LABEL label_number
...
GOTO label-number

```

或者：

```

LABEL text_label
...
GOTO text_label

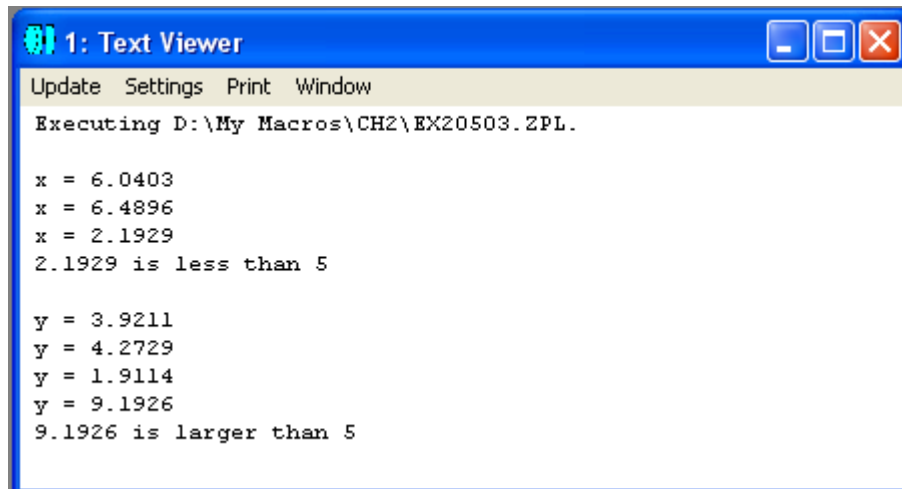
```

其中，LABEL 后面可以跟任意数字或字符串（此处可以理解为数字也是一种字符），并可置于程序中的任意行。当程序执行到 GOTO 语句时，转向相应的 LABEL 语句，继续执行 LABEL 语句后面的语句。

例 2.5-3 给出一个 GOTO 语句的例子：

```
1 ! ex20503
2 ! This program shows how to use GOTO/LABEL key words
3
4 PRINT
5 LABEL 01.23
6 x = RAND(10)
7 PRINT "x = ", x
8 IF x >= 5 THEN GOTO 01.23
9 PRINT x, " is less than 5"
10 PRINT
11
12 LABEL anotherLabel
13 y = RAND(10)
14 PRINT "y = ", y
15 IF y <= 5 THEN GOTO anotherLabel
16 PRINT y, " is larger than 5"
```

其运行结果如下：



```
1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH2\EX20503.ZPL.

x = 6.0403
x = 6.4896
x = 2.1929
2.1929 is less than 5

y = 3.9211
y = 4.2729
y = 1.9114
y = 9.1926
9.1926 is larger than 5
```

图 2.5-2 程序 ex20503.ZPL 运行结果

需要指出的是，在结构化的程序设计中，一般不推荐使用 GOTO 语句，因为这很容易造成程序结构不清晰，出错时难于诊断。但遗憾的是，ZPL 中不支持类似于 C 语言中的 WHILE 条件循环语句，只能用 GOTO 语句配合 LABEL 语句来实现，因此，我们在 ZPL 中使用 GOTO 语句时应该加倍小心。

ZPL 中还有一个关键词 PAUSE，用于暂停程序的执行，并在消息窗口中显示相应信息，等待用户的响应。在用户按 OK 按钮后，程序从暂停处继续执行。关键词 PAUSE 的用法如下：

PAUSE
或
PAUSE message

其中，message 可以是任何数值量，也可以是字符串。

例 2.5-4 给出一个 PAUSE 语句的例子：

```
1 ! ex20504
2 ! This program shows how to use PAUSE key word
3
4 PRINT
5 FOR i, 1, 5, 1
6   PRINT "i = ", i
7   IF i == 5
8     PRINT "Waiting for approval ..."
9     PAUSE "We just finished i = 5, please responde..."
10  ENDIF
11 NEXT
12 PRINT
13 PRINT "APPROVED!"
14
```

程序运行到 $i = 5$ 时，将会显示一个消息窗口，并暂停运行，如图 2.5-3 所示：

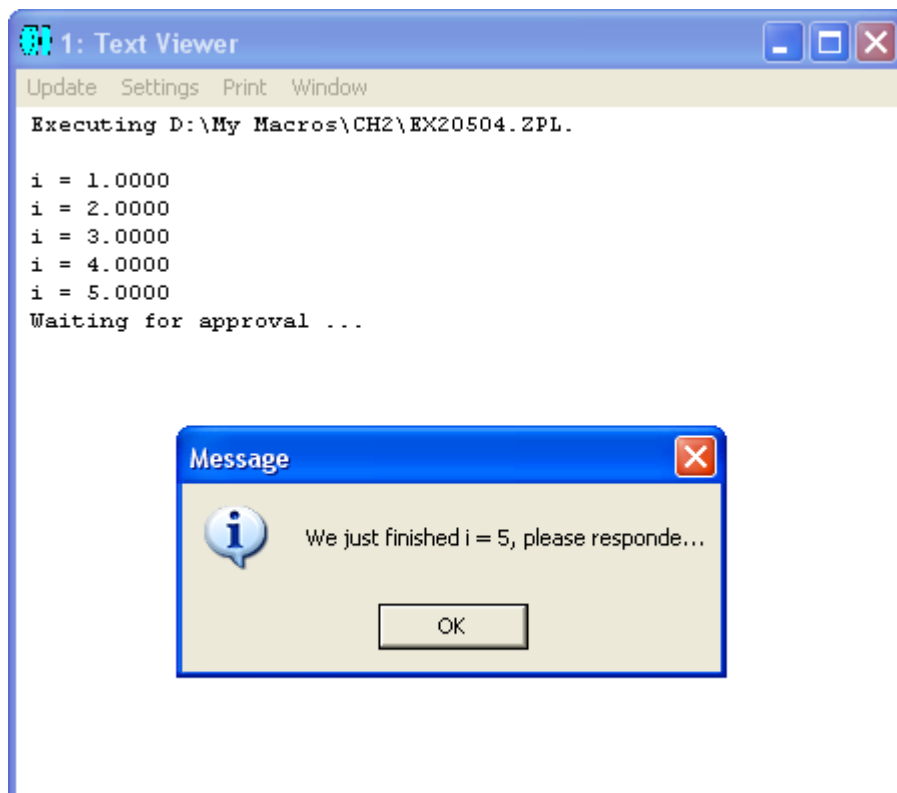
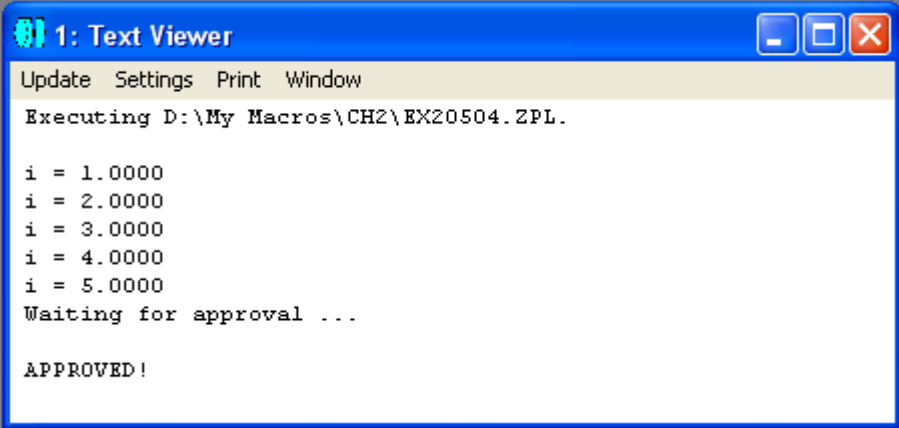


图 2.5-3 程序 ex20504.ZPL 运行结果(暂停处)

在按 OK 按钮后，程序从暂停处继续执行，显示结果如下：



```
1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH2\EX20504.ZPL.

i = 1.0000
i = 2.0000
i = 3.0000
i = 4.0000
i = 5.0000
Waiting for approval ...

APPROVED!
```

图 2.5-4 程序 ex20504.ZPL 的最后运行结果

第六节 ZPL 中的子程序

在 ZPL 中，可以定义子程序，并在主程序或其它子程序中对其进行调用。子程序的定义方法如下：

```
SUB sub_name
(commands)
RETURN
```

子程序由关键词 SUB 开始，并在 SUB 语句中给出子程序的名称 sub_name。SUB 后面的 commands 部分，是子程序的主体，用于完成子程序的特定功能。子程序必须以 RETURN 语句结束，但在子程序体的其它地方，也可以使用另外的 RETURN 语句。有的时候，为了使程序结构清晰，可以在 RETURN 后面加上子程序名。但要注意的，RETURN 后面的子程序名没有任何实质上的意义，甚至可以是其它的任何名称，所以在使用时要注意别引起混淆。

ZPL 中规定，如果程序中用到子程序，那么至少要用一个 END 语句标明主程序的结束，而且主程序应置于子程序之前。

另外，ZPL 中的变量为全局变量，因此，如果在子程序中对某一变量进行修改，在整个程序的其它地方此变量值也会改变。

例 2.6-1 给出一个子程序的例子：

```
1 ! ex20601
2 ! This program shows how to use GOSUB-SUB-RETURN-END keywords
3
4 ! This is the main program
5 x = 3
6 y = 2
7 GOSUB findMax
8 PRINT
9 PRINT " x = ", x, ", y = ", y, ", and ", max, " is larger than ", min
10 END # This is the end of the main program
11
12 ! This is the sub-routine
13 SUB findMax
14 x = RAND(10)
15 y = RAND(10)
16 IF x > y
17   max = x
18   min = y
19   RETURN # This is another RETURN
20 ENDIF
21 max = y
22 min = x
23 RETURN # This is the end of the sub-routine
```

注意我们在主程序中先定义了 x 和 y 的值，但在子程序中，由随机函数产生的新的 x 和 y 的值覆盖了原来的值，因此在主程序最后显示出来的值是新的值。另外，在子程序中我们用到了两个 **RETURN** 语句，如果 $x > y$ ，则子程序在第一个 **RETURN** 处就会结束，返回主程序，而不必继续运行子程序后面的部分。

图 2.6-1 给出此程序的运行结果：

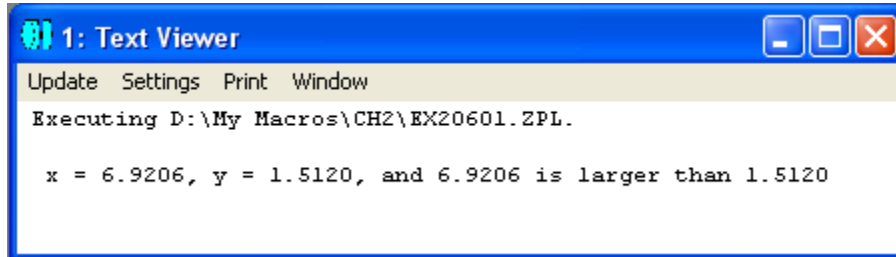


图 2.6-1 程序 ex20601.ZPL 的运行结果

第七节 ZPL 中的输入输出和文件操作

ZPL 提供了一个关键词 INPUT，允许在程序运行过程中由用户通过键盘输入数值信息或字符串信息。INPUT 的用法如下：

```
INPUT "Prompt String", variable
INPUT variable
INPUT "Prompt String", string_variable$
INPUT string_variable$
```

例 2.7-1 给出一个使用关键词 INPUT 的例子：

```
1 ! ex20701
2 ! This program shows how to use INPUT keyword
3
4 INPUT "Enter value for x:", x
5 PRINT "x = ", x
6
7 INPUT "Enter string:", string$
8 PRINT "string is '", string$, "'"
```

当程序执行到每一个语句时，都会弹出一个输入窗口，显示相应的信息，并等待用户输入，如图 2.7-1 所示：

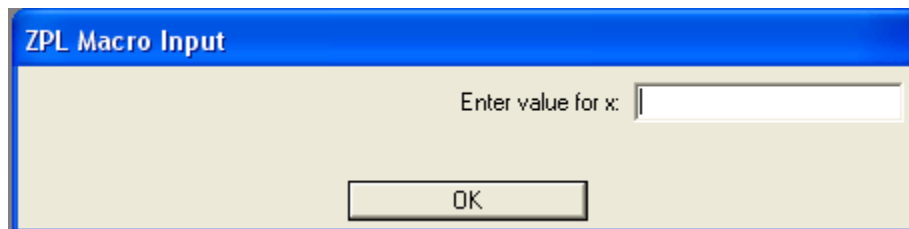


图 2.7-1 程序 ex20701.ZPL 的运行时弹出的第一个输入窗口

当用户输入相应的数值或字符串信息并按 OK 按钮确认后，用户输入的值就会被储存在到相应的变量中，然后程序继续运行。

我们在前面已经知道，ZPL 通过关键词 PRINT 可以在消息窗口中输出数值或字符串信息。实际上，PRINT 既可以往屏幕上输出信息，也可以往文件中输出信息，这通过关键词 OUTPUT 来控制。OUTPUT 的用法如下：

```
OUTPUT SCREEN
OUTPUT filename$
OUTPUT filename$, APPEND
```

如果 OUTPUT 后面跟 SCREEN，则随后的 PRINT 语句将把结果显示在屏幕上，而如果 OUTPUT 后面跟文件名 filename\$，则随后的 PRINT 语句将把结果输出到相应的文件中。另外，如果在 OUTPUT 中用到 APPEND，则将结果输出到相应文件的最后而不覆盖文件已有的内容。

例 2.7-2 给出一个使用关键词 OUTPUT 的例子：

```
1 ! ex20702
2 ! This program shows how to use OUTPUT keyword
3
4 filename$ = "D:\My Macros\ch2\output_files\test1.txt"
5
6 OUTPUT filename$
7 PRINT "You will see this line in the file, but not on the screen."
8
9 OUTPUT SCREEN
10 PRINT
11 PRINT "You will see this line on the screen, but not in the file."
12
13 OUTPUT filename$, APPEND
14 PRINT "You will see this line in the file as a new line."
15
16 OUTPUT "D:\My Macros\ch2\output_files\test2.txt"
17 PRINT "You will see this line in a different file."
```

其中我们假定文件夹 “D:\My Macros\ch2\output_files\” 已经存在，否则 ZEMAX 将会给出出错信息。图 2.7-2 为程序运行后在屏幕窗口及不同文件中看到的内容：

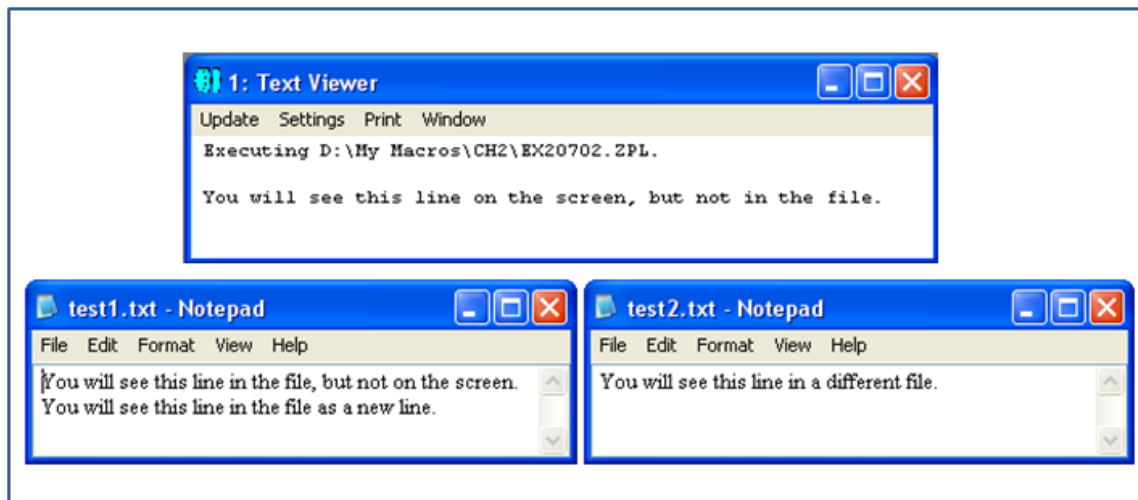


图 2.7-2 程序 ex20702.ZPL 的运行结果

关键词 PRINT 常和另外一个关键词 FORMAT 配合使用。FORMAT 可以控制打印到文本窗口或文件中的数值量的格式，其用法为：


```

FORMAT m.n
FORMAT m.n EXP
FORMAT m INT
FORMAT "C_format_string" LIT

```

其中，m 和 n 为整数，由小数点 "." 分开。m 代表打印的总字符数，包括空格，n 代表打印在小数点后的位数。如果 m.n 后跟有 EXP，表示打印格式为科学计数法，如果 m 后跟有 INT，表示打印的数值量应先转换为整数形式再打印出来。如果用到 LIT，则表明所打印的格式为 C 语言中的输出控制格式，由字符串 "C_format_string" 决定。

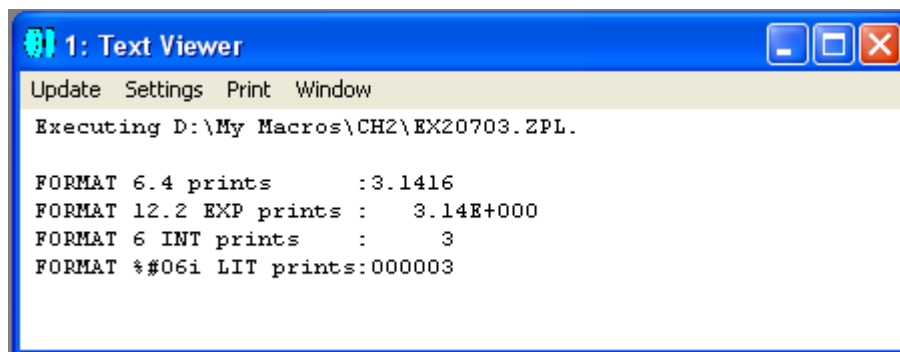
例 2.7-3 给出了一些应用关键词 FORMAT 的例子。

```

1 ! ex20703
2 ! This program shows how to use keyword FORMAT
3
4 x = 3.14159
5
6 PRINT
7 FORMAT 6.4
8 PRINT "FORMAT 6.4 prints      :", x
9
10 FORMAT 12.2 EXP
11 PRINT "FORMAT 12.2 EXP prints :", x
12
13 FORMAT 6 INT
14 PRINT "FORMAT 6 INT prints    :", x
15
16 FORMAT "%#06i" LIT
17 PRINT "FORMAT %#06i LIT prints:", x

```

图 2.7-3 为程序运行后在屏幕窗口中看到的内容：



```

1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH2\EX20703.ZPL.

FORMAT 6.4 prints      :3.1416
FORMAT 12.2 EXP prints :  3.14E+000
FORMAT 6 INT prints    :      3
FORMAT %#06i LIT prints:000003

```

图 2.7-3 程序 ex20703.ZPL 的运行结果

除了前面介绍的用 INPUT 通过屏幕和键盘输入信息外，ZPL 还支持打开文本文件并从中读取数值或字符串信息。对文本文件进行操作可能用到的关键词及函数如下：

OPEN, READ, READNEXT, READSTRING, CLOSE, EOF()

其中，OPEN 用于打开文本文件，其用法为：

OPEN "filename"

或

OPEN filename\$

在文件打开后，可通过 READ、READNEXT 或 READSTRING 读取信息。其中，READ 用于读取整行信息，并将所读的数值量存于 READ 后面所跟的数值变量中，用法为：

READ x, y, z, ...

在使用 READ 读取数值信息时，READ 后面的变量的个数应该和数据文件中读取行的数据个数相同，否则 ZEMAX 会给多出的变量赋 0。

除了用 READ 整行读取数据外，也可以用关键词 READNEXT 读取数据，其与 READ 的区别在于，READNEXT 读取数据的个数仅与其后面所跟的变量个数相同。READNEXT 的用法为：

READNEXT x, y, z, ...

另外，如果要读取的信息为字符信息，可用 READSTRING 关键词。READSTRING 的用法如下：

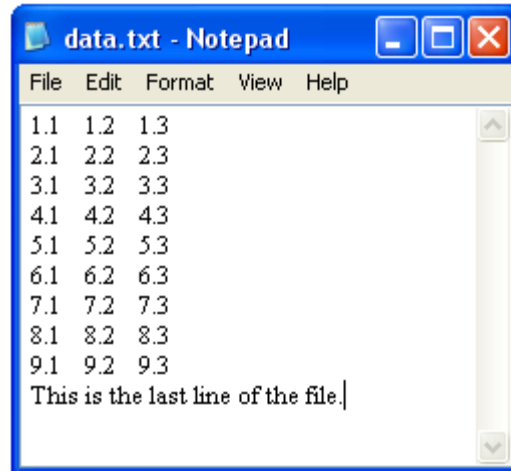
READSTRING textString\$

READSTRING 将整个读取行的字符信息存入其后所跟的字符串变量 textString\$ 中。

有些时候，我们需要判断是否已经读到文件末尾，这时，可以用 ZPL 所提供的函数 EOF()。如果已经读到文件末尾，此函数返回值 1，否则返回值 0。EOF() 只有在 READ、READNEXT 或 READSTRING 后面使用才有意义。

需要记住的是，当文件读取完毕后，应使用关键词 CLOSE 将文件关闭。

现在，假设我们有一个文本文件 “data.txt”，其内容如下：



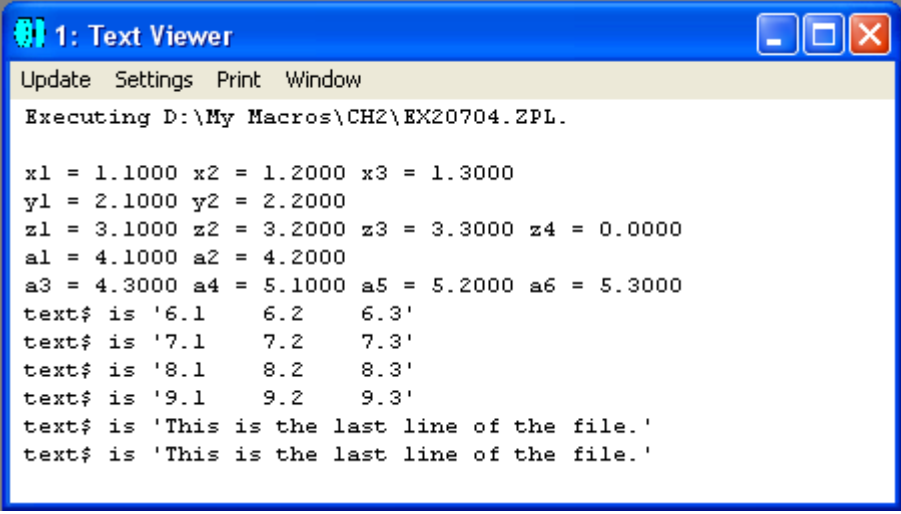
在例 2.7-4 中，我们用上面所介绍到的关键词从 “data.txt” 中读取信息：

```

1 ! ex20704
2 ! This program shows how to use READ series keyword
3
4 filename$ = "D:\My Macros\CH2\data_files\data.txt"
5
6 OPEN filename$
7
8 PRINT
9 READ x1, x2, x3
10 PRINT "x1 = ", x1, " x2 = ", x2, " x3 = ", x3
11
12 READ y1, y2
13 PRINT "y1 = ", y1, " y2 = ", y2
14
15 READ z1, z2, z3, z4
16 PRINT "z1 = ", z1, " z2 = ", z2, " z3 = ", z3, " z4 = ", z4
17
18 READNEXT a1, a2
19 PRINT "a1 = ", a1, " a2 = ", a2
20
21 READNEXT a3, a4, a5, a6
22 PRINT "a3 = ", a3, " a4 = ", a4, " a5 = ", a5, " a6 = ", a6
23
24 LABEL 1
25 IF (EOFF() != 1)
26     READSTRING text$
27     PRINT "text$ is '", text$, "'"
28     GOTO 1
29 ENDIF
30
31 CLOSE

```

其运行结果如图 2.7-4 所示：



The screenshot shows a window titled "1: Text Viewer" with a menu bar containing "Update", "Settings", "Print", and "Window". The main text area displays the output of a ZPL macro execution. The text is as follows:

```
Executing D:\My Macros\CH2\EX20704.ZPL.  
  
x1 = 1.1000 x2 = 1.2000 x3 = 1.3000  
y1 = 2.1000 y2 = 2.2000  
z1 = 3.1000 z2 = 3.2000 z3 = 3.3000 z4 = 0.0000  
a1 = 4.1000 a2 = 4.2000  
a3 = 4.3000 a4 = 5.1000 a5 = 5.2000 a6 = 5.3000  
text$ is '6.1    6.2    6.3'  
text$ is '7.1    7.2    7.3'  
text$ is '8.1    8.2    8.3'  
text$ is '9.1    9.2    9.3'  
text$ is 'This is the last line of the file.'  
text$ is 'This is the last line of the file.'
```

图 2.7-4 程序 ex20704.ZPL 的运行结果

(注意最后一行屏幕输出了两遍同样的信息，这可能是 ZEMAX 的一个 BUG。)

有关更多的输入输出及文件操作的内容，我们将在下一章中作进一步的介绍。

第三章 ZPL 常用指令详解

ZPL 提供了很丰富的关键词和函数，特别是与光学设计相关的部分。在 ZEMAX 的用户指南中，按字母顺序对这些关键词和函数逐条进行了介绍。在本章中，我们准备重复 ZEMAX 用户指南中的内容，而将从一个不同的角度，把在光学设计中常用到的关键词和函数分类进行介绍，以方便读者学习。在本书最后的附录中，我们会将本书中所介绍到的关键词和函数按字母顺序列出，并指出在本书中对其介绍和使用的章节，以便于读者查阅。

在本章中某些地方，我们将不特别区分关键词或函数，而是把它们统称为指令。关键词和函数的主要区别在于，前者没有返回值，而后者有返回值。

需要指出的是，随着 ZEMAX 的不断发展，其指令集也在更新和完善。有一些旧版本中的指令，已经被新版本中的其它指令所取代，而新版本中的某些指令，很可能是旧版本中所没有的。由于种种不同的原因，读者手中的 ZEMAX 软件版本可能不是最新版本，因此，建议读者参阅相应的 ZEMAX 用户手册，并以其中的说明为准。

第一节 ZPL 中的数值运算函数

在上一章中，我们已经接触过 ZPL 中的数值运算函数。表 3.1-1 列出了 ZPL 所支持的数值运算函数：

表 3.1-1 ZPL 中的数值运算函数

函数	返回值
ABS0(x)	变量的绝对值
ACOS(x)	变量的反余弦值，单位为弧度
ASIN(x)	变量的反正弦值，单位为弧度
ATAN(x)	变量的反正切值，单位为弧度
COSI(x)	单位为弧度的变量的余弦值
EXPE(x)	e 为底的指数函数
EXPT(x)	10 为底的指数函数
GAUS(x)	分布为高斯函数(平均值为 0，标准偏差为所给变量)的随机数
INTE(x)	不大于所给变量的最大整数
LOGE(x)	e 为底的对数函数
LOGT(x)	10 为底的对数函数
MAGN(x,y)	(x^2+y^2) 的平方根，x 和 y 为实数
POWR(x,y)	$ x ^y$ ，x、y 为任意实数
RAND(x)	在 0 和变量之间均匀分布的随机数
SCOM(A\$, B\$)	如果两字符串 A\$和 B\$相等，则返回值 0 如果 A\$ < B\$，则返回值 -1 如果 A\$ > B\$，则返回值 +1
SIGN(x)	如果变量值小于 0，返回值-1 如果变量值等于 0，返回值 0 如果变量值大于 0，返回值+1
SLEN(A\$)	字符串变量 A\$ 中字符的数目
SINE(x)	单位为弧度的变量的正弦值
SQRT(x)	变量的平方根
SVAL(A\$)	字符串变量 A\$ 相应的浮点数值
TANG(x)	单位为弧度的变量的正切值

第二节 ZPL 中的字符串函数

在上一章中我们介绍到，ZPL 中也定义了许多字符串函数，其返回值为字符串。表 3.2-1 列出了 ZPL 所支持的字符串函数：

表 3.2-1 ZPL 中的字符串函数

字符串函数	返回值
\$BUFFER()	镜头缓存区中的当前字符串。此函数用于提取字符串
\$COAT(i)	第 i 表面的镀膜名称
\$COMMENT(i)	第 i 表面的注释字符串
\$DATE()	当前日期和时间的字符串，其格式由 ZEMAX 设定
\$EXTENSIONPATH()	ZEMAX 扩展模块的路径名
\$FILENAME()	当前镜头文件的名称，不包含路径
\$FILEPATH()	当前镜头文件的名称，包含完整路径
\$GETSTRING(A\$, n)	字符串 A\$ 的第 n 个子字符串。空格和单引号为分隔符
\$GLASS(i)	第 i 表面的玻璃名称
\$LEFTSTRING(A\$, n)	字符串 A\$ 的左起前 n 个字符。如果 A\$ 中的字符数少于 n，则以空格填充。
\$LENSNAME()	General System 中定义的镜头名称
\$NOTE(line#)	General System 中定义的第 line# 行注释信息。当遇到换行符或者所在行的总字符数超过 100 时，则新起一行。如果所指定的行中没有内容，则返回一个空字符。
\$OBJECTPATH()	NSC 物体的路径名
\$PATHNAME()	当前镜头文件的路径名
\$RIGHTSTRING(A\$, n)	字符串 A\$ 的最右边 n 个字符。如果 A\$ 中的字符数少于 n，则以空格填充。
\$STR(expression)	将表达式的值以字符串形式返回，其格式由关键词 FORMAT 定义
\$TEMPFILENAME()	返回一个临时文件的全名，包含路径。此临时文件可用于文本或二元数据。
\$UNITS()	当前镜头系统的单位，为 MM、CM、IN 或 M

第三节 ZEMAX 运行环境和系统参数的设置与读取

熟悉 ZEMAX 的读者都知道，在使用 ZEMAX 进行光学设计之前，一般都需要告诉 ZEMAX 一些基本的系统参数，如工作波长、镜头单位、孔径类型等。ZPL 提供了一个关键词 SETSYSTEMPROPERTY (或其缩写形式 SYSP)，可以对许多系统参数进行设置。此关键词的使用方法为：

SETSYSTEMPROPERTY code, value1, value2

或

SYSP code, value1, value2

其中，code 为表 3.3-1 中所列的代码之一，value1 和 value2 为想要设置的参数，可以是数值量或字符串。大多数代码只需要一个参数，还有一些代码需要两个参数。

表 3.3-1

代码	说明
10	Aperture type, 系统孔径类型, 0 为入瞳直径, 1 为像空间 F/#, 2 为物空间数值孔径, 3 为由光阑尺寸确定, 4 为近轴系统像空间工作 F/#, 5 为物空间锥形张角。
11	Aperture Value, 系统孔径值
12	Apodization Type, 变迹类型, 0 为均匀类型, 1 为高斯类型, 2 为余弦立方类型
13	Apodization Factor, 变迹因子
14	Telecentric Object Space, 远心物空间, 0 为关闭此选项, 1 为打开此选项
15	Iterate Solves When Updating, 反复求解, 0 为关闭此选项, 1 为打开此选项
16	镜头标题
17	镜头注释
18	Afocal Image Space, 非聚焦系统像空间, 0 为关闭此选项, 1 为打开此选项
21	全域变量参考表面
23	玻璃目录列表, 参数要用字符串, 如 "SCHOTT"、"SCHOTT HOYA OHARA"
24	系统温度, 单位为摄氏度
25	系统相对大气压, 0.0 为真空, 1.0 为海平面大气压
30	镜头单位, 0 为毫米, 1 为厘米, 2 为英寸, 3 为米
31	光源单位前缀, 0 为飞(Femto-), 1 为皮(Pico-), 2 为纳(Nano-), 3 为微

	(Micro-), 4 为毫(Milli-), 5 表示没有前缀, 6 为千(Kilo), 7 为兆(Mega-), 8 为吉(Giga-), 9 为太(Tera)
32	光源单位, 0 为瓦(Watt), 1 为流明(Lumen), 2 为焦耳(Joule)
33	分析单位前缀, 0 为飞(Femto-), 1 为皮(Pico-), 2 为纳(Nano-), 3 为微(Micro-), 4 为毫(Milli-), 5 表示没有前缀, 6 为千(Kilo), 7 为兆(Mega-), 8 为吉(Giga-), 9 为太(Tera)
34	分析面积单位, 0 为平方毫米, 1 为平方厘米, 2 为平方英寸, 3 为平方米, 4 为平方英尺
35	调制传递函数单位, 0 为周期每毫米, 1 为周期每毫弧度
40	镀膜文件名
41	散射轮廓名 (Scatter Profile)
42	ABg 数据文件名
43	GRADIUM 变折射率轮廓名
50	非顺序元件每条光线最大相交点数 NSC Maximum Intersections Per Ray
51	非顺序元件每条光线最大分节数 NSC Maximum Segments Per Ray
52	非顺序元件最大套叠或接触物体数目
53	非顺序元件最小相对光线强度
54	非顺序元件最小绝对光线强度
55	非顺序元件胶合层厚度, 单位为镜头单位
56	非顺序元件遗失光线描绘距离, 单位为镜头单位
57	非顺序元件模式中打开文件时重新进行光线追迹, 0 为关闭此选项, 1 为打开此选项
58	非顺序元件模式中内存所容纳的光源文件的最大光线数
59	简单光线分叉, 0 为关闭此选项, 1 为打开此选项
60	偏振光参数 Jx
61	偏振光参数 Jy
62	偏振光参数 X-Phase
63	偏振光参数 Y-Phase
64	转换薄膜相位为光线相位, 0 为关闭此选项, 1 为打开此选项
65	非偏振光模式, 0 为关闭此选项, 1 为打开此选项
66	二维 Jones 矢量转换为三维电场矢量的方法, 0 为 X 轴参考, 1 为 Y 轴参考, 2 为 Z 轴参考
70	光线定位, 0 为关闭此选项, 1 为打开此选项, 2 为选择光瞳畸变
71	光线定位光瞳位移 x
72	光线定位光瞳位移 y
73	光线定位光瞳位移 z
74	使用光线定位高速缓存, 0 为关闭此选项, 1 为打开此选项
75	稳健光线定位(Robust Ray Aiming), 0 为关闭此选项, 1 为打开此选项
76	按照视场调整光瞳位移因子, 0 为关闭此选项, 1 为打开此选项

77	光线定位光瞳压缩因子 x
78	光线定位光瞳压缩因子 y
100	视场类型代码, 0 为角度, 1 为物高度, 2 为近轴系统像高, 3 为实际系统像高
101	视场总数
102	value1 为视场序号, value2 为视场 x 坐标
103	value1 为视场序号, value2 为视场 y 坐标
104	value1 为视场序号, value2 为视场权重
105	value1 为视场序号, value2 为视场渐晕偏移 x
106	value1 为视场序号, value2 为视场渐晕偏移 y
107	value1 为视场序号, value2 为视场渐晕压缩 x
108	value1 为视场序号, value2 为视场渐晕压缩 y
109	value1 为视场序号, value2 为视场渐晕角度
110	视场归一方法, 0 为径向归一, 1 为矩形归一
200	主波长序号
201	波长总数
202	value1 为波长序号, value2 为波长值, 单位为微米
203	value1 为波长序号, value2 为波长权重
901	用于多线程任务的计算机 CPU 的数目, 若为 0 则设为缺省值

与设置系统参数相反, 如果需要读取系统参数, 可用 ZPL 提供的函数 SYPR()。通过此函数, 可以读取大部分利用关键词 SETSYSTEMPROPERTY 设置的参数。

SYPR() 的用法如下:

returnValue = SYPR(code)

其中, code为表3.3-1中所列的为关键词SETSYSTEMPROPERTY所定义的代码, returnValue 为相应的系统参数, 可能是数值量或字符串。如果是字符串, 可以用字符串函数 \$buffer() 将其读出。需要注意的是, 函数 SYPR() 不支持 SETSYSTEMPROPERTY 中需要两个参数的部分。对于需要两个参数的系统信息, ZPL定义了另外的一些函数, 如WAVL(n)用于读取第 n 个波长的值, WWGT(n)用于读取第 n 个波长的权重, 等等。

除了用上述函数读取系统参数, 也可以通过关键词 GETSYSTEMDATA, 其用法为:

GETSYSTEMDATA vector_expression

其中 `vector_expression` 为 ZPL 所提供的四个一维数组 (VEC1, VEC2, VEC3, VEC4) 的序号, 例如, GETSYSTEMDATA 3 说明将返回的系统参数储存于数组 VEC3 内, 其顺序如表 3.3-2 所示:

表 3.3-2

数组元素序号	返回系统参数
0	数组中所储存的系统参数的数目
1	孔径值
2	Apodization Factor, 变迹因子
3	Apodization Type, 变迹类型 (0: 无, 1: 高斯函数, 2: 三角函数)
4	使用环境数据(0 为伪, 1 为真)
5	温度, 单位为摄氏度(仅当使用环境数据为真时有效)
6	相对大气压(仅当使用环境数据为真时有效)
7	有效焦距
8	像空间 F/#
9	物空间数值孔径
10	工作 F/# (Working F/#)
11	入瞳直径
12	入瞳位置
13	出瞳直径
14	出瞳位置
15	近轴像高
16	近轴放大率
17	角放大率
18	系统总长度
19	使用光线定位(真为 1, 伪为 0)
20	光瞳位移 x
21	光瞳位移 y
22	光瞳位移 z
23	光阑表面序号
24	全域坐标系统参考表面序号
25	Telecentric Object Space, 远心物空间, 0 为关闭此选项, 1 为打开此选项
26	组态(configuration) 的数目
27	多组态运算元的数目
28	评价函数(merit function)运算元的数目
29	公差运算元的数目
30	Afocal Image Space, 非聚焦系统像空间, 0 为关闭此选项, 1 为打

	开此选项
31	光瞳压缩因子 x
32	光瞳压缩因子 y

大家可能已经注意到，使用不同的关键词或函数可能会得到相同的系统信息，例如，要读取系统的变迹类型(Apodization Type)，既可以通过函数 SYPR(12)，也可通过关键词 GETSYSTEMDATA 3 来实现。需要指出的是，通过函数可以直接读取数值信息，而通过关键词则将所读取的数值信息储存于 ZPL 预先定义的某个一维数组中供调用。

下面，我们将通过一些具体的例子，来介绍光学设计中一些常用系统参数的设置与读取。

例 3.3-1： 工作波长的设置与读取

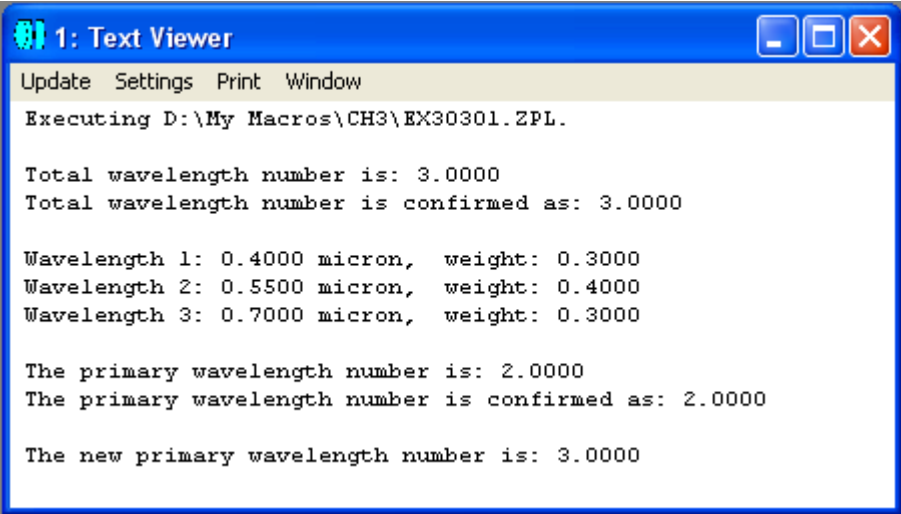
```

1 ! ex30301
2 ! This program shows how to Set and Read working wavelengths
3
4 PRINT
5
6 SYSP 201, 3 # set total wavelength number as 3
7 totalWavelengthNumber = SYPR(201) # read total wavelength number
8 PRINT "Total wavelength number is: ", totalWavelengthNumber
9
10 totalWavelengthNumber = NWAV() # read total wavelength number again
11 PRINT "Total wavelength number is confirmed as: ", totalWavelengthNumber
12
13 SYSP 202, 1, 0.40 # set the 1st wavelength as 0.40 micron
14 SYSP 202, 2, 0.55 # set the 2nd wavelength as 0.55 micron
15 SYSP 202, 3, 0.70 # set the 3rd wavelength as 0.70 micron
16
17 SYSP 203, 1, 0.3 # set the 1st wavelength weight as 0.3
18 SYSP 203, 2, 0.4 # set the 2nd wavelength weight as 0.4
19 SYSP 203, 3, 0.3 # set the 3rd wavelength weight as 0.3
20
21 wavelength1 = WAVL(1) # read the 1st wavelength
22 wavelength2 = WAVL(2) # read the 2nd wavelength
23 wavelength3 = WAVL(3) # read the 3rd wavelength
24
25 wavelengthWeight1 = WWGT(1) # read the 1st weight
26 wavelengthWeight2 = WWGT(2) # read the 2nd weight
27 wavelengthWeight3 = WWGT(3) # read the 3rd weight
28
29 PRINT
30 PRINT "Wavelength 1: ", wavelength1, " micron, weight: ", wavelengthWeight1
31 PRINT "Wavelength 2: ", wavelength2, " micron, weight: ", wavelengthWeight2
32 PRINT "Wavelength 3: ", wavelength3, " micron, weight: ", wavelengthWeight3
33
34 SYSP 200, 2 # set the 2nd wavelength as the primary wavelength
35 primaryNumber = SYPR(200) # read the primary wavelength number
36
37 PRINT
38 PRINT "The primary wavelength number is: ", primaryNumber
39
40 primaryNumber = PWAV() # read the primary wavelength number again
41 PRINT "The primary wavelength number is confirmed as: ", primaryNumber
42
43 PWAV 3 # set the 3rd wavelength as the primary wavelength
44 primaryNumber = PWAV() # read the new primary wavelength number
45
46 PRINT
47 PRINT "The new primary wavelength number is: ", primaryNumber

```

在这个例子中，我们首先用 SYSP 201, 3 设置系统工作波长总数目为 3(第 6 行)，并通过不同的方法 SYPR(201) 及 NWAV() 将此信息读出(第 7、10 行)。接着，我们用 SYSP 202, 1, 0.40 及 SYSP 203, 1, 0.3 设置第一个工作波长及权重，类似地，设置第二、第三个工作波长及权重(第 13~19 行)，然后用 WAVL() 及 WWGT() 将波长值和权重读出(第 21~27 行)。最后，我们用两种不同的方法 SYSP 200, 2 和 PWAV 3 设置系统主波长的序号(第 34、43 行)，然后用不同的方

法 SYPR(200) 及 PWAV() 将主波长的序号读出(第 35、44 行)。程序的运行结果如图 3.3-1 所示:



```
1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH3\EX30301.ZPL.

Total wavelength number is: 3.0000
Total wavelength number is confirmed as: 3.0000

Wavelength 1: 0.4000 micron, weight: 0.3000
Wavelength 2: 0.5500 micron, weight: 0.4000
Wavelength 3: 0.7000 micron, weight: 0.3000

The primary wavelength number is: 2.0000
The primary wavelength number is confirmed as: 2.0000

The new primary wavelength number is: 3.0000
```

图 3.3-1 程序 ex30301.ZPL 的运行结果

例 3.3-2: 单位的设置与读取

```

1 ! ex30302
2 ! This program shows how to Set and Read units
3
4 PRINT
5 FORMAT 1.0
6
7 SYSP 30, 0 # set lens unit as mm
8 lensUnitCode = SYPR(30) # get lens unit code
9 PRINT "Lens unit code is: ", lensUnitCode
10
11 lensUnitCode = UNIT() # get lens unit code with a different method
12 PRINT "Lens unit code is confirmed as: ", lensUnitCode
13
14 lensUnit$ = $UNITS() # get the actual lens unit in string format
15 PRINT "The actual lens unit is ", lensUnit$
16
17 SYSP 31, 4 # set source unit prefix as Milli
18 SYSP 32, 0 # set source unit as Watts
19 sourceUnitPrefix = SYPR(31) # get the source unit prefix code
20 sourceUnit = SYPR(32) # get the source unit code
21 PRINT
22 PRINT "Source unit prefix code is: ", sourceUnitPrefix, ", means Milli"
23 PRINT "Source unit code is: ", sourceUnit, ", means Watts"
24
25 SYSP 33, 3 # set analysis unit prefix as Micro
26 SYSP 34, 0 # set analysis unit "per" area as square mm
27 analysisUnitPrefix = SYPR(33) # get the analysis unit prefix code
28 analysisUnitPerArea = SYPR(34) # get the analysis unit "per" area code
29 PRINT
30 PRINT "The analysis unit prefix code is: ", analysisUnitPrefix
31 PRINT "The analysis unit 'per' area code is: ", analysisUnitPerArea
32 PRINT "Actual analysis unit is (micro-Watts)/(square mm)." # see line 18
33
34 SYSP 35, 0 # set MTF unit
35 mtfUnit = SYPR(35) # get the MTF unit code
36 PRINT
37 PRINT "The MTF unit code is: ", mtfUnit, ", which means cycles/mm"

```

在这个例子中，我们首先设置了镜头单位，并用不同的方法将其读出；接着，我们设置并读取了光源前缀和光源单位；然后，我们设置并读取了分析单位的前缀和面积部分(分析单位的光强度部分已经在前面第 18 行设置)；最后，我们设置并读取了调制传递函数 MTF 的单位。程序的运行结果如图 3.3-2 所示：

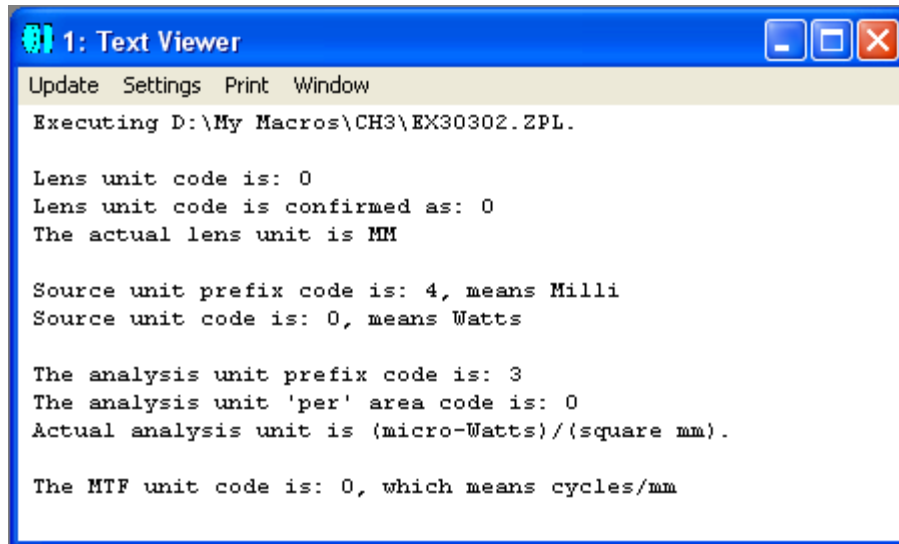


图 3.3-2 程序 ex30302.ZPL 的运行结果

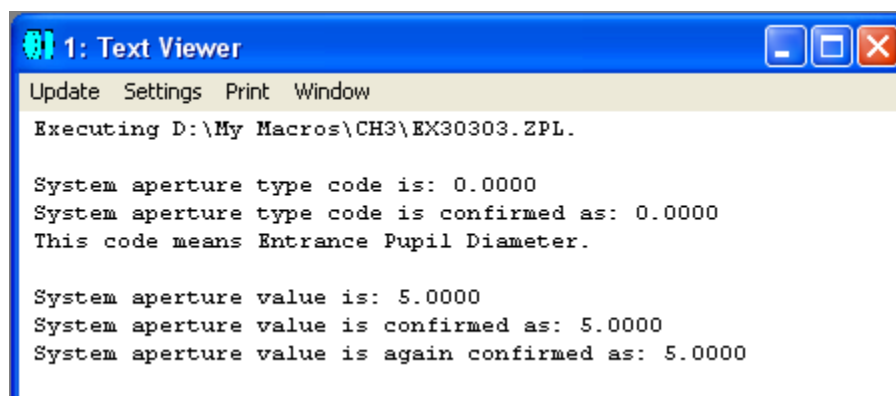
例 3.3-3: 系统孔径的设置与读取

```

1 ! ex30303
2 ! This program shows how to Set and Read aperture
3
4 PRINT
5
6 SYSP 10, 0 # set system aperture as Entrance Pupil Diameter
7 apertureType = SYPR(10) # get system aperture type code
8 PRINT "System aperture type code is: ", apertureType
9
10 apertureType = ATYP() # get system aperture type code again
11 PRINT "System aperture type code is confirmed as: ", apertureType
12 PRINT "This code means Entrance Pupil Diameter."
13
14 SYSP 11, 5 # set system aperture value as 5 lens units
15 apertureValue = SYPR(11) # get system aperture value
16 PRINT
17 PRINT "System aperture value is: ", apertureValue
18
19 apertureValue = AVAL() # get system aperture value
20 PRINT "System aperture value is confirmed as: ", apertureValue
21
22 GETSYSTEMDATA 1 # get system information and store them in VEC1
23 apertureValue = VEC1(1) # get system aperture value
24 PRINT "System aperture value is again confirmed as: ", apertureValue

```

在这个例子中，我们设置了系统孔径的类型和数值，并用三种不同的方法将系统孔径值读出。注意系统孔径值不同于某一镜头表面的孔径值，我们将在下一节中对后者做进一步的介绍。程序的运行结果如图3.3-3所示。



The image shows a screenshot of a '1: Text Viewer' window. The window has a blue title bar with the text '1: Text Viewer' and standard Windows window controls (minimize, maximize, close). Below the title bar is a menu bar with 'Update', 'Settings', 'Print', and 'Window'. The main text area displays the following output:

```
Executing D:\My Macros\CH3\EX30303.ZPL.  
  
System aperture type code is: 0.0000  
System aperture type code is confirmed as: 0.0000  
This code means Entrance Pupil Diameter.  
  
System aperture value is: 5.0000  
System aperture value is confirmed as: 5.0000  
System aperture value is again confirmed as: 5.0000
```

图 3.3-3 程序 ex30303.ZPL 的运行结果

例 3.3-4: 视场的设置与读取

```

1 ! ex30304
2 ! This program shows how to set and read field
3
4 PRINT
5
6 SYSP 100, 1 # set field as Object Height
7 fieldType = SYPR(100) # get field type code
8 PRINT "Field type code is: ", fieldType
9
10 fieldType = FTYP() # get field type code
11 PRINT "Field type code is confirmed as: ", fieldType
12
13 SYSP 101, 3 # set total number of fields as 3
14 totalFieldNum = SYPR(101) # get total number of fields
15 PRINT
16 PRINT "Total number of fields is: ", totalFieldNum
17 totalFieldNum = NFLD() # get total number of fields
18 PRINT "Total number of fields is confirmed as: ", totalFieldNum
19
20 SYSP 102, 1, 0 # set x of field 1 as 0
21 SYSP 103, 1, 0 # set y of field 1 as 0
22 SYSP 104, 1, 1 # set weight of field 1 as 1
23 SYSP 102, 2, 0 # set x of field 2 as 0
24 SYSP 103, 2, 0.707 # set y of field 2 as 0.707
25 SYSP 104, 2, 0.5 # set weight of field 2 as 0.5
26 SYSP 102, 3, 0 # set x of field 3 as 0
27 SYSP 103, 3, 1 # set y of field 3 as 1
28 SYSP 104, 3, 0.25 # set weight of field 3 as 0.25
29
30 fld1X = FLDX(1) # get x of field 1
31 fld1Y = FLDY(1) # get y of field 1
32 fldWeight1 = FWGT(1) # get weight of field 1
33 fld2X = FLDX(2) # get x of field 2
34 fld2Y = FLDY(2) # get y of field 2
35 fldWeight2 = FWGT(2) # get weight of field 2
36 fld3X = FLDX(3) # get x of field 3
37 fld3Y = FLDY(3) # get y of field 3
38 fldWeight3 = FWGT(3) # get weight of field 3
39
40 PRINT
41 PRINT "Field 1, X = ", fld1X, ", Y = ", fld1Y, ", weight = ", fldWeight1
42 PRINT "Field 2, X = ", fld2X, ", Y = ", fld2Y, ", weight = ", fldWeight2
43 PRINT "Field 3, X = ", fld3X, ", Y = ", fld3Y, ", weight = ", fldWeight3

```

在此例中，我们先设置了视场的类型，并用两种不同的方法读取了此参数；然后，我们设置了视场的总数目，并用两种方法读取了此参数；最后，我们设置并读取了每个视场的 x、y 坐标和权重。程序的运行结果如下：

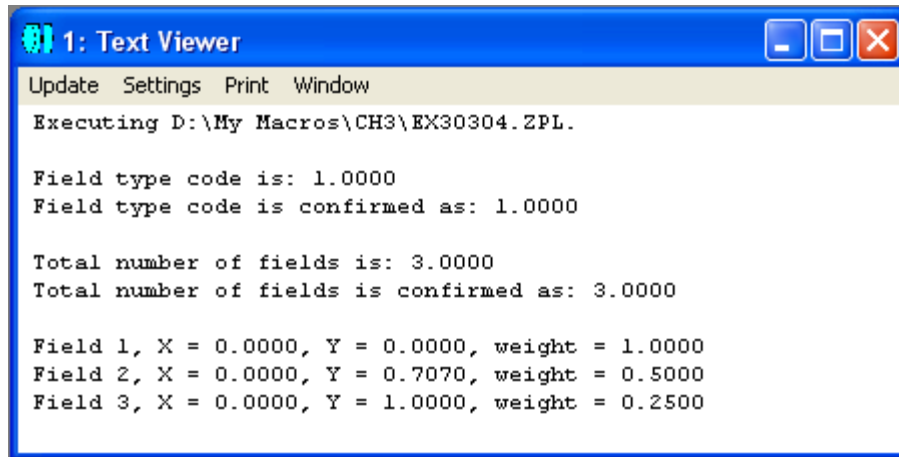


图 3.3-4 程序 ex30304.ZPL 的运行结果

例 3.3-5: 镜头标题和说明的设置与读取

```

1 ! ex30305
2 ! This program shows how to set and read lens title and lens note
3
4 PRINT
5
6 SYSP 16, "New Design for X Project" # set lens title
7 SYSP 17, "In this design, a Cooke Triplet is used to get the image quality."
8 ! line 7 sets the lens note
9
10 dummy = SYPR(16) # use a dummy variable to call SYPR() function
11 lensTitle$ = $BUFFER() # the string information is put in a buffer
12 PRINT "The lens title is: ", lensTitle$
13
14 lensTitle$ = $LENSNAME() # get the lens title using a different method
15 PRINT "The lens title is confirmed as: ", lensTitle$
16
17 dummy = SYPR(17) # use a dummy variable to call SYPR() function
18 lensNote$ = $BUFFER() # string information can be obtained from $BUFFER()
19 PRINT
20 PRINT "The lens note is: "
21 PRINT " ", lensNote$
22
23 lensNote$ = $NOTE(1) # get the lens note using a different method
24 PRINT "The lens note is confirmed as: "
25 PRINT " ", lensNote$

```

在这个例子中，我们设置了镜头的标题，并用两种不同的方法读取了此信息。同样地，我们设置了镜头的说明，也用两种不同的方法读取了此信息。值得注意的是，如果要用 SYPR() 函数读取字符串信息，那么，此信息返回后存于缓存区内，需要通过字符串函数 \$BUFFER() 将所需信息读出。程序的运行结果如图 3.3-5 所示：

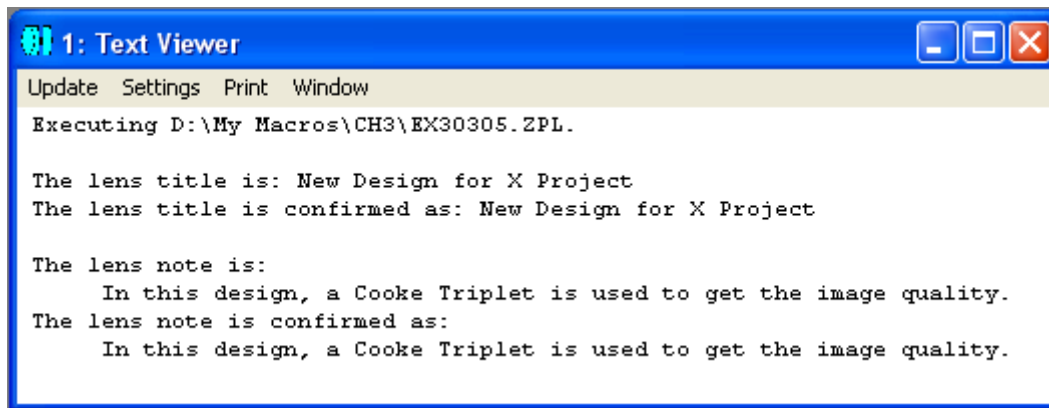


图 3.3-5 程序 ex30305.ZPL 的运行结果

通过上面这些例子，我们介绍了在 ZPL 程序中对光学系统中一些重要参数进行设置与读取的基本方法。需要指出的是，很多通过程序可以设置和读取的参数，也可以在 ZEMAX 的运行环境中直接设置和读取，而究竟何种方法更为有效，需要由读者根据具体的情况作出判断。

另外，由于篇幅所限，我们不可能介绍完与系统参数相关的全部指令。在后面的章节中，我们还会根据需要继续介绍一些新的指令，但是，我们鼓励读者查阅 ZEMAX 用户手册，并以其中的内容为准。

第四节 镜头参数的设置和读取

镜头数据编辑器 (Lens Data Editor) 是通过 ZEMAX 进行镜头设计的一个重要场所。我们知道，在顺序光学系统中，ZEMAX 将光学系统定义为由许多表面组成，大部分与表面相关的参数，都在镜头数据编辑器中设置。在本节中，我们将通过不同的例子，介绍如何在 ZPL 中设置和读取镜头表面参数。

对于镜头表面参数的设置，ZPL 提供了一个重要的关键词 SETSURFACEPROPERTY(缩写形式 SURP)，其用法为：

SETSURFACEPROPERTY surface, code, value1, value2

或

SURP surface, code, value1, value2

其中，surface 为镜头表面的序号，code 为镜头表面参数的代码，为一整数或字符串助记码，如表 3.4-1 所示。value1 和 value2 为相应的参数，可能为字符串或者数值量。大多数 code 只需要一个参数 value1，其它一些 code 则需要 value1 和 value2 两个参数。

表 3.4-1

表面参数代码	性质
<i>0 ~ 18 为基本表面数据</i>	
0 或 TYPE	表面类型。参数值应为相应的 ASCII 码名字，如用"STANDARD"代表标准表面。各个表面的 ASCII 码名字列于 Prescription Report 中的表面数据摘要内。
1 或 COMM	表面注释
2 或 CURV	表面曲率，单位为镜头单位的倒数，0 代表无穷大
3 或 THIC	表面厚度，单位为镜头单位
4 或 GLAS	玻璃名称。参见代码 18
5 或 CONI	二次曲线的 Conic 常数
6 或 SDIA	半直径(Semi-diameter)。如为 0 或正数，则半直径的求解被设为“固定值”；如为负数，则半直径的求解被设为“自动”，但执行到下一个 UPDATE 指令时被重新计算。
7 或 TCE	热膨胀系数
8 或 COAT	镀膜名称。可用空字符串去掉镀膜。
9 或 SDLL	用户定义的表面 DLL 名称
10 或 PARM	参数值。value1 为新的参数值，value2 为参数序号。

11 或 EDVA	附加数据值。value1 为新的附加数据值，value2 为附加数据序号。
12	表面颜色，0 为缺省值。
13	表面不透明度
14	镜头数据编辑器中代表表面的行的颜色
15	表面不能为超半球 (hyperhemispheric)。设为 1 以避免表面成为超半球。
16	忽略表面。1 为忽略表面，0 为不忽略表面。
17 或 CODE	表面类型代码，为整数。此整数代码可代替表面类型的 ASCII 代码名字。参见表面参数代码 0。
18 或 GLAN	玻璃序号。参见表面参数代码 4。
20 ~ 25 为表面孔径数据	
20 或 ATYP	表面孔径类型代码
21 或 APP1	表面孔径参数 1
22 或 APP2	表面孔径参数 2
23 或 APDX	表面孔径中心偏移 x
24 或 APDY	表面孔径中心偏移 y
25 或 UDA	用户定义孔径 (User Defined Aperture, UDA) 名称
30 ~ 43 为物理光学传播设定	
30	物理光学设定“用光线传播至下一表面”。1 为真，0 为伪。
31	物理光学设定“不用光线数据重新设定光束尺寸”。1 为真，0 为伪。
32	物理光学设定“使用角度谱传播因子(Angular Spectrum Propagator)”。1 为真，0 为伪。
33	物理光学设定“在阴影模型中画出 ZBF”。1 为真，0 为伪。
34	物理光学设定“重新计算先导光束(Pilot Beam)参数”。1 为真，0 为伪。
35	物理光学设定“折射后重新采样”。1 为真，0 为伪。
36	物理光学设定“自动采样”。1 为真，0 为伪。
37	物理光学设定“新的 X 采样”。用 1 代表 32，2 代表 64，等等。
38	物理光学设定“新的 Y 采样”。用 1 代表 32，2 代表 64，等等。
39	物理光学设定“新的 X 宽度”。新的 X 方向的阵列总宽度。
40	物理光学设定“新的 Y 宽度”。新的 Y 方向的阵列总宽度。
41	物理光学设定“输出先导光束半径”。0 为最佳拟合，1 为较短值，2 为较长值，3 为 x 值，4 为 y 值，5 为平面，6 为用户定义。
42	物理光学设定“X 半径”。
43	物理光学设定“Y 半径”。
50 ~ 56 为镀膜设定	
50	使用镀膜层放大因子和折射率偏移。1 为真，0 为伪。

51	镀膜层放大因子值。Value1 为新值，Value2 为镀膜层序号。
52	镀膜层放大因子状态。Value 1 为状态值，0 代表固定值，1 代表变量，n+1 代表从层 n 取值。Value 2 为镀膜层序号。
53	镀膜层折射率偏移值。Value1 为新值，Value2 为镀膜层序号。
54	镀膜层折射率偏移状态。Value 1 为状态值，0 代表固定值，1 代表变量，n+1 代表从层 n 取值。Value 2 为镀膜层序号。
55	镀膜层折射率消光偏移(Extinction Offset)值。Value1 为新值，Value2 为镀膜层序号。
56	镀膜层折射率消光偏移(Extinction Offset)状态。Value 1 为状态值，0 代表固定值，1 代表变量，n+1 代表从层 n 取值。Value 2 为镀膜层序号。
<i>60 ~ 66 为表面倾斜和偏移数据</i>	
60 或 BOR	表面前倾斜和偏移顺序。0 为偏移/倾斜，1 为倾斜/偏移。
61 或 BDX	表面前偏移 x
62 或 BDY	表面前偏移 y
63 或 BTX	表面前倾斜 x
64 或 BTY	表面前倾斜 y
65 或 BTZ	表面前倾斜 z
66 或 APU	表面后的摘取状态：0 为详细定义，1/2 为摘取/反转当前表面，3/4 为摘取/反转当前表面序号减 1，5/6 为摘取/反转当前表面序号减 2，依此类推...
70 或 AOR	表面后倾斜和偏移顺序。0 为偏移/倾斜，1 为倾斜/偏移。
71 或 ADX	表面后偏移 x
72 或 ADY	表面后偏移 y
73 或 ATX	表面后倾斜 x
74 或 ATY	表面后倾斜 y
75 或 ATZ	表面后倾斜 z
<i>80 ~ 83 为表面散射数据</i>	
80	设定散射代码：0 为无，1 为 Lambertian，2 为 Gaussian，3 为 ABg。
81	设定散射比例，应为 0.0 ~ 1.0 之间的数值。
82	设定 Gaussian 散射因子 sigma
83	设定 ABg 散射数据名称。
<i>90 ~ 98 为表面绘制数据</i>	
90	设定“隐藏至此表面的光线”选择框状态：0 为关闭，1 为打开。
91	设定“忽略至此表面的光线”选择框状态：0 为关闭，1 为打开。
92	设定“不绘制此表面”选择框状态：0 为关闭，1 为打开。
93	设定“不从此表面绘制边缘线”选择框状态：0 为关闭，1 为打开。

96	设定“将边缘线绘制为”状态：0为直角过度，1为平滑过度，2为平行过度。
97	设定“镜面基板”状态：0为无，1为平面，2为曲面。
98	设定镜面基板厚度值。

如果要删除某个表面，可以用ZPL提供的关键词DELETE，其用法为：

DELETE n

其中 n 为所要删除的表面的序号。

如果要在某个位置插入一个表面，则可以用到 ZPL 提供的关键词 INSERT，其用法为：

INSERT n

其中 n 为所要插入的位置的表面序号。新插入的表面在原来的第 n 个表面之前。

如要将某一表面设为系统光阑，可用关键词 STOPSURF，其用法为：

STOPSURF n

表明将第 n 个表面设为系统光阑。

需要注意的是，通常在设定或修改表面参数后，需要用关键词 UPDATE 将表面参数进行更新。

与设置镜头表面参数相似，ZPL 提供了两个重要函数 SPRO() 和 SPRX()，可以用来读取镜头表面参数，其用法为：

SPRO(surf, code)

和

SPRX(surf, code, value2)

其中，surf 为所要读取参数表面的序号，code 为表 3.4-1 中为关键词 SURP 所定义的代码。需要注意的是此处 code 只能为数值，不能为字母助记符。函数 SPRO() 和 SPRX() 的区别在于，前者支持表 3.4-1 中有一个值的代码，后者支持表 3.4-1 中有两个值的代码。

除了 SPRO() 和 SPRX() 以外, ZPL 还提供了其它一些重要函数以读取和表面相关的参数, 如表 3.4-2 所示:

表 3.4-2 用于读取表面参数的一些重要函数

函数	说明
NSUR()	用于读取所定义的表面总数。
APMN(n)	用于读取表面 n 的最小半径值。
APMX(n)	用于读取表面 n 的最大半径值。
APXD(n)	用于读取表面 n 的 x 孔径偏移值。
APYD(n)	用于读取表面 n 的 y 孔径偏移值。
APTP(n)	用于读取表面 n 的孔径类型。
CONI(n)	用于读取表面 n 的Conic常数。
CURV(n)	用于读取表面 n 的曲率。
EDGE(n)	用于读取表面 n 半直径处的边缘厚度。
GLCA(n)	用于读取表面 n 的全域坐标顶点的 x 矢量值。
GLCB(n)	用于读取表面 n 的全域坐标顶点的 y 矢量值。
GLCC(n)	用于读取表面 n 的全域坐标顶点的 z 矢量值。
GLCM(n, item)	用于读取表面 n 的全域坐标变换参数。item 为 1 ~ 12 间的整数, 当 item 为 1 ~ 9 时, 返回值为 R11、R12、R13、R21、R22、R23、R31、R32 或 R33, 当 item 为 10 ~ 12 时, 返回值为全域坐标平移矢量的 x、y 或 z 分量。
GLCX(n)	用于读取表面 n 的全域坐标顶点的 x 坐标值。
GLCY(n)	用于读取表面 n 的全域坐标顶点的 y 坐标值。
GLCZ(n)	用于读取表面 n 的全域坐标顶点的 z 坐标值。
PARM(p,n)	用于读取表面 n 的参数 p。
RADI(n)	用于读取表面 n 的曲率半径值。如果表面为平面, 则半径值为 0。
SDIA(n)	用于读取表面 n 的半直径值。
STDD(n, parm)	用于读取表面 n 的倾斜或偏移参数的值。parm 为参数序号。
SURC(A\$)	用于读取注释为字符串 A\$ 的表面的序号。如果没有相符合的表面, 则返回值为 -1。
THIC(n)	用于读取表面 n 的厚度。

在和表面相关的参数中, 还有一个重要的内容, 即表面的材料, 在这里统称为玻璃材料。表 3.4-3 列出了一些重要的表面材料参数的相关函数:

表 3.4-3 用于读取表面玻璃材料参数的一些重要函数

函数	说明
MAXG()	用于读取在当前设计中所用到的玻璃总数。
GIND(n)	用于读取表面 n 的玻璃材料在 d 光波长处的折射率。
GNUM(A\$)	如果字符串 A\$ 为有效的玻璃名称，则返回此玻璃在材料目录中的序号；如果玻璃材料目录中没有与 A\$ 相符合的玻璃名称，则返回值为 0；如果材料为镜面，则返回值为 -1。
GPAR(n)	用于读取表面 n 的玻璃材料的部分色散(partial dispersion)系数。
GRIN(n, w, x, y, z)	用于读取表面 n 的玻璃材料在波长 w 处位于坐标点 x, y, z 的折射率。
INDX(n)	用于读取表面 n 的玻璃材料在主波长处的折射率。
ISMS(n)	如果表面为第偶数个镜面，或者不是镜面但跟在第偶数个镜面之后，则返回值为 1，否则返回值为 0。
TMAS()	用于读取从表面 1 到像面的镜头总质量。
GABB(n)	用于读取表面 n 的玻璃材料的 Abbe 数。

除了上表所列的函数外，ZPL 还提供了一个关键词 GETGLASSDATA 用于读取玻璃材料参数，其用法为：

GETGLASSDATA vector_expression, glass_number

其中 vector_expression 为 ZPL 所提供的四个一维数组(VEC1, VEC2, VEC3, VEC4)的序号，glass_number 为玻璃材料目录中所指定的玻璃的序号，一般通过函数 GNUM() 读取。数组 VECn (n 为 1、2、3 或 4) 中的玻璃参数按照表 3.4-4 的形式存放。

表3.4-4 玻璃材料参数读取函数

数组元素序号	返回玻璃材料参数
0	数组中所储存的玻璃材料参数的数目
1	色散公式序号：1 为 Schott，2 为 Sellmeier 1，3 为 Herzberger，4 为 Sellmeier 2，5 为 Conrady，6 为 Sellmeier 3，7 为 Handbook of Optics 1，8 为 Handbook of Optics 2，9 为 Sellmeier 4，10 为 Extended，11 为 Sellmeier 5，12 为 Extended 2
2	6 位 MIL 数 (MIL number)
3	D 线折射率 Nd
4	Abbe 数 Vd

5	-30 ~ +70 °C 热膨胀系数
6	+20 ~ 300°C 热膨胀系数
7	密度, 单位为 g/cm^3
8	部分色散 $\Delta P_g \square \square F$
9	最小波长
10	最大波长
11~16	色散系数(取决于不同的色散公式)
17~22	色散的热系数
23+	每毫米材料内部光透过率 α (不包含表面损耗)。波长 1 的透过率存于数组元素 23, 波长 2 的透过率存于数组元素 24, 等等。

下面, 我们将通过一些具体的例子, 来介绍光学设计中和表面相关的一些常用参数的设置与读取。

例 3.4-1: 构造一个双透镜(doublet)。

在这个例子中, 我们将从头开始, 通过 ZPL 程序, 在 ZEMAX 的镜头数据编辑器(LDE)中构造一个双透镜。假设此透镜的基本参数为:

工作波长: $F = 0.48613270 \mu\text{m}$, $d = 0.58756180 \mu\text{m}$, $C = 0.65627250 \mu\text{m}$

入瞳直径: 50mm

F/#: F/8

全视场角: 10°

边界制约条件: 边缘及中心最小厚度 4 mm, 中心最大厚度 18 mm

玻璃材料: BK7 和 F2

经过一些简单的计算(此处略), 我们可以得到此双透镜各个表面的初始值, 本例中我们的任务就是利用这些初始值构造这个双透镜。在本书后面的不同例子中, 我们还有机会对其进行分析和优化。

首先, 通过选择 ZEMAX 文件菜单下的新建(new), 可以得到一个空的镜头数据编辑器, 其中有一个物表面、一个像表面及一个镜头表面。在我们的程序中, 首先要进行一些系统参数设定, 如系统孔径类型和大小, 视场的类型和大小, 工作波长等, 然后要在镜头数据编辑器中添加足够的镜头表面, 并输入镜头参数。下面是相应的程序:

```
1 ! ex30401
2 ! This program shows how to create a doublet from scratch
3
4 ! set system parameters
5 SYSP 30, 0 # set lens unit as mm
6
7 SYSP 10, 0 # set system aperture as Entrance Pupil Diameter
8 SYSP 11, 50 # set system aperture value as 50mm
9
10 SYSP 201, 3 # set total wavelength number as 3
11 SYSP 202, 1, 0.48613270 # set the 1st wavelength as 0.48613270 micron
12 SYSP 202, 2, 0.58756180 # set the 2nd wavelength as 0.58756180 micron
13 SYSP 202, 3, 0.65627250 # set the 3rd wavelength as 0.65627250 micron
14 SYSP 203, 1, 1 # set the 1st wavelength weight as 1
15 SYSP 203, 2, 1 # set the 2nd wavelength weight as 1
16 SYSP 203, 3, 1 # set the 3rd wavelength weight as 1
17
18 SYSP 200, 2 # set the 2nd wavelength as the primary wavelength
19
20 SYSP 100, 0 # set the field type as Angle
21 SYSP 101, 3 # set the total field number as 3
22 SYSP 102, 1, 0 # set field 1 as x = 0 degree
23 SYSP 103, 1, 0 # set field 1 as y = 0 degree
24 SYSP 104, 1, 1 # set field 1 as weight = 1
25 SYSP 102, 2, 0 # set field 2 as x = 0 degree
26 SYSP 103, 2, 3.5 # set field 2 as y = 3.5 degree
27 SYSP 104, 2, 1 # set field 2 as weight = 1
28 SYSP 102, 3, 0 # set field 3 as x = 0 degree
29 SYSP 103, 3, 5 # set field 3 as y = 5 degree
30 SYSP 104, 3, 1 # set field 3 as weight = 1
31
```

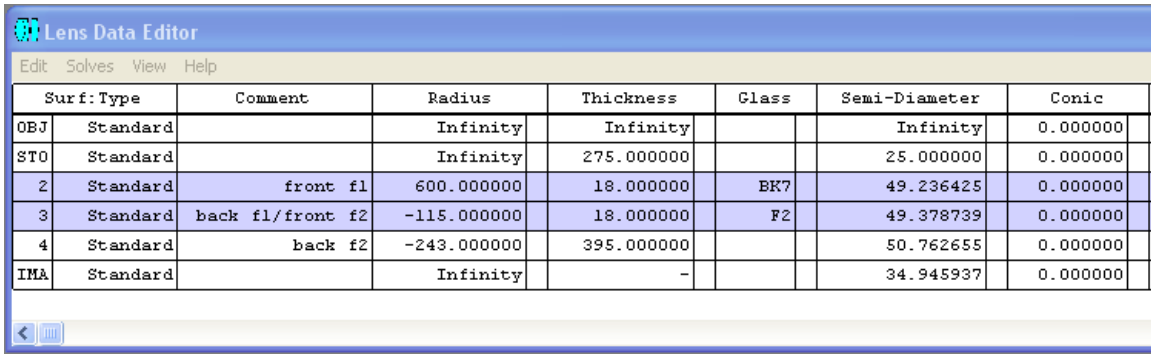
```

31
32 ! set surface 1 as stop
33 STOPSURF 1
34
35 ! insert 3 surfaces after stop
36 INSERT 2
37 INSERT 2
38 INSERT 2
39
40 ! set surface parameters
41 SURP 1, THIC, 275 # set surface 1 thickness as 275
42
43 SURP 2, TYPE, "STANDARD" # set surface 2 type as "STANDARD", can be omitted
44 SURP 2, COMM, "front f1" # set surface 2 comment
45 SURP 2, CURV, 1/600 # set surface 2 curvature as 1/600
46 SURP 2, THIC, 18 # set surface 2 thickness as 18
47 SURP 2, GLAS, "BK7" # set surface 2 glass type as BK7
48
49 SURP 3, COMM, "back f1/front f2" # set surface 3 comment
50 SURP 3, CURV, -1/115 # set surface 3 curvature as -1/115
51 SURP 3, THIC, 18 # set surface 3 thickness as 18
52 SURP 3, GLAS, "F2" # set surface 3 glass type as F2
53
54 SURP 4, COMM, "back f2" # set surface 4 comment
55 SURP 4, CURV, -1/243 # set surface 4 curvature as -1/243
56 SURP 4, THIC, 395 # set surface 4 thickness as 395
57
58 UPDATE

```

程序的第一部分(第 5 ~ 30 行)为系统参数设定, 其中我们对镜头单位(第 5 行)、系统孔径(第 7、8 行)、工作波长(第 10 ~ 18 行)、视场(第 20 ~ 30 行)等参数进行了设定。程序的第二部分(第 32 ~ 58 行)为镜头表面参数设定, 其中我们定义了光阑表面(第 33 行), 插入了三个新的镜头表面(第 36 ~ 38 行), 并对各个表面的类型、注释、曲率、厚度、玻璃材料类型进行了设定(第 41 ~ 56 行)。在程序的最后, 我们对系统进行了更新(第 58 行), 以保证所设定的参数为系统确认。需要指出的是, 很多参数的设定可以直接在镜头数据编辑器中完成, 并不一定需要通过程序进行。我们在此只是想说明通过程序如何来完成这些工作。另外, 对于许多镜头数据编辑器中缺省的设定, 如程序第 51 行中的“标准表面”, 也完全可以在程序中省略。

程序 ex30401.ZPL 的运行结果如图 3.4-1 所示:



Lens Data Editor							
Edit Solves View Help							
Surf	Type	Comment	Radius	Thickness	Class	Semi-Diameter	Conic
OBJ	Standard		Infinity	Infinity		Infinity	0.000000
STO	Standard		Infinity	275.000000		25.000000	0.000000
2	Standard	front f1	600.000000	18.000000	BK7	49.236425	0.000000
3	Standard	back f1/front f2	-115.000000	18.000000	F2	49.378739	0.000000
4	Standard	back f2	-243.000000	395.000000		50.762655	0.000000
IMA	Standard		Infinity	-		34.945937	0.000000

图 3.4-1 程序 ex30401.ZPL 运行后镜头数据编辑器的内容

对于程序 ex30401 中所构造的双透镜，我们可以通过程序读取其各种参数。在下面的程序中我们给出了一些例子。

例 3.4-2：读取双透镜的表面参数。

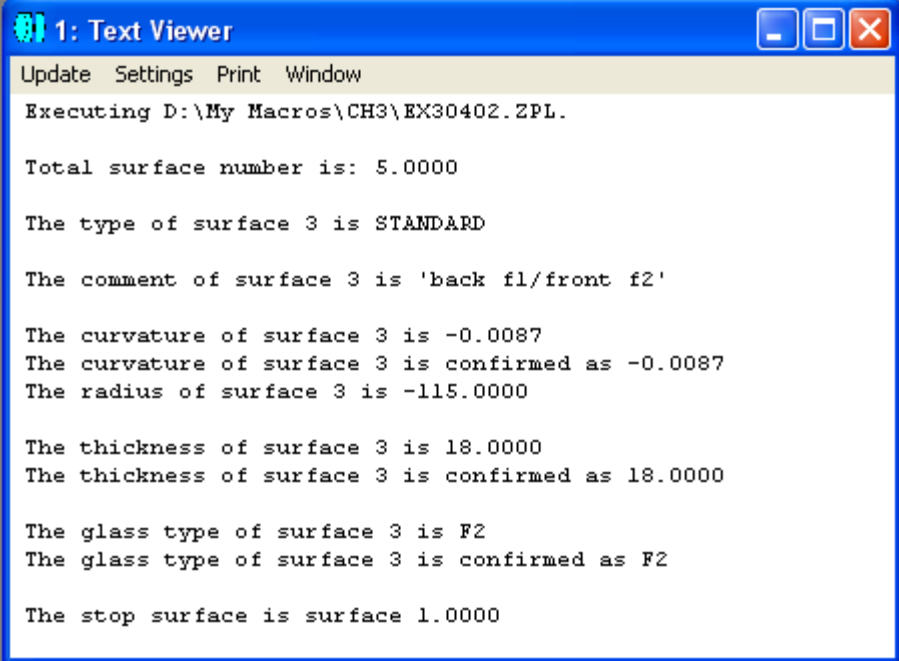
在这个程序中，我们首先读取了表面总数(第 7 行)，然后读取了表面 3 的类型(第 11、12 行)、注释(第 16、17 行)、曲率(第 21、23 行)、半径(第 25 行)、厚度(第 29、31 行)和玻璃类型(第 35、36、38 行)，最后我们还读取了光阑的表面序号(第 42、43 行)。读者可以注意到，当通过函数 `SPRO()` 读取字符串信息(如表面类型、注释等)时，返回的信息存于数据缓存区中，可通过缓存区字符串函数 `$buffer()` 将所需信息读出。

```

1 ! ex30402
2 ! This program shows how to read surface data
3 ! Assume the lens is defined in ex30401
4
5 PRINT
6
7 surfTotalNum = NSUR() # get the total surface number
8 PRINT "Total surface number is: ", surfTotalNum
9 PRINT
10
11 dummy = SPRO(3,0) # get the type of surface 3
12 surfaceType$ = $buffer() # read the type of surface 3 from buffer
13 PRINT "The type of surface 3 is ", surfaceType$
14 PRINT
15
16 dummy = SPRO(3,1) # get the comment of surface 3
17 surfaceComment$ = $buffer() # read the comment from buffer
18 PRINT "The comment of surface 3 is '", surfaceComment$, "'"
19 PRINT
20
21 curvOfSurf = SPRO(3,2) # get the curvature of surface 3
22 PRINT "The curvature of surface 3 is ", curvOfSurf
23 curvOfSurf = CURV(3) # get the curvature by a different way
24 PRINT "The curvature of surface 3 is confirmed as ", curvOfSurf
25 radiOfSurf = RAD(3) # get the radius of surface 3
26 PRINT "The radius of surface 3 is ", radiOfSurf
27 PRINT
28
29 thickness = SPRO(3,3) # get the thickness of surface 3
30 PRINT "The thickness of surface 3 is ", thickness
31 thickness = THIC(3) # get the thickness by a different way
32 PRINT "The thickness of surface 3 is confirmed as ", thickness
33 PRINT
34
35 dummy = SPRO(3,4) # get the glass type of surface 3
36 glassType$ = $buffer() # read the glass type from buffer
37 PRINT "The glass type of surface 3 is ", glassType$
38 glassType$ = $GLASS(3) # get the glass type by a different way
39 PRINT "The glass type of surface 3 is confirmed as ", glassType$
40 PRINT
41
42 GETSYSTEMDATA 1 # get system information and save into VEC1
43 stopNum = VEC1(23) # read the stop surface number from VEC1
44 PRINT "The stop surface is surface ", stopNum
45

```

程序 ex30402.ZPL 的运行结果如图 3.4-2 所示:



1: Text Viewer

Update Settings Print Window

```
Executing D:\My Macros\CH3\EX30402.ZPL.  
  
Total surface number is: 5.0000  
  
The type of surface 3 is STANDARD  
  
The comment of surface 3 is 'back f1/front f2'  
  
The curvature of surface 3 is -0.0087  
The curvature of surface 3 is confirmed as -0.0087  
The radius of surface 3 is -115.0000  
  
The thickness of surface 3 is 18.0000  
The thickness of surface 3 is confirmed as 18.0000  
  
The glass type of surface 3 is F2  
The glass type of surface 3 is confirmed as F2  
  
The stop surface is surface 1.0000
```

图 3.4-2 程序 ex30402.ZPL 的运行结果

例 3.4-3: 读取玻璃材料的各种参数

下面这个程序调用了不同的函数及关键词读取玻璃材料的各种参数。

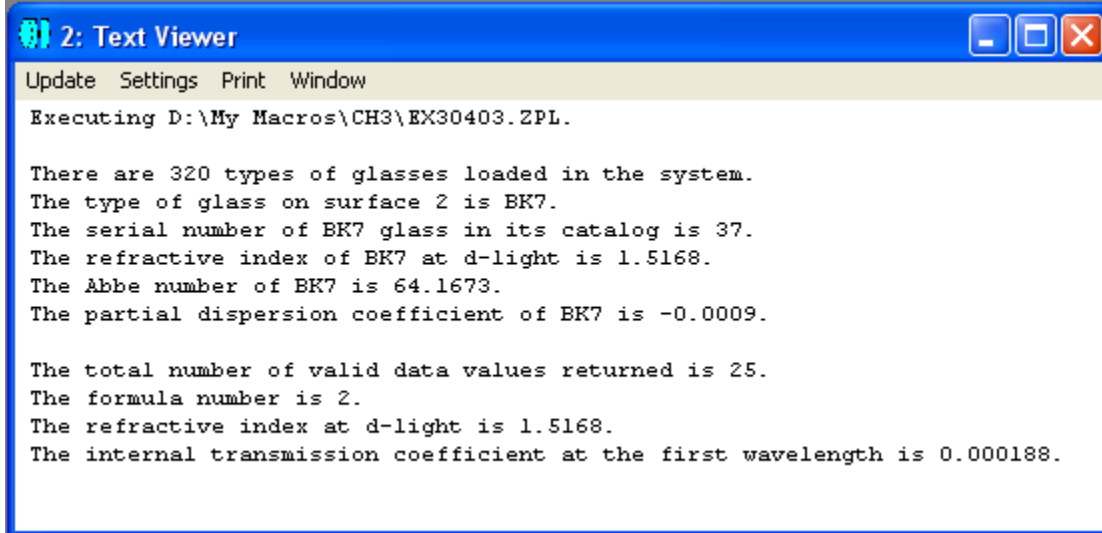

```

1 ! ex30403
2 ! This program shows how to read glass data
3 ! Assume the lens system is defined in ex30401
4
5 PRINT
6
7 ! get the total number of glass types currently loaded in the system
8 FORMAT 1 INT
9 PRINT "There are ", MAXG(), " types of glasses loaded in the system."
10
11 ! get the glass name
12 surfaceNum = 2
13 glassName$ = $GLASS(surfaceNum)
14 PRINT "The type of glass on surface ", surfaceNum, " is ", glassName$, "."
15
16 ! get the glass number
17 PRINT "The serial number of ", glassName$, " glass in its catalog ",
18 PRINT "is ", GNUM(glassName$), "."
19
20 ! get the refractive index of glass at d-light
21 PRINT "The refractive index of ", glassName$, " at d-light is ",
22 FORMAT 5.4
23 PRINT GIND(surfaceNum), "."
24
25 ! get the Abbe number of glass
26 PRINT "The Abbe number of ", glassName$, " is ", GABB(surfaceNum), "."
27
28 ! get the partial dispersion coefficient of glass
29 PRINT "The partial dispersion coefficient of ", glassName$, " is ",
30 PRINT GPAR(surfaceNum), "."
31
32 ! use keyword GETGLASSDATA to get glass data
33 GETGLASSDATA 1, GNUM(glassName$)
34 PRINT
35 FORMAT 1 INT
36 PRINT "The total number of valid data values returned is ", VEC1(0), "."
37 PRINT "The formula number is ", VEC1(1), "."
38 FORMAT 5.4
39 PRINT "The refractive index at d-light is ", VEC1(3), "."
40 PRINT "The internal transmission coefficient at the first wavelength is ",
41 FORMAT 7.6
42 PRINT VEC1(23), "."

```

程序第 9 行通过函数 MAXG() 读取了系统中装载的玻璃总数。程序第 12~14 行调用函数 \$GLASS() 读取了指定表面的玻璃名称。注意与程序第 9 行直接调用数值函数不同，由于 PRINT 语句不能直接调用字符串函数，所以程序第 13 行先将结果存于字符串变量中，第 14 行再打印显示出来。程序第 17~18 行调用函数 GNUM() 读取玻璃材料在相应目录中的序号，第 21~23 行调用函数 GIND() 读取指定表面的玻璃材料在 d 线处的折射率。程序第 26 行调用函数 GABB() 读取指定表面的玻璃材料的 Abbe 数。程序第 29~30 行调用函数 GPAR() 读取指定表面玻璃材料的部分色散。如前所述，除了通过函数读取玻璃材料的相关参数外，也可以

通过关键词 GETGLASSDATA 读取。程序第 33 ~ 42 行就说明了这一方法。通过关键词读取的信息存于缺省数组 VEC1 中，后续程序可以很方便地对其进行调用。程序运行的结果如图 3.4-3 所示：



```
2: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH3\EX30403.ZPL.

There are 320 types of glasses loaded in the system.
The type of glass on surface 2 is BK7.
The serial number of BK7 glass in its catalog is 37.
The refractive index of BK7 at d-light is 1.5168.
The Abbe number of BK7 is 64.1673.
The partial dispersion coefficient of BK7 is -0.0009.

The total number of valid data values returned is 25.
The formula number is 2.
The refractive index at d-light is 1.5168.
The internal transmission coefficient at the first wavelength is 0.000188.
```

图 3.4-3 程序 ex30403.ZPL 的运行结果

第五节 评价函数(merit function)

在利用 ZEMAX 进行光学设计和优化时，常会用到评价函数 (merit function)。评价函数是用户所定义的一个数值量，用于衡量光学系统相对于一系列设计指标的偏差。ZEMAX 中有一个评价函数编辑器，其中包含一系列的运算元(operand)。不同的运算元用于衡量不同的系统限制或设计指标。整个评价函数就由评价函数编辑器中的具有不同权重的各个运算元共同组成。

ZPL 提供了一些相应的关键词和函数，可以对评价函数进行设置和读取。

关键词 DEFAULTMERIT 用于生成一个缺省评价函数，其用法为：

DEFAULTMERIT type, data, reference, method, rings, arms, grid, delete, axial, lateral, start, xweight, oweight

其中各个参数的内容如表 3.5-1 所示：

参数	说明
type	0 为 RMS，1 为 PTV
data	0 为波前，1 为光斑半径，2 为光斑 x 长度，3 为光斑 y 长度，4 为光斑 x+y 长度
reference	0 为中心参照，1 为主光线参照，2 为无参照
method	1 为高斯正交(Gaussian quadrature)，2 为矩形阵列
rings	同心圆环的数目(仅适用于高斯正交方法)
arms	径向分支的数目(仅适用于高斯正交方法)，须为偶数并不小于 6。
grid	格子尺寸。用整数 n 代表 n x n 格子。n 必须为偶数并不小于 4。
delete	0 为不删除渐晕光线，1 为删除渐晕光线
axial	1 为假设系统轴向对称，0 为不假设系统轴向对称，-1 选择，即只有当系统为轴向对称时才使用对称性。
lateral	1 为忽略横向颜色，0 为不忽略。
start	-1 为自动，即将缺省评价函数加在任何现存的 DMFS 运算元之后，否则其它数值则为运算元的序号，表明将缺省评价函数加在此序号的位置，此序号之前的原有运算元保持不变，其余运算元被覆盖。
xweight	x 方向权重，只有“光斑 x+y 长度”用到此权重
oweight	总的权重

如果要删除评价函数编辑器中的某一个运算元，可用关键词 DELETEMFO，其用法为：

DELETEMFO row

或

DELETMFO ALL

其中，row 为运算元所在的行数，必须为大于 0 并小于现有总运算元数目的整数。如果用 ALL，则删除现有的所有运算元。

如果要在评价函数编辑器中插入一个运算元，可用关键词 INSERTMFO，其用法为：

INSERTMFO row

其中，row 为运算元所插入的行数，必须为大于 0 并小于现有总运算元数目的整数。插入新的运算元后，行数 row 上原有的运算元及其后面的运算元在评价函数编辑器中的位置顺序后移。需要指出的是，新插入的运算元为空的运算元，其具体的内容需要通过其它关键词如 SETOPERAND 来设置。

如果要设置或更新评价函数编辑器中某一运算元的内容，可用关键词 SETOPERAND，其用法为：

SETOPERAND row, col, value

其中，row 为评价函数编辑器中的行数，col 为评价函数编辑器中不同的列，value 为由 row 和 col 所决定的位置的值。col 的意义由具体的运算元决定，一般而言，为 1 代表运算元类型，为 2 代表 Int1，为 3 代表 Int2，为 4~7 代表 data1~data4，为 8 代表目标值 target，为 9 代表权重 weight。图 3.5-1 所示为直接在 ZEMAX 中设置运算元的对话框，其中 Operand 为运算元类型，对运算元 CNAX 而言，Surf 和 Wave 相当于上面所说的 Int1 和 Int2，Hx、Hy、Pol 和 Samp 则相当于上面所说的 data1~data4。

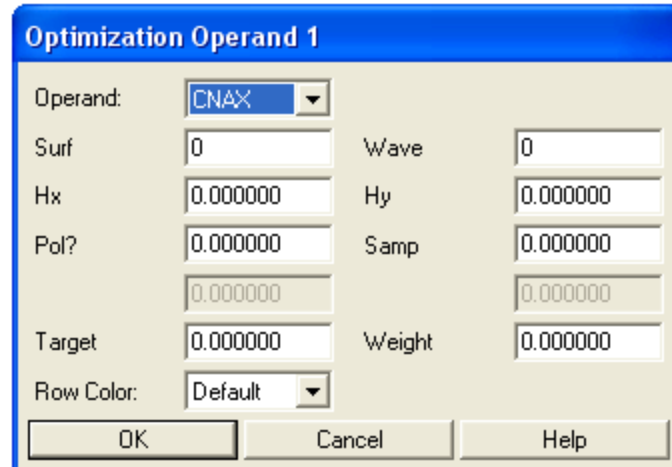


图 3.5-1 运算元设置对话框

当通过 `col = 1` 设置运算元的类型时，`value` 的值应为运算元相应的整数值，例如 1 代表“ACOS”，2 代表“ABSO”，4 代表“DENC”，367 代表“CNAX”等等。各个运算元相应的整数值可由函数 `ONUM(A$)` 的返回值确定，其中 `A$` 代表运算元相应的字符串。

除了通过上述的方法 (`col = 1`) 设置运算元的类型外，也可以用 `col = 11`，并将 `value` 值直接设为代表运算元类型的字符串，如：

```
SETOPERAND 1, 11, "CNAX"
```

或

```
A$ = "CNAX"
SETOPERAND 1, 11, A$
```

如果要读取评价函数编辑器中某一运算元的内容，可用函数 `OPER(row, col)`，其中 `row` 为评价函数编辑器中的行数，代表运算元在评价函数编辑器中的位置，`col` 为评价函数编辑器中不同的列，为 1 代表运算元类型，为 2 代表 `Int1`，为 3 代表 `Int2`，为 4~7 代表 `data1`~`data4`，为 8 代表目标值 `target`，为 9 代表权重 `weight`，为 10 代表运算元的值，为 11 代表对总的评价函数的百分比贡献。需要指出，函数 `OPER()` 只读取评价函数编辑器中的现有内容，并不对其进行更新。

如果要计算总的评价函数的值，可用函数 `MFCN()`。此函数将更新镜头数据，判断评价函数的有效性，计算其值，并将最后结果返回。

下面我们将通过例子来说明对评价函数编辑器中不同参数的设置与读取。

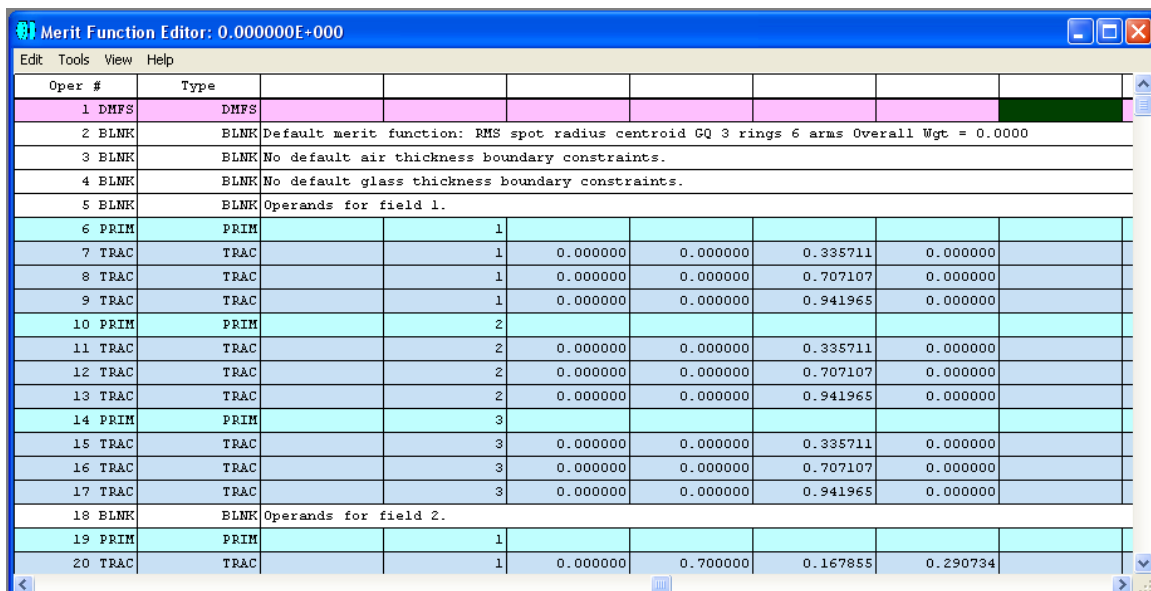
例3.5-1: 缺省评价函数的设置与读取。在此例中, 我们假设光学系统为程序 ex30401 中所构造的双透镜。我们想定义一个评价函数来衡量此光学系统的成像质量, 具体地说, 则是想衡量从各个视场点发出的光在像平面上形成的光斑的总质量。一个最直接的也是最常用的方法就是利用 ZEMAX 提供的缺省评价函数。下面的程序给出了在 ZPL 中的实现方法:

```

1 ! ex30501
2 ! This program shows how to set default merit function
3 ! Assume the lens system is defined in ex30401
4
5 tp = 0 # type is RMS
6 data = 1 # spot radius
7 ref = 0 # centroid reference
8 method = 1 # Gaussian quadrature
9 ring = 3
10 arm = 6
11 grid = 4
12 delOrNot = 0 # not delete vignettted rays
13 ax = 1 # assume axial symmetry
14 lat = 1 # ignore lateral color
15 st = 1 # put the default merit function start at the beginning
16 ow = 1 # no xweight, overall weight = 1
17
18 DEFAULTMERIT tp,data,ref,method,ring,arm,grid,delOrNot,ax,lat,st,ow
19
20 PRINT
21 PRINT "The final merit function value is ", MFCN()

```

程序运行后, 评价函数编辑器中的内容变为如图 3.5-2 所示:



Oper #	Type							
1	DMFS	DMFS						
2	BLNK	BLNK	Default merit function: RMS spot radius centroid GQ 3 rings 6 arms Overall Wgt = 0.0000					
3	BLNK	BLNK	No default air thickness boundary constraints.					
4	BLNK	BLNK	No default glass thickness boundary constraints.					
5	BLNK	BLNK	Operands for field 1.					
6	PRIM	PRIM		1				
7	TRAC	TRAC		1	0.000000	0.000000	0.335711	0.000000
8	TRAC	TRAC		1	0.000000	0.000000	0.707107	0.000000
9	TRAC	TRAC		1	0.000000	0.000000	0.941965	0.000000
10	PRIM	PRIM		2				
11	TRAC	TRAC		2	0.000000	0.000000	0.335711	0.000000
12	TRAC	TRAC		2	0.000000	0.000000	0.707107	0.000000
13	TRAC	TRAC		2	0.000000	0.000000	0.941965	0.000000
14	PRIM	PRIM		3				
15	TRAC	TRAC		3	0.000000	0.000000	0.335711	0.000000
16	TRAC	TRAC		3	0.000000	0.000000	0.707107	0.000000
17	TRAC	TRAC		3	0.000000	0.000000	0.941965	0.000000
18	BLNK	BLNK	Operands for field 2.					
19	PRIM	PRIM		1				
20	TRAC	TRAC		1	0.000000	0.700000	0.167855	0.290734

图 3.5-2 通过程序 ex30501.ZPL 生成的评价函数。

同时，在文本信息输出窗口中通过函数 MFCN() 得到总的评价函数的值，如图 3.5-3 所示。

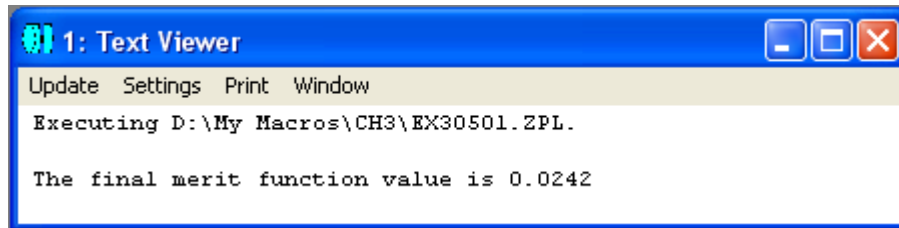


图 3.5-3 程序 ex30501.ZPL 运行后得到的评价函数值。

ZEMAX 的强大的优化功能，就表现为能通过改变不同的变量值，将评价函数的值最小化。我们将在后面的章节中介绍如何通过编程对光学系统进行优化。

除了用到上面所介绍的 ZEMAX 缺省评价函数外，很多时候用户也需要自定义评价函数，以达到特殊的设计目的。我们通过下面的例子介绍怎样在 ZPL 程序中自定义评价函数。

例 3.5-2: 自定义评价函数

评价函数如何定义，完全取决于光学系统的设计目的。在这个例子中，我们仍然假设光学系统为程序 ex30401 中所构造的双透镜。假设由于像平面探测器尺寸的限制，我们希望视场 3 ($\theta_y = 5^\circ$) 在像平面上成像点的高度为 30。这就要求我们对上一个例子中的缺省评价函数作一些修改，构造一个用户自定义的评价函数。我们可以从上一例中的缺省评价函数出发，在评价函数编辑器中插入一个新的运算元 CENY，将其目标值设定为 30。程序如下所示：

```

1 ! ex30502
2 ! This program shows how to set user defined merit function
3 ! Assume the lens system is defined in ex30401
4 ! Also assume the default merit function was first created as in ex30501
5
6 INSERTMFO 1           # insert a blank row in the merit function editor
7 INSERTMFO 1           # insert another blank row
8 SETOPERAND 1, 11, "CENY" # set operand type
9 SETOPERAND 1, 2, 0      # set surface number, 0 for image surface
10 SETOPERAND 1, 3, 0     # set wavelength number, 0 for polychromatic
11 SETOPERAND 1, 4, 3     # set field number
12 SETOPERAND 1, 8, 30    # set target
13 SETOPERAND 1, 9, 1     # set weight
14
15 PRINT
16 PRINT "The final merit function value is ", MFCN()

```

我们在评价函数编辑器内插入两个空行，将第一行的运算元类型设为“CENY”，然后设定其相应参数如表面序号、波长序号、视场序号、目标值、权重等，最后在文本信息窗口中输出当前评价函数值，如图 3.5-4 所示。

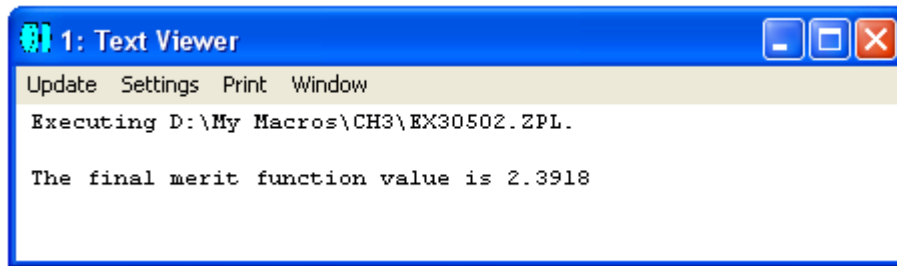


图 3.5-4 程序 ex30502.ZPL 运行后得到的评价函数值。

如果我们将双透镜的几个表面(表面 2、3、4)及其后表面到像平面的距离设为变量，则很容易利用我们定义的评价函数将系统进行优化，优化后的评价函数值可以达到 0.02，感兴趣的读者可以自行验证。

第六节 求解(solve)

ZEMAX 的一个很强大的光学设计功能，就是对现有的光学系统进行求解和优化。在 ZPL 中，也提供了许多相应的关键词和函数，可以让用户在程序中使用 ZEMAX 求解和优化的强大功能。在本节和下一节中，我们将分别介绍如何在 ZPL 程序中实现求解和优化。

一个常用的设置和修改求解参数的关键词为 SOLVETYPE，其用法为：

SOLVETYPE surf, CODE, arg1, arg2, arg3, arg4

其中，surf 为所需要设定的表面序号，为大于 0 及小于最大表面数的整数，CODE 为表 3.6-1 中所列的相应于求解类型的 ASCII 码助记符，arg1~ arg4 为不同求解类型相应的参数。注意各个求解类型所要求的参数数目和类型不等。

表 3.6-1 求解类型助记符及参数

求解类型	助记符	参数 1	参数 2	参数 3	参数 4
曲面：固定(即取消求解)	CF				
曲面：变量	CV				
曲面：边缘光线夹角	CM	夹角			
曲面：主光线夹角	CC	夹角			
曲面：摘取	CP	表面数	比例因子		列数*
曲面：垂直于边缘光线	CN				
曲面：垂直于主光线	CO				
曲面：消球面差	CA				
曲面：元件放大率	CE	放大率			
曲面：与表面共心	CQ	与其共心的表面数			
曲面：与半径共心	CR	与其半径共心的表面数			
曲面：F/#	CG	近轴系统 F/#			
曲面：ZPL 宏	CZ	宏程序名			
厚度：固定(即取消)	TF				

求解)					
厚度: 变量	TV				
厚度: 边缘光线高度	TM	高度	光瞳区域		
厚度: 主光线高度	TC	高度			
厚度: 边缘厚度	TE	厚度	径向高度 (0 代表半直径)		
厚度: 摘取	TP	表面数	比例因子	偏移量	列数*
厚度: 光程差	TO	光程差	光瞳区域		
厚度: 位置	TL	表面数	自指定表面的长度		
厚度: 补偿器	TX	表面数	表面总厚度		
厚度: 曲率中心	TY	位于曲率中心的表面数			
厚度: 光瞳位置	TU				
厚度: ZPL 宏	TZ	宏程序名			
玻璃: 固定(即取消求解)	GF				
玻璃: 模型	GM	D 线折射率 Nd	Abbe 数 Vd	部分色散 $\Delta P_g \square \square F$	
玻璃: 摘取	GP	表面数			
玻璃: 取代	GS	目录名称			
玻璃: 偏移	GO	D 线折射率 Nd 偏移	Abbe 数 Vd 偏移		
半直径: 自动	SA				
半直径: 用户定义	SU				
半直径: 摘取	SP	表面数	比例因子		列数*
半直径: 最大值	SM				
半直径: ZPL 宏	SZ	宏程序名			
Conic 常数: 固定(即取消求解)	KF				
Conic 常数: 摘取	KP	表面数	比例因子		列数*
Conic 常数: ZPL 宏	KZ	宏程序名			
参数: 固定(即取消求解), 助记符中的 p 为参数序号	PF_p				
参数: 摘取, 助记符	PP_p	表面数	比例因子	偏移量	列数*

中的 p 为参数序号					
参数：主光线，助记符中的 p 为参数序号	PC_p	视场序号	波长序号		
参数：ZPL 宏	PZ_p	宏程序名			
热膨胀系数：固定 (即取消求解)	HF				
热膨胀系数：摘取	HP	表面数			
附加数据值：固定 (即取消求解)，助记符中的 e 为附加数据序号	EF_e				
附加数据值：摘取，助记符中的 e 为附加数据序号	EP_e	表面数	比例因子	偏移量	列数*
附加数据值：ZPL 宏，助记符中的 e 为附加数据序号	EZ_e	宏程序名			
非顺序元件 X 坐标、Y 坐标、Z 坐标、X 偏移、Y 偏移、Z 偏移、材料：摘取，助记符中的 o 为元件序号	NSC_PX_o NSC_PY_o NSC_PZ_o NSC_PTX_o NSC_PTY_o NSC_PTZ_o NSC_PMAT_o				
非顺序元件 X 坐标、Y 坐标、Z 坐标、X 偏移、Y 偏移、Z 偏移：ZPL 宏，助记符中的 o 为元件序号	NSC_ZX_o NSC_ZY_o NSC_ZZ_o NSC_ZTX_o NSC_ZTY_o NSC_ZTZ_o				
非顺序元件参数：摘取，助记符中的 o 为元件序号，p 为元件参数序号	NSC_PP_o_p				
非顺序元件参数：ZPL 宏，助记符中的 o 为元件序号，p 为元件参数序号	NSC_ZP_o_p	宏程序名			

*其中列数定义如下:

0: 当前列;
1~4: 半径、厚度、conic 常数、半直径;
5~17: 参数1~12;
18~259: 附加数据1~242。

在上表所列的求解类型中, 有多处用到ZPL宏程序, 这允许用户在求解时使用自己编写的ZPL程序。这个程序与其它ZPL程序不同的地方是它需要用到关键词 SOLVERETURN将结果返回到调用此程序的编辑器中, 其用法为:

SOLVERETURN value

其中 value 为返回值。

当程序中出现多于一个的SOLVERETURN关键词时, 只有最后一个 SOLVERETURN指令会被执行, 而其前面的SOLVERETURN指令被忽略。如果程序中出现错误或无效情况时, 程序将不会执行SOLVERETURN指令, 表明光学系统中有错误存在。

需要特别指出的是, 尽管ZPL宏程序求解是一种非常灵活的手段, 但另一方面它也很容易引起死循环或异常中断, 因此在编写程序时应特别小心。

相应于设置和修改求解参数, ZPL也提供了读取求解参数的函数SOLV(), 其用法为:

returnValue = SOLV(surf, code, param)

其中, surf为表面的序号, code 为0代表曲率, 为1代表厚度, 为2代表玻璃, 为3代表conic常数, 为4代表半直径, 为5代表热膨胀系数TCE, param为0~3的整数, 代表指定求解类型下相应参数的序号。当param为0时, 函数返回值returnValue为相应于求解类型的整数, 而当param为1~3之间的整数时, 函数返回值returnValue为指定表面的指定求解类型和参数的设定值。

下面我们将给出一些关于求解的例子。

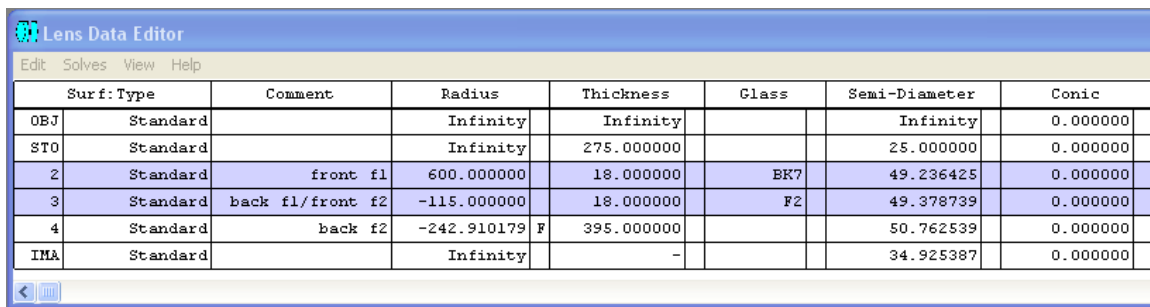
例3.6-1: 求解参数的设置。假设我们已经构造了如例3.4-1所述的双透镜。在那个系统中, 表面4的曲率半径为-243, 系统的近轴F/#并不完全等于设计参数8。我们可以通过求解的方式改变表面4的曲率半径, 迅速让系统的近轴F/#趋近于8。下面是相应的程序:

```

1 ! ex30601
2 ! This program shows how to set solve parameters
3 ! Assume the lens system is defined in ex30401
4
5 surf = 4
6 CODE$ = "CG" # solve type is "Curvature: F/#"
7 arg1 = 8 # F/# = 8
8
9 SOLVETYPE surf, CODE$, arg1 # only 1 argument is needed
10 UPDATE # UPDATE is needed to get the solve result

```

在程序中，我们将表面4设为求解类型“曲面：F/#”，并令F/# = 8。程序运行后，我们看到在镜头数据编辑器中表面4的曲率半径已经改变，并且在数据右边出现字母F，如图3.6-1所示：



Surf	Type	Comment	Radius	Thickness	Glass	Semi-Diameter	Conic
OBJ	Standard		Infinity	Infinity		Infinity	0.000000
STO	Standard		Infinity	275.000000		25.000000	0.000000
2	Standard	front f1	600.000000	18.000000	EK7	49.236425	0.000000
3	Standard	back f1/front f2	-115.000000	18.000000	F2	49.378739	0.000000
4	Standard	back f2	-242.910179 F	395.000000		50.762539	0.000000
IMA	Standard		Infinity	-		34.925387	0.000000

图 3.6-1 程序 ex30601.ZPL 的运行结果

例3.6-2：求解参数的读取。假设要读取上例中所设置的求解参数，我们可以通过函数SOLV()来实现，程序如下：

```

1 ! ex30602
2 ! This program shows how to read solve parameters
3 ! Assume the solve type is defined in ex30601
4
5 surf = 4
6 code = 0 # solve type is "Curvature"
7 param = 1 # want to know the first argument, i.e. F/#
8
9 returnValue = SOLV(surf, code, param)
10
11 PRINT
12 PRINT "The solve parameter is ", returnValue

```

注意，此处的code是一个数值变量，而上例中的CODE\$是一个字符串变量。程序的运行结果为：

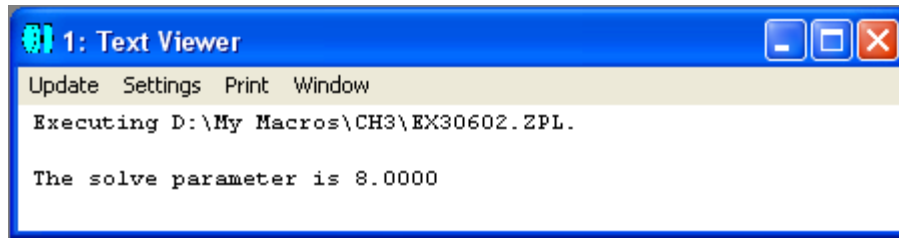


图 3.6-2 程序 ex30602.ZPL 的运行结果

结果表明我们在上例中设置的表面4的求解参数为 $F/\# = 8$ 。

例3.6-3: ZPL宏求解的应用。在例3.6-1中, 我们假设空气的折射率为1, 得到表面4的曲率半径为-242.910179。现在我们要利用空气的实际折射率 $n = 1.0008$ 和关系 $n_1C_1 = n_2C_2$ (n 为折射率, C 为曲率)对此半径值进行微调, 使计算值更接近实际值。我们可以通过ZPL宏求解来实现。求解参数设置的程序和宏求解程序分别为 ex30603.ZPL及ex30603M.ZPL, 如下所示:

```

1 ! ex30603
2 ! This program shows how to set ZPL macro solve
3 ! Assume the optical system is defined in ex30601
4
5 surf = 4
6 CODE$ = "C2" # solve type is "Curvature: ZPL macro"
7 arg1$ = "\ch3\ex30603M.zpl" # macro name
8
9 SOLVETYPE surf, CODE$, arg1$ # only 1 argument is needed
10 UPDATE # UPDATE is needed to get the solve result

```

```

1 ! ex30603M
2 ! This program shows an example of ZPL macro
3 ! It will be called by program "ex30603.zpl"
4
5 nVacuum = 1
6 nAir = 1.0008
7
8 surf = 4
9 culvatureVacuum = CURV(surf) # get the curvature of surface 4
10
11 culvatureAir = culvatureVacuum*nVacuum/nAir # modify the culvature
12
13 temp = 1/123.456 # create a number
14 SOLVERETURN temp # try to add more than one SOLVERETURN
15
16 SOLVERETURN culvatureAir # return the culvature, not the radius!!

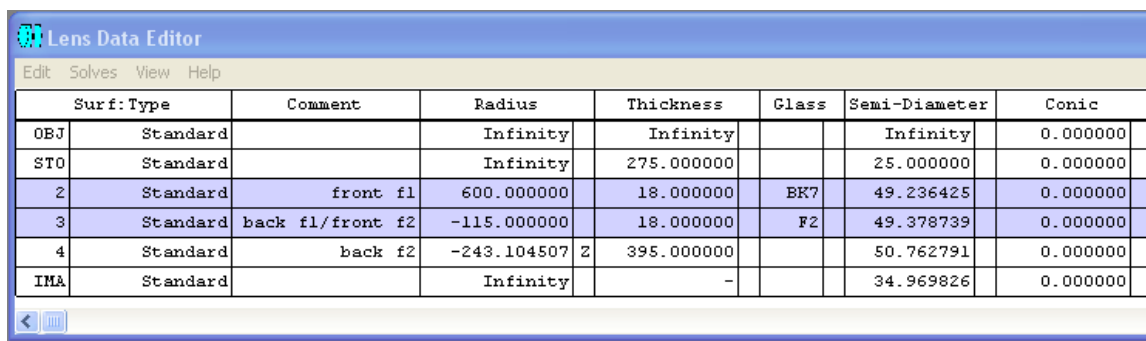
```

注意在程序ex30603中的第7行，我们在宏程序名“ex30603M.zpl”前加上了一个路径名“\ch3\”，这是因为执行程序ex30603.zpl时，ZEMAX只在其指定路径下寻找要执行的求解宏程序，如果指定路径下还有目录结构时，需要进一步指明。

还需要注意，程序ex30603M中计算的数值为曲率，将此曲率值返回后，ZEMAX会自动换算出曲率半径值。

另外，程序ex30603M的第13和14行不具有任何功能，可以从程序中删除。我们将其插入程序中，只是想说明，如果求解宏程序中有多于一个的关键词SOLVERRETURN时，只有最后一个SOLVERRETURN有效。

程序ex30603的运行结果如下：



Surf	Type	Comment	Radius	Thickness	Glass	Semi-Diameter	Conic
OBJ	Standard		Infinity	Infinity		Infinity	0.000000
STO	Standard		Infinity	275.000000		25.000000	0.000000
2	Standard	front f1	600.000000	18.000000	BK7	49.236425	0.000000
3	Standard	back f1/front f2	-115.000000	18.000000	F2	49.378739	0.000000
4	Standard	back f2	-243.104507 Z	395.000000		50.762791	0.000000
IMA	Standard		Infinity	-		34.969826	0.000000

图 3.6-3 程序 ex30603.ZPL 的运行结果

我们可以看到，表面4的新的曲率半径值为-243.104507，并且数值右边出现字母Z，表明用到的求解类型为ZPL宏求解。

ZPL还提供了一个和自动求解相似的关键词QUICKFOCUS，用于对系统进行快速聚焦，其用法为：

QUICKFOCUS mode, centroid

其中mode为0、1、2或3，分别代表均方根(RMS)光斑半径、光斑x值、光斑y值及波前光程差，centroid为0或1，分别代表RMS是相对于主光线或成像中心。

第七节 优化(optimization)

ZEMAX强大的光学设计功能之一，就表现在它具有很强的优化能力。在本节中我们将着重介绍在ZPL程序中如何使用优化指令。关于在ZEMAX中进行优化的详细介绍，请参阅ZEMAX用户指南。

我们知道，通过ZEMAX对光学系统进行优化，就是将一些特定的镜头参数设置为变量，让ZEMAX自动改变变量的值以使设定的评价函数趋于最小。ZPL提供了一个设置和修改优化变量的关键词SETVAR，其用法为：

SETVAR surf, code, status, objectNum

或

SETVAR config, M, status, operand

其中，surf为镜头表面序号，config为多组态编辑器中的组态序号，code为表3.7-1所列的字符串之一。如果status的值为0，则取消变量设置，否则将与code相应的值设为变量。如果code为Nn或On，则需提供非顺序元件的编号objectNum。如果code为M，则需提供多组态中的运算元operand。

表3.7-1：关键词SETVAR相应的代码

代码	说明
R	曲率半径
T	厚度
C	conic常数
G	玻璃种类
I	玻璃折射率
J	玻璃Abbe数
K	玻璃部分色散 $\Delta P_g \square \square F$
Pn	第n个参数
D	热膨胀系数
En	第n个附加数据
M	多组态数据
Nn	非顺序元件位置数据，n = 1~6，分别为x, y, z, tx, ty, tz
On	非顺序元件第n个参数的数据

如果要取消对所有优化变量的设置，可用关键词REMOVEVARIABLES，将所有已设置的变量改为固定量。

如果要读取已设置的优化变量，可用关键词GETVARDATA，其用法为：

GETVARDATA vector

其中vector = 1~4，为ZPL所提供的4个一维数组之一。各数组元素存放的数据如表3.7-2所示：

表3.7-2：通过GETVARDATA得到的各数组元素存放的数据

数组元素序号	说明
0	变量的总数目n
1	第一个变量的类型代码
2	第一个变量的表面序号
3	第一个变量的参量序号
4	第一个变量的元件序号
5	第一个变量的值
5*q-4	第一个变量的类型代码
5*q-3	第q个变量的表面序号
5*q-2	第q个变量的参量序号
5*q-1	第q个变量的元件序号
5*q	第q个变量的值

不同变量类型的代码和其它参数说明如表3.7-3所示。对于不同的变量类型，某些参数可能没有意义。

表3.7-3：GETVARDATA关键词用到的变量类型代码和其它参数

变量类型	代码	表面	参量	元件
曲率	1	表面序号	-	-
厚度	2	表面序号	-	-
Conic常数	3	表面序号	-	-
折射率Nd	4	表面序号	-	-
Abbe系数 Vd	5	表面序号	-	-
部分色散 $\Delta P_g \square \square F$	6	表面序号	-	-
热膨胀系数	7	表面序号	-	-
参量值	8	表面序号	参量序号	-
附加数据值	9	表面序号	附加数据序号	-
多组态运算元的值	10	运算元的序号	组态序号	-
非顺序元件位置x	11	表面序号	-	元件序号
非顺序元件位置y	12	表面序号	-	元件序号

非顺序元件位置Z	13	表面序号	-	元件序号
非顺序元件倾斜X	14	表面序号	-	元件序号
非顺序元件倾斜Y	15	表面序号	-	元件序号
非顺序元件倾斜Z	16	表面序号	-	元件序号
非顺序元件参量	17	表面序号	参量序号	元件序号

例3.7-1给出了在实际程序中设置和读取变量的例子。在此例中，我们假设光学系统为程序ex30401.ZPL中所构造的双透镜。我们希望将表面4的曲率半径和厚度设为变量，并将所设置的变量读出。相应的程序如下所示：

```

1 ! ex30701
2 ! This program shows how to set and read variables for optimization
3 ! Assume the lens system is defined in ex30401
4
5 surf = 4
6 CODE1$ = "R" # Radius
7 CODE2$ = "T" # Thickness
8 status = 1 # set variable
9
10 SETVAR surf, CODE1$, status # variable 1, only 3 arguments
11 SETVAR surf, CODE2$, status # variable 2, only 3 arguments
12
13 GETVARDATA 1 # read variable data and put in VEC1
14 totalVarNum = VEC1(0)
15
16 PRINT
17 PRINT "Total variable number is: ", totalVarNum
18
19 for q = 1, totalVarNum, 1
20 PRINT
21 PRINT "Variable ", q, ", code number : ", VEC1(5*q-4)
22 PRINT "Variable ", q, ", surface number : ", VEC1(5*q-3)
23 PRINT "Variable ", q, ", current value : ", VEC1(5*q)
24 NEXT

```

在程序运行后，我们可以看到，在镜头数据编辑器中，表面4的曲率半径和厚度相应的数据旁边出现字母“V”，表明这两个参数已被设为变量。另外，在文字窗口中，我们可以看到变量的读取结果，如图3.7-1所示。

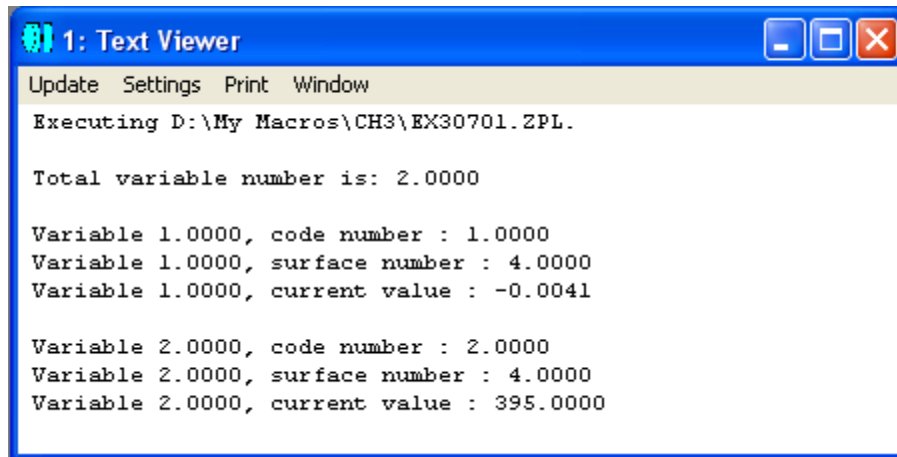


图 3.7-1 程序 ex30701.ZPL 的运行结果

我们注意到，变量1的值为表面4的曲率的值，和镜头数据编辑器中所示的曲率半径的值互为倒数关系。

在设置完变量和评价函数后，进行优化就很简单了。ZPL提供了关键词OPTIMIZE和HAMMER，其用法为：

```
OPTIMIZE
OPTIMIZE number_of_cycles
OPTIMIZE number_of_cycles, algorithm
```

及

```
HAMMER
HAMMER number_of_cycles
HAMMER number_of_cycles, algorithm
```

其中，number_of_cycles为1~99之间的整数，代表执行优化的次数，algorithm = 0为阻尼最小均方差法(Damped Least Squares，为缺省方法)，1为正交递减法(Orthogonal Descent)。如果OPTIMIZE后面不带参数，则使用缺省优化方法，自动执行；如果HAMMER后面不带参数，则使用缺省优化方法，执行优化次数为1。

有些时候，我们想直接计算某些优化运算元的值，而不需要将其加入评价函数内，这时我们可以使用ZPL提供的函数OPEV或OPEW(两者的作用很相似，只是参数的数目不同)，其用法分别为：

```
OPEV(code, int1, int2, data1, data2, data3, data4)
```

及

OPEW(code, int1, int2, data1, data2, data3, data4, data5, data6)

其中，code为优化运算元代码，int1~int2及data1~data6为各运算元相应的参数。一般而言，代码code由函数OCOD(A\$)返回，其中A\$为优化运算元相应的字符串，如"EFFL"等。

例3.7-2给出了在实际程序中进行优化的例子。在此例中，我们假设光学系统为程序ex30401.ZPL中所构造的双透镜，相应的变量如程序ex30701.ZPL所设置，评价函数如程序ex30501.ZPL所设置。我们希望将现有的光学系统作进一步的优化，并求出系统的有效焦距。相应的程序如下所示：

```

1 ! ex30702
2 ! This program shows how to do optimization and calculate operand
3 ! Assume the lens system is defined in ex30401
4 ! Assume the variables are defined in ex30701
5 ! Assume the merit function is defined in ex30501
6
7 code = OCOD("EFFL") # get the code of operand "EFFL"
8 wavelengthNum = 2
9 efflValue = OPEV(code, 0, wavelengthNum, 0, 0, 0, 0) # only int2 is needed
10
11 PRINT
12 PRINT "The effective focal length before optimization is : ", efflValue
13
14 OPTIMIZE # use automatic optimization, default algorithm
15
16 efflValue = OPEV(code, 0, wavelengthNum, 0, 0, 0, 0) # only int2 is needed
17
18 PRINT
19 PRINT "The effective focal length after optimization is : ", efflValue
20

```

在程序中，我们用到函数OCOD()读取有效焦距运算元“EFFL”的相应代码，并用函数OPEV()直接求出有效焦距。我们还用到OPTIMIZE对系统进行优化，并比较了优化前后有效焦距的值。程序的运行结果如下：

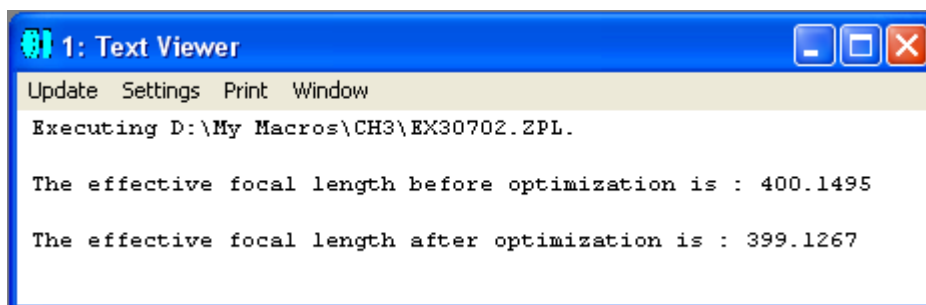


图3.7-2程序ex30702.ZPL的运行结果

ZPL还提供了一个和自动优化相似的关键词TESTPLATEFIT，用于在设计中自动套用镜头生产厂商提供的标准样板库。

TESTPLATEFIT的用法为：

TESTPLATEFIT tpd_file, log_file, method, number_cycles

其中tpd_file为样板数据文件名，log_file为输出log文件名，method为0~4间的整数，分别表示尝试所有方法、从优到劣、从劣到优、从长到短及从短到长，number_cycles为0表示自动，否则表示最大优化次数。

第八节 光线追迹

我们知道，ZEMAX的大部分计算，都是基于光线追迹，因此，光线追迹是ZEMAX的一个极其重要的功能。ZPL提供了两个关键词RAYTRACE 和 RAYTRACEX，用于支持顺序光学系统中的光线追迹。对于非顺序光学系统中的光线追迹，我们将在第十节中进行介绍。

关键词RAYTRACE用于对特定的视场和经过系统孔径上的特定位置的光线进行追迹，其用法如下：

RAYTRACE hx, hy, px, py, wavelength

其中，hx 和 hy是归一化的视场坐标，其值在-1~+1之间；px和 py是归一化的系统孔径上的坐标，其值也在-1~+1之间；wavelength是可选项，为系统中工作波长的序号，缺省则为主波长。

关键词 RAYTRACEX 用于对特定的坐标点和传播方向上的光线进行追迹，其用法如下：

RAYTRACEX x, y, z, l, m, n, surf, wavelength

其中，x、 y、 z 为光线起始点相对于起始表面的位置，l、 m、 n 为光线的方向余弦，surf 为进行光线追迹的起始表面的序号，wavelength 是可选项，为系统中工作波长的序号，缺省则为主波长。

在使用关键词RAYTRACE或RAYTRACEX进行光线追迹后，所得到的结果可以用不同的函数进行读取，如表3.8-1所示：

表3.8-1：光线追迹结果的读取函数

函数	说明
RAYX(n)	光线与表面相交点相对于表面顶点的坐标，其中 n 为表面序号
RAYY(n)	
RAYZ(n)	
RAGX(n)	光线与表面相交点的全域坐标，其中 n 为表面序号
RAGY(n)	
RAGZ(n)	
RAYL(n)	光线与表面相交点光线的方向余弦，其中 n 为表面序号
RAYM(n)	
RAYN(n)	
RANX(n)	光线与表面相交点表面法线的方向余弦，其中 n 为表面序号
RANY(n)	
RANZ(n)	
RAYT(n)	从表面 n-1 到表面 n 的光线的总路程，可能为负值。对于非顺序光学系统，返回值为光线所经过的物体的总路程。
RAYO(n)	从表面 n-1 到表面 n 的光线的总光程(总路程乘以折射率)，可能为负值。对于非顺序光学系统，返回值为光线所经过的物体的总光程。
RAYV()	返回值为0表明光线没有发生渐晕，否则返回值为发生渐晕的表面的序号。
RAYE(n)	返回值为0表明光线追迹没有出错，为负整数 -x 表明在第 x 个表面发生了全内反射，为正整数 +x 表明光线错过了第 x 个表面。

另外，ZPL还提供了以下几个和光线追迹有关的关键词：SCATTER、SETAIM、SETAIMDATA，分别说明如下：

SCATTER用于控制在顺序光线追迹中是否使用散射，其用法为SCATTER ON 或 SCATTER OFF。系统的缺省状态为SCATTER OFF，表明不使用散射。如果运行指令SCATTER ON，则在随后的光线追迹中起用散射功能。

SETAIM 用于控制在顺序光线追迹中是否使用光线定位，其用法为SETAIM state，其中state为0表示不使用光线定位，为1则使用光线定位。

SETAIMDATA用于设置光线定位的参数，其用法为：

SETAIMDATA code, value

其中，code和value的内容分别为表3.8-2所示：

表3.8-2: SETAIMDATA关键词的代码code和参数value说明

代码code	说明
1	使用光线定位缓存, value为1代表真, 为0代表伪。
2	使用稳健光线定位, value为1代表真, 为0代表伪。
3	按照视场调整光瞳位移因子, value为1代表真, 为0代表伪。
4、5、6	分别设置光瞳位移x、y和z
7、8	分别设置光瞳压缩因子x和y

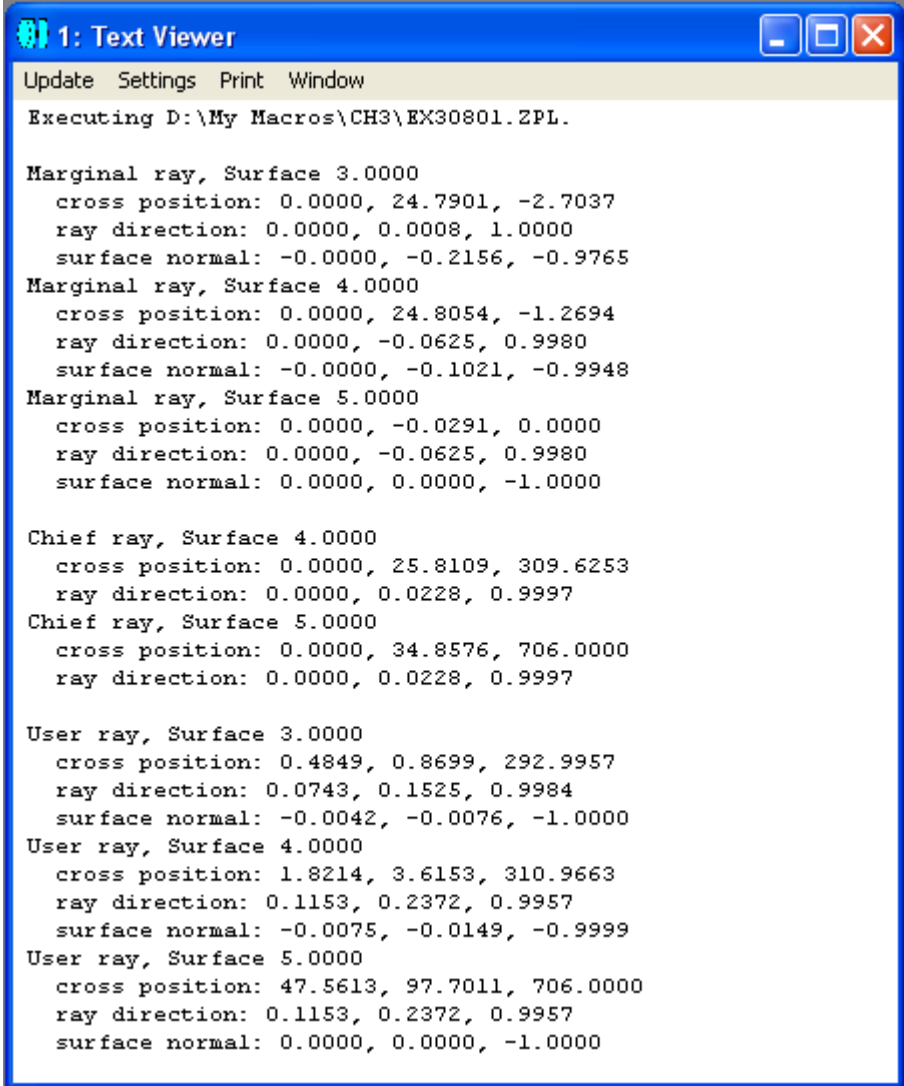
下面我们给出一个在程序中使用光线追迹的例子。在本例中, 我们假设光学系统为程序ex30401.ZPL中所构造的双透镜, 我们将对边缘光线(marginal ray)、主光线(chief ray)和一条任意光线进行追迹, 并读取光线和不同表面相交点的信息。程序如下:


```

1 ! ex30801
2 ! This program shows how to do ray tracing
3 ! Assume the lens system is defined in ex30401
4
5 ! define the marginal ray
6 hx = 0
7 hy = 0
8 px = 0
9 py = 1
10 wavelength = 2
11
12 RAYTRACE hx, hy, px, py, wavelength # trace the marginal ray
13
14 PRINT
15 FOR n = 3, NSUR(), 1 # go through the last 3 surfaces
16   PRINT "Marginal ray, Surface ", n
17   PRINT " cross position: ", RAYX(n), ", ", RAYY(n), ", ", RAYZ(n)
18   PRINT " ray direction: ", RAYL(n), ", ", RAYM(n), ", ", RAYN(n)
19   PRINT " surface normal: ", RANX(n), ", ", RANY(n), ", ", RANZ(n)
20 NEXT n
21
22 ! define the chief ray
23 hx = 0
24 hy = 1
25 px = 0
26 py = 0
27
28 RAYTRACE hx, hy, px, py # trace the chief ray, use primary wavelength
29
30 PRINT
31 FOR n = 4, NSUR(), 1 # go through the last 2 surfaces
32   PRINT "Chief ray, Surface ", n
33   PRINT " cross position: ", RAGX(n), ", ", RAGY(n), ", ", RAGZ(n)
34   PRINT " ray direction: ", RAYL(n), ", ", RAYM(n), ", ", RAYN(n)
35 NEXT n
36
37 ! define an arbitrary ray on surface 2
38 x = 0.2
39 y = 0.3
40 z = -0.5
41 l = 0.1
42 m = 0.2
43 z = 0.9
44 surf = 2
45 wavelength = 3
46
47 RAYTRACEX x, y, z, l, m, n, surf, wavelength
48 PRINT
49 FOR n = 3, 5, 1 # go through surfaces 3, 4 and 5
50   PRINT "User ray, Surface ", n
51   PRINT " cross position: ", RAGX(n), ", ", RAGY(n), ", ", RAGZ(n)
52   PRINT " ray direction: ", RAYL(n), ", ", RAYM(n), ", ", RAYN(n)
53   PRINT " surface normal: ", RANX(n), ", ", RANY(n), ", ", RANZ(n)
54 NEXT n

```

在程序中，我们定义了一条边缘光线、一条主光线和一条用户自定义光线，并对这几条光线进行了追迹，然后，用不同的函数将相应的信息读出。注意，在我们的程序中读取光线与表面交点的位置时，对于边缘光线，我们读到的是相对于表面顶点的坐标(程序第17行)，而对于主光线，我们读到的是全域坐标(程序第33行)。整个程序的运行结果如下所示：



```
1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH3\EX30801.ZPL.

Marginal ray, Surface 3.0000
  cross position: 0.0000, 24.7901, -2.7037
  ray direction: 0.0000, 0.0008, 1.0000
  surface normal: -0.0000, -0.2156, -0.9765
Marginal ray, Surface 4.0000
  cross position: 0.0000, 24.8054, -1.2694
  ray direction: 0.0000, -0.0625, 0.9980
  surface normal: -0.0000, -0.1021, -0.9948
Marginal ray, Surface 5.0000
  cross position: 0.0000, -0.0291, 0.0000
  ray direction: 0.0000, -0.0625, 0.9980
  surface normal: 0.0000, 0.0000, -1.0000

Chief ray, Surface 4.0000
  cross position: 0.0000, 25.8109, 309.6253
  ray direction: 0.0000, 0.0228, 0.9997
Chief ray, Surface 5.0000
  cross position: 0.0000, 34.8576, 706.0000
  ray direction: 0.0000, 0.0228, 0.9997

User ray, Surface 3.0000
  cross position: 0.4849, 0.8699, 292.9957
  ray direction: 0.0743, 0.1525, 0.9984
  surface normal: -0.0042, -0.0076, -1.0000
User ray, Surface 4.0000
  cross position: 1.8214, 3.6153, 310.9663
  ray direction: 0.1153, 0.2372, 0.9957
  surface normal: -0.0075, -0.0149, -0.9999
User ray, Surface 5.0000
  cross position: 47.5613, 97.7011, 706.0000
  ray direction: 0.1153, 0.2372, 0.9957
  surface normal: 0.0000, 0.0000, -1.0000
```

图3.8-1程序ex30801.ZPL的运行结果

除了上面所述的光线追迹指令外，ZPL还专门提供了两个用于偏振光线追迹的关键词POLDEFINE和POLTRACE，其中，POLDEFINE用于设置初始偏振状态，POLTRACE用于进行偏振光线追迹。

关键词POLDEFINE的用法如下：

POLDEFINE Jx, Jy, PhaX, PhaY

其中，Jx和Jy为Jones矢量中的电场振幅，PhaX和PhaY为以度为单位的相位。ZEMAX会自动将输入的数值归一化，以保证合振幅的大小为1。缺省的Jx、Jy、PhaX和PhaY的值分别为0、1、0 和 0。

关键词POLTRACE的用法如下：

POLTRACE Hx, Hy, Px, Py, wavelength, vec, surf

其中，Hx 和Hy为归一化的物表面坐标，介于-1 和1之间；Px 和Py 为归一化的入瞳坐标，也是介于-1 和1之间；wavelength为波长序号；vec为1~4之间的整数，代表计算结果所存的缺省一维数组的序号；surf为表面序号。光线的起始偏振状态由关键词POLDEFINE定义。光线追迹的结果存于由vec指定的一维数组中，存放格式如下：

表3.8-3：偏振光线追迹结果存放说明

数组元素序号	说明
0	数组中数据总数n。如果 n = 0，说明有错误发生。
1	离开表面的光线强度。
2	电场X分量，实部。
3	电场Y分量，实部。
4	电场Z分量，实部。
5	电场X分量，虚部。
6	电场Y分量，虚部。
7	电场Z分量，虚部。
8	S-偏振反射幅度，实部。
9	S-偏振反射幅度，虚部。
10	S-偏振透射幅度，实部。
11	S-偏振透射幅度，虚部。
12	P-偏振反射幅度，实部。
13	P-偏振反射幅度，虚部。
14	P-偏振透射幅度，实部。
15	P-偏振透射幅度，虚部。
16	电场X方向相位Px。
17	电场Y方向相位Py。
18	电场Z方向相位Pz。
19	偏振椭圆长轴长度。

20	偏振椭圆短轴长度。
21	偏振椭圆方向，单位为弧度。

下面我们通过例子介绍关键词POLDEFINE和POLTRACE的用法。

例3.8-2：偏振光线追迹。

```

1 ! ex30802
2 ! This program shows how to do polarization ray tracing
3 ! Assume the lens system is defined in ex30401
4
5 ! define the initial polarization
6 Jx = 1
7 Jy = 1
8 PhaiX = 0
9 PhaiY = 0
10 POLDEFINE Jx, Jy, PhaiX, PhaiY
11
12 ! define the chief ray
13 hx = 0
14 hy = 1
15 px = 0
16 py = 0
17
18 ! choose the vector number
19 vec = 1
20
21 ! ray tracing
22 POLTRACE hx, hy, px, py, pwav(), vec, nsur()
23
24 PRINT
25 PRINT "Transmission of chief ray at primary wavelength is ", vec1(1)
26 PRINT "Angle of polarization ellipse is ", vec1(21), " radians"

```

在此例中，我们假设光学系统为例 3.4-1 所定义的系统。我们在程序中定义了初始偏振状态(第 6~10 行)和主光线(第 13~16 行)，选择了将结果存于缺省数组 vec1 中，然后进行了偏振光线追迹。程序的运行结果如图 3.8-2 所示：

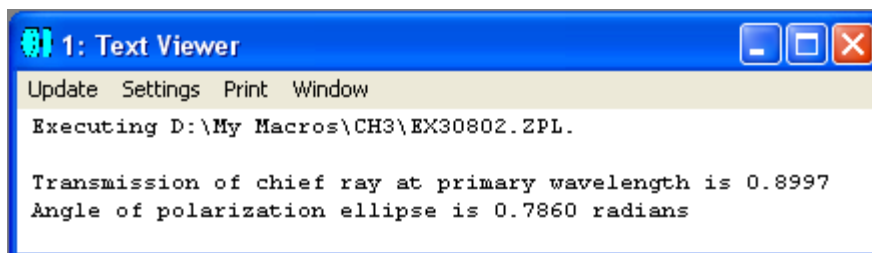
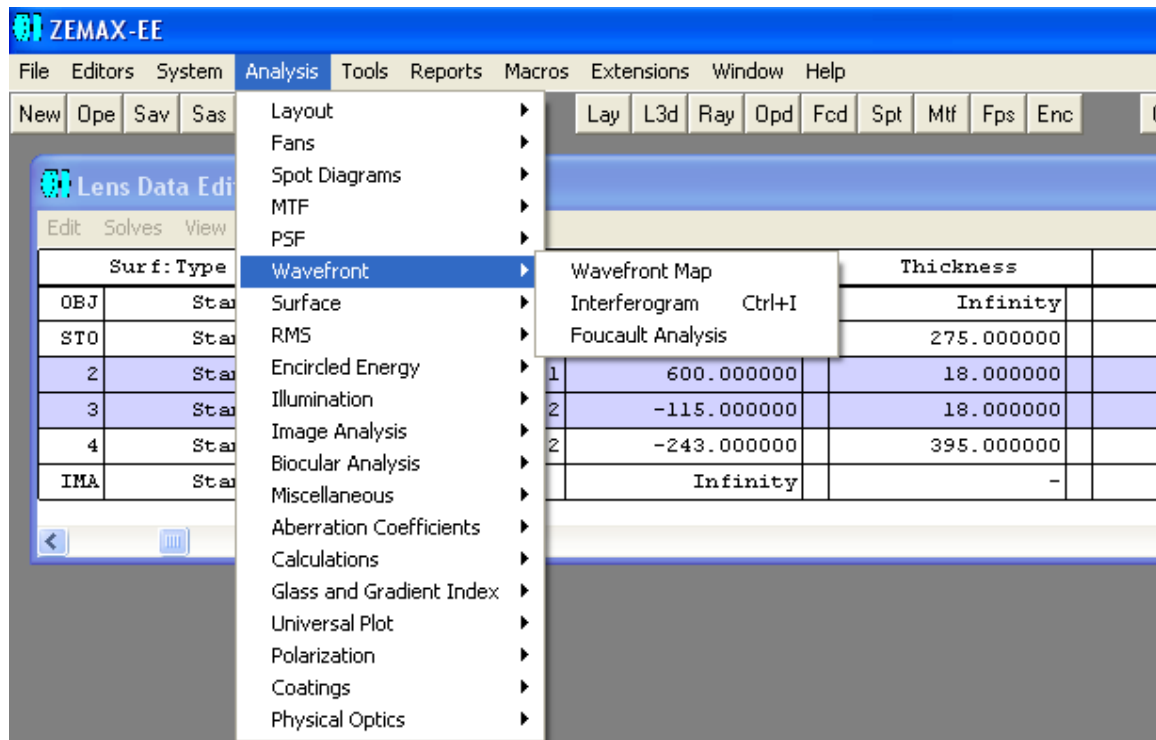


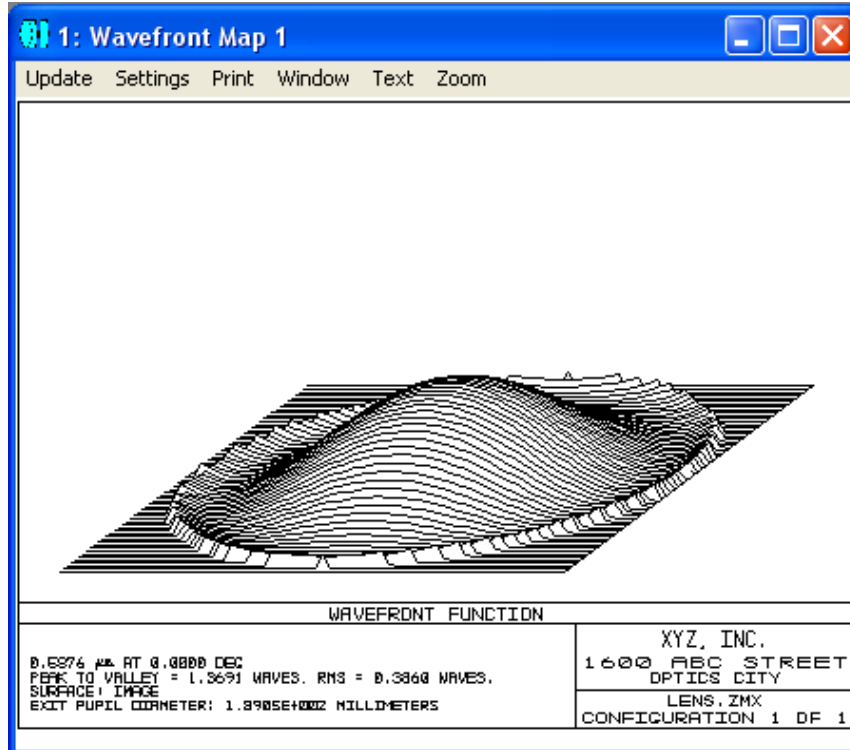
图3.8-2程序ex30802.ZPL的运行结果

第九节 系统分析

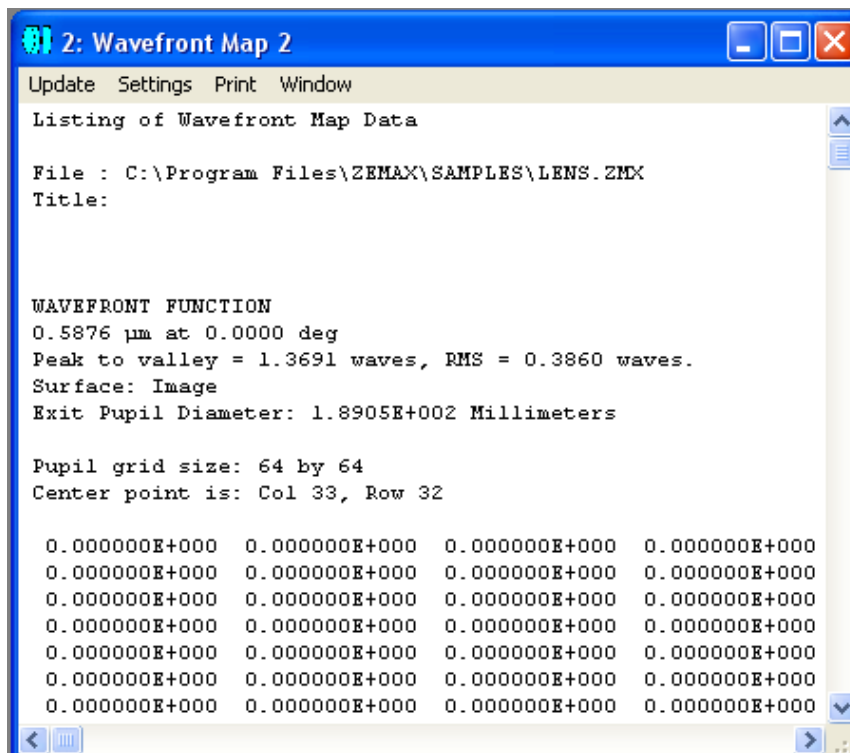
ZEMAX 提供了许多的分析工具，用于衡量系统的光学质量。其中，有很多分析工具提供文字输出信息，如图 3.9-1(a)所示的 Wavefront Map 选择项，就可以提供在某一表面上的波前形貌图，如图 3.9-1(b)所示，同时，相应的波前形貌图还提供文字信息输出，如图 3.9-1(c)所示。此例中我们假设光学系统为程序 ex30401.ZPL 中所构造的双透镜。



(a) ZEMAX 的分析工具选项



(b) 分析结果的图形输出



(c) 分析结果的文字输出

图 3.9-1: ZEMAX 分析选项及相应结果的图形和文字输出

对于这些文字输出结果，ZPL 提供了关键词 GETTEXTFILE 用于读取相应的信息并将结果存于文本文件中，其用法为：

GETTEXTFILE textfilename\$, type, settingsfilename\$, flag

其中，textfilename\$为字符串，代表指定的目标文件名(包含路径名)；type 为由三个字母组成的标记符，代表不同的分析类型，如表 3.9-1 所示；settingsfilename\$为字符串，代表与分析计算相关的参数设置；flag 为参数设置文件的标识符。如果 type 代表的分析类型无效，则执行一次光线追迹。如果 flag 的值为 0，则进行分析计算时使用缺省参数设置，即如果当前镜头文件有自己的缺省参数设置(存于文件 "lensfilename.cfg"内)，则使用此参数设置，否则使用 ZEMAX 的通用缺省参数设置(存于文件 "ZEMAX.CFG"内)，如果没有找到事先设置好的通用参数，则用 ZEMAX 的工厂缺省参数设置。如果 flag 的值为 1，则使用文件 settingsfilename\$中的参数设置(前提是参数设置文件有效，否则使用缺省参数设置)。如果 flag 的值为 2，则使用文件 settingsfilename\$中的参数设置并在屏幕上弹出对话框以允许用户对参数进行修改。

表3.9-1: GETTEXTFILE关键词中用到的类型代码及说明

代码	Description	说明
Bfv	Beam File Viewer	光束文件浏览器，用于浏览和分析ZEMAX光束文件(ZBF)
Caa	Coating Abs. vs Angle	镀膜的吸收率和入射角的关系
Car	Cardinal Points	光学系统的基点
Caw	Coating Abs. vs Wavelength	镀膜的吸收率和波长的关系
Cda	Coating Diattenuation vs Angle	镀膜的双衰减和入射角的关系
Cdw	Coating Diatten. vs Wavelength	镀膜的双衰减和波长的关系
Cfs	Chromatic Focal Shift	色差焦点偏移
Chk	System Check	系统检查
Cls	Coating List	镀膜列表
Cna	Coating Retardation vs Angle	镀膜的相位差和入射角的关系
Cnw	Coating Retardation vs Wavelength	镀膜的相位差和波长的关系
Con	Conjugate Surface Analysis	共轭表面分析
Cpa	Coating Phase vs Angle	镀膜的相位和入射角的关系
Cpw	Coating Phase vs Wavelength	镀膜的相位和波长的关系

Cra	Coating Refl. vs Angle	镀膜的反射率和入射角的关系
Crw	Coating Refl. vs Wavelength	镀膜的反射率和波长的关系
Cta	Coating Tran. vs Angle	镀膜的透射率和入射角的关系
Ctw	Coating Tran. vs Wavelength	镀膜的透射率和波长的关系
Dim	Diffraction Image Analysis	衍射图像分析
Dip	Dipvergence/Convergence Data	垂直视差/水平视差数据
Dis	Dispersion Diagram	色散图
Enc	Diff Encircled Energy	衍射能量圈图
Fcd	Field Curvature/ Distortion	场曲和畸变
Fcl	Fiber Coupling	光纤耦合分析
Fmm	FFT MTF Map	FFT调制传递函数曲面图
Foa	Foucault Analysis	Foucault分析
Foo	Footprint Analysis	光线痕迹分析
Fps	FFT PSF	FFT点扩散函数
Gbp	Paraxial Gaussian Beam	近轴高斯光束
Gbs	Skew Gaussian Beam	斜高斯光束
Gee	Geom Encircled Energy	几何能量圈图
Gip	Grin Profile	变折射率表面的折射率分布图
Gmm	Geometric MTF map	几何调制传递函数曲面图
Gmp	Glass Map	玻璃图
Gpr	Gradium Profile	Gradium表面的折射率分布图
Grd	Grid Distortion	网格畸变
Gtf	Geometric MTF	几何调制传递函数
Gvf	Geometric MTF vs Field	几何调制传递函数和视场关系
Hcs	Huygens PSF Cross Section	惠更斯点扩散函数横截面
Hmf	Huygens MTF	惠更斯调制传递函数
Hps	Huygens PSF	惠更斯点扩散函数
Hsm	Huygens Surface MTF	惠更斯表面调制传递函数
Htf	Huygens Through Focus MTF	惠更斯离焦调制传递函数
Ibm	Geomatic Bitmap Ima. Analysis	几何点图成像分析
Ilf	Illumination Surface	二维表面照度
Ils	Illumination XY Scan	X、Y方向照度
Ima	Geometric Energy Analysis	几何像能量分析
Imv	IMA/BIM File Viewer	IMA/BIM 文件浏览器
Int	Interferogram	干涉图
Lat	Lateral Color	横向色差
Lin	Geom Line/Edge Spread	几何直线/边缘响应
Lon	Longitudinal Aberration	纵向像差
Lsf	FFT Line/Edge Spread	FFT直线/边缘响应

Mfl	Merit Function List	评价函数列表
Mtf	FFT MTF	FFT调制传递函数
Mth	FFT MTF vs field	FFT调制传递函数和视场关系
Opd	Opd Fan	光程像差特性曲线
Pab	Pupil Aberration Fan	光瞳像差特性曲线
Pal	Power Field Map	放大率与视场关系图
Pcs	FFT PSF cross section	FFT点扩散函数横截面
Pha	Polarization Phase Abberation	偏振相位畸变
Pmp	Polarization Pupil Map	偏振与光瞳位置关系图
Pol	Polarization Ray Trace	偏振光线追迹
Pop	Physical Optics Propagation	物理光学传播
Ppm	Power Pupil Map	放大率与光瞳位置关系图
Pre	Prescription Data	配镜数据
Ptf	Polarization Transmission Fan	偏振传输特性曲线
Ray	Ray Fan	光线像差特性曲线
Rel	Relative Illumination	相对照度
Rfm	RMS Field Map	均方根视场图
Rmf	RMS vs. Focus	作为离焦量函数的均方根
Rms	RMS vs. Field	作为视场函数的均方根
Rmw	RMS vs. Wavelength	作为波长函数的均方根
Rtr	Ray Trace	光线追迹
Sag	Sag Table	矢高表
Sei	Seidel Coefficients	Seidel系数
Smf	Surface MTF	FFT表面调制传递函数
Spt	Spot Diagram	点列图
Srp	Surface Phase	表面相位
Srs	Surface Sag	表面矢高
Sur	Surface Data	表面数据
Sys	System Data	系统数据
Tfg	Geometric through focus MTF	几何离焦调制传递函数
Tfm	FFT Through Focus MTF	FFT离焦调制传递函数
Tls	Tolerance List	公差列表
Tpl	Test Plate List	套样板列表
Tra	Polarization Transmission Data	偏振透过率数据
Trw	Internal Transmission vs Lambda	内部透过率与波长关系
Tsm	Tolerance Data Summary	公差数据总结
Uni	Universal Plot – 1D	1维通用图
Un2	Universal Plot – 2D	2维通用图
Vig	Vignetting Plot	渐晕图

Wfm	Wavefront Map	波前图
Xdi	Extended Diffra Image Analysis	扩展衍射图像分析
Xse	Extended source encircled energ	扩展光源能量圈图
Yni	YNI Contribution	YNI权重
Yyb	Y-Ybar	Y-Y bar图
Zat	Zernike Annular Coefficients	Zernike环状系数
Zfr	Zernike Fringe Coefficients	Zernike条纹系数
Zst	Zernike Standard Coefficients	Zernike标准系数

例 3.9-1 说明如何在程序中得到图 3.9-1(c) 的分析结果：

```

1 ! ex30901
2 ! This program shows how to get analysis information
3 ! Assume the lens system is defined in ex30401
4
5 ! Get a temporary file name
6 A$ = $TEMPFILENAME()
7
8 ! Compute the data and place in the temp file
9 GETTEXTFILE A$, Wfm
10
11 ! Open the temp file and print it out
12 OPEN A$
13 LABEL 1
14 READSTRING B$
15 IF (!EOFF())
16     PRINT B$
17     GOTO 1
18 ENDIF
19 CLOSE

```

在这个程序中，我们通过函数 \$TEMPFILENAME() 自动生成一个临时文件，然后通过关键词 GETTEXTFILE 读取波前形貌图的分析结果并将结果存于前面自动生成的临时文件中。随后，只要打开临时文件逐行读取并打印到屏幕上，即可得到波前形貌图的文字信息，如图 3.9-2 所示：

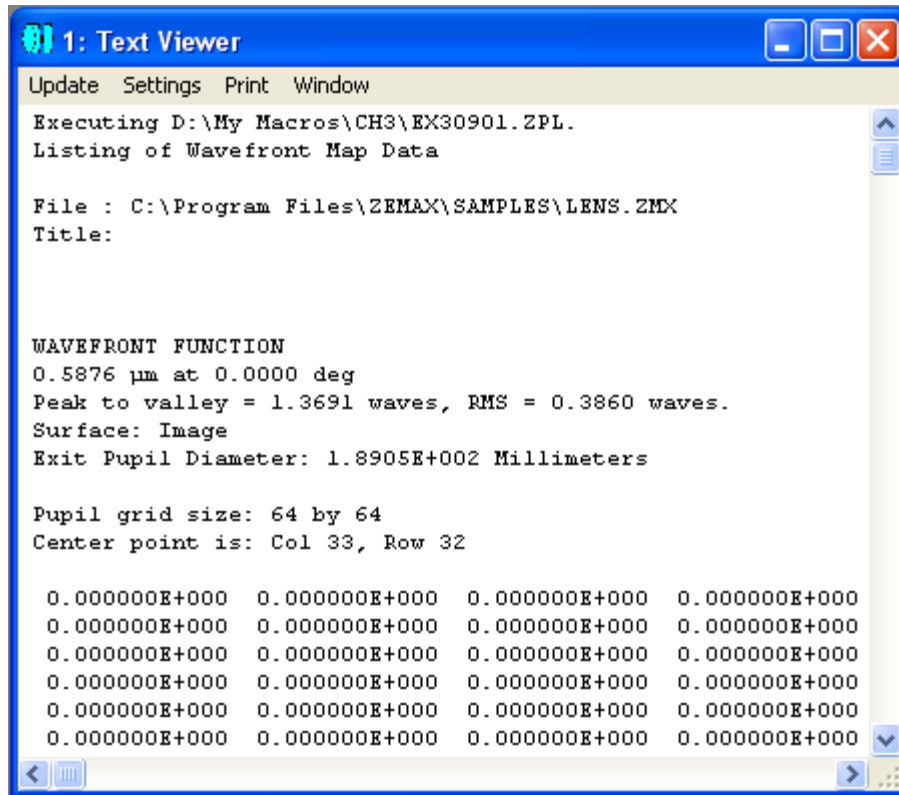


图 3.9-2：通过程序读取的波前形貌图的分析结果

比较一下图 3.9-1 和图 3.9-2，可以发现两者是一致的。

在使用关键词 GETTEXTFILE 进行分析并读取信息时，如果我们想要修改参数设置文件中的设置，可用关键词 MODIFYSETTINGS，其用法为：

MODIFYSETTINGS settingsfilename\$, type, value

其中，settingsfilename\$ 为参数设置文件名，包含路径，type 为类型代码字母助记符，如表 3.9-2 所示，代表所要修改的参数，value 为新的参数值。执行完此命令后，老的参数文件将被更新。

表3.9-2: MODIFYSETTINGS关键词中用到的类型代码及说明

分析类型	代码助记符	说明
2D 外形图	LAY_RAYS	光线数目
探测器浏览器	DVW_SURFACE	表面序号，用1代表非顺序模式。
	DVW_DETECTOR	探测器序号
	DVW_SHOW	显示类型。0为灰度模式，1为反转灰度模式，2为伪彩色模式，3为反转伪彩色模式，4为横截面行，5为横截面列。
	DVW_ROWCOL	横截面图的行或列序号。
	DVW_ZPLANE	体探测器的Z平面序号。
	DVW_SCALE	比例尺模式。0为线性，1为Log -5，2为Log -10，3为Log -15。
	DVW_SMOOTHING	整数平滑值
	DVW_DATA	0为非相干辐射照度，1为向干辐射照度，2为相干相位，3为辐射强度，4为位置空间辐射亮度，5为角度空间辐射亮度。
	DVW_ZRD	光线数据文件名，如无则为空。
	DVW_FILTER	过滤器字符串。
	DVW_MAXPLOT	最大比例尺。
	DVW_MINPLOT	最小比例尺。
衍射图像分析	DIA_FIELD	视场序号。
	DIA_FILESIZE	文件大小。
	DIA_WAVE	波长序号。
扩展衍射图像分析	EXD_DISPLAYSIZE	显示大小。
	EXD_FIELD	视场序号。
	EXD_FILESIZE	文件大小。
	EXD_WAVE	波长序号。
FFT线性/边缘扩散函数	LSF_COHERENT	0为非相干，1为相干。
	LSF_TYPE	0-9分别为X线性、Y线性、X对数、Y对数、X相位、Y相位、X实部、Y实部、X虚部、Y虚部。
	LSF_SAMP	采样值，1为32 x 32，2为64 x 32，等等。
	LSF_SPREAD	0为直线，1为边缘。
	LSF_WAVE (仅用于非相干)	波长序号。0代表白光。

	LSF_FIELD	视场序号。
	LSF_POLARIZATION	0为非偏振，1为偏振。
	LSF_PLOTSIZE	绘图比例尺。
FFT 点扩散函数	PSF_TYPE	0-4分别代表线性、对数、相位、实部及虚部。
	PSF_SAMP	采样值，1为32 x 32，2为64 x 32，等等。
	PSF_WAVE	波长序号。0代表白光。
	PSF_FIELD	视场序号。
	PSF_SURFACE	表面序号。0代表像平面。
	PSF_POLARIZATION	0为非偏振，1为偏振。
	PSF_NORMALIZE	0表示没有归一化，1表示归一化。
FFT 点扩散函数横截面	PSF_IMAGEDELTA	成像点间距，单位为微米。
	PSF_TYPE	0-9分别为X线性、Y线性、X对数、Y对数、X相位、Y相位、X实部、Y实部、X虚部、Y虚部。
	PSF_ROW	行序号(如果为 X 扫描)或列序号(如果为 Y 扫描)。0为中心。
	PSF_SAMP	采样值，1为32 x 32，2为64 x 32，等等。
	PSF_WAVE	波长序号。0代表白光。
	PSF_FIELD	视场序号。
	PSF_POLARIZATION	0为非偏振，1为偏振。
	PSF_NORMALIZE	0表示没有归一化，1表示归一化。
光线痕迹分析	PSF_PLOTSIZE	绘图比例尺。
	FOO_SURFACE	表面序号。
几何点列图成像分析	GBM_FIELDSIZE	视场Y大小。
	GBM_RAYS	每个光源像素所发出的光线数。
	GBM_XPIX	X像素数目。
	GBM_YPIX	Y像素数目。
	GBM_XSIZ	X像素尺寸。
	GBM_YSIZ	Y像素尺寸。
	GBM_INPUT	输入文件名。
	GBM_OUTPUT	输出文件名。
照明的 XY方向扫描	GBM_SURFACE	表面序号。
	ILL_SOURCE	光源尺寸。
	ILL_SMOOTH	平滑值。
	ILL_DETSIZE	探测器尺寸。
	ILL_SURFACE	表面序号。

成像分析	IMA_FIELD	视场大小。
	IMA_IMAGESIZE	像平面大小。
	IMA_IMANAME	图像文件名。
	IMA_OUTNAME	输出文件名。
	IMA_SURFACE	表面序号。
非顺序元件浏览器	SHA_ROT_X	绕x轴旋转角度，单位为度。
	SHA_ROT_Y	绕y轴旋转角度，单位为度。
	SHA_ROT_Z	绕z轴旋转角度，单位为度。
非顺序元件阴影模型	SHA_ROT_X	绕x轴旋转角度，单位为度。
	SHA_ROT_Y	绕y轴旋转角度，单位为度。
	SHA_ROT_Z	绕z轴旋转角度，单位为度。
物理光学传播-通用菜单条	POP_END	中止表面。
	POP_FIELD	视场序号。
	POP_START	起始表面。
	POP_WAVE	波长序号。
物理光学传播-光束定义菜单条	POP_AUTO	用于模拟"auto"按钮，根据采样和其它设置自动选取合适的X及Y光束宽度。
	POP_BEAMTYPE	选择光束类型。0为Gaussian束腰，1为Gaussian角度，2为Gaussian尺寸及角度，3为平顶，4为文件，5为DLL，6为多模。
	POP_PARAMn	设置第n个光束参数。
	POP_PEAKIRRAD	设置用峰值辐射照度进行归一化。
	POP_POWER	设置用总的光束能量进行归一化。
	POP_SAMPX	X方向采样值，1为32，2为64，等等。
	POP_SAMPY	Y方向采样值，1为32，2为64，等等。
	POP_SOURCEFILE	文件名，用于当起始光束通过ZBF文件、DLL或多模文件定义时。
	POP_WIDEX	X方向宽度。
	POP_WIDEY	Y方向宽度。
物理光学传播-光纤数据菜单条	POP_COMPUTE	1为打开光纤耦合积分，0为关闭光纤耦合积分。
	POP_FIBERFILE	文件名，用于当光线模型通过 ZBF 或 DLL 定义时。
	POP_FIBERTYPE	与上面所列的POP_BEAMTYPE相同，多模时除外。
	POP_FPARAMn	设置第n个光纤参数。
	POP_IGNOREPOL	1为忽略偏振，0为考虑偏振。
	POP_POSITION	光纤位置设定，0为主光线，1为表面顶点。

	POP_TILTX	X倾斜。
	POP_TILTY	Y倾斜。
	POP_TILTZ	Z倾斜。
阴影模型	SHA_ROT X	绕x轴旋转角度，单位为度。
	SHA_ROT Y	绕y轴旋转角度，单位为度。
	SHA_ROT Z	绕z轴旋转角度，单位为度。
点列图	SPT_RAYS	光线密度。
波前图	WFM_SAMP	采样值，1为32x32，2为64x64，等等。
	WFM_FIELD	视场序号。
	WFM_WAVE	波长序号。

例 3.9-2 说明如何在程序中通过关键词 **MODIFYSETTINGS** 修改参数设置。这个程序基本上和例 3.9-1 相同，只是在程序最前面加上了修改参数设置的语句 (第 5、6 行)，如下所示：

```

1 ! ex30902
2 ! This program shows how to modify setting file for analysis
3 ! Assume the lens system is defined in ex30401
4
5 settingFile$ = "C:\Program Files\ZEMAX\Samples\LENS.CFG"
6 MODIFYSETTINGS settingFile$, WFM_SAMP, 3
7
8 ! Get a temporary file name
9 A$ = $TEMPFILENAME()
10
11 ! Compute the data and place in the temp file
12 GETTEXTFILE A$, Wfm
13
14 ! Open the temp file and print it out
15 OPEN A$
16 LABEL 1
17 READSTRING B$
18 IF (!EOFF())
19     PRINT B$
20     GOTO 1
21 ENDIF
22 CLOSE

```

注意参数设置文件与镜头文件放在同一个文件夹内。在程序中，我们将波前图的采样值设为 3，即 128 x 128。程序运行结果如图 3.9-3 所示：

```

1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH3\EX30902.ZPL.
Listing of Wavefront Map Data

File : C:\Program Files\ZEMAX\SAMPLES\LENS.ZMX
Title:

WAVEFRONT FUNCTION
0.5876  $\mu\text{m}$  at 0.0000 deg
Peak to valley = 1.3691 waves, RMS = 0.3852 waves.
Surface: Image
Exit Pupil Diameter: 1.8905E+002 Millimeters

Pupil grid size: 128 by 128
Center point is: Col 65, Row 64

0.000000E+000 0.000000E+000 0.000000E+000 0.000000E+000
0.000000E+000 0.000000E+000 0.000000E+000 0.000000E+000
0.000000E+000 0.000000E+000 0.000000E+000 0.000000E+000
0.000000E+000 0.000000E+000 0.000000E+000 0.000000E+000
0.000000E+000 0.000000E+000 0.000000E+000 0.000000E+000
0.000000E+000 0.000000E+000 0.000000E+000 0.000000E+000

```

图 3.9-3: 修改参数后通过程序读取的波前形貌图的分析结果

比较一下此结果和图 3.9-2 所示的结果，可以发现采样值已经改变为“Pupil grid size: 128 by 128”。

除了通过上面所介绍的方式读取各种系统分析结果外，ZPL还提供了一些函数和关键词如IMAE()、OPDC()、OPTH()、GETLSF、GETMTF、GETPSF、GETZERNIKE、POP、XDIFFIA、GETT()等，用于更直接地读取一些常用的分析信息。下面我们将对这些函数和关键词作一些简要介绍。

函数IMAE(seed)用于读取几何成像分析的效率。如果seed的值为0，则每次调用此函数均使用相同的随机数，如果seed的值不为0，则每次调用此函数使用不同的随机数。

函数OPDC()用于计算相对于主光线的光程差，仅在执行完RAYTRACE命令后才有效。如果不能追迹主光线，则此函数无效。注意不管镜头单位为什么，函数返回值的单位均为mm。例3.9-4给出一个在程序中使用此函数的例子：


```

1 ! ex30904
2 ! This program shows how to use function OPDC()
3 ! Assume the lens system is defined in ex30401
4
5 ! first define the marginal ray
6 hx = 0
7 hy = 0
8 px = 0
9 py = 1
10 wavelength = 2
11
12 RAYTRACE hx, hy, px, py, wavelength # trace the marginal ray
13
14 ! then calculate the optical path difference
15 ! between the marginal ray and the chief ray
16
17 opticalPathDiff = OPDC()
18
19 PRINT
20 PRINT "The optical path difference is ", opticalPathDiff, " mm"

```

我们假设光学系统为程序ex30401.ZPL中所构造的双透镜，程序中首先定义了一条任意光线(此处我们使用边缘光线)，然后用RAYTRACE对其进行光线追迹，最后调用函数OPDC()，比较了所定义光线和主光线之间的光程差。程序运行结果如下：

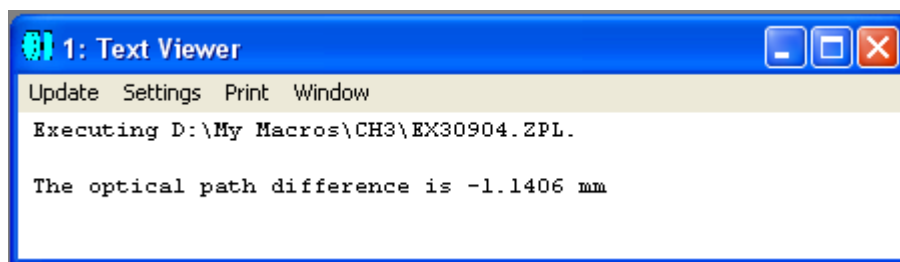


图 3.9-5: 程序 ex30904.ZPL 的运行结果

函数OPTH(n)用于计算至表面n的光程，在计算中考虑到衍射表面如光栅、全息及二元光学元件等引起的相位变化。仅在执行完RAYTRACE命令后才有效。如果不能追迹主光线，则此函数无效。注意与函数OPDC()不同，函数OPTH(n)返回值的单位为当前镜头单位。例3.9-5给出一个在程序中使用此函数的例子：

```

1 ! ex30905
2 ! This program shows how to use function OPTH()
3 ! Assume the lens system is defined in ex30401
4
5 ! first define the ray (we use marginal ray here)
6 hx = 0
7 hy = 0
8 px = 0
9 py = 1
10 wavelength = 2
11
12 RAYTRACE hx, hy, px, py, wavelength # trace the ray
13
14 ! find out the lens unit
15 u = UNIT()
16 IF u == 0 THEN lensUnit$ = "mm"
17 IF u == 1 THEN lensUnit$ = "cm"
18 IF u == 2 THEN lensUnit$ = "inch"
19 IF u == 3 THEN lensUnit$ = "m"
20
21 ! then calculate the optical path length (OPL) to a given surface
22
23 PRINT
24 FOR sn, 1, 5, 1
25     opticalPath = OPTH(sn)
26     PRINT "The OPL to surface ", sn, " is ", opticalPath, " ", lensUnit$
27 NEXT

```

我们假设光学系统为程序ex3401.ZPL中所构造的双透镜，程序中首先定义了一条任意光线(此处我们使用边缘光线)，然后用RAYTRACE对其进行光线追迹，最后调用函数OPTH()，计算了所定义光线至各个表面的光程。程序中我们还用到函数UNIT()读取当前镜头单位。程序运行结果如下：

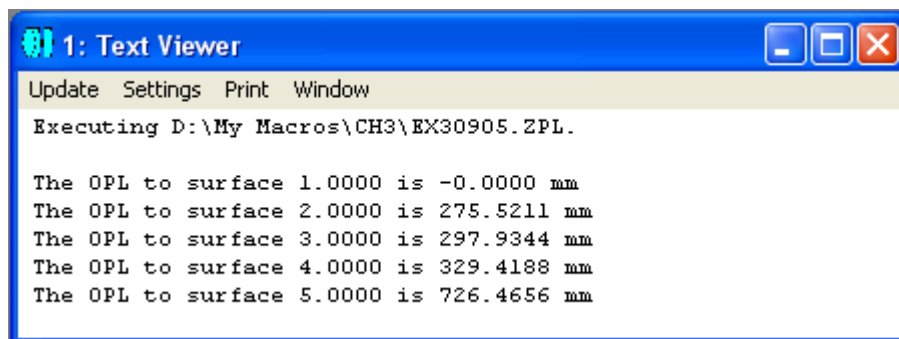


图 3.9-6: 程序 ex30905.ZPL 的运行结果

关键词GETLSF用于计算几何直线及边缘响应函数，其用法为：

GETLSF wave, field, sampling, vector, maxradius, use_polarization

其中，wave为波长序号，如为0代表白光；field为视场序号；sampling为采样值，如为1表示32 x 32，2表示64 x 64，3表示128 x 128，等等，最大采样可至2048 x 2048；vector的值为1至4，表明计算值存于哪个一维数组内；maxradius为直线或边缘响应函数的最大半径坐标，为0表示缺省宽度；use_polarization表明是否采用偏振计算，1为真，0为伪。如果上述参数不在有效取值范围之内，则使用最接近的有效值。返回的计算结果存于vector所指定的数组内，数组元素0~3分别储存数据点数N、起始点的x坐标、步长及偏移，其中偏移指数组中储存直线或边缘响应数据的初始位置，其第一组N个数据为子午直线扩散响应值，随后N个数据为弧矢直线扩散响应值，接下来两组N个数据分别为子午和弧矢边缘扩散响应值。如果当前数组的尺寸不够大，ZEMAX会自动增加数组的尺寸。

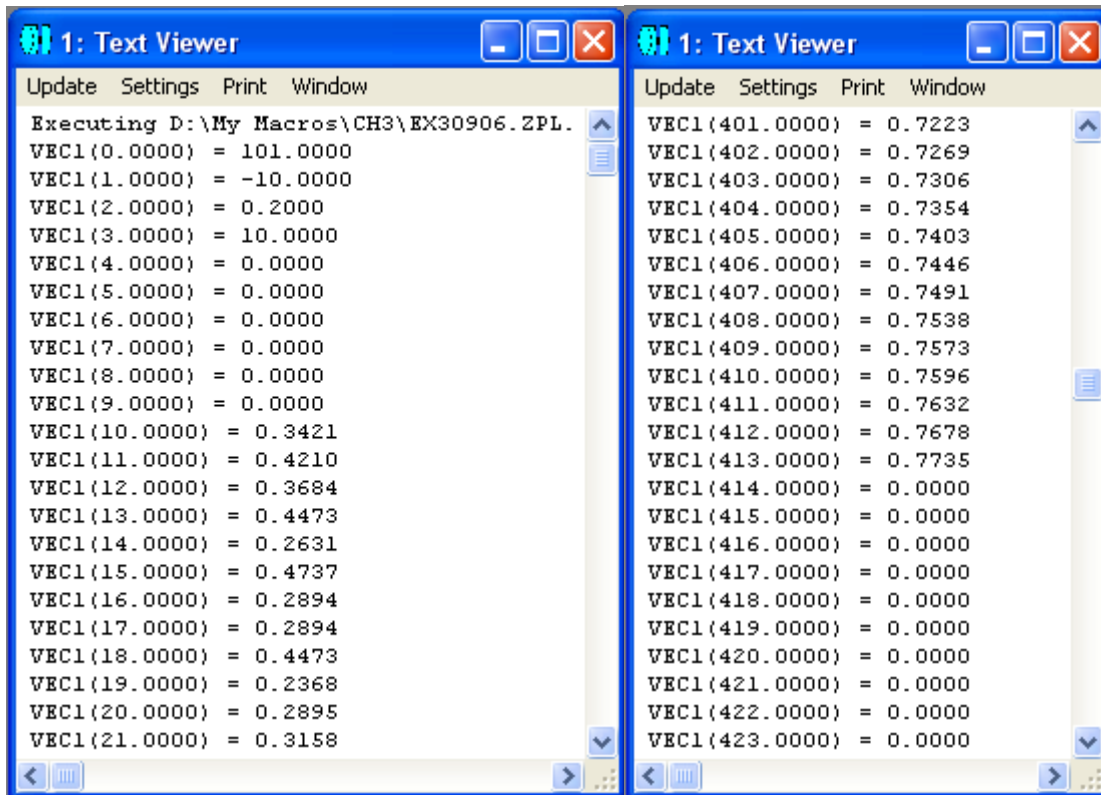
例3.9-6给出一个在程序中使用此函数的例子：

```

1 ! ex30906
2 ! This program shows how to use key word GETLSF
3 ! Assume the lens system is defined in ex30401
4
5 wave = 2
6 field = 1
7 sampling = 2
8 vector = 1
9 maxradius = 10
10 use_polarization = 0
11
12 GETLSF wave, field, sampling, vector, maxradius, use_polarization
13
14 FOR n, 0, 1000, 1
15   PRINT "VEC1(", n, ") = ", VEC1(n)
16 NEXT

```

假设光学系统为程序ex30401.ZPL中所构造的双透镜，我们通过关键词GETLSF计算了系统的几何直线及边缘响应函数，并将存于数组VEC1中的结果显示出来。程序运行结果如下所示：



```

1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH3\EX30906.ZPL.
VEC1(0.0000) = 101.0000
VEC1(1.0000) = -10.0000
VEC1(2.0000) = 0.2000
VEC1(3.0000) = 10.0000
VEC1(4.0000) = 0.0000
VEC1(5.0000) = 0.0000
VEC1(6.0000) = 0.0000
VEC1(7.0000) = 0.0000
VEC1(8.0000) = 0.0000
VEC1(9.0000) = 0.0000
VEC1(10.0000) = 0.3421
VEC1(11.0000) = 0.4210
VEC1(12.0000) = 0.3684
VEC1(13.0000) = 0.4473
VEC1(14.0000) = 0.2631
VEC1(15.0000) = 0.4737
VEC1(16.0000) = 0.2894
VEC1(17.0000) = 0.2894
VEC1(18.0000) = 0.4473
VEC1(19.0000) = 0.2368
VEC1(20.0000) = 0.2895
VEC1(21.0000) = 0.3158

1: Text Viewer
Update Settings Print Window
VEC1(401.0000) = 0.7223
VEC1(402.0000) = 0.7269
VEC1(403.0000) = 0.7306
VEC1(404.0000) = 0.7354
VEC1(405.0000) = 0.7403
VEC1(406.0000) = 0.7446
VEC1(407.0000) = 0.7491
VEC1(408.0000) = 0.7538
VEC1(409.0000) = 0.7573
VEC1(410.0000) = 0.7596
VEC1(411.0000) = 0.7632
VEC1(412.0000) = 0.7678
VEC1(413.0000) = 0.7735
VEC1(414.0000) = 0.0000
VEC1(415.0000) = 0.0000
VEC1(416.0000) = 0.0000
VEC1(417.0000) = 0.0000
VEC1(418.0000) = 0.0000
VEC1(419.0000) = 0.0000
VEC1(420.0000) = 0.0000
VEC1(421.0000) = 0.0000
VEC1(422.0000) = 0.0000
VEC1(423.0000) = 0.0000

```

图 3.9-7: 程序 ex30906.ZPL 的运行结果

从结果中我们可以看到，数组VEC1的初始下标从0开始，与用户自定义的数组下标从1开始不同，这个差异我们已经在上一章中介绍数组时提到过。VEC1(0)~ VEC1(3)分别存放了数据点数N = 101、起始点的坐标 x = -10、步长 = 0.2 及偏移 = 10，真正的数据由偏移值决定，从VEC1(10)开始，第一组N = 101个数据为子午直线扩散响应值，随后N个数据为弧矢直线扩散响应值，接下来两组N个数据分别为子午和弧矢边缘扩散响应值，所有的数据至VEC1(413)结束，如图所示。

关键词GETMTF用于计算当前镜头文件的子午及弧矢MTF、实部、虚部、相位或方波响应，并将计算结果返回到缺省的4个一维数组之一，其用法为：

GETMTF freq, wave, field, sampling, vector, type

其中，freq为MTF空间频率，如果此频率小于0或大于截止频率，则GETMTF返回值为0；wave为波长序号；field为视场序号；sampling为采样值，如为1表示32 x 32，2表示64 x 64，3表示128 x 128，等等，最大采样可至2048 x 2048；vector的值为1至4，表明计算值存于哪个一位数组内；type为类型代码，1表示MTF，2表示实部，3表示虚部，4表示以弧度为单位的相位，5表示方波响应MTF。如果上述参数不在有效取值范围之内，则使用最接近的有效值。返回的计算结果存于vector所指定的数组内，数组元素0为子午响应，1为弧矢响应。

例3.9-7给出一个在程序中使用此函数的例子：

```

1 ! ex30907
2 ! This program shows how to use key word GETMTF
3 ! Assume the lens system is defined in ex30401
4
5 freq = 30
6 wave = 2
7 field = 1
8 sampling = 2
9 vector = 1
10 type = 1
11
12 GETMTF freq, wave, field, sampling, vector, type
13
14 PRINT
15 PRINT "Tangential response: ", vec1(0)
16 PRINT "Sagittal response: ", vec1(1)

```

假设光学系统为程序ex30401.ZPL中所构造的双透镜，我们通过关键词GETMTF计算了系统的子午及弧矢MTF响应，并将存于数组VEC1中的结果显示出来。程序运行结果如下所示：

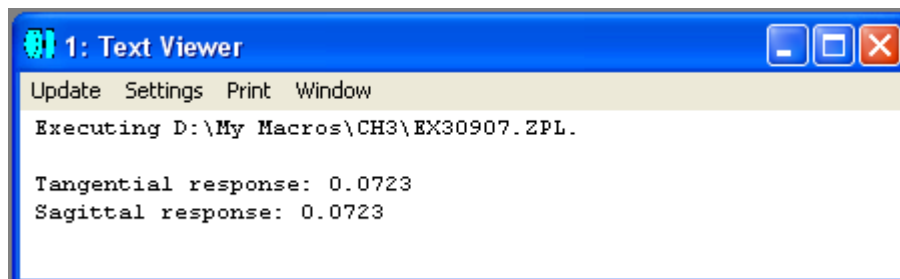


图 3.9-8: 程序 ex30907.ZPL 的运行结果

关键词GETPSF用于计算当前镜头文件的衍射点扩散函数，并将计算结果返回到缺省的4个一维数组之一，其用法为：

GETPSF wave, field, sampling, vector, unnormalized, phaseflag, imagedelta

其中，wave为波长序号，如为0代表白光；field为视场序号；sampling为采样值，如为1表示32 x 32，2表示64 x 64，3表示128 x 128，等等，最大采样可至2048 x 2048；vector的值为1至4，表明计算值存于哪个一位数组内；unnormalized如为0表示返回数据归一化，如为1表示返回数据未归一化；phaseflag为0表示返回数据为强度值，为1表示返回数据为以度为单位的相位值；imagedelta为以微米为单位的点扩散函数的间距，如为0代表缺省间距。如要计算相位数据，则波长须为单色光。如

果上述参数不在有效取值范围之内，则使用最接近的有效值。返回的计算结果存于vector所指定的数组内，数组元素0为PSF数据点总数，通常这个数为 $4*n*n$ ，其中n为采样尺寸(如32、64等)。数组元素0也有可能为出错信息：0表示计算中止，-1表示数组尺寸不够大，可用SETVECSIZE增加数组尺寸，-2表示系统内存不够，-3为通用出错信息。数组元素1至 $4*n*n$ 为点扩散函数的强度数据，数组元素 $4*n*n+1$ 为以微米为单位的点扩散函数的间距。

例3.9-8给出一个在程序中使用此函数的例子：

```

1 ! ex30908
2 ! This program shows how to use key word GETPSF
3 ! Assume the lens system is defined in ex30401
4
5 wave = 0 # use polychromatic light
6 field = 1
7 sampling = 2
8 vector = 3 # result saved in VEC3
9 unnormalized = 1
10 phaseflag = 0 # no phase data
11 imagedelta = 0 # default image delta
12
13 vecSize = 1000 # this is the default vector size
14
15 PRINT
16 FORMAT 5.0
17 PRINT "The original vector size is ", vecSize
18
19 label RESIZE
20 vecSize = vecSize+100
21 SETVECSIZE vecSize
22
23 GETPSF wave, field, sampling, vector, unnormalized, phaseflag, imagedelta
24 pointNum = VEC3(0)
25 IF pointNum == -1 THEN GOTO RESIZE
26
27 PRINT "The final vector size is ", vecSize
28 PRINT "There are ", pointNum, " data points."
29 FORMAT 3.4
30 PRINT "They spaced ", VEC3(pointNum+1), " micrometers apart."

```

假设光学系统为程序ex30401.ZPL中所构造的双透镜，我们通过关键词GETPSF 计算了系统的衍射点扩散函数，并将存于数组VEC3中的结果显示出来。注意数组VEC3的初始缺省长度为1000，不够存储计算结果，因此VEC3(0)的返回结果为-1。我们在程序第25行加入一个判断语句，如果检测到-1的结果，则转回到第19行增加数组的长度，重新计算，直到获得大于0的正确结果为止。程序运行结果如下所示：

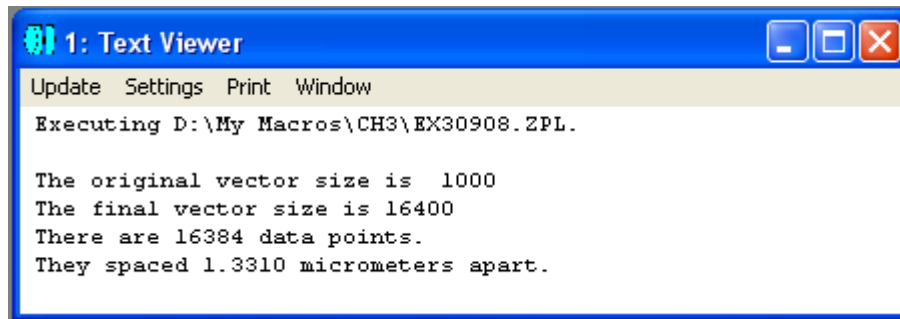


图 3.9-8: 程序 ex30908.ZPL 的运行结果

关键词GETZERNIKE用于计算当前镜头文件的Zernike条纹、标准及环形系数，并将计算结果返回到缺省的4个一维数组之一，其用法为：

GETZERNIKE maxorder, wave, field, sampling, vector, zerntype, epsilon, reference

其中，maxorder为Zernike系数的最高阶数，对条纹系数可为1~37，对标准或环形系数可为1~231；wave为波长序号；field为视场序号；sampling为采样值，如为1表示32 x 32，2表示64 x 64，3表示128 x 128，等等，最大采样可至2048 x 2048；vector的值为1至4，表明计算值存于哪个一位数组内；zerntype为系数类型，0表示条纹，1表示标准，2表示环形；对环形系数而言，epsilon为环形比率，对其它系数忽略此参数；reference为0表示相对于主光线计算光程差，为1表示相对于表面定点计算光程差。如果上述参数不在有效取值范围之内，则使用最接近的有效值。计算结果以下面的形式存于指定的一维数组中：数组元素1用于储存以波长为单位的波峰至波谷的值，数组元素2用于储存以波长为单位的相对于0光程差线的均方根值，数组元素3用于储存以波长为单位的相对于主光线的均方根值，数组元素4用于储存以波长为单位的相对于图像中心的均方根值，数组元素5用于储存以波长为单位的方差，数组元素6用于储存Strehl比率，数组元素7用于储存以波长为单位的均方根拟合误差，随后的数组元素用于存储实际的Zernike系数。

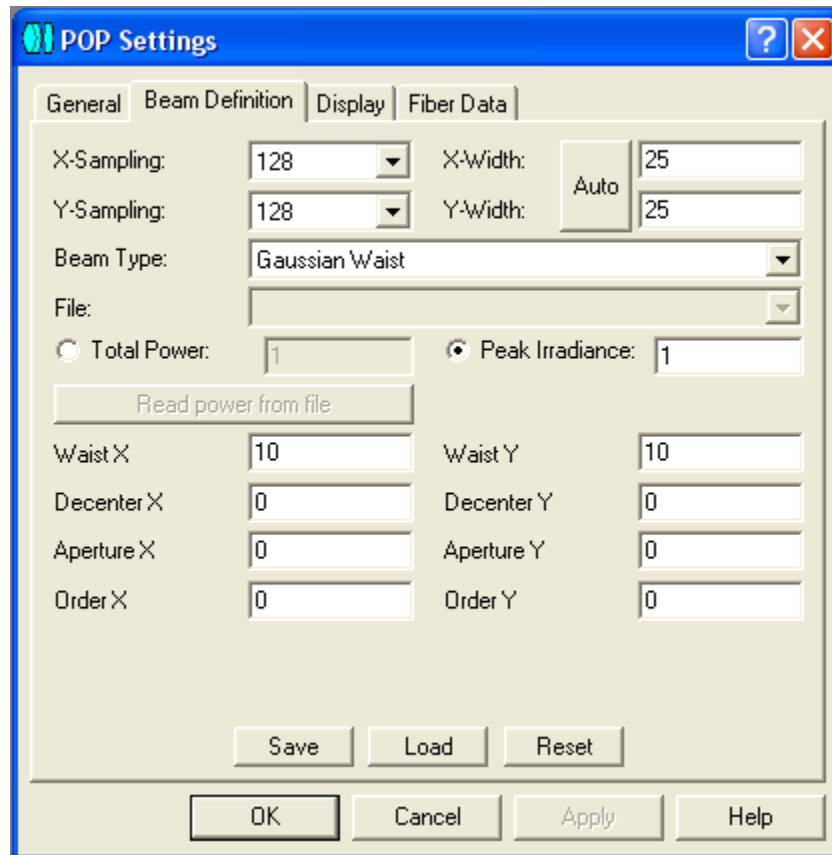
关键词 POP 用于计算光束在光学系统中的物理光学传播，并将各个表面的结果存于ZBF文件内。其用法为：

POP outfilename\$, lastsurface

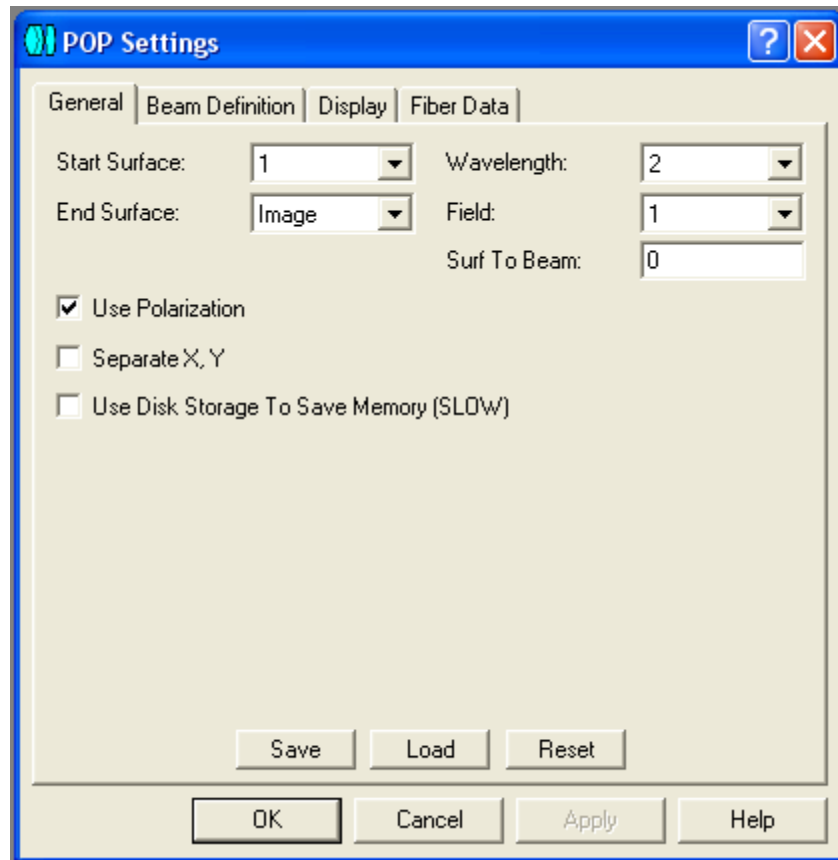
其中，outfilename\$为输出ZBF文件名，lastsurface为最后一个表面的序号。输出文件将被存于"...\\POP\\Beamfiles " 文件夹内，因此在文件名outfilename\$中不需要指明路径。进行计算所使用的参数为事先保存好的当前镜头文件的设置参数。如果要

修改参数设置，可以通过打开一个物理光学传播窗口来进行。在本章第十四节中，我们还要进一步介绍与ZBF文件相关的指令。

下面我们通过一个例子来说明关键词 POP 的用法。首先，我们假设光学系统为程序 ex30401.ZPL 中所构造的双透镜。另外，我们假定物理光学传播的参数设置如图 3.9-10 所示：



(a)



(b)

图3.9-10(a)(b): 程序 ex30910.ZPL 中所用到的物理光学传播的参数设置

```

1 ! ex30910
2 ! This program shows how to use key word POP
3 ! Assume the lens system is defined in ex30401
4
5 outfilename1$ = "ex30910a.ZBF"
6 outfilename2$ = "ex30910b.ZBF"
7
8 lastsurface = 4
9 POP outfilename1$, lastsurface
10
11 lastsurface = 5
12 POP outfilename2$, lastsurface

```

在程序中，我们定义了两个不同的文件名，并将不同表面 4 及 5 上的物理光学传播计算结果分别存于这两个文件中。注意程序中没有设置路径名，因为所有的 ZBF 文件都被存于 "... \POP\Beamfiles " 文件夹内。我们在本章第十四节还会用到这两个文件。

关键词XDIFFIA用于计算扩展衍射成像分析并将结果存于ZBF文件内。其用法为：

XDIFFIA outfile\$, infile\$

其中，outfile\$为输出ZBF文件名，infile\$为输入IMA 或 BIM文件名。输出文件的扩展名如果省略，则自动设为ZBF，但输入文件的扩展名要给出。输出文件将被存于"...\\POP\\Beamfiles " 文件夹内，输入文件则应存于"...\\IMAFiles " 文件夹内，文件名中不需要指明路径。进行计算所使用的参数为事先保存好的当前镜头文件的设置参数。如果要修改参数设置，可以通过打开一个扩展衍射成像分析窗口来进行。

函数GETT(window_num, line, column)用于从任意打开的第window_num个文本窗口中读取指定行line及指定列column上的数值，注意每一行中的不同列之间由空格界定。

第十节 非顺序元件

在第一章中我们提到过，在很多情况下，对光学系统的分析只有通过非顺序元件模型来进行，例如照明系统或杂散光分析等。为此，ZEMAX开发了很强大的非顺序元件分析功能。相应地，ZPL中也提供了很多相关函数及关键词。

我们知道，非顺序元件编辑器(Non-Sequential Component Editor)是定义和修改非顺序光学系统的一个重要场所。在这里，我们可以添加和删除各种非顺序元件，并对其参数进行修改。

如果要在非顺序元件编辑器中添加一个元件，可用关键词INSERTOBJECT，其用法为：

INSERTOBJECT surf, object

其中，surf 为非顺序元件所在的表面序号，如果为完全非顺序模式，则其值为 1；object 为非顺序元件所要插入的位置序号，必须介于 1 和当前元件总数 + 1 之间，可包含此两值。如果在插入的位置后面有其它元件，则新元件插入后其后面的各元件重新编号。

如果要从非顺序元件编辑器中删除一个元件，可用关键词DELETEOBJECT，其用法为：

DELETEOBJECT surf, object

其中，surf 为非顺序元件所在的表面序号，如果为完全非顺序模式，则其值为1；object 为要删除的非顺序元件所在的位置序号，如果其后有其它元件，元件删除后其后面的各元件重新编号。

通常，在新元件插入后，我们需要定义其空间位置和其它参数，这需要用关键词SETNSCPOSITION、SETNSCPROPERTY 及 SETNSCPARAMETER。

SETNSCPOSITION 用于定义非顺序元件的空间位置和倾斜，其用法为：

SETNSCPOSITION surface, object, code, value

其中，surf为非顺序元件所在的表面序号，如果为完全非顺序模式，则其值为1；object为非顺序元件所在的位置序号；code为位置代码，用1~6分别代表x、y、z、x倾斜、y倾斜及z倾斜；value为相应的值。

SETNSCPROPERTY用于定义非顺序元件编辑器及属性对话框中的相应属性，其用法为：

SETNSCPROPERTY surface, object, code, face, value

其中，surf为非顺序元件所在的表面序号，如果为完全非顺序模式，则其值为1；object为非顺序元件所在的位置序号；code为如表3.10-1所示的代码，用于指明所需要设定的元件属性；face为元件上的表面组序号，如果不需要指明表面则为0；value为相应的属性值。

表3.10-1 关键词SETNSCPROPERTY的相应代码

代码	属性
<i>以下代码用于设置非顺序元件编辑器的值</i>	
1	设置元件注释
2	设置参照元件序号
3	设置被包含(inside of)元件序号
4	设置元件材料
<i>以下代码用于设置非顺序元件属性对话框中类型标签上的值</i>	
0	设置元件类型。其值应为元件类型的ASCII名称，如NSC_SLEN代表标准镜头等。非顺序元件编辑器中各元件的ASCII名称可从配镜单报表(Prescription Report)中读取。所有的非顺序元件的ASCII名称均以NSC_开头。
13	设置是否使用用户自定义光阑，1为选用，0为取消。
14	设置用户自定义光阑文件名。
15	设置“使用全域XYZ旋转顺序”选择框，1为选中，0为取消。
16	设置“光线忽略此元件”选择框，1为选中，0为取消。
17	设置“元件为探测器”选择框，1为选中，0为取消。
18	设置“考虑元件”列表。其值应为与元件序号相应的字符串，各序号之间用空格隔开，例如：“1 5 12”。
19	设置“忽略元件”列表。其值应为与元件序号相应的字符串，各序号之间用空格隔开，例如：“2 6 11”。
20	设置“使用像素插值”选择框，1为选中，0为取消。
<i>以下代码用于设置非顺序元件属性对话框中镀膜/散射标签上的值</i>	
5	设置指定表面组的镀膜文件名称。
6	设置指定表面组的散射轮廓文件名称
7	设置指定表面组的散射模式，0为无，1为Lambertian，2为Gaussian，3为ABg，4为用户自定义。
8	设置指定表面组的散射比例因子。
9	设置指定表面组的散射光线数目。

10	设置指定表面组的Gaussian sigma因子。
11	设置指定表面组的ABg反射数据名。
12	设置指定表面组的ABg透射数据名。
<i>代码21~26用于设置用户自定义散射DLL的参数</i>	
<i>以下代码用于设置非顺序元件属性对话框中体散射标签上的值</i>	
81	设置体散射模式值。0为无，1为角散射，2为DLL定义散射。
82	设置用于体散射的平均自由程。
83	设置用于体散射的角度。
84	设置用于体散射的DLL文件名。
85	设置传递给DLL的参数值。表面序号face的值用于指定哪一个参数被赋值，1代表第一个参数，2代表第二个参数，等等。
<i>以下代码用于设置非顺序元件属性对话框中衍射标签上的值</i>	
91	设置“分裂”值。0为“不按衍射级数分裂”，1为“按下表分裂”，3为“按DLL函数分裂”。
92	设置用于衍射分裂的DLL文件名。
93	设置初始衍射级数。
94	设置中止衍射级数。
95	设置衍射标签上的参数值。这些参数为传递给衍射DLL的值及用于“按下表分裂”的各衍射级数效率。表面序号face的值用于指定哪一个参数被赋值，1代表第一个参数，2代表第二个参数，等等。
<i>以下代码用于设置非顺序元件属性对话框中光源标签上的值</i>	
101	设置光源元件的随机偏振选项，1为选中，0为不选。
102	设置光源元件的光线反向选项，1为选中，0为不选。
103	设置光源元件的Jones矩阵X值。
104	设置光源元件的Jones矩阵Y值。
105	设置光源元件的相位X值。
106	设置光源元件的相位Y值。
107	设置光源元件的以度为单位的初始相位。
108	设置光源元件的相干长度值。
109	设置光源元件的预传播值。
110	设置光源元件的采样方法，0为随机采样，1为Sobol采样。
111	设置光源元件的体散射模式，0为多次，1为1次，2为无。
<i>以下代码用于设置非顺序元件属性对话框中渐变折射率GRIN标签上的值</i>	
121	设置“用DLL定义Grin介质”选项，1为选中，0为不选。
122	设置最大步长值。
123	设置DLL文件名。
124	设置Grin DLL参数。这些参数为传递给DLL的值。表面序号face的值用于指定哪一个参数被赋值，1代表第一个参数，2代表第二个参数，等等。

以下代码用于设置非顺序元件属性对话框中绘制标签上的值	
141	设置不绘制元件选项，1为选中，0为不选。
142	设置元件不透明度，0为100%，1为90%，2为80%，等等。
以下代码用于设置非顺序元件属性对话框中散射至标签上的值	
151	设置散射至方法，0为散射至列表，1为重点采样。
152	设置重点采样目标数据。参数应为一串数字字符，列出光线序号、元件序号、大小及限定值，各数字之间以空格隔开，如“3 6 3.5 0.6”代表光线3、元件6、大小为3.5、限定值为0.6的设置。
153	设置“散射至列表”的值。参数应为一串数字字符，列出各元件序号，各数字之间以空格隔开，如“5 8 12”。
代码200仅用于函数NPRO	
200	在函数NPRO中使用此代码，用于返回元件的折射率，用法为：NPRO(surface, object, 200, wavenumber)，其中surface为元件所在的表面序号(如果为完全非顺序模式，则其值为1)，object为元件序号，wavenumber为波长序号。

SETNSCPARAMETER用于设置非顺序元件编辑器中的不同参数值，其用法为：

SETNSCPARAMETER surface, object, parameter, value

其中，surface为非顺序元件所在的表面序号，如果为完全非顺序模式，则其值为1；object为非顺序元件所在的位置序号；parameter为所需设置的参数序号；value为新设定的参数值。

ZPL定义了一系列的函数，用于读取非顺序元件的各种参数。

如果要知道某一表面中非顺序元件的数目，可用函数NOBJ(surface)，其中surface为表面序号，函数返回值为所指定的非顺序表面中的元件数目。

如果想要读取某一表面中非顺序元件的位置和倾斜，可用函数NPOS(surf, object, code)，其中surface为表面序号，object为元件序号，code为1~6分别代表x、y、z、x倾斜、y倾斜及z倾斜，函数返回值为code相应的数值。

函数NPRO(surf, object, code, face)用于读取非顺序元件编辑器中相应元件的属性，其中surf为表面序号，object为元件序号，code为表3.10-1中的各个代码，face为元件上的表面组序号，函数返回值为代码code的相应属性值，可为数值量或字符串。如果返回属性值为字符串，则在调用此函数后，可用函数\$buffer()读取相应字符串。

函数NPAR(surf, object, param)用于读取非顺序元件编辑器中参数列的值，其中surf为表面序号，object为元件序号，param为参数序号。

下面我们将通过一些例子来说明以上所介绍的一些关键词及函数的用法。

例3.10-1说明如何在非顺序元件编辑器中添加、删除元件及设置元件参数。

```

1 ! ex31001
2 ! This program shows how to Set NSC parameters
3 ! Assume the mode is total Non-Sequential Mode
4
5 ! this part is used to clean the NSC editor
6 totalObjNum = NOBJ(1)
7 for i, totalObjNum, 1, -1
8     DELETEOBJECT 1, i      # delete all the objects
9 next                      # there is still a null object in the editor at last
10                          # because the editor cannot be empty
11
12 ! add two more objects in the editor, so the total is 3
13 INSERTOBJECT 1, 1
14 INSERTOBJECT 1, 1
15
16 ! define the type of the first object
17 surface = 1
18 object = 1
19 code = 0
20 face = 0
21 value$ = "NSC_SLEN"
22 SETNSCPROPERTY surface, object, code, face, value$
23
24 ! define the position of the first object
25 SETNSCPOSITION 1, 1, 1, 0      # x
26 SETNSCPOSITION 1, 1, 2, 0      # y
27 SETNSCPOSITION 1, 1, 3, 0      # z
28 SETNSCPOSITION 1, 1, 4, 0      # x tilt
29 SETNSCPOSITION 1, 1, 5, 0      # y tilt
30 SETNSCPOSITION 1, 1, 6, 0      # z tilt
31
32 ! define the comment of the first object
33 SETNSCPROPERTY 1, 1, 1, 0, "first lens"
34
35 ! define the material of the first object
36 SETNSCPROPERTY 1, 1, 4, 0, "BK7"
37
38 ! define curvature, clear aperture, edge and thickness of the first object
39 SETNSCPARAMETER 1, 1, 1, 10      # radius 1
40 SETNSCPARAMETER 1, 1, 3, 1.8      # clear 1
41 SETNSCPARAMETER 1, 1, 4, 2      # edge 1
42 SETNSCPARAMETER 1, 1, 5, 1      # thickness
43 SETNSCPARAMETER 1, 1, 6, -10      # radius 2
44 SETNSCPARAMETER 1, 1, 8, 1.8      # clear 2
45 SETNSCPARAMETER 1, 1, 9, 2      # edge 2
46

```

```

46
47 ! define the type of the second object
48 SETNSCPROPERTY 1, 2, 0, 0, "NSC_SLEN"
49
50 ! define the comment of the second object
51 SETNSCPROPERTY 1, 2, 1, 0, "second lens"
52
53 ! define the position of the second object
54 SETNSCPOSITION 1, 2, 3, 10      # z position
55 SETNSCPOSITION 1, 2, 5, 90      # rotate 90 degrees around y
56
57 ! define the material of the second object
58 SETNSCPROPERTY 1, 2, 4, 0, "BK7"
59
60 ! define curvature, clear aperture, edge and thickness of the second object
61 SETNSCPARAMETER 1, 2, 1, 10      # radius 1
62 SETNSCPARAMETER 1, 2, 3, 1.8     # clear 1
63 SETNSCPARAMETER 1, 2, 4, 2       # edge 1
64 SETNSCPARAMETER 1, 2, 5, 1       # thickness
65 SETNSCPARAMETER 1, 2, 6, -10     # radius 2
66 SETNSCPARAMETER 1, 2, 8, 1.8     # clear 2
67 SETNSCPARAMETER 1, 2, 9, 2       # edge 2
68

```

在此例中，我们通过删除元件，首先清空了非顺序元件编辑器(6~9行)。注意在清空后，编辑器中仍然会有一个缺省元件为空元件(NULL object)。随后，我们在编辑器中插入两个空元件(13~14行)，这样，编辑器中总共有3个元件。程序第17~22行定义了第一个元件类型为标准透镜，第17~22行定义了其位置及倾斜，第33行给出其注释行内容，第36行设置了材料类型，然后，第39~45行设置了透镜的相应参数，分别为第1表面曲率半径、第1表面有效半直径、第1表面边缘半直径、厚度、第2表面曲率半径、第2表面有效半直径及第2表面边缘半直径。类似地，程序第47~67行定义了第2个元件。事实上，在定义完元件类型后，ZEMAX会自动生成一些缺省参数，因此只需要对那些与缺省参数不同的地方进行设置即可。例如对第二个元件，我们只设置了z坐标及y倾斜，而忽略了x坐标、y坐标、x倾斜及z倾斜。在程序 ex31001.ZPL 运行后，我们可以看到，非顺序元件编辑器中的内容已经发生改变，如图 3.10-1 所示：

Non-Sequential Component Editor									
Edit Solves Errors Detectors Database Tools View Help									
Object	Type	Comment	Ref Obj..	Inside Of	X Position	Y Position	Z Position	Tilt About X	Tilt About Y
1	Standard Lens	first lens	0	0	0.0000	0.00000	0.0000	0.000000	0.000000
2	Standard Lens	second lens	0	0	0.0000	0.00000	10.000	0.000000	90.00000
3	Null Object		0	0	0.0000	0.00000	0.0000	0.000000	0.000000

图 3.10-1: 程序 ex31001.ZPL 的运行后非顺序元件编辑器内容

如果我们打开 3D Layout 窗口，可以看到所定义的两个透镜，二者的倾斜不同，如图 3.10-2 所示。

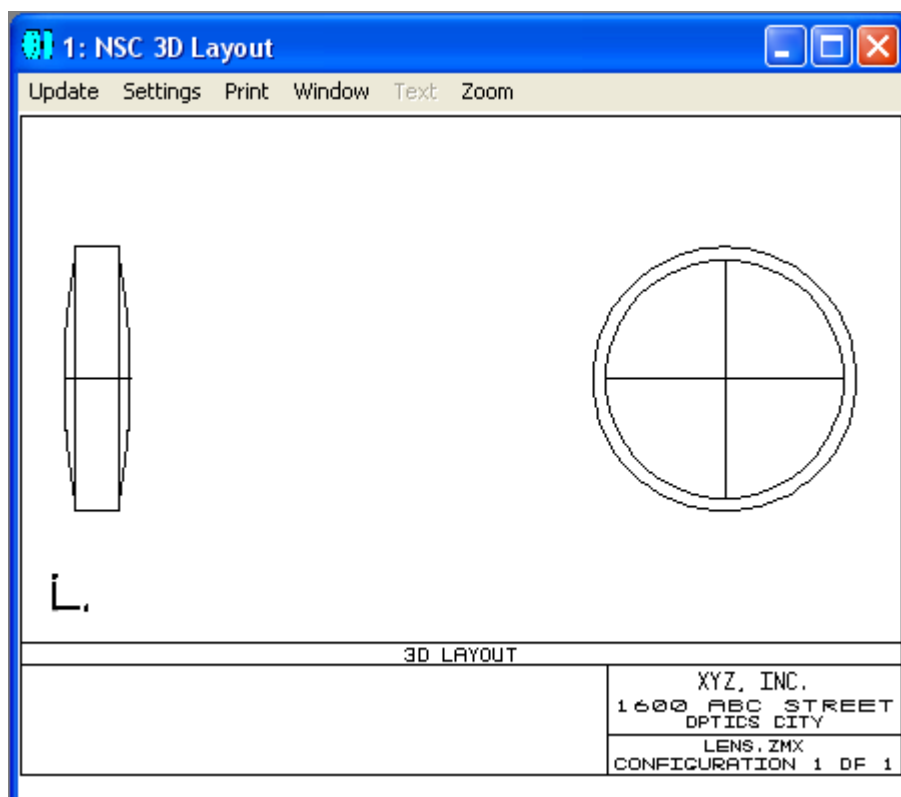


图 3.10-2: 程序 ex31001.ZPL 的运行后 3D Layout 窗口内容

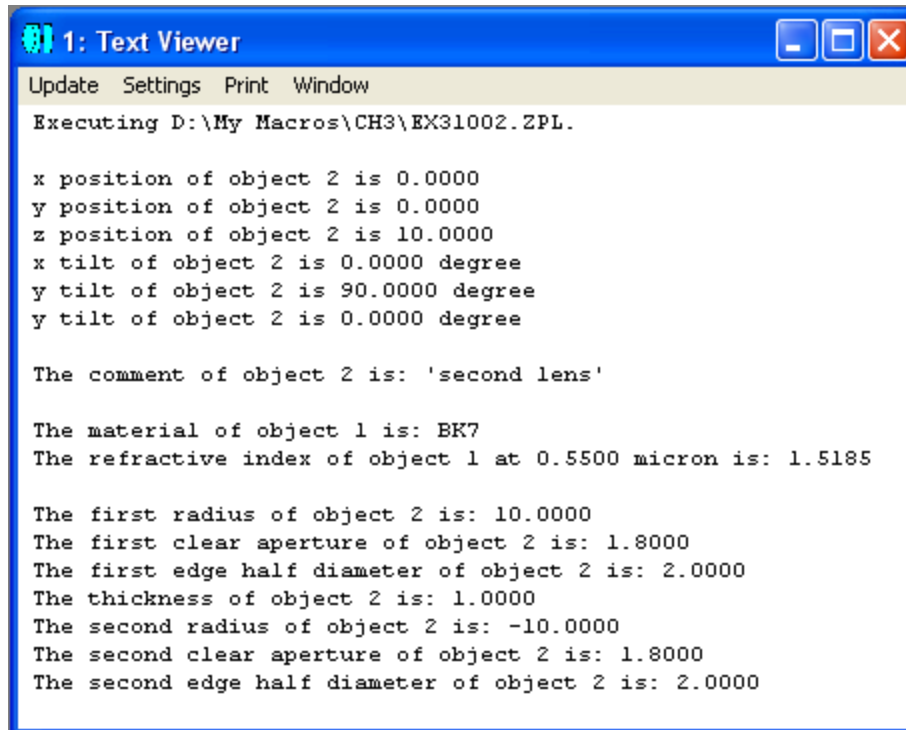
例3.10-2说明如何读取非顺序元件的参数。

```

1 ! ex31002
2 ! This program shows how to Read NSC parameters
3 ! Assume the objects are defined in ex31001
4
5 ! read the position
6 x = NPOS(1, 2, 1)
7 PRINT
8 PRINT "x position of object 2 is ", x
9 PRINT "y position of object 2 is ", NPOS(1, 2, 2)
10 PRINT "z position of object 2 is ", NPOS(1, 2, 3)
11 PRINT "x tilt of object 2 is ", NPOS(1, 2, 4), " degree"
12 PRINT "y tilt of object 2 is ", NPOS(1, 2, 5), " degree"
13 PRINT "y tilt of object 2 is ", NPOS(1, 2, 6), " degree"
14
15 ! read the comment
16 dummy = NPRO(1, 2, 1, 0)
17 a$ = $buffer()
18 PRINT
19 PRINT "The comment of object 2 is: '", a$, "'"
20
21 ! read the material
22 dummy = NPRO(1, 2, 4, 0)
23 a$ = $buffer()
24 PRINT
25 PRINT "The material of object 1 is: ", a$
26 PRINT "The refractive index of object 1 at ",
27 PRINT WAVL(1), " micron is: ", NPRO(1,1,200,1)
28
29 ! read the parameters
30 PRINT
31 PRINT "The first radius of object 2 is: ", NPAR(1,2,1)
32 PRINT "The first clear aperture of object 2 is: ", NPAR(1,2,3)
33 PRINT "The first edge half diameter of object 2 is: ", NPAR(1,2,4)
34 PRINT "The thickness of object 2 is: ", NPAR(1,2,5)
35 PRINT "The second radius of object 2 is: ", NPAR(1,2,6)
36 PRINT "The second clear aperture of object 2 is: ", NPAR(1,2,8)
37 PRINT "The second edge half diameter of object 2 is: ", NPAR(1,2,9)

```

在这个程序中，我们假设光学系统为例 3.10-1 所定义的包含两个透镜的系统。通过函数 NPOS()，可以读出某元件的位置和倾斜，如程序第 6~13 行所示。注意我们既可以先将函数的返回值赋给一个中间变量，然后再在需要的地方使用此中间变量，如第 6、第 8 行所示，也可以直接在需要的地方通过调用函数获得返回值，如第 9~13 行所示。程序中还通过函数 NPRO() 读取了元件特性，通过函数 NPAR() 读取了元件参数。注意在使用函数 NPRO() 时，如果返回值为字符串，则可以先将返回结果存于一个中间变量内(16 行)，然后再通过函数 \$buffer() 读取字符串(17 行)。程序的运行结果如图 3.10-3 所示：



```
1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH3\EX31002.ZPL.

x position of object 2 is 0.0000
y position of object 2 is 0.0000
z position of object 2 is 10.0000
x tilt of object 2 is 0.0000 degree
y tilt of object 2 is 90.0000 degree
y tilt of object 2 is 0.0000 degree

The comment of object 2 is: 'second lens'

The material of object 1 is: BK7
The refractive index of object 1 at 0.5500 micron is: 1.5185

The first radius of object 2 is: 10.0000
The first clear aperture of object 2 is: 1.8000
The first edge half diameter of object 2 is: 2.0000
The thickness of object 2 is: 1.0000
The second radius of object 2 is: -10.0000
The second clear aperture of object 2 is: 1.8000
The second edge half diameter of object 2 is: 2.0000
```

图 3.10-3: 程序 ex31002.ZPL 的运行结果

在非顺序系统中，光源和探测器是两种非常重要的元件。例 3.10-3 说明如何在程序中设置光源和探测器。

```
1 ! ex31003
2 ! This program shows how to set NSC sources and detectors
3
4 ! this part is used to clean the NSC editor
5 for i, NOBJ(1), 1, -1
6   DELETEOBJECT 1, i      # delete all the objects
7 next                    # there is still a null object in the editor at last
8
9 ! add two more objects in the editor, so the total is 3
10 INSERTOBJECT 1, 1
11 INSERTOBJECT 1, 1
12
13 ! define the type of the first object as source ellipse (circular)
14 SETNSCPROPERTY 1, 1, 0, 0, "NSC_SRCE"
15
16 ! define the position
17   # this part is omitted. Use the default setting.
18
19 ! define parameters of the source
20 SETNSCPARAMETER 1, 1, 1, 100      # number of layout rays
21 SETNSCPARAMETER 1, 1, 2, 10000    # number of analyze rays
22 SETNSCPARAMETER 1, 1, 3, 1       # power in Watts
23 SETNSCPARAMETER 1, 1, 6, 1       # x half width
24 SETNSCPARAMETER 1, 1, 7, 1       # y half width
25 SETNSCPARAMETER 1, 1, 8, 0       # source distance, 0 for collimated
26
```

```

26
27 ! define the type of the second object as a standard lens
28 SETNSCPROPERTY 1, 2, 0, 0, "NSC_SLEN"
29
30 ! define the position
31 SETNSCPOSITION 1, 2, 3, 10      # z position
32
33 ! define the material
34 SETNSCPROPERTY 1, 2, 4, 0, "BK7"
35
36 ! define curvature, clear aperture, edge and thickness of the lens
37 SETNSCPARAMETER 1, 2, 1, 10      # radius 1
38 SETNSCPARAMETER 1, 2, 3, 1.8     # clear 1
39 SETNSCPARAMETER 1, 2, 4, 2       # edge 1
40 SETNSCPARAMETER 1, 2, 5, 1       # thickness
41 SETNSCPARAMETER 1, 2, 6, -10     # radius 2
42 SETNSCPARAMETER 1, 2, 8, 1.8     # clear 2
43 SETNSCPARAMETER 1, 2, 9, 2       # edge 2
44
45 ! define the type of the third object as a detector rect
46 SETNSCPROPERTY 1, 3, 0, 0, "NSC_DETE"
47
48 ! define the position
49 SETNSCPOSITION 1, 3, 3, 20      # z position
50
51 ! define the material
52 SETNSCPROPERTY 1, 3, 4, 0, "ABSORB" # assuming the detector is absorbing
53
54 ! define parameters of the detector
55 SETNSCPARAMETER 1, 3, 1, 1       # x half width
56 SETNSCPARAMETER 1, 3, 2, 1       # y half width
57 SETNSCPARAMETER 1, 3, 3, 100     # number of x pixels
58 SETNSCPARAMETER 1, 3, 4, 100     # number of y pixels

```

在程序中，我们先清空了非顺序元件编辑器(5~7行)，然后插入了两个空元件(10~11行)，这样，编辑器中总共有3个空元件。我们将第1个元件设为椭圆光源(14行)，其空间坐标为缺省坐标(0, 0, 0)。随后，我们定义了光源的相应参数(20~25行)。接下来，我们将第2个元件设为标准透镜(28行)，并定义了其z坐标(31行)、材料(34行)及其它参数(37~43行)。类似地，我们将第3个元件设为探测器并定义了相应参数(46~58行)。

在运行程序 ex31003.ZPL 后，如果我们打开 3D Layout 窗口，可以看到程序中所定义的两个元件，同时还可以看到演示光线，如图 3.10-4 所示：

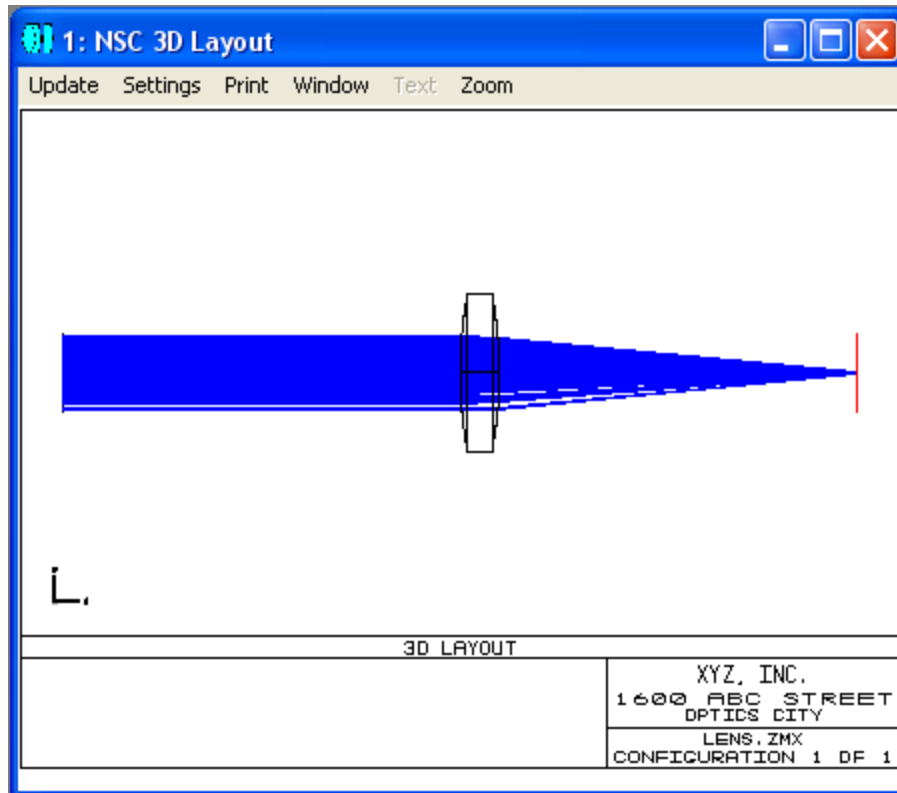


图 3.10-4: 程序 ex31003.ZPL 的运行后 3D Layout 窗口内容

有的时候，在对原有的非顺序元件进行修改和分析之后，我们需要再调回原来的元件，这可以通过关键词RELOADOBJECTS实现，其用法为：

RELOADOBJECTS surf, object

其中，surf为非顺序表面序号，如果为完全非顺序模式，则其值为1；object为非顺序元件所在的位置序号，为0表示重新装载所有的元件。

和顺序系统一样，在非顺序系统中，光线追迹也是一个重要功能。ZPL 提供了相应的关键词 NSTR 和函数 NSDC()及 NSDD()，用于进行光线追迹和读取探测器结果。

关键词 NSTR 的用法为：

NSTR surf, source, split, scatter, usepol, ignore_err, rand_seed, save, filename\$, filter

其中，surf 为非顺序表面序号，如果为完全非顺序模式，则其值为 1；source 为用于进行光线追迹的光源的元件序号，若为 0 则表示使用当前系统中所有的光源；

split 为非 0 整数表示打开光线分裂选择开关，为 0 表示关闭；scatter 为非 0 整数表示打开散射选择开关，为 0 表示关闭；usepol 为非 0 整数表示打开偏振选择开关，为 0 表示关闭；ignore_err 为非 0 整数表示忽略光线追迹中的错误，为 0 表示出错时中止光线追迹，退出 ZPL 程序并报告出错信息；rand_seed 若为 0，使用一个随机数为其基数，每次执行 NSTR 将会产生不同的随机光线，若为非 0 整数，则随机函数发生器使用此给定值为其基数，每次执行 NSTR 将会产生相同的光线，这样可以对每次光线追迹进行比较，在对系统进行优化时常用；save 为 0 或被省略，则其后面的参数 filename 及 filter 也不需要，否则，若为非 0 整数，则光线追迹的结果将被保存于一个 ZRD 文件中，文件名由 filename\$指定(不包含路径)，扩展名为 ZRD，文件保存于与镜头文件相同的文件夹中；filter\$为可选项，对应于不同的光线过滤器，如表 3.10-2 所示。在每次执行 NSTR 命令时，ZEMAX 总是会先调用 UPDATE 命令对当前系统进行更新。

表 3.10-2: 光线过滤器字符串

过滤器	说明
Bn	第 n 个元件内的体散射光线。如果 n 为 0 则包括所有元件。
Dn	遇到第 n 个元件后衍射的光线。
En	其前一级光线段遇到第 n 个元件后发生衍射的光线。仅当光线分裂发生于衍射元件、衍射级数不为 0 以及光线分裂选项打开时有效。
Fn	其前一级光线段遇到第 n 个元件后发生散射的光线。仅当光线分裂发生于散射元件以及光线分裂选项打开时有效。仅对散射光线有效，而对反射光线无效。如果 n 为 0 则包括所有元件。
Gn	其前一级光线段遇到第 n 个元件后发生鬼影反射的光线。仅当光线分裂发生于折射元件以及光线分裂选项打开时有效。如果 n 为 0 则包括所有元件。
Hn	遇到第 n 个元件的光线。
Jn	类似于 Gn，不同之处在于在鬼影反射发生前的所有光线段其强度被设为 0，这有助于在探测器上只观测鬼影反射能量而不是直接入射的能量。设为 0 的强度值只影响探测器的观测器，而不影响光线数据库观测器。
Ln	最后遇到第 n 个元件的光线。
Mn	没有遇到第 n 个元件的光线。
On	从元件序号为 n 的光源发出的光线。n 为 0 表明包含所有的光源。
Rn	遇到第 n 个元件后反射的光线。
Sn	遇到第 n 个元件后散射的光线。
Tn	折射进入或离开第 n 个元件的光线。
Wn	波长序号为 n 的光线。n 为 0 表明包含任意的波长。

X_GHOST(n,b)	恰好发生 b 次鬼影反射，并至少遇到第 n 个元件 1 次的光线。如果 n 为 0，则包含所有恰好发生 b 次鬼影反射的光线。
X_HIT(n,b)	遇到第 n 个元件恰好 b 次的光线。
X_IAGT(n,v)	遇到第 n 个元件时其绝对强度大于 v 的光线。
X_IALT(n,v)	遇到第 n 个元件时其绝对强度小于 v 的光线。
X_IRGT(n,v)	遇到第 n 个元件时其与初始强度的相对值大于 v 的光线。
X_IRLT(n,v)	遇到第 n 个元件时其与初始强度的相对值小于 v 的光线。
X_LGT(n,v)	遇到第 n 个元件时其局域坐标的 x 方向余弦大于 v 的光线。
X_LLT(n,v)	遇到第 n 个元件时其局域坐标的 x 方向余弦小于 v 的光线。
X_MGT(n,v)	遇到第 n 个元件时其局域坐标的 y 方向余弦大于 v 的光线。
X_MLT(n,v)	遇到第 n 个元件时其局域坐标的 y 方向余弦小于 v 的光线。
X_NGT(n,v)	遇到第 n 个元件时其局域坐标的 z 方向余弦大于 v 的光线。
X_NLT(n,v)	遇到第 n 个元件时其局域坐标的 z 方向余弦小于 v 的光线。
X_SCATTER(n,b)	恰好发生 b 次散射，并至少遇到第 n 个元件 1 次的光线。如果 n 为 0，则包含所有恰好发生 b 次散射的光线。
X_XGT(n,v)	遇到第 n 个元件时其局域 x 坐标大于 v 的光线。
X_XLT(n,v)	遇到第 n 个元件时其局域 x 坐标小于 v 的光线。
X_YGT(n,v)	遇到第 n 个元件时其局域 y 坐标大于 v 的光线。
X_YLT(n,v)	遇到第 n 个元件时其局域 y 坐标小于 v 的光线。
X_ZGT(n,v)	遇到第 n 个元件时其局域 z 坐标大于 v 的光线。
X_ZLT(n,v)	遇到第 n 个元件时其局域 z 坐标小于 v 的光线。
Z	发生致命错误的光线。

函数 NSDC() 的用法为：

$returnValue = NSDC(surf, object, pixel, data)$

其中，surf 为非顺序表面序号，如果为完全非顺序模式，则其值为 1；object 为探测器的元件序号；pixel 为探测器上的像素序号，为 0 表示所有像素之和；data 为 0 表示实部，1 表示虚部，2 表示振幅之和，3 表示相干强度。函数的返回结果存于变量 returnValue 中。

函数 NSDD() 的用法为：

$returnValue = NSDD(surf, object, pixel, data)$

其中，surf 为非顺序表面序号，如果为完全非顺序模式，则其值为 1；object 为探测器的元件序号；pixel 为探测器上的像素序号，为 0 表示所有像素之和；data 为 0 表示函数返回值为光通量 flux，1 表示返回值为单位面积上的光通量，2 表示返回值为单位立体角上的光通量。如果 object 的值为 0，则清空所有探测器，并返回值

0。需要注意的是，如果 data 为 2，则像素序号 pixel 必须大于 0，否则函数返回值为 0。对于表面探测器，仅支持 data 为 0 和 1 的情况。对于体探测器，像素序号 pixel 为体像素序号，同时，data 为 0、1、2 分别代表函数返回值为入射光通量、吸收光通量及单位体积上的吸收光通量。同样，pixel 为 0 表示所有像素之和。函数的返回结果存于变量 returnValue 中。

有的时候，我们想要对矩形探测器或表面探测器上某个像素的相干或非相干光强度值进行设置或修改，这可以通过关键词 SETDETECTOR 实现，其用法为：

SETDETECTOR surf, object, pixel, datatype, value

其中，surf 为非顺序表面序号，如果为完全非顺序模式，则其值为 1；object 为探测器的元件序号，探测器只能为矩形探测器或表面探测器；pixel 为探测器上的像素序号，介于 1 和最大像素数目之间；datatype 为 0 表示非相干强度，为 1 表示角度空间的非相干强度，为 2 表示相干实部，为 3 表示相干虚部，为 4 表示相干振幅；value 为所要设定的值。

下面我们举例说明如何在程序中对非顺序系统进行光线追迹。

例3.10-4：非顺序系统中的光线追迹：

```

1 ! ex31004
2 ! This program shows how to do ray tracing in NSC mode
3 ! Assume the lens system is defined in ex31003
4
5 surf = 1
6 source = 1
7 split = 1
8 scatt = 1
9 pol = 1
10 ignore_err = 1
11 random_seed = 1
12 save = 0
13 object = 0
14 obj = 3
15 pix = 0
16 data = 0
17
18 temp = NSDD(surf, object, pix, data)    # clear the detector
19 NSTR surf, source, split, scatt, pol, ignore_err, random_seed, save
20
21 PRINT
22 PRINT "Reading on detector is:", NSDD(surf, obj, pix, data)
23
24 NSTR surf, source, split, scatt, pol, ignore_err, random_seed, save
25 PRINT "Reading on detector without clearing: ", NSDD(surf, obj, pix, data)
26
27 temp = NSDD(1, 0, 0, 0)                # clear the detector
28 NSTR surf, source, split, scatt, pol, ignore_err, random_seed, save
29 PRINT "Reading on detector should be: ", NSDD(surf, obj, pix, data)

```

在此例中，第 18 行先清空探测器，第 19 行进行光线追迹，第 22 行读取并打印出探测器上的光强度值。整个光线追迹的过程就这么简单。不过需要注意的是，如果探测器中原来有数值没有清空，那么进行光线追迹后探测器上的读数还会包含原来的值。程序第 24~29 行比较了没有清空和清空探测器后再进行光线追迹所读到的探测器上光强度值的区别。程序的运行结果如图 3.10-5 所示：

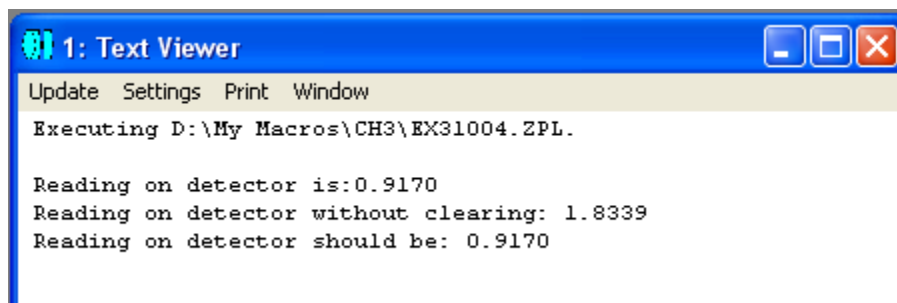


图 3.10-5: 程序 ex31004.ZPL 的运行结果

如果我们打开探测器浏览器，可以看到探测器上的光强分布，如图 3.10-6 所示：

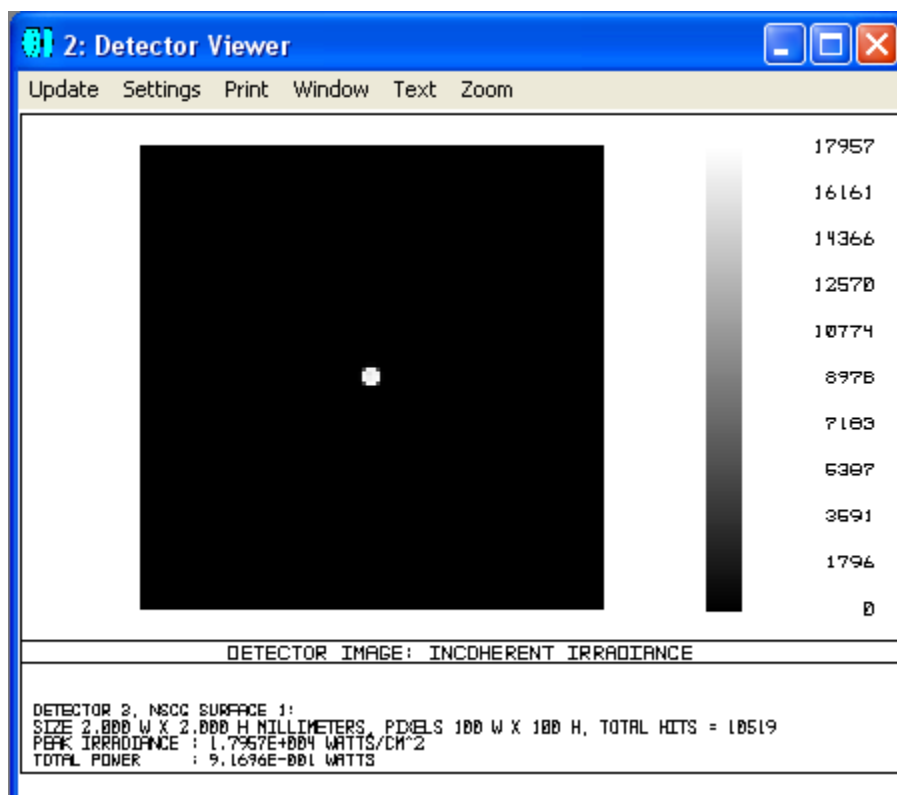


图 3.10-6: 程序 ex31004.ZPL 运行后探测器浏览器上看到的光强分布

例3.10-5: 修改非顺序系统中探测器的值:

```

1 ! ex31005
2 ! This program shows how to use SETDETECTOR key word
3 ! Assume the lens system is defined in ex31003
4
5 surf = 1
6 object = 3
7 datatype = 0
8
9 temp = NSDD(1, 0, 0, 0)    # clear the detector
10 NSTR 1, 1, 1, 1, 1, 1, 1, 0 # ray tracing, as in ex31004
11
12 PRINT
13 PRINT "Reading on detector: ", NSDD(surf, object, 0, datatype)
14
15 radius = 25
16 width = 2
17 value = 0.05
18
19 FOR i, 1, 100, 1
20   FOR j, 1, 100, 1
21     r2 = (i-50)*(i-50)+(j-50)*(j-50)
22     ra = (radius+width/2)
23     rb = (radius-width/2)
24     IF (r2<ra*ra) & (r2>rb*rb)
25       pix = i*100+j
26       SETDETECTOR surf, object, pix, datatype, value
27     ENDIF
28   NEXT
29 NEXT
30
31 PRINT "Modified reading on detector: ", NSDD(surf, object, 0, datatype)

```

在此例中，我们假设系统与前面两个例子一致，程序第9行清空探测器，第10行进行光线追迹，探测器上所得读数与例3.10-4相同。随后，我们在探测器上围绕其中心加一个圆环，方法是判断每一个像素与探测器中心的距离，如果在圆环的圆周上，则修改像素的读数。程序的运行结果如图3.10-7所示：

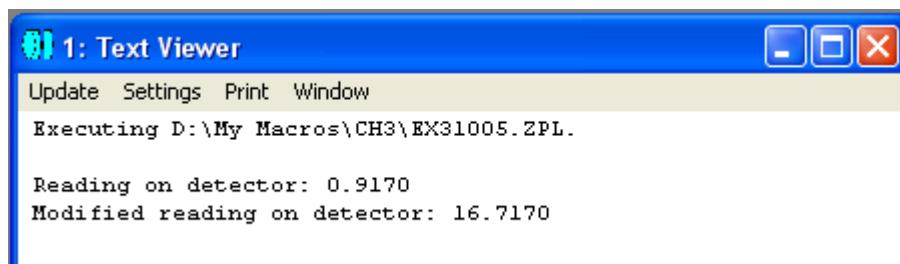


图 3.10-7: 程序 ex31005.ZPL 的运行结果

运行结果表明，整个探测器的读数已经被修改。如果我们打开探测器浏览器，可以看到我们所加的圆环，如图 3.10-8 所示：

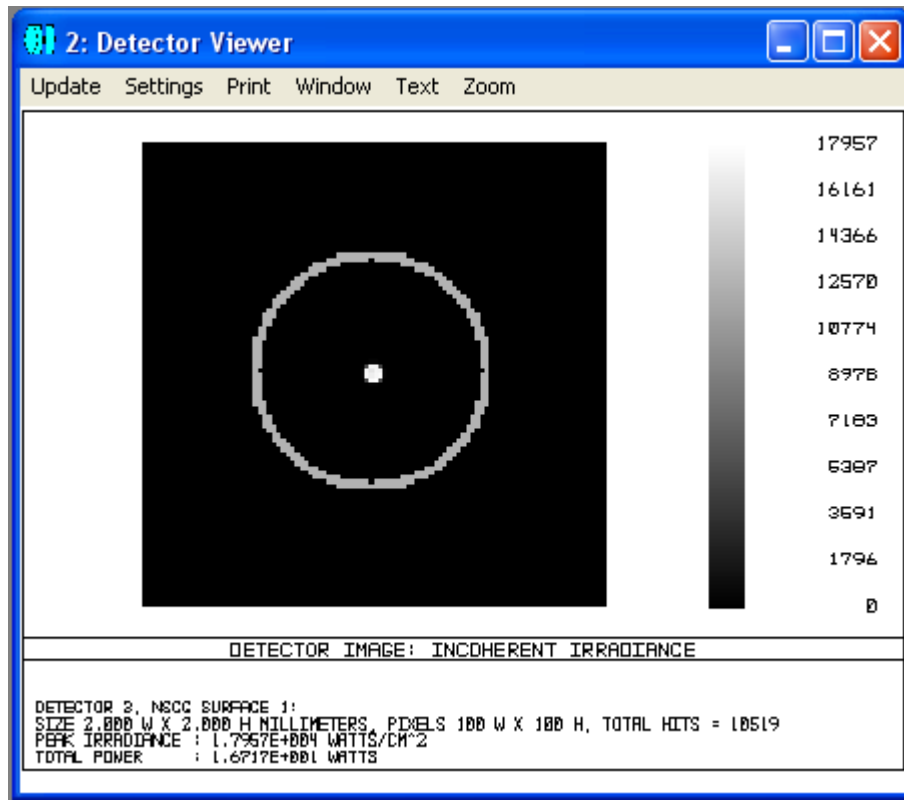


图 3.10-8：程序 ex31005.ZPL 运行后探测器浏览器上看到的光强分布

第十一节 多组态(multi-configuration)

在光学设计中，常常会涉及到需要对系统中的一部分元件或参数针对多个不同的应用环境进行增删或修改的情况，例如可变焦镜头的设计、对不同波长分别进行系统优化等。ZEMAX 所支持的多组态就可以广泛用于这些场合。在本节中，我们将介绍在 ZPL 程序设计与多组态相关的指令，而多组态的详细使用请参阅 ZEMAX 用户指南。

我们知道，多组态编辑器是用于对多组态进行设置和修改的地方。如果想要通过程序对多组态编辑器中的组态或运算元进行增删，可以使用ZPL提供的关键词 INSERTCONFIG、INSERTMCO、DELETECONFIG 和 DELETEMCO。

INSERTCONFIG 用于在多组态编辑器中增加一个组态，其用法为：

INSERTCONFIG config

其中，config 为大于0及小于或等于现有组态数加1的整数。

INSERTMCO 用于在多组态编辑器中增加一个运算元，其用法为：

INSERTMCO row

其中，row 为大于0及小于或等于现有运算元数目加1的整数。

DELETECONFIG 用于从当前多组态编辑器中删除一个组态，其用法为：

DELETECONFIG config

其中，config 为大于0及小于或等于现有组态数的整数。

DELETEMCO 用于从当前多组态编辑器中删除一个运算元，其用法为：

DELETEMCO row

其中，row 为大于0及小于或等于现有运算元数目的整数。

如果要对多组态编辑器中的参数进行设置或修改，可用关键词SETCONFIG 及 SETMCOPERAND。

SETCONFIG用于将多组态编辑器中的某一组态设为当前组态，其用法为：

SETCONFIG config

其中，config 为大于0及小于或等于现有组态数的整数。

SETMCOPERAND用于设置多组态编辑器某一组态中的某一运算元的值，其用法为：

SETMCOPERAND row, config, value, datatype

其中，row为多组态编辑器中运算元所在的行数，config为组态的序号。如果config的值为0，则其余两个参数的意义如下：

datatype = 0 时，value (应为value \$) 为代表运算元类型的字符串，参见表 3.11-1；

datatype = 1、2 或 3 时，value 为相应运算元中第1、2 或 3 个参数的值。

如果 config 的值不为 0，则其余两个参数的意义如下：

datatype = 0 时，value 为相应运算元的值；

datatype = 1 时，value 为摘取的运算元值的偏移；

datatype = 2 时，value 为摘取的运算元值的比例因子；

datatype = 3 时，value 为相应运算元的状态，0为固定，1为变量，2为摘取，3为热摘取；

datatype = 4 时，value 为摘取的组态序号；

datatype = 5 时，value 为摘取的行号。

下表列出了各种运算元的代码及相应参数：

表3.11-1各种运算元的代码及相应参数

类型代码	参数	说明
AFOC	无	非聚焦像空间模式。
APDF	无	系统变迹因子。
APDT	无	系统变迹类型。0为物，1为 Gaussian，2为正切函数。
APDX	表面序号	表面孔径 X- 中心偏移。
APDY	表面序号	表面孔径Y- 中心偏移。
APER	无	系统孔径值。如果系统孔径类型为按光阑尺寸浮动，则此值为光阑表面的半直径。
APMN	表面序号	表面孔径的最小值。此表面必须有一个给定的孔径而不是半直径。此运算元也用于控制所有表面孔径类型的第一个参数，如矩形或椭圆形孔径的X半宽。
APMX	表面序号	表面孔径的最大值。此表面必须有一个给定的孔径而不是半直径。此运算元也用于控制所有表面孔径类型的第二个

		参数，如矩形或椭圆形孔径的Y半宽。
APTP	表面序号	表面孔径类型。整数0~10分别代表孔径类型为：无、圆形、圆形障碍、蜘蛛形、矩形、矩形障碍、椭圆形、椭圆形障碍、用户自定义形、用户自定义形障碍及浮动孔径。
CONN	表面序号	Conic 常数。
COTN	表面序号	表面的镀膜名称。
CROR	表面序号	返回坐标取向。0为无，1为仅返回取向，2为返回取向及XY偏移，3为返回取向及XYZ偏移。
CRSR	表面序号	返回坐标表面序号。
CRVT	表面序号	表面曲率。
CSP1	表面序号	曲率求解参数1。
CSP2	表面序号	曲率求解参数2。
CWGT	无	组态的相对权重。
EDVA	表面序号， 附加数据序号	用于对多个附加数据赋值。
FLTP	无	视场类型。0为角度，单位为度，1为物高度，2为近轴像高，3为实际像高。
FLWT	视场序号	视场权重。
FVAN	视场序号	渐晕因子VAN。
FVCX	视场序号	渐晕因子VCX。
FVCY	视场序号	渐晕因子VCY。
FVDX	视场序号	渐晕因子VDX。
FVDY	视场序号	渐晕因子VDY。
GCRS	无	全域坐标参考表面。
GLSS	表面序号	玻璃类型。
GPJX	无	全域 Jones 矢量元 Jx。
GPJY	无	全域 Jones 矢量元 Jy。
GPIU	无	全域偏振状态是否为“非偏振”。1表明为非偏振状态，否则为偏振状态。
GPPX	无	全域偏振状态相位x。
GPPY	无	全域偏振状态相位y。
HOLD	无	保留数据于多组态缓冲器中，但无其它影响。可用于暂时关闭某个运算元而不丢失相应数据。
IGNR	表面序号	是否忽略此表面。0为不忽略，1为忽略。
LTTL	无	镜头名称。字符串长度限于32个字符。
MABB	表面序号	模型玻璃 Abbe 数。
MCOM	表面序号	表面注释。
MDPG	表面序号	模型玻璃部分色散 dP _g F。
MIND	表面序号	模型玻璃折射率。

MOFF	无	未用到的运算元，可用于加入注释。
NCOM	表面序号， 元件序号	非顺序元件编辑器中指定元件的注释，字符串长度限于32个字符。
NCOT	表面序号， 元件序号， 表面组序号	非顺序元件编辑器中指定元件指定表面组的镀膜类型。
NGLS	表面序号， 元件序号	非顺序元件编辑器中指定元件的材料类型。
NPAR	表面序号， 元件序号， 参数序号	修改非顺序元件编辑器中指定元件的指定参数的值。
NPOS	表面序号， 元件序号， 位置参数	修改非顺序元件编辑器中指定元件的指定位置参数的值。 位置参数1~6分别代表 x、y、z、x 倾斜、y 倾斜和z 倾斜。
NPRO	表面序号， 元件序号， 性质参数	修改非顺序元件编辑器中指定元件的指定性质参数的值。 性质参数为整数，意义如下： 1 – 包含于其内的元件序号 2 – 参考元件序号 3 – 不绘制元件选项 (0 为不选，1为选中) 4 – 光线忽略元件选项 (0 为不选，1为选中) 5 – 使用像素插值选项 (0 为不选，1为选中) 201-212 – 用户自定义渐变折射率参数 301-312 – 用户自定义衍射参数 401-412 – 用户自定义体散射参数
PRAM	表面序号， 参数序号	指定表面的指定参数值。
PRES	无	大气压。0代表真空，1代表标准大气压。
PRWV	无	主波长序号。
PSCX	无	光线定位光瞳压缩因子X。
PSCY	无	光线定位光瞳压缩因子Y。
PSHX	无	光线定位光瞳位移X。
PSHY	无	光线定位光瞳位移Y。
PSHZ	无	光线定位光瞳位移Z。
PSP1	表面序号， 参数序号	用于参数求解的参数1 (摘取表面序号)。
PSP2	表面序号	用于参数求解的参数2 (放大比例因子)。
PSP3	表面序号	用于参数求解的参数3 (偏移量)。
PUCN	无	用于从指定的前面某个组态中摘取一系列数据。如果组态序号为正整数，则PUCN 运算元以下的所有值都从指定组态摘取；如果组态序号为负整数，则摘取值变为负数；如

		果组态序号为0，则PUCN 运算元以下的值不进行摘取求解。注意可用两个PUCN 运算元定义进行摘取求解的值的起始位置和中止位置。所有指定运算元的序号都必须小于使用PUCN 运算元的组态序号。
RAAM	无	光线定位选项。0 为不选，1为选中。
SATP	无	系统孔径类型。0为入瞳直径，1为像空间F/#，2为物空间数值孔径，3为由光阑尺寸确定，4为近轴系统像空间工作F/#，5为物空间锥形张角。
SDIA	表面序号	表面半直径。
STPS	无	光阑表面序号。
TCEX	表面序号	热膨胀系数。
TELE	无	远心物空间，0为关闭此选项，1为打开此选项。
TEMP	无	以度为单位的摄氏温度。
THIC	表面序号	表面厚度。
TSP1	表面序号	用于厚度求解的参数1。
TSP2	表面序号	用于厚度求解的参数2。
TSP3	表面序号	用于厚度求解的参数3。
UDAF	表面序号	用户自定义孔径文件。表面必须为孔径或障碍。
WAVE	波长序号	波长。
WLWT	波长序号	波长权重。
XFIE	视场序号	视场X值。
YFIE	视场序号	视场Y值。

下面我们通过例子来说明如何在程序中定义和修改多组态系统。

例3.11-1：定义和修改多组态系统。

```

1 ! ex31101
2 ! This program shows how to create a multiconfiguration system from scratch
3 ! Before start this program, make sure a new lens file is created
4
5 ! add 5 more surfaces
6 FOR i, 1, 5, 1
7     INSERT 1
8 NEXT
9
10 ! set system parameters
11 SYSP 30, 0 # set lens unit as mm
12 SYSP 10, 0 # set system aperture as Entrance Pupil Diameter
13 SYSP 11, 15 # set system aperture value as 15mm
14
15 SYSP 201, 1 # set total wavelength number as 1
16 SYSP 202, 1, 0.55 # set the wavelength as 0.55 micron
17
18 SYSP 100, 0 # set the field type as Angle
19 SYSP 101, 3 # set the total field number as 3
20 SYSP 102, 1, 0 # set field 1 as x = 0 degree
21 SYSP 103, 1, 0 # set field 1 as y = 0 degree
22 SYSP 104, 1, 1 # set field 1 as weight = 1
23 SYSP 102, 2, 0 # set field 2 as x = 0 degree
24 SYSP 103, 2, 0.5 # set field 2 as y = 0.5 degree
25 SYSP 104, 2, 1 # set field 2 as weight = 1
26 SYSP 102, 3, 0 # set field 3 as x = 0 degree
27 SYSP 103, 3, 1 # set field 3 as y = 1 degree
28 SYSP 104, 3, 1 # set field 3 as weight = 1
29

```

```

29
30 ! set surface parameters
31 SURF 1, THIC, -37 # set surface 1 thickness as -37
32 SURF 2, THIC, 50 # set surface 2 thickness as 50
33
34 SURF 3, CURV, -1/65.8 # set surface 3 curvature as -1/65.8
35 SURF 3, THIC, -12.34 # set surface 3 thickness as -12.34
36 SURF 3, GLAS, "MIRROR" # set surface 3 glass type as MIRROR
37
38 SURF 4, CURV, -1/124.6 # set surface 4 curvature as -1/124.6
39 SURF 4, THIC, 60.62 # set surface 4 thickness as 60.62
40 SURF 4, GLAS, "MIRROR" # set surface 4 glass type as MIRROR
41
42 SURF 5, CURV, -1/21.8 # set surface 5 curvature as -1/21.8
43 SURF 5, THIC, -7.01 # set surface 5 thickness as -7.01
44 SURF 5, GLAS, "MIRROR" # set surface 5 glass type as MIRROR
45
46 SURF 6, CURV, -1/25.7 # set surface 6 curvature as -1/25.7
47 SURF 6, THIC, 48.02 # set surface 6 thickness as 48.02
48 SURF 6, GLAS, "MIRROR" # set surface 6 glass type as MIRROR
49
50 ! set surface 3 as stop
51 STOPSURF 3
52
53 ! insert 4 more configurations
54 FOR i, 1, 4, 1
55     INSERTCONFIG 1
56 NEXT
57
58 ! insert 4 more operand rows in the multi-configuration editor
59 FOR i, 1, 4, 1
60     INSERTMCO 1
61 NEXT
62
63 ! set the operand types in different rows
64 row = 1
65 config = 0
66 datatype = 0
67 value$ = "THIC"
68 value1 = 1
69 datatype1 = 1
70 SETMCOOPERAND row, config, value$, datatype # set operand type
71 SETMCOOPERAND row, config, value1, datatype1 # set operand surface #
72 SETMCOOPERAND 2, 0, "APMN", 0 # set operand type
73 SETMCOOPERAND 2, 0, 1, 1 # set operand surface #
74 SETMCOOPERAND 3, 0, "THIC", 0 # set operand type
75 SETMCOOPERAND 3, 0, 3, 1 # set operand surface #
76 SETMCOOPERAND 4, 0, "THIC", 0 # set operand type
77 SETMCOOPERAND 4, 0, 4, 1 # set operand surface #
78 SETMCOOPERAND 5, 0, "THIC", 0 # set operand type
79 SETMCOOPERAND 5, 0, 5, 1 # set operand surface #
80

```

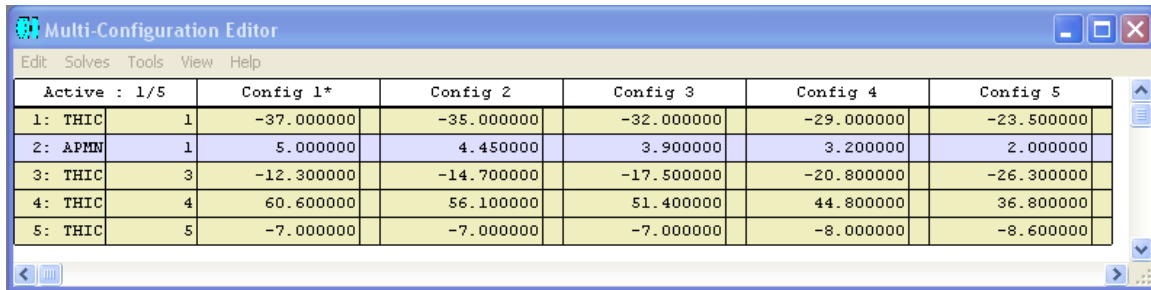
```

80
81 ! set the operand values in different configurations
82 SETMCOPIERAND 1, 1, -37, 0 # set the first operand in each configuration
83 SETMCOPIERAND 1, 2, -35, 0
84 SETMCOPIERAND 1, 3, -32, 0
85 SETMCOPIERAND 1, 4, -29, 0
86 SETMCOPIERAND 1, 5, -23.5, 0
87 SETMCOPIERAND 2, 1, 5, 0 # set the second operand in each configuration
88 SETMCOPIERAND 2, 2, 4.45, 0
89 SETMCOPIERAND 2, 3, 3.9, 0
90 SETMCOPIERAND 2, 4, 3.2, 0
91 SETMCOPIERAND 2, 5, 2, 0
92 SETMCOPIERAND 3, 1, -12.3, 0 # set the third operand in each configuration
93 SETMCOPIERAND 3, 2, -14.7, 0
94 SETMCOPIERAND 3, 3, -17.5, 0
95 SETMCOPIERAND 3, 4, -20.8, 0
96 SETMCOPIERAND 3, 5, -26.3, 0
97 SETMCOPIERAND 4, 1, 60.6, 0 # set the fourth operand in each configuration
98 SETMCOPIERAND 4, 2, 56.1, 0
99 SETMCOPIERAND 4, 3, 51.4, 0
100 SETMCOPIERAND 4, 4, 44.8, 0
101 SETMCOPIERAND 4, 5, 36.8, 0
102 SETMCOPIERAND 5, 1, -7, 0 # set the fifth operand in each configuration
103 SETMCOPIERAND 5, 2, -7, 0
104 SETMCOPIERAND 5, 3, -7, 0
105 SETMCOPIERAND 5, 4, -8, 0
106 SETMCOPIERAND 5, 5, -8.6, 0
107
108 UPDATE

```

在此例中，我们从头开始建立一个多组态系统，所以在运行程序之前，要先在 ZEMAX 主窗口的文件菜单下选择新建一个镜头系统。在新建的系统中，从镜头编辑器中可以看到有三个表面，程序的第 6~8 行再插入 5 个表面，这样系统中总共有 8 个表面，其中包括物表面和像表面。程序的第 11~28 行设置了一些基本的系统参数，包括镜头单位（第 11 行）、入瞳种类和尺寸（第 12、13 行）、波长数量和值（第 15、16 行），还定义了物方视场（第 18~28 行）。程序第 31~48 行定义了各表面参数，第 51 行定义了光阑表面。在基本的光学系统设置好后，程序第 54~56 行在组态编辑器中插入 4 个组态，这样总共有 5 个组态。程序第 59~61 行在组态编辑器中插入 4 行，这样总共有 5 行。程序第 64~79 行定义了组态编辑器中各运算元的种类，并在第 82~106 行定义了各组态中各运算元的值。程序最后一行通过关键词 UPDATE 对光学系统进行更新，以保证系统中各个参数为最后设定值。

程序 ex31101.ZPL 运行后，如果打开多组态编辑器，可以看到其中的内容如图 3.11-1 所示：



Active : 1/5		Config 1*	Config 2	Config 3	Config 4	Config 5
1: THIC	1	-37.000000	-35.000000	-32.000000	-29.000000	-23.500000
2: APMN	1	5.000000	4.450000	3.900000	3.200000	2.000000
3: THIC	3	-12.300000	-14.700000	-17.500000	-20.800000	-26.300000
4: THIC	4	60.600000	56.100000	51.400000	44.800000	36.800000
5: THIC	5	-7.000000	-7.000000	-7.000000	-8.000000	-8.600000

图 3.11-1: 程序 ex31101.ZPL 运行后多组态编辑器的内容

如果打开 3D Layout 窗口，可以看到各个组态如图 3.11-2 所示：

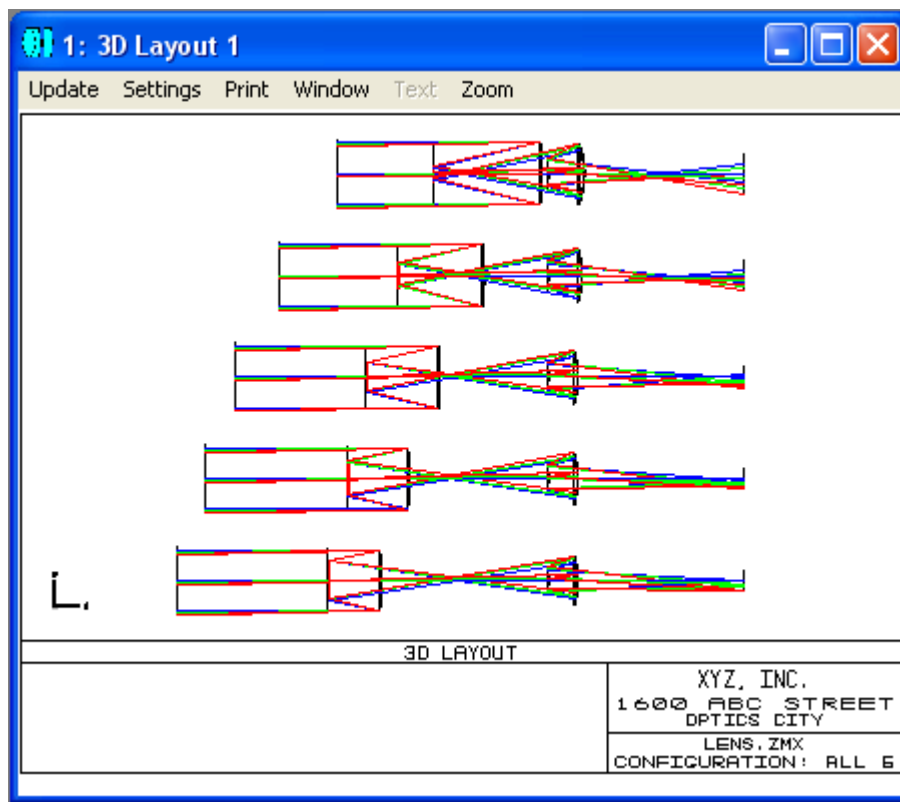


图 3.11-2: 程序 ex31101.ZPL 运行后 3D Layout 窗口的内容

当然，这个多组态系统尚未进行优化，我们可以根据设计要求，选择适当的变量和评价函数对其进行优化，这里就不进行赘述。

除了对多组态系统进行参数设置外，也可以通过 ZPL 提供的函数如 NCON()、CONF()、MCOP()、MCON()等对各种参数进行读取。

函数 `NCON()` 用于读取组态的数量，其用法为：

$$\text{returnValue} = \text{NCON}()$$

函数 `CONF()` 用于读取当前组态的序号，其用法为：

$$\text{returnValue} = \text{CONF}()$$

返回值 `returnValue` 介于 1 和最大组态数之间。

函数 `MCOP()` 用于读取组态编辑器的指定组态中指定行(即运算元)的数据，其用法为：

$$\text{returnValue} = \text{MCOP}(\text{row}, \text{config})$$

其中 `row` 为运算元所在的行数，`config` 为组态序号。如果 `config` 的值为 0，则代表当前组态。

函数 `MCON()` 用于读取组态编辑器的指定组态中指定行(即运算元)的数据，其作用与函数 `MCOP()` 相似，但功能更为强大。函数 `MCON()` 的用法为：

$$\text{MCON}(\text{row}, \text{config}, \text{data})$$

其中，`row` 为运算元所在的行数，`config` 为组态序号。如果 `row` 和 `config` 的值都为 0，则当 `data = 0、1 或 2` 时，函数返回值分别为运算元数目、组态数目或当前组态序号。如果 `row` 的值在 1 和运算元总数之间，同时 `config` 的值为 0，则当 `data = 0、1、2、3 或 4` 时，函数返回值分别为运算元类型、运算元相应参数 1、运算元相应参数 2、运算元相应参数 3 及字符串标志。字符串标志如果为 1，表明运算元相应数据为字符串，字符串标志如果为 0，表明运算元相应数据为数值量。如果 `row` 的值在 1 和运算元总数之间，同时 `config` 的值为大于 0 的有效值，则函数返回值为指定运算元的相应字符串数据或数值数据。注意如果为字符串数据，则应通过缓存区字符串函数 `$buffer()` 将返回的字符串读出。

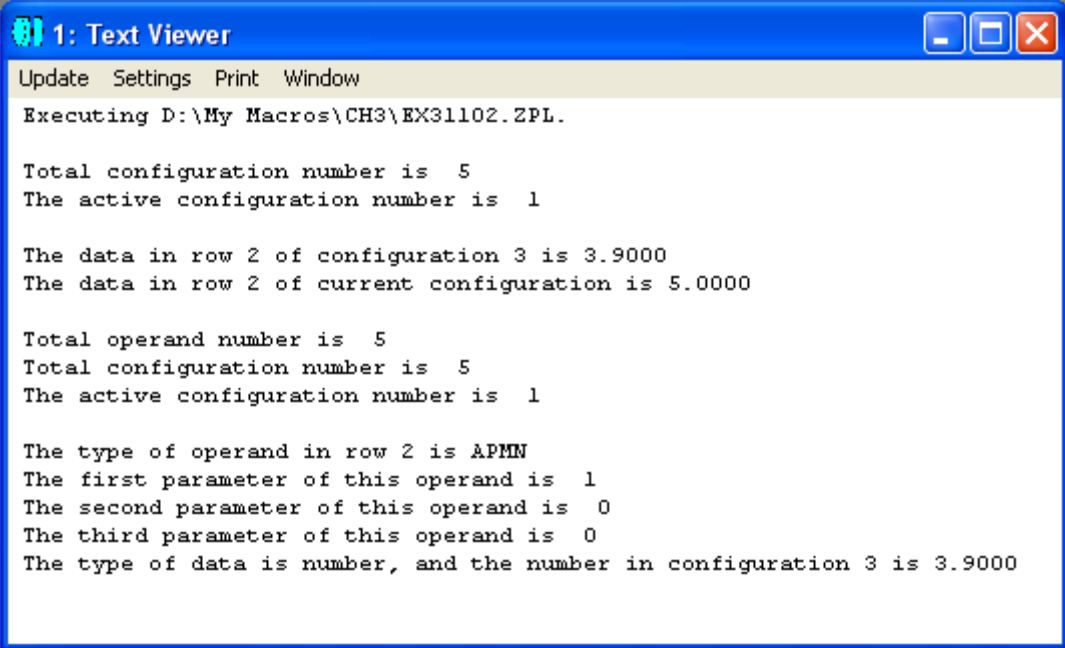
例3.11-2：读取多组态系统数据。

```

1 ! ex31102
2 ! This program shows how to read data from multiconfiguration editor
3 ! Assume the multi-configuration system is defined in ex31101
4
5 PRINT
6 FORMAT 2.0
7 PRINT "Total configuration number is ", NCON()
8 PRINT "The active configuration number is ", CONF()
9
10 FORMAT 6.4
11 PRINT
12 PRINT "The data in row 2 of configuration 3 is ", MCOP(2,3)
13 PRINT "The data in row 2 of current configuration is ", MCOP(2,0)
14
15 FORMAT 2.0
16 PRINT
17 PRINT "Total operand number is ", MCON(0,0,0)
18 PRINT "Total configuration number is ", MCON(0,0,1)
19 PRINT "The active configuration number is ", MCON(0,0,2)
20
21 dummy = MCON(2,0,0)
22 a$ = $buffer()
23 PRINT
24 PRINT "The type of operand in row 2 is ", a$
25 PRINT "The first parameter of this operand is ", MCON(2,0,1)
26 PRINT "The second parameter of this operand is ", MCON(2,0,2)
27 PRINT "The third parameter of this operand is ", MCON(2,0,3)
28 IF MCON(2,0,4) == 1 # if data in this operand is string
29     PRINT "The type of data is string, ",
30     dummy = MCON(2,3,0)
31     a$ = $buffer()
32     PRINT "and the string in configuration 3 is ", a$
33 ELSE # if data in this operand is number
34     FORMAT 6.4
35     PRINT "The type of data is number, ",
36     PRINT "and the number in configuration 3 is ", MCON(2,3,0)
37 ENDIF

```

在此例中，我们假设多组态系统为例3.11-1中所定义的系统。程序第7行通过函数NCON()读取了总的组态数目，第8行通过函数CONF()读取了当前组态序号，第12、13行通过函数MCOP()分别读取了组态3第2行的数据及当前组态第2行的数据，第17~37行通过函数MCON()读取了不同的数据，分别为总的运算元的数目(第17行)、总的组态数(第18行)、当前组态序号(第19行)、多组态编辑器第2行运算元的类型(第21~24行)及其相应第一、二、三个参数(第25~27行)。在程序第28行，先判断运算元相应的数据是字符串还是数值，如果是字符串，则通过缓存区字符串函数将其读出(第29~32行)，如果是数值，则直接将其读出(第36行)。程序的运行结果如图3.11-3所示：



```
1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH3\EX31102.ZPL.

Total configuration number is 5
The active configuration number is 1

The data in row 2 of configuration 3 is 3.9000
The data in row 2 of current configuration is 5.0000

Total operand number is 5
Total configuration number is 5
The active configuration number is 1

The type of operand in row 2 is APMN
The first parameter of this operand is 1
The second parameter of this operand is 0
The third parameter of this operand is 0
The type of data is number, and the number in configuration 3 is 3.9000
```

图 3.11-3: 程序 ex31102.ZPL 的运行结果

从程序结果中可以看到，函数MCOP()和函数MCON()可以用于读取同样的数据，如总的组态数(程序第7行、第18行)、当前组态数(程序第8行、第19行)或指定组态中指定运算元的具体数值(程序第12行、第36行)，得到相同的结果。但是，函数MCON()还可以用来读取更多的数据，如运算元类型(程序第21~24行)及相应参数(程序第25~27行)等。对于我们所举的这个具体例子而言，由于运算元只涉及一个参数，因此其第二、第三个参数的返回值为0。另外，需要指出的是，在使用函数MCON()读取指定组态指定运算元的值时，如果事先不知道返回值的类型，则可以通过读取字符串标志来进行判断，以决定采用什么形式将数据读出，如程序第28~37行所示。

第十二节 屏幕显示

屏幕是计算机系统中最常用的终端设备。ZPL中提供了很多用于控制和读取屏幕信息的关键词和函数。事实上，在前面的章节里，我们已经介绍和使用了一些相应的关键词及函数，如INPUT用于在程序运行过程中由用户通过键盘输入数值信息或字符串信息，OUTPUT用于控制将结果输出到文本窗口还是文件中，FORMAT用于控制打印到文本窗口或文件中的数值量的格式，PRINT用于将结果在屏幕上的文本窗口中显示出来或输出到文件中，等等。在这一节中，我们将继续介绍一些其它的常用指令。需要指出的是，ZPL还在不断丰富和扩展其指令集，因此，建议读者多查阅最新版本的ZEMAX用户指南。

在第一章中我们提到过，图形窗口是ZEMAX的一种重要数据输出工具。ZPL提供的关键词GRAPHICS可以为设计人员提供一个标准的图形输出窗口，设计人员可以根据需要在其中绘制自己的图形。

GRAPHICS的用法为：

```
GRAPHICS  
GRAPHICS NOFRAME  
GRAPHICS OFF
```

如果单独使用关键词GRAPHICS，则在屏幕上显示一个标准的带有标题框的ZEMAX 图形窗口。如果GRAPHICS 后面跟NOFRAME，则此标准图形窗口中的标题框会被压缩(即不绘出)。在这两种情况下，后续程序中的所有图形指令都使用这个新绘制的图形窗口。GRAPHICS OFF将关闭所有打开的图形窗口，然后显示关闭的窗口。

如果要在图形窗口中加入标题，可以使用关键词GTITLE，其用法为：

```
GTITLE user_title$
```

其中user_title\$为用户定义的标题字符串，此字符串将会显示在图形窗口的标题框中，并中心对齐。

如果要在图形窗口中指定位置和方向显示字符串，可以用关键词GTEXT，其用法为：

```
GTEXT x, y, angle, user_text$
```

其中，x、y为字符串起始端的坐标，angle为字符串相对于图形框的旋转角度，单位为度，user_text\$为用户定义的字符串，0度表示水平方向。

如果要在图形窗口中某一行以中心对齐的方式显示字符串，可以用关键词 **GTEXTCENT**，其用法为：

GTEXTCENT y, user_text\$

其中y为纵坐标，user_text\$为用户定义的字符串。

在显示字符串的时候，也可以通过关键词**SETTEXTSIZE**对其大小进行设置，方法为：

SETTEXTSIZE xsize, ysize

其中，xsize表示每个字符的横向尺寸为图形窗口横向尺寸的1/ xsize，ysize表示每个字符的纵向尺寸为图形窗口纵向尺寸的1/ ysize。如果这两个参数值为0或省略，则字符大小为ZEMAX缺省值。

另外，在图形窗口中也可以通过关键词 **GDATE** 显示当前日期，其格式由ZEMAX主菜单中 **File** 菜单条下的 **Preferences** 选项设定。

有的版本的ZEMAX还可以通过关键词 **GLENSNAME** 将镜头文件名显示在图形窗口中，但一般而言，由**GRAPHICS**创建的标准图形窗口中已包含此内容(具体设置请参见ZEMAX用户指南)。

在所创建的图形窗口中，用户可以通过关键词 **LINE** 及 **PIXEL** 绘制线段或像素点。

关键词**LINE**的用法为：

LINE oldx, oldy, newx, newy

其中，oldx、oldy为线段起始点的坐标，newx、newy为线段终止点的坐标。坐标可以为实数，但在实际绘制中，ZEMAX会自动取最接近的整数。另外，坐标定义于当前图形窗口中，因此不能超出其范围。

关键词**PIXEL**的用法为：

PIXEL xcoord, ycoord

其中xcoord和ycoord为所绘制像素在当前图形窗口中的横坐标和纵坐标。

在图形窗口中绘制字符、像素点或线段时，可以通过关键词**COLOR**控制画笔的颜色，其用法为：

COLOR n

其中 *n* 为0~24之间的整数，代表不同的颜色，0为黑色，其余 24 种颜色由 ZEMAX 主菜单中 File 菜单条下的 Preferences 选项设定。

在ZPL的屏幕显示中，还常用到函数XMIN()、XMAX()、YMIN()和YMAX()，分别用于读取当前图形窗口的最小横坐标、最大横坐标、最小纵坐标和最大纵坐标。注意，坐标的原点在图形窗口的左上角，横坐标从左往右，纵坐标从上往下。另外，可以通过函数ASPR()读取当前图形设备的高宽比例。这几个函数都不需要输入参数。

对于打开的窗口，我们可以通过关键词 LOCKWINDOW 将其锁定，其用法为：

LOCKWINDOW winnum

其中，*winnum*为窗口的序号，如果为0，则锁定所有的窗口，如果为-1，则当前窗口将在程序运行结束后锁定。

如果要对窗口解除锁定，可用关键词UNLOCKWINDOW，其用法为：

UNLOCKWINDOW winnum

其中，*winnum*为窗口的序号，如果为0，则解除对所有窗口的锁定，如果为-1，则当前窗口将在程序运行结束后解除锁定。

如果想要关闭打开的窗口，可用关键词CLOSEWINDOW，其用法为：

CLOSEWINDOW

或

CLOSEWINDOW winnum

如果 CLOSEWINDOW 单独使用，则ZPL程序将以“安静”模式执行，即在程序结束后不再显示通常显示的文字窗口。如果CLOSEWINDOW后面加上参数*winnum*，则将关闭序号为*winnum*的窗口。

在第二章第七节中介绍关键词 PRINT 和OUTPUT 时我们提到，通过OUTPUT可以控制将结果打印输出到文件中。那么，对于保存到文本文件中的信息，如何将其显示到屏幕上呢？这可以通过关键词SHOWFILE实现，其用法为：

SHOWFILE filename\$, saveflag

其中, filename\$为有效文件名, 文件类型必须为文本文件, 并且必须存放在当前文件夹中(由ZEMAX主菜单下的File → Preferences → Directories 设定)。saveflag为是否保存文件标志, 如果为 0 或被省略, 则语句执行完后文件filename\$被删除, 如果为其它非 0 数字, 则语句执行完后文件仍然保存。但是, 笔者发现, 如果文件 filename\$在程序执行之前已经存在, 则执行此语句后文件不会被删除。

除了显示文本文件, ZPL还可以显示图形文件。例如, 通过关键词IMASHOW, 可以将IMA 或 BIM格式的文件显示在一个图形窗口中, 其用法为:

IMASHOW filename\$

其中, filename\$为图形文件名, 必须包含扩展名。文件必须存放在 \ImaFiles 文件夹中。当程序执行到这个指令时, 会打开一个新的窗口, 显示指定文件的内容。

下面我们通过例子来说明有关屏幕显示的函数及关键词的用法。

例3.12-1: 屏幕显示。

```
1 ! ex31201
2 ! This program shows how to use display-related keywords and functions
3
4 ! open a graphic window
5 GRAPHICS
6
7 ! get the coordinates
8 xmx = XMAX()
9 xmn = XMIN()
10 ymx = YMAX()
11 ymn = YMIN()
12 xWidth = xmx-xmn
13 yWidth = ymx-ymn
14 xLeft = xmn + ( 0.1 * xWidth )
15 xRight = xmn + ( 0.9 * xWidth )
16 yTop = ymn + ( 0.1 * yWidth )
17 yBot = ymn + ( 0.7 * yWidth )
18
19 ! draw a frame
20 LINE xLeft,yTop,xRight,yTop
21 LINE xRight,yTop,xRight,yBot
22 LINE xRight,yBot,xLeft,yBot
23 LINE xLeft,yBot,xLeft,yTop
24
25 ! add some text
26 SETTEXTSIZE 80, 80
27 GTEXT xmx*0.2,ymx*0.75,0, " X axis"
28 SETTEXTSIZE 50, 50
29 GTEXTCENT ymx*0.75, " X axis"
30 SETTEXTSIZE 30, 30
31 GTEXT xmx*0.6,ymx*0.75,0, " X axis"
32 SETTEXTSIZE 0, 0
33 GTEXT xmx*0.05,ymx*0.45,90, " Y axis"
34 GDATE # add data/time
35
```

```
35
36 ! draw a curve
37 pi = 3.1416
38 FOR theta, 1, 3600, 1
39   r = yWidth/4/3600*theta
40   x = r*COSI(theta*pi/180)+xmn+0.5*xWidth
41   y = r*SINE(theta*pi/180)+ymn+0.4*yWidth
42   COLOR theta - INTE(theta/25)*25
43   PIXEL x,y
44 NEXT
45
46 ! draw title
47 COLOR 0
48 myTitle$ = "This is a user created graphic window"
49 GTITLE myTitle$
50
51 PRINT "This line will not be displayed!"
52 CLOSEWINDOW
53 GRAPHICS OFF
```

在这个程序中，我们先通过关键词GRAPHICS打开一个图形窗口，然后读取窗口的基本坐标参数(第8~11行)，随后，我们定义了四个角的坐标并画出一个矩形框(第12~23行)，然后输出了一些自定义的字符串信息(第26~34行)。在字符串信息的输出中，我们用到了字符大小控制(第26、28、30、32行)，字符串横坐标方向中心对齐(第29行)，字符串旋转90°(第33行)，还输出了日期信息(第34行)。接下来，我们在图形窗口中绘制了一条螺旋曲线(第37~44行)，其中用到了画笔颜色的控制(第42行)。程序的最后，我们给图形窗口加上一个标题(第47~49行)。虽然我们在程序第51行加入一条打印语句，但由于下面一行CLOSEWINDOW设定程序以“安静”模式运行，用于显示运行结果的文字窗口被关闭，因此我们实际上看不到所打印的内容。

程序运行结束后，所得到的图形输出结果如图3.12-1所示：

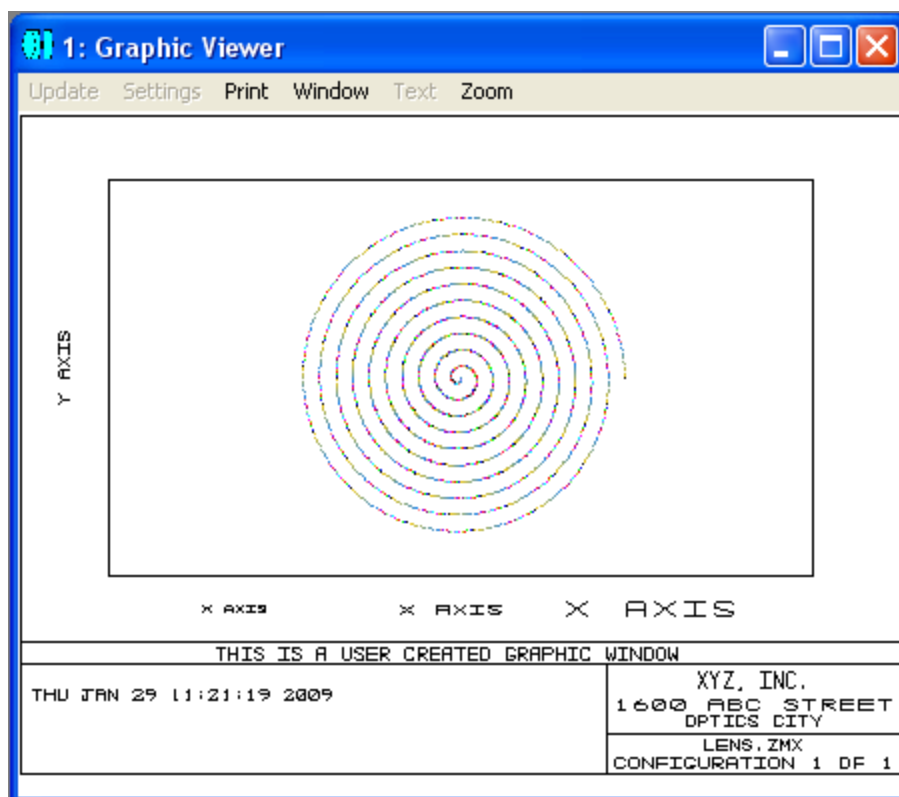


图 3.12-1: 程序 ex31201.ZPL 的运行结果

第十三节 文件操作

在利用ZEMAX进行光学设计的过程中，常常会涉及到很多文件操作。我们在第二章第七节中已经介绍过一些基本的文件操作指令，在本节中，我们将进一步介绍更多的文件操作指令。

在程序设计中，大多数情况下，我们不需要从头开始构造光学系统，而往往是载入已经存在的镜头文件，或在程序执行过程中重新装载镜头文件。ZPL提供了一个关键词LOADLENS来实现此功能，其用法为：

LOADLENS filename\$, appendflag, session

其中，filename\$为所要装载文件的名称，可以包含路径，如果路径省略，则使用缺省路径，由ZEMAX主菜单下的File → Preferences → Directories 设定；appendflag为附加标志，如果为0或省略，则装入指定文件，覆盖掉内存中的原有内容，而如果为大于0，则装入的文件附加在由appendflag指定的表面之后；如果 session 为非0值，则所装载镜头相应的 session 文件也被装入，同时所有窗口被更新，而如果 session 为0或省略，则忽略所装入镜头的 session 文件。

在装入镜头文件时，相应的玻璃目录及数据文件，包括镀膜数据文件，都会被装入。但如果在程序执行过程中，这些目录及文件被修改，而又需要重新载入，则可用关键词LOADCATALOG，将玻璃目录及数据文件重新装载。LOADCATALOG后面不需要跟参数。

如果需要重新装载评价函数文件，则可用关键词LOADMERIT，其用法为：

LOADMERIT filename\$

其中，filename\$为所要装载的评价函数文件的名称，可以包含路径，如果路径省略，则使用缺省路径，由ZEMAX主菜单下的File → Preferences → Directories 设定。

还有一个相似的关键词为IMPORTEXTRADATA，作用是将指定文件内容载入附加数据编辑器，其用法为：

IMPORTEXTRADATA surface, filename\$

其中，surface为表面序号，filename\$为所要装载的附加数据文件的名称，应包含路径。

与装载镜头文件相反，如果需要将当前内存中的镜头文件保存，可用关键词SAVELENS，其用法为：

SAVELENS filename\$, session

其中，filename\$为指定文件名，指令执行后，内存中的镜头文件名也会更新。如果filename\$省略，则使用内存中的当前镜头文件名。如果session的值非0，则相应的session文件也会保存。

如果要将当前评价函数保存，可用关键词SAVEMERIT，其用法为：

SAVEMERIT filename\$

其中，filename\$为所要保存的评价函数文件的名称，可以包含路径，如果路径省略，则使用缺省路径，由ZEMAX主菜单下的File → Preferences → Directories 设定。

另外一个保存文件的关键词为SAVEWINDOW，其作用是将指定的文本窗口的内容保存到指定的文件中，其用法为：

SAVEWINDOW winnum, filename\$

其中，winnum为指定文本窗口的序号，filename\$为所要保存的文件的名称，可以包含路径，如果路径省略，则使用缺省路径，由ZEMAX主菜单下的File → Preferences → Directories 设定。

在进行文件操作的时候，我们常常会涉及到拷贝、更名、删除及搜索。为此，ZPL提供了相应的关键词COPYFILE、RENAMEFILE、DELETEFILE及FINDFILE，分别介绍如下：

关键词COPYFILE用于将源文件拷贝到目标文件，其用法为：

COPYFILE sourcefilename\$, newfilename\$

其中，sourcefilename\$为源文件名，newfilename\$为目标文件名。文件名中可以包含路径，如果路径省略，则使用缺省路径，由ZEMAX主菜单下的File → Preferences → Directories 设定。另外，如果目标文件已存在，则将被直接覆盖。

关键词RENAMEFILE用于修改文件名，其用法为：

RENAMEFILE oldfilename\$, newfilename\$

其中，oldfilename\$为旧文件名，newfilename\$为新文件名。

关键词DELETEFILE用于删除指定文件，其用法为：

DELETEFILE filename\$

其中filename\$为所要删除文件的名称。

关键词FINDFILE用于搜索存放在磁盘上的文件，其用法为：

FINDFILE TEMPNAME\$, FILTER\$

其中，FILTER\$为进行搜索的条件，通常包含路径名和文件类型，TEMPNAME\$用于存放搜索到的第一个文件名。如果相同的指令再执行一次，则搜索的结果为下一个满足条件的结果。如果需要重新定位到第一个满足条件的文件，可以临时改变搜索条件并执行搜索指令，然后重新用原来的搜索条件进行一次搜索，因为每次使用新的搜索条件，搜索过程都会从头开始。FINDFILE常用于在某个目录中找出相同类型的文件，或对大量的相似镜头文件进行分析。

下面我们通过例子来说明上述有关文件操作指令的用法。首先，我们假设镜头文件及相应的session文件已经存在，为“ex31301.ZMX”和“ex31301.SES”。如果没有这两个文件，我们可以在ZEMAX中运行一遍程序“ex30401.ZPL”，然后将当前内存中的光学系统保存到磁盘上，命名为“ex31301”即可。

```

1 ! ex31301
2 ! This program shows file operation
3 ! Assume files "ex31301.ZMX" and "ex31301.SES" have already been created,
4 ! otherwise run "ex30401.ZPL" to build the lens system and save files.
5
6 LOADLENS "D:\My Macros\ch3\ex31301.ZMX"
7 LOADLENS "D:\My Macros\ch3\ex31301.ZMX", 6
8
9 SAVELENS "D:\My Macros\ch3\temp.ZMX", 1
10 SAVEMERIT "D:\My Macros\ch3\temp.MF"
11
12 PRINT
13 PRINT "This line is printed on the screen and will be saved in a file."
14 SAVEWINDOW 1, "D:\My Macros\ch3\tempWin.txt"
15
16 COPYFILE "D:\My Macros\ch3\temp.ZMX", "D:\My Macros\ch3\temp1.ZMX"
17 COPYFILE "D:\My Macros\ch3\temp.SES", "D:\My Macros\ch3\temp1.SES"
18 COPYFILE "D:\My Macros\ch3\temp.MF", "D:\My Macros\ch3\temp1.MF"
19
20 RENAMEFILE "D:\My Macros\ch3\temp.MF", "D:\My Macros\ch3\temp2.MF"
21
22 path$ = "D:\My Macros\ch3\"
23 FILTER$ = path$ + "temp*.*"
24 PRINT
25 PRINT "Listing all files created in this program:"
26 FINDFILE TEMPFILE$, FILTER$
27 LABEL 1
28 IF (SLEN(TEMPFILE$))
29 PRINT TEMPFILE$
30 FINDFILE TEMPFILE$, FILTER$
31 GOTO 1
32 ENDIF
33
34 FILTER1$ = path$ + "e*.*"
35 FINDFILE TEMPFILE$, FILTER1$
36 FINDFILE TEMPFILE$, FILTER$
37 LABEL 2
38 IF (SLEN(TEMPFILE$))
39 TEMPFILE$ = path$ + TEMPFILE$
40 DELETEFILE TEMPFILE$
41 FINDFILE TEMPFILE$, FILTER$
42 GOTO 2
43 ENDIF
44 PRINT
45 PRINT "All the above shown files have been deleted."

```

程序第6行首先调用已经存在的镜头文件，第7行再次调用相同的文件并附加在原有的光学系统之后。程序第9行将当前镜头系统保存到文件“temp.ZMX”及“temp.SES”中，第10行将当前评价函数保存到文件“temp.MF”中。程序第13行输出一条信息到文本窗口，第14行将文本窗口中的内容保存到文件“tempWin.txt”中(假设文本窗口序号为1)。程序第16~18行拷贝文件，第20行重命名文件。程序第22~23行设置了搜索条件，第26行进行第一次搜索，第28行判断如果搜索结果不为空，则打印文件名(第29行)，第30行再次进行搜索，然后进入循环重新判断，直

到所有满足搜索条件的文件都被列出为止。程序第34行重新设定一个搜索条件，用于第35行进行搜索，其目的只是重新定位第一个搜索文件。第36~43行实际只是重新运行一遍第22~32行的搜索，并将搜索到的文件删除。程序运行结果如图3.13-1所示：

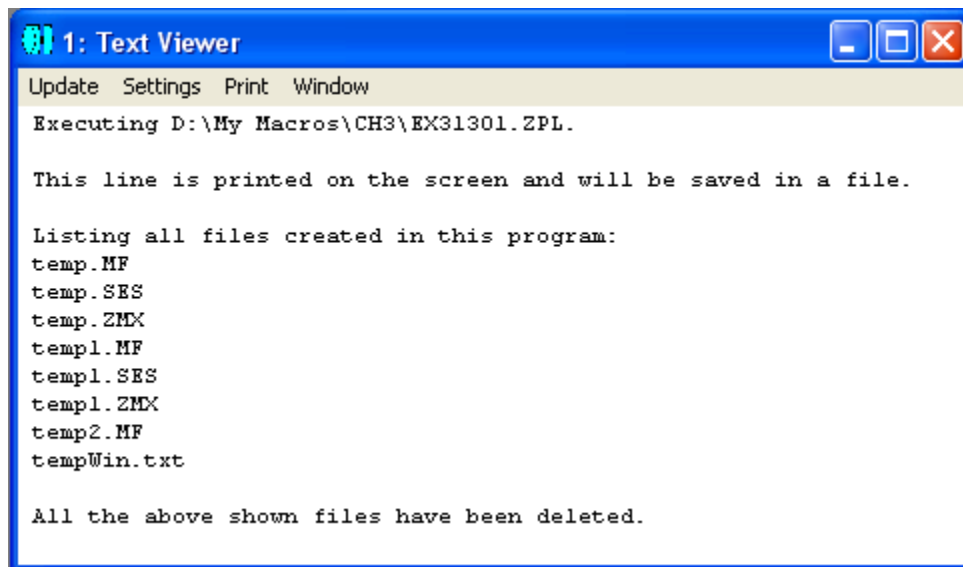


图 3.13-1: 程序 ex31301.ZPL 的运行结果

我们可以看到，程序运行过程中产生了一系列的文件，但这些文件最后都被删除了，所以如果用文件浏览器查看，这些文件并不存在。

前面我们涉及到的打印函数PRINT，实际上是将结果输出到屏幕上或文件中，但如果想要通过打印机输出结果，可以用关键词PRINTFILE或PRINTWINDOW。

关键词PRINTFILE的作用是打印一个文本文件，其用法为：

PRINTFILE filename\$

其中，filename\$为有效文件名，可包含路径，如省略路径则表明为当前文件夹，文件格式必须为文本文件。

关键词PRINTWINDOW的作用是将任何打开的文本窗口或图形窗口的内容输出到打印机，其用法为：

PRINTWINDOW winnum

其中，winnum为窗口序号。

如果要将图形窗口的内容保存到文件中，根据不同的文件格式，可用关键词 EXPORTBMP、EXPORTJPG 或 EXPORTWMF。

关键词EXPORTBMP用于将图形窗口的内容保存为BMP格式文件，其用法为：

EXPORTBMP winnum, filename\$, delay

其中，winnum为窗口序号，filename\$为指定的文件名，应包含路径，但不包含扩展名(ZEMAX会自动加上BMP扩展名)。delay为可选参数，表示延迟时间，单位为毫秒。在绘制一些复杂图形时，常常需要足够的延迟时间(例如 500 ~ 2500毫秒)以保证图形绘制完成。需要注意的是，ZEMAX 输出到文件中的图形是通过屏幕截取的方式得到的，如果在屏幕截取时ZEMAX处于后台工作状态(例如屏幕上有其它程序覆盖掉ZEMAX的图形窗口)，则输出文件中得不到需要的结果。

关键词EXPORTJPG用于将图形窗口的内容保存为JPG格式文件，其用法为：

EXPORTJPG winnum, filename\$, delay

其中，winnum为窗口序号，filename\$为指定的文件名，应包含路径，但不包含扩展名(ZEMAX会自动加上BMP扩展名)。delay为可选参数，表示延迟时间，单位为毫秒。

关键词EXPORTWMF用于将图形窗口的内容保存为WMF格式文件，其用法为：

EXPORTWMF winnum, filename\$

其中，winnum为窗口序号，filename\$为指定的文件名，应包含路径。与EXPORTBMP 及 EXPORTJPG不同的是，EXPORTWMF要求在文件名中指明扩展名。

除了将图形窗口的内容输出到文件，ZPL也可以通过关键词 EXPORTCAD 将整个镜头系统输出为 IGES、STEP、SAT 或 STL 格式的文件，以供CAD 软件使用。

EXPORTCAD的用法为：

EXPORTCAD filename\$

其中filename\$为指定的文件名，应包含路径。输出文件的格式由一系列参数决定，这些参数置于一维数组 VEC1 中，各数组元素所代表参数如表3.13-1所示：

表3.13-1 EXPORTCAD 输出文件的控制参数

数组元素	说明
VEC1(1)	文件类型。0 为 IGES，1 为 STEP，2 为 SAT，3 为 STL。
VEC1(2)	样条曲线点数(如果需要设定的话)。
VEC1(3)	输出的第一个表面。
VEC1(4)	输出的最后一个表面。
VEC1(5)	光线数据放置的层数。
VEC1(6)	镜头数据放置的层数。
VEC1(7)	1为输出 dummy 表面，否则为 0。
VEC1(8)	1为输出表面为实体模型，否则为 0
VEC1(9)	光线图案。0 为 XY，1 为 X，2 为Y，3 为环状，4 为列表，5 为无。
VEC1(10)	光线数目。
VEC1(11)	波长序号。0 代表全部。
VEC1(12)	视场序号。0 代表全部。
VEC1(13)	1 为删除渐晕光线，否则为0。
VEC1(14)	dummy 表面厚度，单位为镜头单位。
VEC1(15)	1为从非顺序元件光源分裂光线，否则为0。
VEC1(16)	1为从非顺序元件光源散射光线，否则为0。
VEC1(17)	1为进行非顺序光线追迹时使用偏振光线追迹，否则为0。如果使用分裂光线，则自动使用偏振光线追迹。
VEC1(18)	所使用组态序号。如果为 0 表示输出当前组态，如果为 n+1 (n 为最大组态序号)表示按文件输出所有组态，如果为 n+2 表示按层输出所有组态，如果为 n+3 表示同时输出所有组态。

例3.13-2 给出一个使用EXPORTCAD的例子：

```

1 ! ex31302
2 ! This program shows how to use EXPORTCAD key word
3 ! Assume the optical system is defined in "ex31301.zpl"
4
5 VEC1(1) = 1          # define STEP file
6 VEC1(3) = 1          # start surface
7 VEC1(4) = 5          # end surface
8 VEC1(7) = 1          # export dummy surfaces if any
9 VEC1(8) = 1          # export solid model
10 VEC1(18) = 0         # current configuration,
11
12 EXPORTCAD "D:\My Macros\ch3\ex31302.stp"

```

程序假设当前光学系统为例3.13-1中所定义的光学系统，我们想要把表面1～表面5中定义的双透镜输出为STEP文件。程序运行后，我们得到一个文件“ex31302.stp”。如果用CAD软件打开这个文件，可看到输出的双透镜如图3.13-2所示：

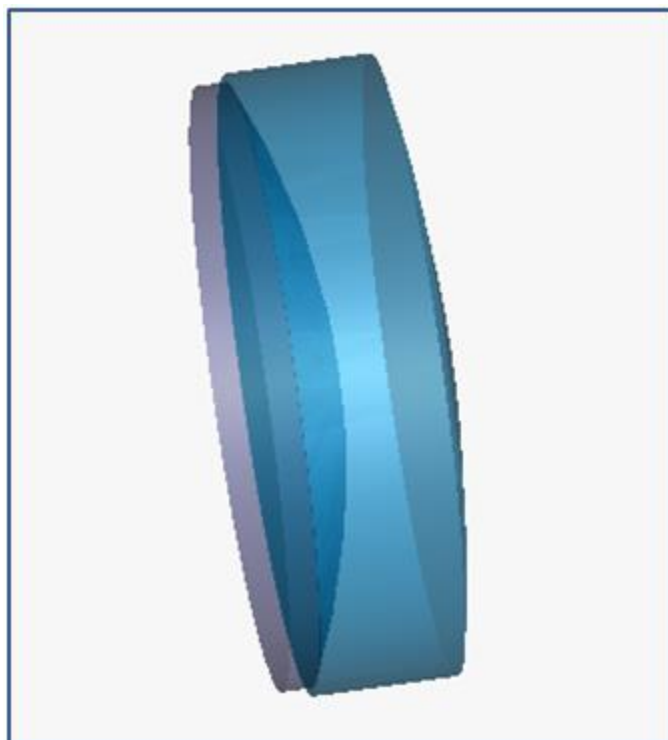


图 3.13-2：程序 ex31302.ZPL 输出的双透镜

第十四节 ZBF 文件

在本章第九节中，我们介绍过关键词POP，用于分析物理光学传播，分析结果被保存于ZEMAX光束文件(ZBF)中。ZBF是进行物理光学分析的一个重要部分，ZPL提供了一系列与其相关的关键词，包括ZBFCLR、ZBFMULT、ZBFPROPERTIES、ZBFREAD、ZBFRESAMPLE、ZBFSHOW、ZBFSUM、ZBFTILT、ZBFWRITE等，我们下面将逐一介绍。需要指出的是，ZEMAX将所有的ZBF文件都保存在“...\POP\Beamfiles\”文件夹中，其扩展名虽然可为任意，但建议统一使用“.ZBF”。

关键词ZBFCLR用于将ZBF文件中复振幅的数据清除，其用法为：

ZBFCLR filename\$

其中filename\$为相应的文件名。

关键词ZBFMULT用于将ZBF文件中复振幅的数据乘上一个复数因子，其用法为：

ZBFMULT filename\$, Ax, Bx, Ay, By

其中filename\$为相应的文件名，A和B分别为复数因子的实部和虚部，x和y代表光的不同偏振方向。

关键词ZBFPROPERTIES用于打开ZBF文件并将光束的相关特性参数存入指定一维数组中，其用法为：

ZBFPROPERTIES filename\$, vector

其中，filename\$为相应的文件名，vector为1~4，代表ZEMAX提供的4个一维数组之一。指令执行后，指定一维数组的第1~14个元素分别存放以下数据：x方向采样点数nx、y方向采样点数ny、x方向点间距dx、y方向点间距dy、x方向先导光束束腰waist_x(单位为镜头单位)、y方向先导光束束腰waist_y(单位为镜头单位)、x方向相对于先导光束束腰的z坐标位置position_x、y方向相对于先导光束束腰的z坐标位置position_y、x方向先导光束的Rayleigh距离rayleigh_x、y方向先导光束的Rayleigh距离rayleigh_y、当前媒质中的波长wavelength(单位为镜头单位)、总光能量total power、峰值辐照度peak irradiance(即单位面积光能量)以及偏振标志(0为非偏振，1为偏振)。

关键词ZBFREAD用于打开ZBF文件并将电场数据及光束的特性参数分别存入两个用户自定义数组中，其用法为：

ZBFREAD filename\$, beamname, propertyname

其中, filename\$ 为相应的文件名; beamname 为用户自定义的三维数组, 如果光束为非偏振光, 其最小长度为 (nx, ny, 2), 如果光束为偏振光, 其最小长度为 (nx, ny, 4); propertyname 为用户自定义的一维数组, 长度为14。指令执行后, 自定义一维数组 propertyname 的第1~14个元素分别存放以下数据: x 方向采样点数 nx、y 方向采样点数 ny、x 方向点间距 dx、y 方向点间距 dy、x 方向先导光束束腰 waist_x (单位为镜头单位)、y 方向先导光束束腰 waist_y (单位为镜头单位)、x 方向相对于先导光束束腰的 z 坐标位置 position_x、y 方向相对于先导光束束腰的 z 坐标位置 position_y、x 方向先导光束的 Rayleigh 距离 rayleigh_x、y 方向先导光束的 Rayleigh 距离 rayleigh_y、当前媒质中的波长 wavelength (单位为镜头单位)、总光能量 total power、峰值辐照度 peak irradiance (即单位面积光能量)以及偏振标志 (0 为非偏振, 1 为偏振)。电场数据将被存于数组 beamname 中, 数组第一、二个元素分别为电场 Ex 实部、电场 Ex 虚部, 如果为偏振光, 数组第三、四个元素分别为电场 Ey 实部、电场 Ey 虚部。

关键词 ZBFRESAMPLE 用于对指定的 ZBF 文件按新的参数进行采样, 其用法为:

ZBFRESAMPLE filename\$, nx, ny, wx, wy, decenterx, decentery

其中, filename\$ 为相应的文件名, nx 为 x 方向采样点数, ny 为 y 方向采样点数, wx 和 wy 分别为光束在 x 方向和 y 方向的总宽度, decenterx 和 decentery 为可选参数, 表示新光束相对于老光束在 x 方向和 y 方向上的偏移。nx 和 ny 的值必须为 2 的指数, 如 32、64、128 等。如果 nx 或 ny 的值为 0, 则不修改原来的值, 同样, 如果 wx 或 wy 的值为 0, 则不修改原来的值。ZEMAX 会自动将文件中原来的长度单位转换为当前镜头单位。指令执行后, 新的数据会被写回原来的文件。

关键词 ZBFSHOW 用于在观察器窗口中显示 ZBF 文件, 其用法为:

ZBFSHOW filename\$

其中, filename\$ 为相应的文件名。此指令将会打开一个新的图形窗口, 并显示光束文件。

关键词 ZBFSUM 用于将两个 ZBF 文件中的数据相干相加或非相干相加, 并将结果存于第三个 ZBF 文件中, 其用法为:

ZBFSUM coherent, filename1\$, filename2\$, outfilename\$

其中, coherent 如果为 0 表示非相干相加, 否则为相干相加, filename1\$ 和 filename2\$ 为源文件名, outfilename\$ 为结果文件名。如果非相干相加, 则输出结果只有实数值。如果两个源文件的数据点数、点间距、x 方向及 y 方向的参考半径不

一致，则第二个源文件filename2\$ 首先会被缩放及插值，然后调整参考半径，以和第一个源文件filename1\$ 一致，最后再进行计算。源文件中的长度单位会被自动调整为当前镜头单位。

关键词 ZBFTILT 用于将 ZBF 文件中的数据乘以一个复数相位因子，以在光束中引入相位倾斜，其用法为：

ZBFTILT filename\$, cx, cy, tx, ty

其中，filename\$ 为相应的文件名，cx 和 cy 为相位倾斜中心坐标，tx 和 ty 为相位倾斜斜率，单位为弧度/镜头长度单位。引入的光束相位角度为： $\theta = (x-cx)*tx + (y-cy)*ty$ 。坐标 x 和 y 指光束文件内的位置，其中心 ($x = 0, y = 0$) 对应于光束文件中的点 ($n_x/2 + 1, n_y/2 + 1$)，其中 n_x 和 n_y 分别为 x 方向和 y 方向的数据点数。源文件中的长度单位会被自动调整为当前镜头单位。指令执行后，结果将会写回原来的文件。

关键词 ZBFWRITE 用于将电场及光束的相关数据保存为 ZBF 文件，其用法为：

ZBFWRITE filename\$, beamname, propertyname

其中，filename\$ 为相应的文件名；beamname 为用户自定义的三维数组，如果光束为非偏振光，其最小长度为 ($n_x, n_y, 2$)，如果光束为偏振光，其最小长度为 ($n_x, n_y, 4$)；propertyname 为用户自定义的一维数组，长度为14。自定义一维数组 propertyname 的第1~14个元素分别存放以下数据：x 方向采样点数 n_x 、y 方向采样点数 n_y 、x 方向点间距 dx、y 方向点间距 dy、x 方向先导光束束腰 waist_x (单位为镜头单位)、y 方向先导光束束腰 waist_y (单位为镜头单位)、x 方向相对于先导光束束腰的 z 坐标位置 position_x、y 方向相对于先导光束束腰的 z 坐标位置 position_y、x 方向先导光束的 Rayleigh 距离 rayleigh_x、y 方向先导光束的 Rayleigh 距离 rayleigh_y、当前媒质中的波长 wavelength (单位为镜头单位)、总光能量 total power、峰值辐照度 peak irradiance (即单位面积光能量)以及偏振标志 (0 为非偏振，1 为偏振)。电场数据存于数组 beamname 中，数组第一、二个元素分别为电场 Ex 实部、电场 Ex 虚部，如果为偏振光，数组第三、四个元素分别为电场 Ey 实部、电场 Ey 虚部。指令执行后，存于数组 beamname 和 propertyname 中的数据将被保存到 ZBF 文件 filename\$ 中。

下面我们通过例子来说明如何在程序中使用与 ZBF 相关的指令。假设光学系统为例3.04-1中所定义的光学系统，如图3.14-1所示。当然，在下面的这个例子中，我们用到以前已经保存好的文件，因此运行程序时对光学系统没有要求。

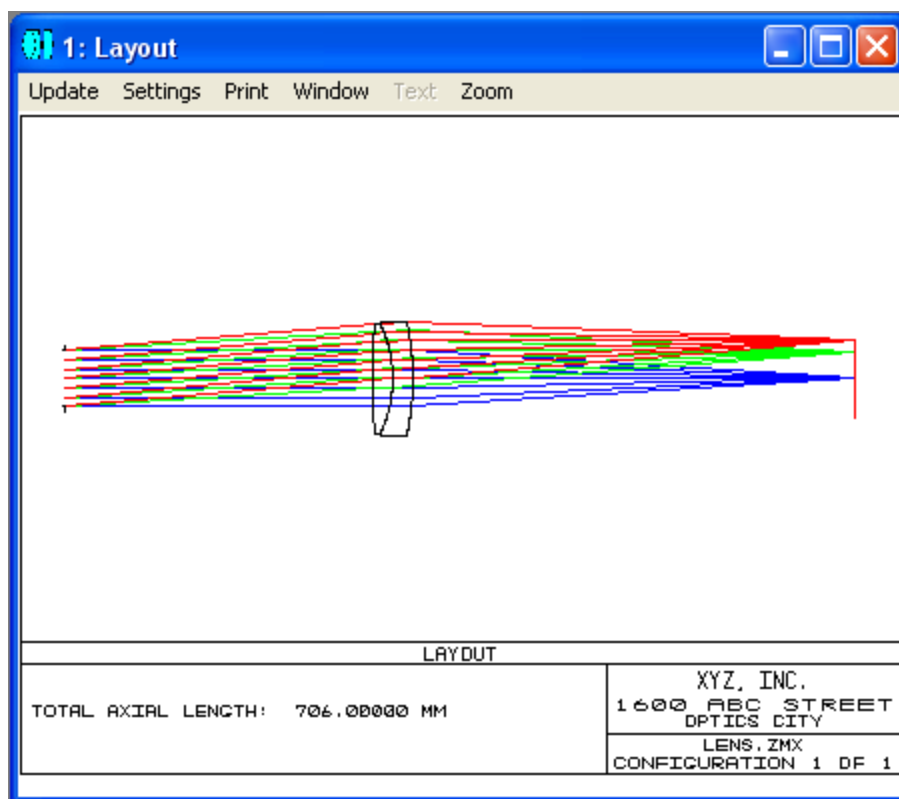


图3.14-1: 文件“ex30910b.ZBF”所对应的光学系统

在本章第九节中介绍关键词 POP 时，我们曾经保存了文件“ex30910b.ZBF”，文件中的数据是入射角为0的光传播到最后一个表面时的光束数据。下面的例子中将会用到这个文件。注意 ZBF 文件保存在“...\POP\Beamfiles ”文件夹内。

```

1 ! ex31401
2 ! This program shows application of some ZBF related key words.
3 ! Assume file "ex30910b.ZBF" already exists.
4
5 oldFile$ = "ex30910b.ZBF"
6 newFile1$ = "ex31401a.ZBF"
7 newFile2$ = "ex31401b.ZBF"
8 newFile3$ = "ex31401c.ZBF"
9 newFile4$ = "ex31401d.ZBF"
10
11 ZBFPROPERTIES oldFile$, 1
12 nx = vec1(1)
13 ny = vec1(2)
14 ip = vec1(14)           # check if is polarization
15
16 ! Allocate enough memory to hold the beam data
17 IF (ip == 0) THEN DECLARE beamArray, DOUBLE, 3, nx, ny, 2
18 IF (ip == 1) THEN DECLARE beamArray, DOUBLE, 3, nx, ny, 4
19 DECLARE propertyArray, DOUBLE, 1, 15
20
21 ZBFREAD oldFile$, beamArray, propertyArray
22 ZBFWRITE newFile1$, beamArray, propertyArray
23 ZBFWRITE newFile2$, beamArray, propertyArray
24 ZBFSHOW newFile1$
25
26 decenterx = 0
27 decentery = 0.015
28 ZBFRESAMPLE newFile2$, 0, 0, 0, 0, decenterx, decentery
29 ZBFSHOW newFile2$
30
31 ZBFSUM 1, newFile1$, newFile2$, newFile3$      ! coherent summation
32 ZBFSHOW newFile3$
33
34 ZBFSUM 0, newFile1$, newFile2$, newFile4$      ! incoherent summation
35 ZBFSHOW newFile4$
36
37 CLOSEWINDOW      # run in "quiet" mode

```

在程序中，我们通过关键词 ZBFPROPERTIES 读取了源文件中光束的特性参数，然后根据得到的参数定义了两个数组 beamArray 和 propertyArray (第 11 ~ 19 行)。程序第 21 行读取了源文件中的光束数据，第 22、23 行将读取的数据存到两个新的文件中，供后续的程序使用。在接下来的程序中，我们利用关键词 ZBFRESAMPLE 将原来的光束在 Y 方向移动 0.015 镜头单位(第 26 ~ 28 行)，并将移动后的光束与原来的光束相干相加(第 31 行)及非相干相加(第 34 行)，结果保存到不同的文件中。程序中还用关键词 ZBFSHOW 打开浏览器显示了不同的文件，结果如图所示：

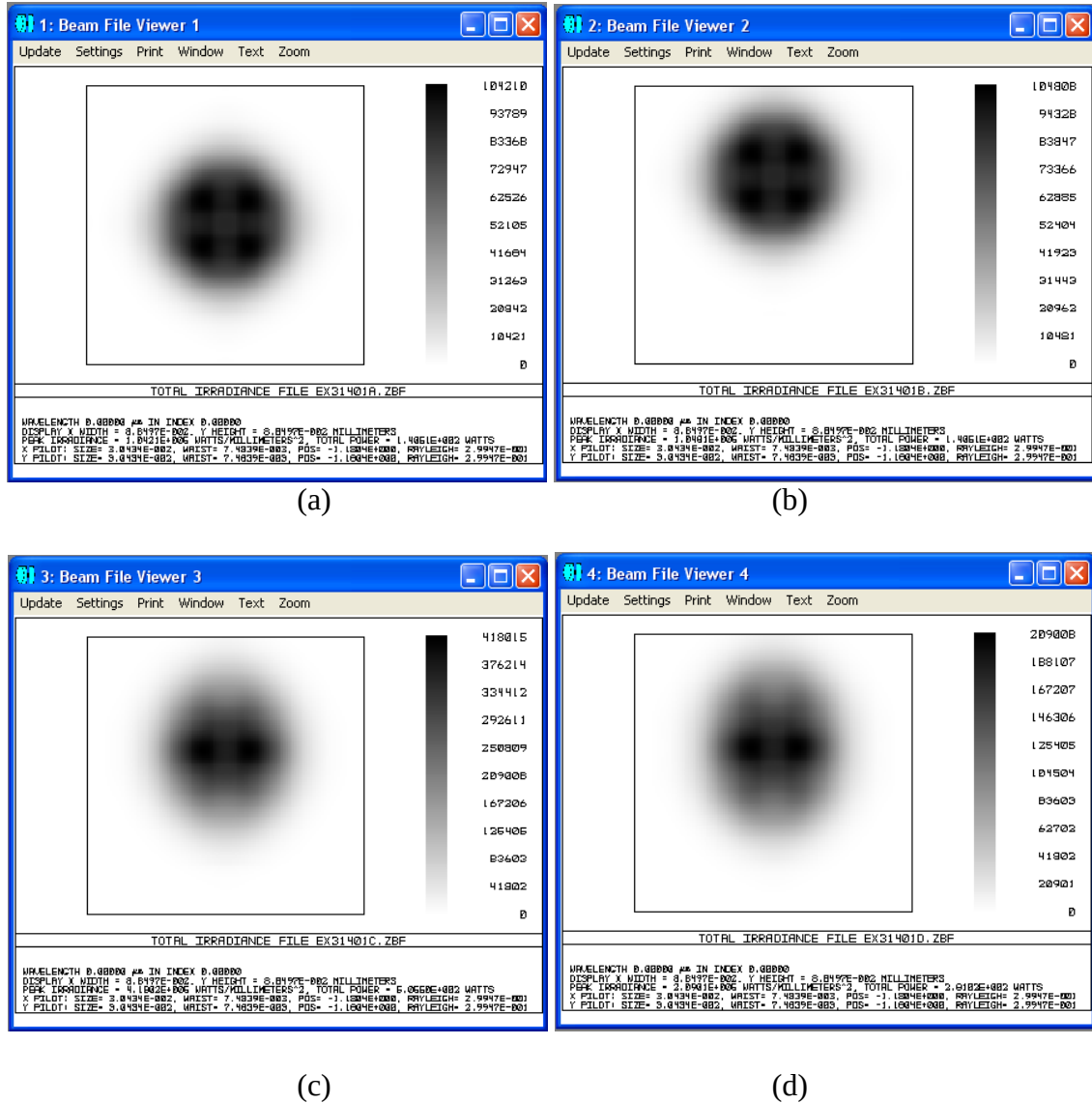


图3.14-2：程序运行后光束文件浏览器中的内容。(a)~(d)分别对应原光束、移动后光束、相干相加结果及非相干相加结果。

第四章 ZPL 应用实例

在本章中，我们将给出一些 ZPL 实际应用的例子。通过这些例子我们可以看到，借助于 ZPL，我们可以将大量繁琐的程序化的工作交给计算机去完成，从而可以大幅度地提高工作效率。作为一个光学设计师，如果能够熟练掌握 ZPL，则在我们的工具宝库中，无疑又添加了一项得心应手的利器，可以使我们更加出色地完成我们的工作。

第一节 顺序光学系统

这一节中的例子所涉及到的光学系统，均为顺序光学系统。

例4.1-1：光线追迹基本参数。

在本例中，我们让用户通过屏幕输入 H_x 、 H_y 、 P_x 、 P_y 定义一条光线，然后计算出当光线通过光学系统中的每一个表面时，光线和表面相交点的坐标、入射角和出射角。程序如下所示：

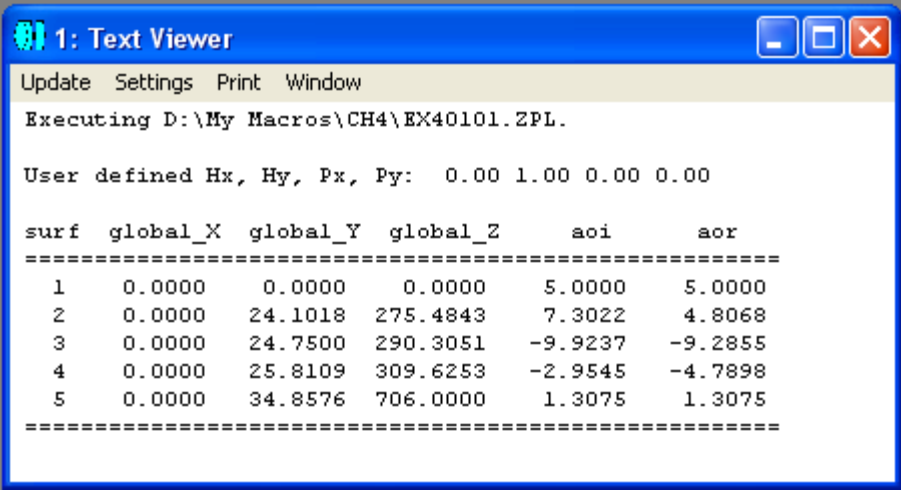
```

1 ! ex40101
2 ! This program traces a given ray and generate the table of ray data.
3
4 ! first, let user define the ray
5 INPUT "Please enter the value of Hx: ", hx
6 INPUT "Please enter the value of Hy: ", hy
7 INPUT "Please enter the value of Px: ", px
8 INPUT "Please enter the value of Py: ", py
9
10 ! print the top frame
11 PRINT
12 FORMAT 5.2
13 PRINT "User defined Hx, Hy, Px, Py: ", hx, hy, px, py
14 PRINT
15 PRINT "surf  global_X  global_Y  global_Z      aoi      aor"
16 PRINT "=====
17
18 ! trace the ray for entire system:
19 RAYTRACE hx, hy, px, py
20
21 ! calculate ray data at each surface
22 FOR s = 1, NSUR(), 1
23   normal = 57.29577951*ATAN(RANY(s)/RANZ(s))
24   slope   = 57.29577951*ATAN(RAYM(s-1)/RAYN(s-1))
25   aoi = (slope - normal)                                # in degrees
26   slope = 57.29577951*ATAN(RAYM(s)/RAYN(s))
27   aor = (slope - normal)                                # in degrees
28   FORMAT 2 INT
29   PRINT " ", s,
30   FORMAT 10.4
31   PRINT RAGX(s), RAGY(s), RAGZ(s), aoi, aor
32 NEXT
33
34 ! print the bottom frame
35 PRINT "=====

```

程序第5~8行提示用户输入所定义的光线，第19行对光线进行追迹，第22~32行计算光线和各表面相交点的参数，其中，第23行计算出表面的法线方向，第24行计算出光线入射表面前的方向，第26行计算出光线离开表面后的方向。

此程序是一个通用程序，对不同的光学系统均有效。如果假设光学系统为例3.4-1中所定义的系统，用户自定义光线为 $Hx = 0$ 、 $Hy = 1$ 、 $Px = 0$ 、 $Py = 0$ ，则程序运行结果为：



```

1: Text Viewer
Update Settings Print Window
Executing D:\My Macros\CH4\EX40101.ZPL.

User defined Hx, Hy, Px, Py:  0.00 1.00 0.00 0.00

surf  global_X  global_Y  global_Z      aoI      aor
=====
  1    0.0000    0.0000    0.0000    5.0000    5.0000
  2    0.0000    24.1018   275.4843    7.3022    4.8068
  3    0.0000    24.7500   290.3051   -9.9237   -9.2855
  4    0.0000    25.8109   309.6253   -2.9545   -4.7898
  5    0.0000    34.8576   706.0000    1.3075    1.3075
=====

```

图 4.1-1: 程序 ex40101.ZPL 的运行结果

例4.1-2: 焦平面附近光斑图形。

在本例中，我们通过在焦平面附近移动像平面(即改变像平面前一平面的厚度)，观察像平面上光斑尺寸的改变。程序如下所示：

```

1 ! ex40102
2 ! This program shows how to modify lens data and update graphic window.
3 ! It can be used to create animation.
4
5 surface = NSUR()-1      # serial number of the second to the last surface
6 z0 = THIC(surface)      # the original thickness of that surface
7
8 str1$ = "Please enter the shift from focus (0 ~ " + $STR(z0) + "):"
9 str2$ = "Please enter the total steps (positive integer): "
10
11 ! let user define the offset and total steps
12 LABEL 1
13 INPUT str1$, shift
14 IF ((shift < 0) | (shift > z0)) THEN GOTO 1
15
16 LABEL 2
17 INPUT str2$, stepNum
18 IF (stepNum < 1) THEN GOTO 2
19
20 stepNum = INTE(stepNum)
21
22 startZ = z0 - shift
23 endZ = z0 + shift
24 stepSize = shift*2/stepNum
25
26 prefix$ = "D:\My Macros\ch4\ex40102 shift "
27 ext$ = ".WMF"
28
29 FOR z = startZ, endZ, stepSize
30     FORMAT 3.1
31     shift$ = $STR(z-z0)
32     fileName$ = prefix$ + shift$ + ext$
33     value = z
34     SURP surface, THIC, value
35     UPDATE 1      # assume serial number of the graphic window is 1
36     EXPORTBMP 1, fileName$
37 NEXT
38
39 value = z0      # put back the original thickness
40 SURP surface, THIC, value
41 UPDATE ALL
42
43 CLOSEWINDOW

```

这个程序也是一个通用程序。作为说明，我们假定当前光学系统为例 3.4-1 中所定义的系统。运行程序之前，在 ZEMAX 中打开 Spot Diagram 图形分析窗口，并按图 4.1-2 进行设定：

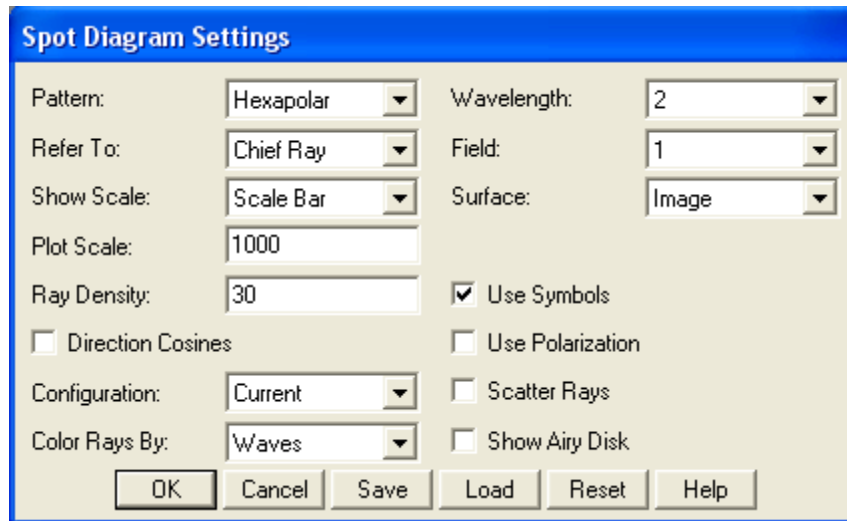


图4.1-2: Spot Diagram 图形分析窗口设定

在程序运行过程中，首先让用户输入像平面移动的范围和步数(第12~18行)。程序对用户输入的数据进行判断，要求移动范围为正值但不能大于像平面前一表面厚度，并要求移动步数为正值。如果用户输入数据不符合要求，则会被要求重新输入。程序第20行保证输入的步数为整数。接下来，程序计算像平面起始坐标、终止坐标和步长(第22~24行)，然后在每一次循环中，根据像平面的偏移值生成一个新的文件名(第32行)，改变像平面的位置(第34行)，更新Spot Diagram 图形分析窗口(第35行)，并将其内容输出到指定文件中(第36行)。程序第39~41行恢复原来的像平面位置，其目的是保证原有光学系统不受程序影响。

程序运行中我们可以看Spot Diagram 图形分析窗口中的光斑尺寸在发生改变。程序运行结束后，指定文件夹中出现了下列文件(假设我们的输入为 $\text{shift} = 5$, $\text{step} = 10$):

```
ex40102 shift -5.0.BMP
ex40102 shift -4.0.BMP
ex40102 shift -3.0.BMP
ex40102 shift -2.0.BMP
ex40102 shift -1.0.BMP
ex40102 shift 0.0.BMP
ex40102 shift 1.0.BMP
ex40102 shift 2.0.BMP
ex40102 shift 3.0.BMP
ex40102 shift 4.0.BMP
ex40102 shift 5.0.BMP
```

其中第1、3、6个文件的内容如图4.1-3所示:

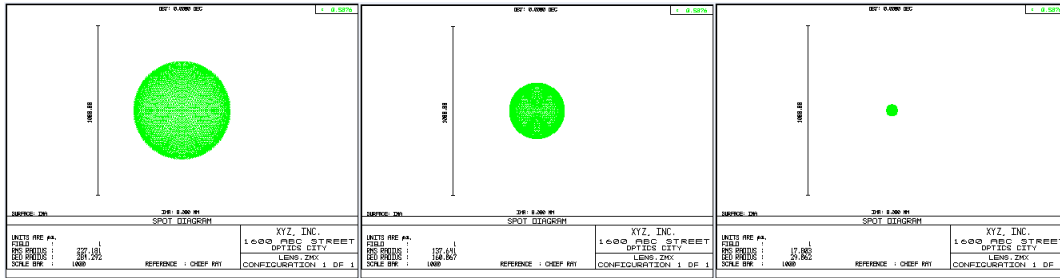


图4.1-3: 程序 ex40102.ZPL 部分输出文件内容

利用程序执行后所输出的这些文件，不难合成一段模拟动画，演示光斑大小如何随着像平面的移动而发生改变，这里就不再赘述。

例4.1-3: 几何光束与高斯光束的比较。

在本例中，我们通过比较几何光束方法与高斯光束方法得到的各个表面上的尺寸（边缘光线与表面相交点的 Y 坐标），介绍了如何读取 ZEMAX 的分析数据。我们假定当前光学系统为例 3.4-1 中所定义的系统。由于 ZEMAX 计算高斯光束时采用的缺省光斑半径为 0.05 镜头单位，因此我们需要根据具体的系统修改这个参数。在 ZEMAX 下新建一个镜头文件，运行程序 EX30401.ZPL，然后通过 Analysis → Physical Optics → Paraxial Gaussian Beam 打开一个近轴高斯光束分析窗口，按鼠标右键调出其设置对话框，将束腰尺寸改为 25，如图 4.1-4 所示，然后保存设置。

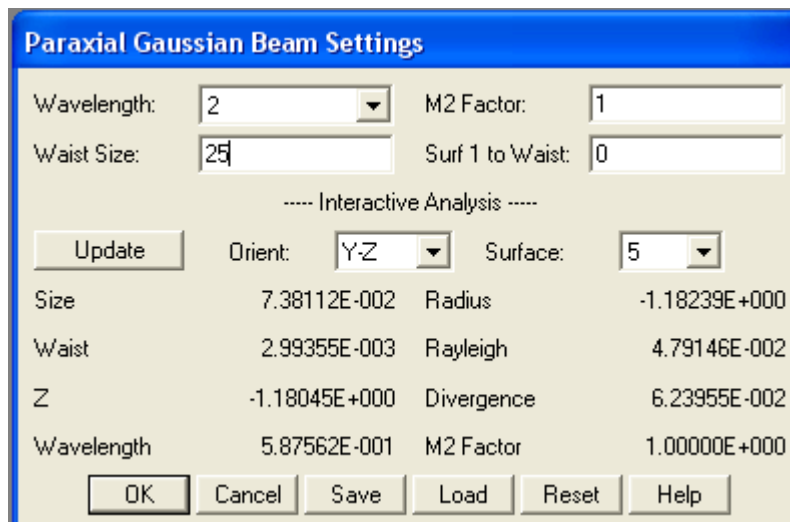


图4.1-4: 例4.1-3 光学系统的近轴高斯光束设置

```

1 ! ex40103
2 ! This program compares the Geometrical beam size and Gaussian beam size
3 ! Assume the lens system is defined in ex30401
4
5 PI = 3.14159265
6
7 ! define a marginal ray
8 hx = 0
9 hy = 0
10 px = 0
11 py = 1
12
13 ! trace the ray for entire system:
14 RAYTRACE hx, hy, px, py
15
16 ! obtain Gaussian beam data
17 A$ = $TEMPFILENAME()      # get a temporary file to store Gaussian beam data
18 GETTEXTFILE A$, Gbp       # store Gaussian beam data in the temp file
19 OPEN A$                   # open the newly created temp file
20 row = 0                   # line number in the file
21 s = 0                     # surface number
22
23 LABEL 1
24
25 READSTRING B$             # read a line in the file
26 row = row + 1             # line number
27 IF row < 28 THEN GOTO 1    # line 28 is the data of the first surface
28 s = s + 1                 # surface number
29 temp$ = $GETSTRING(B$, 2) # the second sub-string
30 VEC1(s) = SVAL(temp$)     # convert to number, and stored in an array
31 temp$ = $GETSTRING(B$, 1) # the first sub-string
32 IF (temp$ <> "IMA") THEN GOTO 1 # repeat until reaching IMA surface
33 CLOSE                     # close file
34 DELETEFILE A$            # delete the temp file
35

```

程序第 14 行对一条边缘光线进行了追迹，用于获取几何光束大小，即边缘光线与各个表面交点的 Y 坐标。程序第 17~34 行用于获取高斯光束数据，其中，第 17 行由 ZEMAX 自动生成一个临时文件，第 18 行将近轴高斯光束分析数据窗口的内容存于临时文件中，第 19 行打开所保存的文件进行读取，第 25 行读取文件中的一整行。由于文件的格式是固定的，我们知道其中的第 28 行为第一个表面的数据，所以如果行数小于 28，则转回到标志 1 处继续读取下一行，直到读到文件第 28 行为止，然后程序第 29、31 行将读取行中的第 2 个字符串(高斯光束半径)转换为数值，保存于缺省数组 VEC1 中。程序第 31、32 行判断读取行中的第 1 个字符串是否为“IMA”，如果是，则表明已经读到最后一个表面的数据，否则转回到标志 1 处继续读取下一行。读完最后一个表面的数据后，程序第 33 行关闭所保存的临时文件，第 34 行删除此文件。

```

35
36 ! print header
37 PRINT
38 PRINT "surf    Geometrical    Gaussian"
39 PRINT "=====
40
41 FOR s = 1, NSUR(), 1
42   FORMAT 1 INT
43   PRINT " ", s,
44   FORMAT 6.4
45   PRINT " ", RAGY(s), " ", VEC1(s)
46 NEXT
47
48 PRINT "=====
49

```

程序第 37~48 行用于在屏幕上打印前面得到的结果，其中函数 RAGY() 为几何光线追迹的结果，数组 VEC1() 为文件读取的高斯光束结果。

程序运行的结果如图 4.1-5 所示：

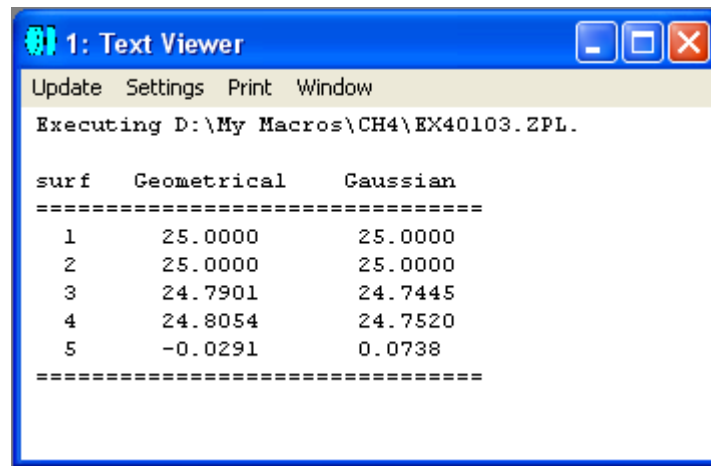


图 4.1-5: 程序 ex40103.ZPL 的运行结果

例4.1-4：玻璃材料透过率特性比较。

在光学设计中，有时候我们会需要寻找一些具有特殊透过率特性的玻璃材料。例如，我们可能会希望某种材料在可见光波段(400nm ~ 700nm)具有最大透过率，而在紫外及红外波段具有最大吸收率。在本例中，我们就将对 ZEMAX 所提供的一系列玻璃库中的上千种玻璃进行比较，寻找这样的玻璃材料。假定波长范围为 380

~ 1000nm, 我们希望在 380 ~ 400nm 及 700 ~ 1000nm 范围内透过率最小, 而在 400 ~ 700nm 范围内透过率最大。假设玻璃材料的厚度为 1mm, 我们将计算在不同波长下的透过率值, 并将在 380 ~ 400nm 及 700 ~ 1000nm 范围内的透过率值改为吸收率值, 然后求出所有这些值的方均根, 将此值定义为玻璃材料的评价值。最后, 我们对所有的玻璃材料按其评价值从大到小进行排序。根据我们的定义, 评价值越大的玻璃, 越接近我们的要求。

```
1 ! ex40104
2 ! This program compares different glasses to find the best profile.
3 ! Assume the target is to get lowest transmission in 380~400nm, 700~1000nm
4 ! and get highest transmission in 400~700nm
5
6 ! define some parameters
7 startWav = 0.38      # start wavelength, 380 nm
8 stopWav = 1          # stop wavelength, 1000 nm
9 stepNum = 500        # total steps between start and stop wavelengths
10 gm = 0              # initial total glass number of all the catalogs
11
12 saveName$ = "D:\My Macros\ch4\ex40104.txt"
13
14 SETVECSIZE 10000
15
16 GOSUB calculate      # calculate the merit value for each glass
17 GOSUB sortResult     # sort the results based on merit value
18 GOSUB reportResult   # print result on screen and save in a file
19
20 END
21
22
```

程序第 1 ~ 20 行为主程序, 其中, 第 7 ~ 10 行定义一些基本参数, 第 12 行为输出文件名。我们用缺省数组来存储玻璃材料的数据。由于总的玻璃材料数目超过 1000, 所以我们在程序第 14 行将缺省数组的大小增加为 10000。程序第 16 行调用子程序 `calculate` 计算各种玻璃材料的评价值, 第 17 行调用子程序 `sortResult` 对结果按评价值进行排序, 第 18 行调用子程序 `reportResult` 输出最后结果。

```

22
23 SUB calculate
24
25 originalWave = WAVL(1)      # store the original first wavelength
26
27 FOR cNum = 1, 16, 1          # go through the catalogs
28   GOSUB getCatalog          # set catalog
29   g = 1                      # initial glass number
30   LABEL 1
31   merit = 0                  # initial merit
32   FOR s = 1, stepNum+1, 1    # go through the wavelengths
33     w = startWav+(s-1)*(stopWav-startWav)/stepNum
34     SYSP 202, 1, w           # set the wavelength
35     GETGLASSDATA 1, g        # store glass data in VEC1
36     alpha = VEC1(23)         # Internal transmission coefficient
37     IF alpha < 0 THEN alpha = -1*alpha    # correct some catalog errors
38     t = EXPE(-1*alpha)       # transmission, assume 1 mm length
39     IF ((w < 0.4) || (w > 0.7)) THEN t = 1-t    # need absorption
40     merit = merit+t*t        # updated merit
41   NEXT s
42   gm = gm + 1                # total glass number increase by 1
43   VEC2(gm) = cNum            # store catalog number
44   VEC3(gm) = g               # store glass number of the catalog
45   VEC4(gm) = SQRT(merit)     # store merit value
46   FORMAT 1 INT
47   REWIND
48   PRINT gm, " processed. ", "Now processing glass ", g, " of ", cName$
49   g = g + 1                  # glass number of the catalog increase by 1
50   GETGLASSDATA 1, g          # store new glass data
51   IF VEC1(1) > 0 THEN GOTO 1  # if formula is valid, go back to process
52 NEXT cNum
53
54 SYSP 202, 1, originalWave    # recover the original first wavelength
55
56 RETURN
57

```

程序第 23~56 行为子程序 calculate，用于计算玻璃材料的评价值。由于我们需要改变系统设置中第一个工作波长的值，因此，程序第 25 行先保留原有的波长值，计算完成后，程序第 54 行再恢复原有的波长值，这样原有的光学系统就可以不受影响。程序第 27~52 行的循环，访问了共 16 个玻璃目录，其中第 28 行调用子程序 getCatalog 按目录序号调入了该目录，第 32~41 行的循环计算了不同波长下的透过率或吸收率，并计算了评价值(注意此处的评价值还没有开平方)。需要指出的是，作者发现，ZEMAX 提供的玻璃库数据中，有些时候透过率系数 alpha 为正值，也有的时候为负值，所以第 37 行统一将它们改为正值，然后第 38 行再按正的透过率系数进行计算。第 39 行判断如果波长在可见光范围之外，则用吸收率而不是透射率参与计算玻璃评价值。在计算完一种玻璃的评价值后，程序第 42 行将玻璃总数增加 1，第 43~45 行再将此种玻璃的目录序号、玻璃在目录中的序号及评价值分别存入缺省数组 VEC2、VEC3 及 VEC4 的相应元素中。由于计算过程需要较长时间，因此程序第 46~48 行在屏幕上显示当前进程，其中用到关键词 REWIND 使显示总在同一行。图 4.1-6 为程序运行过程中的某一时刻显示内容。随

后，第 49 行将目录中的玻璃序号加 1，第 50 行读取新的玻璃参数，第 51 行判断如果玻璃有效(我们采用的方法是判断其色散公式种类代码，如果是有效玻璃，其色散公式代码为一个正整数，而无效玻璃代码为 0)，则跳转到标号 1 处从头开始处理新的玻璃，否则进入循环中的下一个玻璃库。

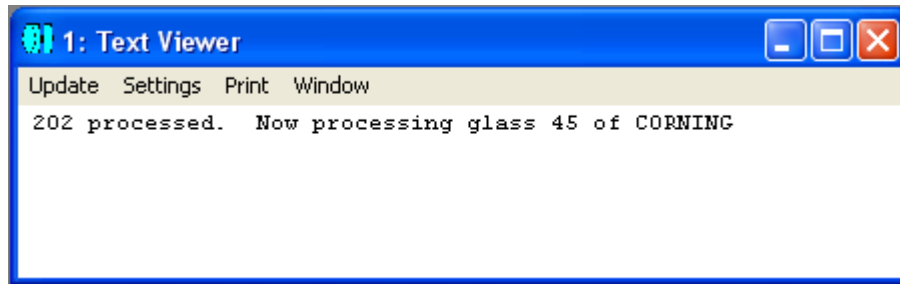


图 4.1-6 玻璃评价值计算过程中的进程显示

```

57
58 SUB getCatalog
59
60 IF cNUM == 1 THEN cName$ = "ARCHER"
61 IF cNUM == 2 THEN cName$ = "CDGM"
62 IF cNUM == 3 THEN cName$ = "CORNING"
63 IF cNUM == 4 THEN cName$ = "HERAEUS"
64 IF cNUM == 5 THEN cName$ = "HIKARI"
65 IF cNUM == 6 THEN cName$ = "HOYA"
66 IF cNUM == 7 THEN cName$ = "LIGHTPATH"
67 IF cNUM == 8 THEN cName$ = "LZOS"
68 IF cNUM == 9 THEN cName$ = "OHARA"
69 IF cNUM == 10 THEN cName$ = "PILKINGTON"
70 IF cNUM == 11 THEN cName$ = "RPO"
71 IF cNUM == 12 THEN cName$ = "SCHOTT"
72 IF cNUM == 13 THEN cName$ = "SUMITA"
73 IF cNUM == 14 THEN cName$ = "TOPAS"
74 IF cNUM == 15 THEN cName$ = "UMICORE"
75 IF cNUM == 16 THEN cName$ = "ZEON"
76
77 SYSP 23, cName$
78 LOADCATALOG
79
80 RETURN
81
82

```

程序第 58~80 行为子程序 getCatalog，其作用是根据玻璃目录编号，赋予相应的目录名称，然后调入该目录。

```

82
83 SUB sortResult
84
85 FOR i = 1, gm, 1           # go through all the stored numbers
86   FOR j = i, 1, -1         # go through all processed numbers
87     IF j > 1               # if j is not the first
88       IF VEC4(j) > VEC4(j-1) # if current merit is larger
89         temp = VEC4(j-1)
90         VEC4(j-1) = VEC4(j)
91         VEC4(j) = temp
92         temp = VEC3(j-1)
93         VEC3(j-1) = VEC3(j)
94         VEC3(j) = temp
95         temp = VEC2(j-1)
96         VEC2(j-1) = VEC2(j)
97         VEC2(j) = temp
98       ENDIF
99     ENDIF
100   NEXT j
101   FORMAT 1 INT
102   REWIND
103   PRINT i, " of total ", gm, " glasses sorted"
104 NEXT i
105
106 RETURN
107
108

```

程序第 83~106 行为子程序 sortResult，其作用是将存入缺省数组中的各种玻璃，按照存入 VEC4 中的评价价值从大到小进行排序，排序的方法为常用的冒泡排序法，这里就不赘述。同样，程序第 101~103 行也在排序过程中显示了进程，如图 4.1-7 所示。

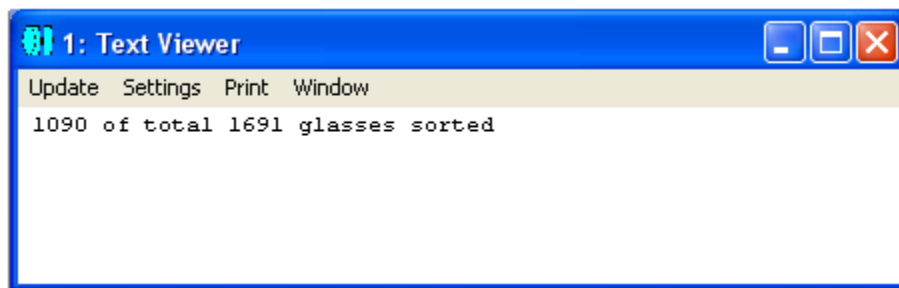


图 4.1-7 玻璃排序过程中的进程显示

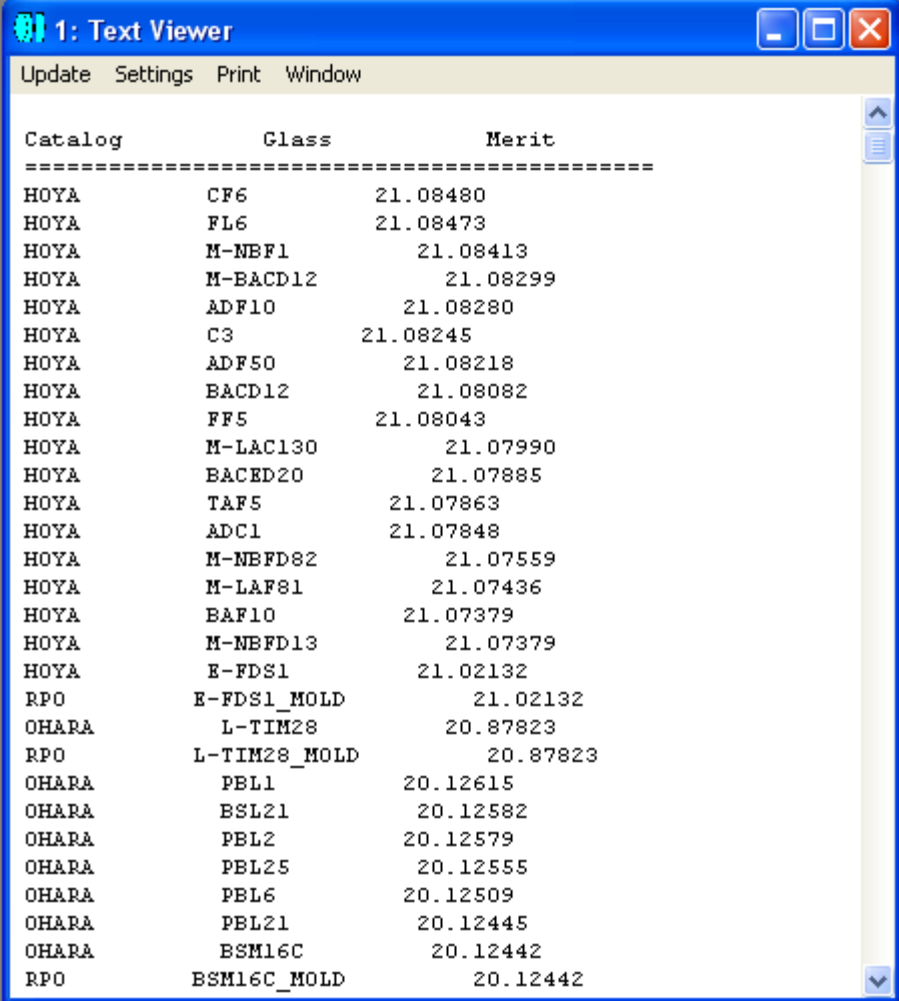
```

108
109 SUB reportResult
110
111 INSERT 1                      # insert a surface temporarily
112
113 REWIND
114 PRINT
115 PRINT "Catalog           Glass           Merit           "
116 PRINT "===== "
117 FORMAT 6.5
118 FOR i = 1, gm, 1
119     cNum = VEC2(i)             # get the catalog number
120     GOSUB getCatalog           # load the catalog
121     g = VEC3(i)                # get the glass number in the catalog
122     SURF 1, GLAN, g            # set the glass material for the surface
123     glassName$ = $GLASS(1)     # read back the glass name
124     PRINT cName$, "           ", glassName$, "           ", VEC4(i)
125 NEXT
126
127 DELETE 1                      # delete the temporary surface
128
129 ! save the result
130 SAVEWINDOW 1, saveName$       # assume there is no other windows open
131
132 RETURN
133

```

程序第 109 ~ 132 行为子程序 reportResult，用于输出计算结果。其中，第 113 ~ 116 行在屏幕上打印表头，第 118 ~ 125 行的循环打印出所有的排序过的玻璃目录名、玻璃名及评价值。由于我们只知道玻璃序号，但想要读取玻璃名，因此就在镜头数据编辑器中临时插入一个表面(第 111 行)，第 122 行将表面材料类型设置为已知序号的玻璃材料，然后第 123 行再通过函数 \$GLASS() 读回玻璃材料名称。当所有的玻璃材料都显示完后，程序第 127 行删除临时加入的表面，还原当前光学系统。程序最后第 130 行将显示在屏幕文本窗口中的内容保存到指定文件中。

图 4.1-8 所示为程序运行后屏幕文本窗口中显示的部分内容。需要指出的是，我们的程序假定 ZEMAX 所给出的玻璃库数据均正确有效，但有些时候这个假设不一定成立。因此，程序运行结果的正确性，需要设计者根据自己的要求进行判断。



Catalog	Glass	Merit
HOYA	CF6	21.08480
HOYA	FL6	21.08473
HOYA	M-NBF1	21.08413
HOYA	M-BACD12	21.08299
HOYA	ADF10	21.08280
HOYA	C3	21.08245
HOYA	ADF50	21.08218
HOYA	BACD12	21.08082
HOYA	FF5	21.08043
HOYA	M-LAC130	21.07990
HOYA	BACED20	21.07885
HOYA	TAF5	21.07863
HOYA	ADC1	21.07848
HOYA	M-NBFD82	21.07559
HOYA	M-LAF81	21.07436
HOYA	BAF10	21.07379
HOYA	M-NBFD13	21.07379
HOYA	E-FDS1	21.02132
RPO	E-FDS1_MOLD	21.02132
OHARA	L-TIM28	20.87823
RPO	L-TIM28_MOLD	20.87823
OHARA	PBL1	20.12615
OHARA	BSL21	20.12582
OHARA	PBL2	20.12579
OHARA	PBL25	20.12555
OHARA	PBL6	20.12509
OHARA	PBL21	20.12445
OHARA	BSM16C	20.12442
RPO	BSM16C_MOLD	20.12442

图 4.1-8 程序 ex40104.ZPL 运行后屏幕文本窗口中显示的部分内容

最后，作为验证，我们给出上图结果中排序由前到后的三种玻璃材料CF6、BK3 及 SF5，将其透过率进行比较，如图 4.1-9 所示。从图中可以看到，我们得到的排序结果是合理的。图 4.1-9 中的透过率数据来自 ZEMAX 玻璃目录库，获取的具体方法我们将在下一个例子中说明。

留给读者思考的问题是，在实际应用中，也许用两种不同的玻璃组合起来实现所需要的透过率特性会得到更好的效果，那么，通过程序如何选取最佳组合呢？

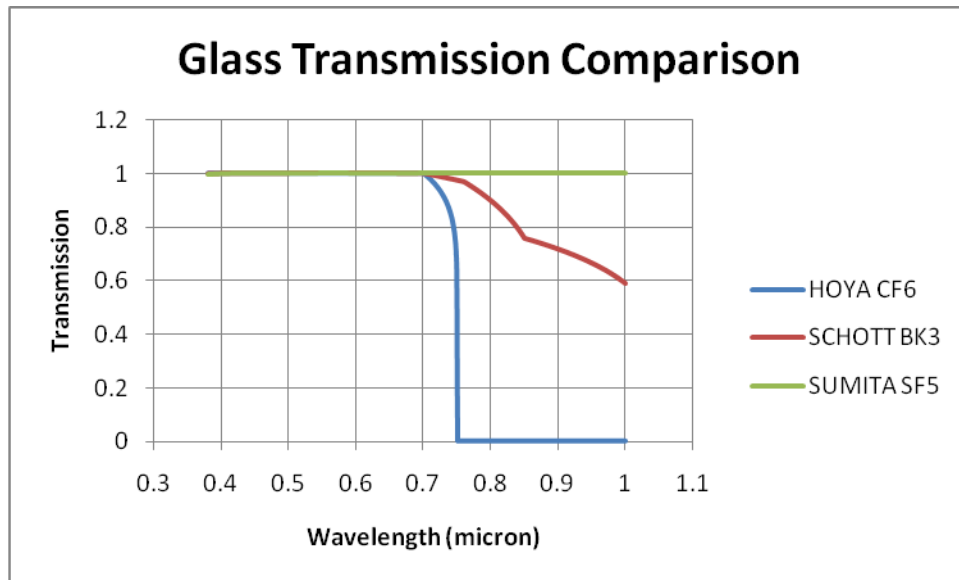


图 4.1-9 厚度为 1mm 的玻璃透过率比较

例4.1-5：玻璃材料折射率和透过率的读取。

ZEMAX 提供的目录库给出了一系列玻璃材料的折射率、透过率等各种参数，在实际应用中很有参考价值。我们可以编写一个简单的程序，很容易地得到指定材料的折射率和透过率数据。

在下面的程序中，第 8~12 行通过提示窗口让用户输入玻璃材料名称、玻璃厚度、起始波长、终止波长及间隔步数，第 15 行将波长1设为主波长，因为后面第 26 行要读取主波长的折射率值。程序第 21~34 行的循环中，第 22 行计算了各个波长值，第 23 行将表面 1 设为用户指定的玻璃材料，第 24 行将波长 1 (即主波长) 设为计算值，第 25 行更新系统使设置生效，第 26 行通过函数 `INDX()` 读取表面 1 主波长相应的折射率值，第 27~29 行读取玻璃的透射率系数，和上一个例子一样，第 30 行对透射率系数做一个判断，将其统一按正值处理，然后第 31 行计算指定厚度玻璃材料的透过率，第 33 行将结果列表输出到屏幕上。程序最后第 36 行将屏幕文本窗口的内容保存到指定文件中。在上一个例子的图 4.1-9 中，我们就用到了通过这个程序读取的数据。

在程序运行之前建议先新建一个镜头文件，程序运行结束后不必保存镜头文件。

```

1 ! ex40105
2 ! This program lists refractive index and internal transmission of glass
3 ! The glass type and wavelength range is given by user
4
5 saveName$ = "D:\My Macros\ch4\ex40105.txt"
6
7 ! let user define the glass type and wavelength range
8 INPUT "Please enter the glass type", glassType$
9 INPUT "Please enter the glass thickness (mm)", glassThickness
10 INPUT "Please enter the starting wavelength (um): ", startWave
11 INPUT "Please enter the stopwavelength (um): ", stopWave
12 INPUT "Please enter the number of steps: ", stepNum
13
14 ! set the first wavelength as primary wavelength
15 PWAV 1
16
17 PRINT
18 PRINT "Wavelength(um)      Refractive Index      Internal Transmission"
19 PRINT "=====
20
21 FOR i = 1, stepNum+1, 1
22   w = startWave+(i-1)*(stopWave-startWave)/stepNum # calculate wavelength
23   SURF 1, GLAS, glassType$ # set the glass material for surface 1
24   SYSP 202, 1, w # set the new wavelength for the primary
25   UPDATE
26   index = INDX(1) # get the refractive index of primary wave at surface 1
27   glassType$ = $GLASS(1)
28   GETGLASSDATA 1, GNUM(glassType$) # store glass data in VEC1
29   alpha = VEC1(23) # Internal transmission coefficient
30   IF alpha < 0 THEN alpha = -1*alpha # correct some catalog errors
31   t = EXPE(-1*alpha*glassThickness) # transmission, length unit is mm
32   FORMAT 8.7
33   PRINT " ", w, " ", index, " ", t
34 NEXT
35
36 savewindow 1, saveName$

```

第二节 非顺序光学系统

例4.2-1：导光管。

在光学设计中，常常会用到导光管。最简单的导光管就是一段圆柱，可以用玻璃或者塑料等光学材料做成，光从圆柱的一头输入，从另一头输出。导光管的横截面除了圆形以外，还可以是其它形状，如三角形、矩形、六角形等。在设计不同形状不同长度的导光管时，通常需要考虑耦合效率、光斑均匀性等。在本例中，我们就比较了不同形状和不同长度导光管的最大输出光强、总光通量及光斑非均匀性。

首先，我们假设有一个圆形平面 Lambertian 光源，置于坐标原点，其尺寸小于导光管横截面积，其法线方向为 +Z 方向。另有一导光管，其横截面面积恒定，形状可能为正三、四、五、六、七、八边形，长度可能为10、40、160、640 镜头单位，材料为 Acrylic，起点在 $Z = 0$ 处，长度方向沿 +Z 方向。最后我们再将一矩形探测器置于导光管之后，探测器尺寸略大于导光管横截面外接圆尺寸。整个系统如图 4.2-1 所示：

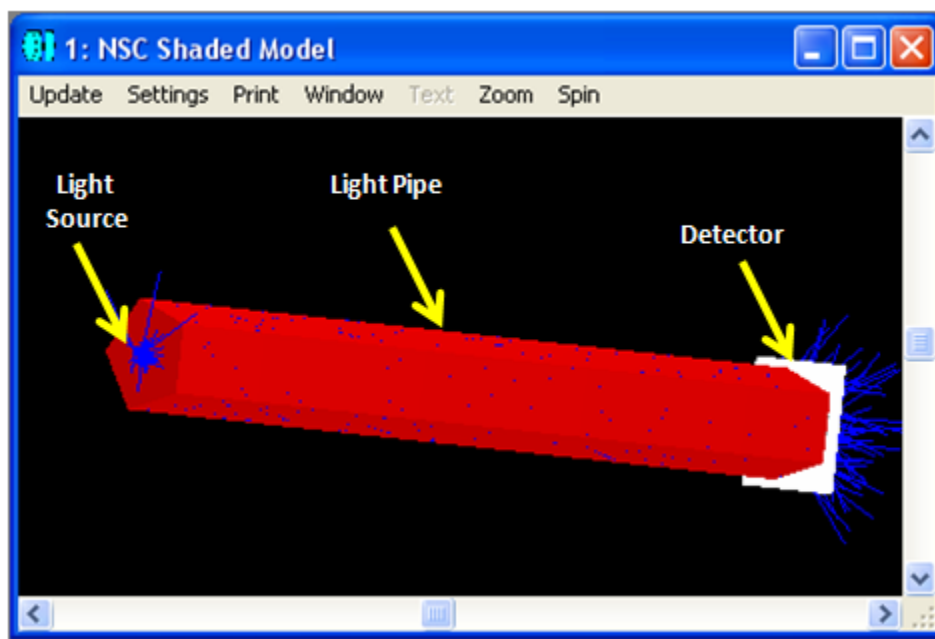


图 4.2-1：程序 ex40201.ZPL 中的光学系统

由于这个光学系统很简单，我们可以很容易地在程序中从头开始构造整个光学系统，然后通过循环改变系统的参数并进行光线追迹，最后再将结果输出。

```

1 ! ex40201
2 ! This program compares output of different shape and length light pipes.
3 ! Assume the mode is total Non-Sequential Mode.
4
5 ! define constant and parameters
6 PI = 3.14159265
7 objPath$ = "D:\My Macros\ch4\"
8 fileName$ = "polygon.txt"
9 outputName$ = objPath$ + fileName$           # polygon UDA file
10 crossArea = 1                                # the area of the cross section of polygon
11 sourceRadius = 0.1                            # the radius of a circular source
12 pNum = 100*100                                # total pixel number
13 rayNum = 1000000                              # number of rays to be traced
14
15 ! define result as a 3-dimension array
16 lengthNum = 4      # assume lengths are 10, 40, 160, 640
17 sideNum = 8         # assume sides are 3, 4, 5, 6, 7, 8
18 paraNum = 3         # assume to analyze maxFlux, totalFlux, nonUniformity
19 DECLARE result, DOUBLE, 3, lengthNum, sideNum, paraNum
20
21 ! clean and prepare the NSC editor
22 GOSUB prepareEditor
23
24 ! define the first object as Source Ellipse
25 GOSUB defineSource
26
27 ! go through different polygons
28 FOR n = 3, 8, 1      # different n sided polygons
29     FOR m = 1, lengthNum, 1  # go through different lengths
30         GOSUB createPolygon
31         GOSUB createDetector
32         GOSUB analyze
33     NEXT m
34 NEXT n
35
36 ! print result
37 GOSUB reportResult
38
39 END

```

程序的第 1~39 行为主程序，其中，第 6~13 行定义了程序中需要用到的一些常数和基本参数，第 16~19 行定义了一个 3 维数组用于存放计算结果。接下来，程序第 22 行调用子程序 prepareEditor 将非顺序元件编辑器设置好，第 25 行调用子程序 defineSource 设置光源，第 28~34 行进入循环，通过改变导光管横截面正多边形的边数和导光管长度，对不同的导光管进行光线追迹并对结果进行分析，其中第 30 行调用子程序 createPolygon 设置导光管，第 31 行调用子程序 createDetector 设置探测器，第 32 行调用子程序进行光线追迹并分析结果。程序第 37 行调用子程序 reportResult 在屏幕上输出结果。我们在整个程序中，通过模块化的设计，将大量具体的设置放到子程序中，这样，整个主程序就显得简单明了，易于理解和调试。


```

40
41
42 SUB prepareEditor
43
44 totalObjNum = NOBJ(1)
45 FOR i, totalObjNum, 1, -1
46     DELETEOBJECT 1, i      # delete all the objects
47 NEXT                      # there is still a null object in the editor at last
48
49 ! add two more objects in the editor, so the total is 3
50 INSERTOBJECT 1, 1
51 INSERTOBJECT 1, 1
52
53 RETURN prepareEditor
54
55

```

程序第 42 ~ 53 行为子程序 `prepareEditor`，其作用是先删除非顺序元件编辑器中所有的元件(删除后只剩下一个空元件)，然后另插入两个空元件，这样编辑器中最后就有 3 个空元件。我们在程序第 53 行 `RETURN` 之后加上子程序名 `prepareEditor`，主要是便于阅读，实际上对 ZEMAX 而言，`RETURN` 之后有没有子程序名都是一样的。

```

55
56 SUB defineSource
57
58 ! define the source type
59 SETNSCPROPERTY 1, 1, 0, 0, "NSC_SRCE"
60
61 ! define the position
62 SETNSCPOSITION 1, 1, 3, -0.1      # z
63
64 ! define the source parameters
65 SETNSCPARAMETER 1, 1, 1, 100      # Layout Rays
66 SETNSCPARAMETER 1, 1, 2, rayNum    # Analysis Rays
67 SETNSCPARAMETER 1, 1, 3, 1        # Power
68 SETNSCPARAMETER 1, 1, 6, sourceRadius # X Half Width
69 SETNSCPARAMETER 1, 1, 7, sourceRadius # Y Half Width
70 SETNSCPARAMETER 1, 1, 9, 1        # Cosine Exponent, assume Lambertian
71
72 RETURN defineSource
73
74

```

程序第 56 ~ 72 行为子程序 `defineSource`，其作用是将非顺序元件编辑器中的第一个元件定义为圆形光源，并进行设置。

```

73
74
75 SUB createPolygon
76
77 ! calculate r1(from center to a side) and r2(from center to a vertex)
78 r1 = SQRT(crossArea/(n*TANG(PI/n)))
79 r2 = r1/COS(PI/n)
80
81 ! write the polygon UDA file
82 OUTPUT outputName$
83 PRINT "POL 0 0 ",
84 FORMAT 8.7
85 PRINT r2,
86 FORMAT 1.0
87 PRINT " ", n, " 0"
88 PRINT "BRK"
89 OUTPUT SCREEN
90
91 ! set the second object as extruded
92 SETNSCPROPERTY 1, 2, 0, 0, "NSC_EXTR"
93
94 ! define the filename in the comment
95 SETNSCPROPERTY 1, 2, 1, 0, fileName$ # notice the path is not needed
96
97 ! define the material
98 SETNSCPROPERTY 1, 2, 4, 0, "ACRYLIC"
99
100 ! define the polygon parameters
101 length = 10*POWR(4, m-1)
102 SETNSCPARAMETER 1, 2, 1, length # length
103 SETNSCPARAMETER 1, 2, 2, 1 # front x scale
104 SETNSCPARAMETER 1, 2, 3, 1 # front y scale
105 SETNSCPARAMETER 1, 2, 4, 1 # rear x scale
106 SETNSCPARAMETER 1, 2, 5, 1 # rear y scale
107
108 RETURN createPolygon
109
110

```

程序第 75~108 行为子程序 createPolygon，其作用是将非顺序元件编辑器中的第二个元件定义为多边形导光管，并进行设置。在这里我们生成多边形导光管的方法是利用 ZEMAX 中的 Extruded 类型元件，通过用户自定义光阑文件(UDA)来生成正多边形并将其延伸为导光管。程序中，第 78 行计算出正多边形内切圆半径，第 79 行计算出正多边形外接圆半径，第 82~89 行定义 UDA 文件，注意第 82 行将打印输出设为文件方式，第 89 行设回为屏幕方式，这样就保证了在后面的程序中，特别是在调试时，打印输出为缺省的屏幕方式。程序第 92 行设置元件类型，第 95 行指定 UDA 文件，ZPL 规定此文件不需指定路径名，应放在 objects 文件夹中(由 ZEMAX 主菜单下的 File → Preferences → Directories 设定)，在此例中为 “D:\My Macros\ch4”。程序第 98~106 行设置材料及其它参数。

```

109
110
111 SUB createDetector
112
113 ! define the type of the third object as Detector Rect
114 SETNSCPROPERTY 1, 3, 0, 0, "NSC_DETE"
115
116 ! define the position of the second object
117 z = length + 0.01
118 SETNSCPOSITION 1, 3, 3, z      # z
119
120 ! define the detector parameters
121 halfWidth = r2*1.1      # set the detector slightly larger than the pipe
122 SETNSCPARAMETER 1, 3, 1, halfWidth      # X Half Width
123 SETNSCPARAMETER 1, 3, 2, halfWidth      # Y Half Width
124 SETNSCPARAMETER 1, 3, 3, 100           # Number of X Pixels
125 SETNSCPARAMETER 1, 3, 4, 100           # Number of Y Pixels
126
127 RETURN createDetector
128
129

```

程序第 111 ~ 127 行为子程序 createDetector，设置了探测器的类型、位置、尺寸及像素数目。

```

128
129
130 SUB analyze
131
132 temp = NSDD(1, 0, 0, 0)      # clear the detector
133 NSTR 1, 1, 1, 1, 1, 1, 1, 0  # ray tracing
134
135 maxFlux = 1E-8
136 sumFlux = 0
137 sumSquare = 0
138 totalCount = 0
139 FOR pix = 1, pNum, 1
140   pFlux = NSDD(1, 3, pix, 0) # check light flux on each pixel
141   IF maxFlux < pFlux THEN maxFlux = pFlux
142   IF pFlux > 1E-8
143     sumFlux = sumFlux + pFlux
144     sumSquare = sumSquare + pFlux*pFlux
145     totalCount = totalCount + 1  # only count those illuminated pixels
146   ENDF
147 NEXT
148
149 temp = totalCount*maxFlux*maxFlux-2*maxFlux*sumFlux+sumSquare
150 ripple = SQRT(temp)/(maxFlux*SQRT(totalCount))
151 result(m, n, 1) = maxFlux/(halfWidth*halfWidth*4/pNum) # intensity on pixel
152 result(m, n, 2) = sumFlux
153 result(m, n, 3) = ripple
154
155 RETURN analyze
156
157

```

程序第 130 ~ 155 行为子程序 analyze，其中，第 132 行清空探测器，第 133 行进行光线追迹，第 139 ~ 147 行逐个读取探测器上每个像素上的光通量(第 140 行)，判断如果当前像素光通量值大于最大光通量值，则取代最大光通量值(第 141 行)，其目的是找出单个像素上的最大光通量。另外在循环中还判断如果光通量不为 0 (第 142 行)，则将其记入总光通量 sumFlux 及 光通量平方和 sumSquare 中，并将有效像素数目增加1(第 143 ~ 145 行)。在子程序中，我们需要计算以下几个值，并将结果存入数组 result 中：单个像素上的最大光强，其方法是将单个像素上的最大光通量除以单个像素面积(第 151 行)；总光通量(第 152 行)；非均匀度 ripple (第 150 行)，我们将其定义为所有单个像素光通量 pFlux 与最大光通量 maxFlux 之差的均方根，其中用到关系 $\sum_i (pFlux_i - maxFlux)^2 = \sum_i (pFlux_i)^2 - 2 * \sum_i (pFlux_i) * maxFlux + \sum_i (maxFlux)^2 = sumSquare - 2 * sumFlux * maxFlux + totalCount * sumSquare$ 。

```

157
158 SUB reportResult
159
160 PRINT
161 PRINT "length side max-Flux total-Flux non-uniformity"
162 PRINT "-----"
163 FOR m = 1, lengthNum, 1
164   FOR n = 3, sideNum, 1
165     length = 10*POWER(4, m-1)
166     FORMAT 3.0
167     PRINT " ", length, " ", n, " ",
168     FORMAT 11.8
169     PRINT result(m, n, 1), " ", result(m, n, 2), " ", result(m, n, 3)
170   NEXT n
171   PRINT "-----"
172 NEXT m
173
174 RETURN reportResult
175
176

```

程序第 158 ~ 174 行为子程序 reportResult，其作用是将存于数组 result 中的结果输出到屏幕上。

整个程序的运行结果如图 4.2-2 所示：

1: Text Viewer

Update Settings Print Window

Executing D:\My Macros\CH4\EX40201.ZPL.

length	side	max-Flux	total-Flux	non-uniformity
10	3	0.94890384	0.83360122	0.47665470
10	4	0.98323281	0.83754776	0.43453269
10	5	1.03984896	0.83606445	0.43815834
10	6	0.98345285	0.83673602	0.42302788
10	7	1.19124229	0.83645425	0.48791640
10	8	1.01125039	0.83681343	0.42774033

40	3	0.94141587	0.83331077	0.47507123
40	4	0.97470743	0.83714738	0.43872912
40	5	0.99361068	0.83589607	0.43236967
40	6	1.00724617	0.83633069	0.43288871
40	7	1.06363062	0.83609228	0.44942962
40	8	0.98955022	0.83634466	0.42173299

160	3	0.94366142	0.83319181	0.47089640
160	4	0.96346162	0.83663494	0.43271867
160	5	0.98934567	0.83544672	0.42888529
160	6	0.98361326	0.83577082	0.42446393
160	7	1.02188224	0.83569761	0.43432436
160	8	0.99222065	0.83583068	0.42282320

640	3	0.92571785	0.83175683	0.46277315
640	4	0.97548388	0.83518598	0.42666194
640	5	0.98219971	0.83409849	0.42370388
640	6	0.98067980	0.83431529	0.41773588
640	7	0.99645881	0.83435704	0.42007982
640	8	1.01831913	0.83447147	0.42445987

图 4.2-2: 程序 ex40201.ZPL 的运行结果

注意因为当导光管比较长时，光线在导光管内发生全内反射的次数较多，因此 ZEMAX 系统设置中的单根光线最大相交次数及光线最大节数(System → General → Non-Sequential)可能需要进行调整，否则有很多光线会因为超过最大反射次数而被损耗掉，从而得不到正确结果。

从图 4.2-2 所示结果中可以看到，当导光管长到一定程度后，其输出光通量就会比较稳定，这很容易理解，因为光线在导光管中的传输主要通过全内反射，损耗很小。另外，还有人专门研究了导光管横截面形状与输出光均匀性的关系，认为如果能用多个横截面铺满整个平面(例如正六边形)，则相应的导光管输出光均匀性要好于其它横截面形状(例如正五边形、正七边形等)，而圆形导光管的均匀性最差。我们将这个问题留给有兴趣的读者去深入研究，在这里就不进行讨论了。

如果我们将程序稍作修改，则不难让程序输出探测器上看到的光斑形状，如图 4.2-3 所示，这里也不再赘述。

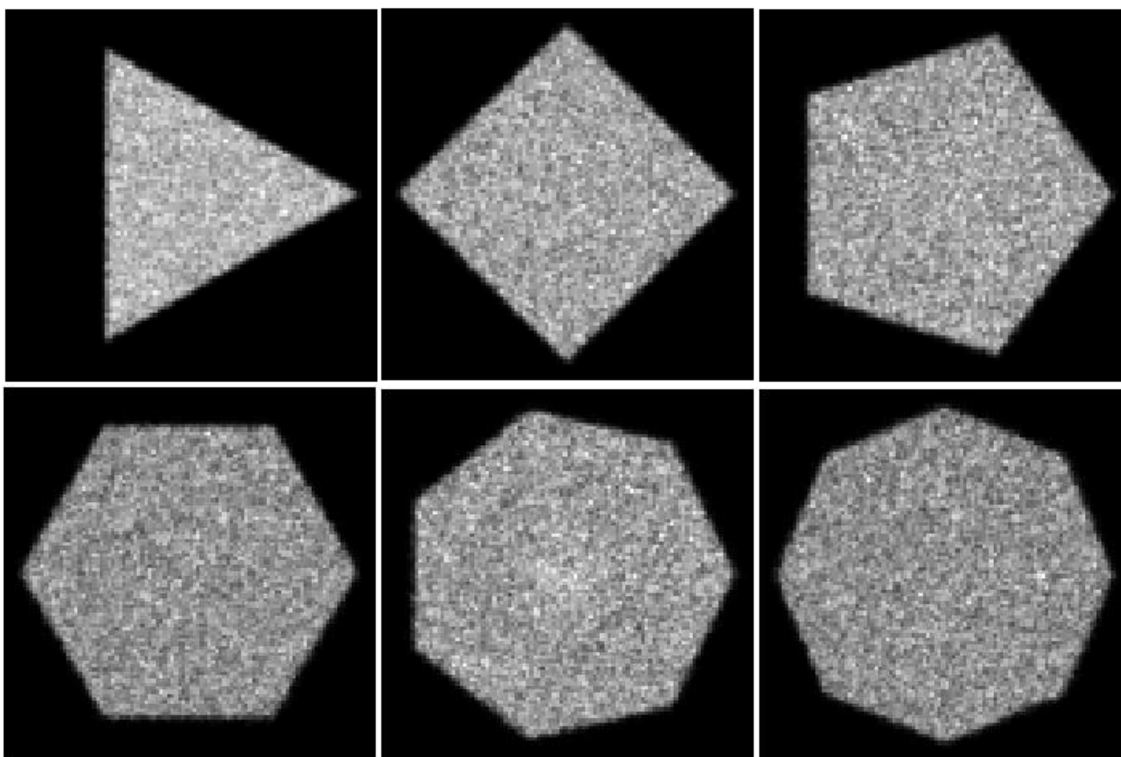


图 4.2-3: 程序 ex40201.ZPL 运行过程中探测器上看到的光斑形状

例4.2-2：余弦四次方规律。

我们知道，在光学成像系统中，即使出瞳均匀照明，没有渐晕，在像平面上，其中心与边缘的照度也会不同，遵循余弦四次方关系，即像平面上偏离中心轴 Θ 角度的点 H 其照度只有轴上点 A 的 $\cos^4(\Theta)$ 倍，如图 4.2-4 所示。

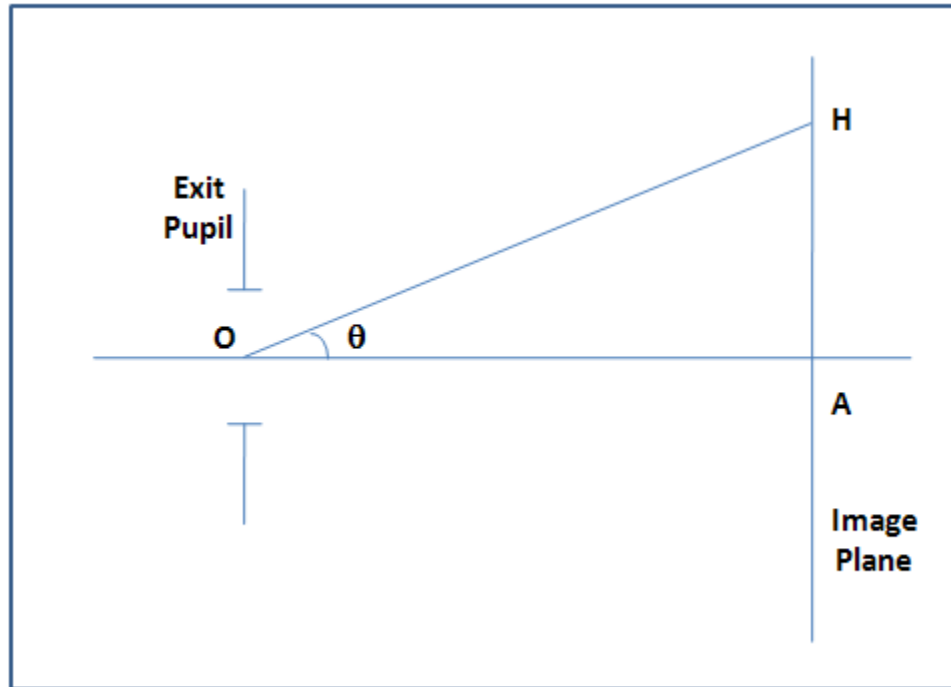


图 4.2-4：遵循余弦四次方规律的光学系统

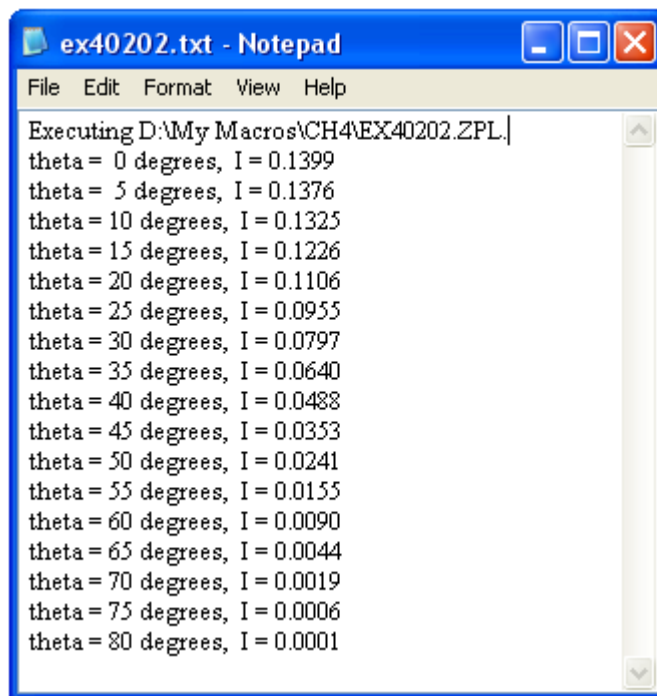
在本例中，我们用一个Lambertian 圆形光源来模拟均匀照明的出瞳，放置一个矩形探测器在成像面上，其 Y 坐标由角度 Θ 确定。我们通过改变角度 Θ 来研究点 H 处的光通量变化。

```

1 ! ex40202
2 ! this program is used to check cosine fourth rule
3
4 ! define some constant and parameters
5 PI = 3.14159
6 sourceR = 1          # source radius
7 detectorHW = 1       # detector half width
8 dO = 15              # on axis distance
9 rayNum = 10000000    # total rays to be traced
10 saveName$ = "D:\My Macros\ch4\ex40202.txt"
11
12 ! create optical system
13 INSERTOBJECT 1, 1    # so the total object number is 2
14
15 ! define the first object
16 SETNSCPROPERTY 1, 1, 0, 0, "NSC_SRCE"    # elliptical source
17 SETNSCPARAMETER 1, 1, 1, 100            # Layout Rays
18 SETNSCPARAMETER 1, 1, 2, rayNum         # Analysis Rays
19 SETNSCPARAMETER 1, 1, 3, 1              # Power
20 SETNSCPARAMETER 1, 1, 6, sourceR        # X Half Width
21 SETNSCPARAMETER 1, 1, 7, sourceR        # Y Half Width
22 SETNSCPARAMETER 1, 1, 9, 1              # Cosine Exponent, assume Lambertian
23
24 ! define the second object
25 SETNSCPROPERTY 1, 2, 0, 0, "NSC_DETE"    # detector rectangular
26 SETNSCPOSITION 1, 2, 3, dO               # z position
27 SETNSCPARAMETER 1, 2, 1, detectorHW      # X Half Width
28 SETNSCPARAMETER 1, 2, 2, detectorHW      # Y Half Width
29 SETNSCPARAMETER 1, 2, 3, 1              # Number of X Pixels
30 SETNSCPARAMETER 1, 2, 4, 1              # Number of Y Pixels
31
32 ! trace the rays
33 FOR theta = 0, 80, 5                      # in degrees
34   y = dO*TANG(theta*PI/180)
35   SETNSCPOSITION 1, 2, 2, y              # y position
36   temp = NSDD(1, 0, 0, 0)                # clear the detector
37   NSTR 1, 1, 1, 1, 1, 1, 1, 0           # ray tracing
38   FORMAT 2 INT                            # same as FORMAT 2.0
39   PRINT "theta = ", theta,
40   FORMAT 6.4
41   PRINT " degrees, I = ", NSDD(1,2,0,1)
42 NEXT theta
43
44 ! save the result
45 SAVEWINDOW 1, saveName$                  # assume there is no other windows open

```

假设程序运行之前新建一个非顺序光学系统，打开一个新的非顺序元件编辑器。程序第 13 行插入一个空元件，这样系统中总共有两个空元件。程序第 16~22 行将第一个元件设置为圆形光源，第 25~30 行将第二个元件设置为矩形探测器。我们假设光源和探测器的尺寸均远小于它们之间的距离。程序第 33~42 行改变角度 θ 的值，并由其确定探测器 Y 坐标位置，然后进行光线追迹，读取探测器上的总光通量并打印到屏幕上。在程序最后，将屏幕上显示的结果(假设显示于文字窗口 1 中)保存于所指定的文件中。图 4.2-5 所示为所保存的文件内容。



```
ex40202.txt - Notepad
File Edit Format View Help
Executing D:\My Macros\CH4\EX40202.ZPL.
theta = 0 degrees, I = 0.1399
theta = 5 degrees, I = 0.1376
theta = 10 degrees, I = 0.1325
theta = 15 degrees, I = 0.1226
theta = 20 degrees, I = 0.1106
theta = 25 degrees, I = 0.0955
theta = 30 degrees, I = 0.0797
theta = 35 degrees, I = 0.0640
theta = 40 degrees, I = 0.0488
theta = 45 degrees, I = 0.0353
theta = 50 degrees, I = 0.0241
theta = 55 degrees, I = 0.0155
theta = 60 degrees, I = 0.0090
theta = 65 degrees, I = 0.0044
theta = 70 degrees, I = 0.0019
theta = 75 degrees, I = 0.0006
theta = 80 degrees, I = 0.0001
```

图 4.2-5: 程序 ex40202.ZPL 运行后所保存文件的内容

程序运行后的结果保存为文本格式的文件，可以很容易用其它软件作进一步处理。图 4.2-6 所示为归一化后的模拟结果与 $\cos^4(\Theta)$ 的比较，可以看到，模拟结果的确遵循余弦四次方规律。当然，如果光源和探测器之间的距离很短，此关系就不再准确，读者可自行探讨。

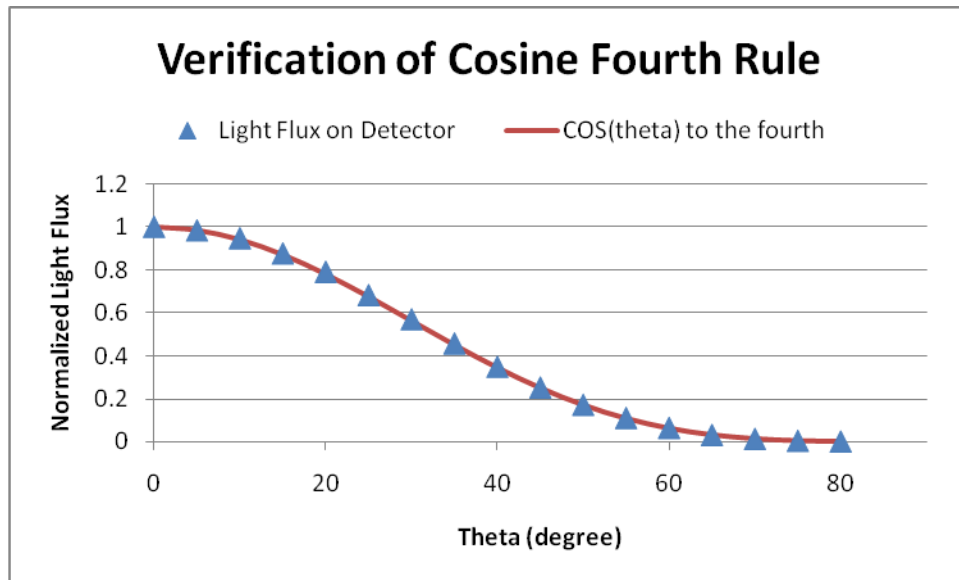


图 4.2-6: 程序 ex40202.ZPL 运行后的结果与余弦四次方函数的比较

例4.2-3: 重点采样(importance sampling)。

我们在作散射分析时，常常会碰到这样的情况：样品向空间各个方向散射光，而探测器只能收集其中比例极小的一部分，这样如果要通过光线追迹进行分析，需要对数目极大的光线进行追迹，在很多情况下并不现实。为了解决这个问题，ZEMAX 提供了一种重点采样的方法，可以人为地增加散射至目标物体的光线数，而不影响实际能量分布。这样就大大提高了分析效率，因此这种方法在实际工作中经常会被用到。本例中我们就会对重点采样的编程方法进行详细介绍。

如图 4.2-7 所示，假设有一准直光源将一束平行光照射到一个 Lambertian 样品，样品将入射光散射至空间各个方向。我们在空间同一平面上均匀放置了 8 个探测器，用于测量散射光强度。每个探测器与光源及样品的距离均相同。我们想要研究当样品倾斜时，各个探测器上的光强分布如何变化。

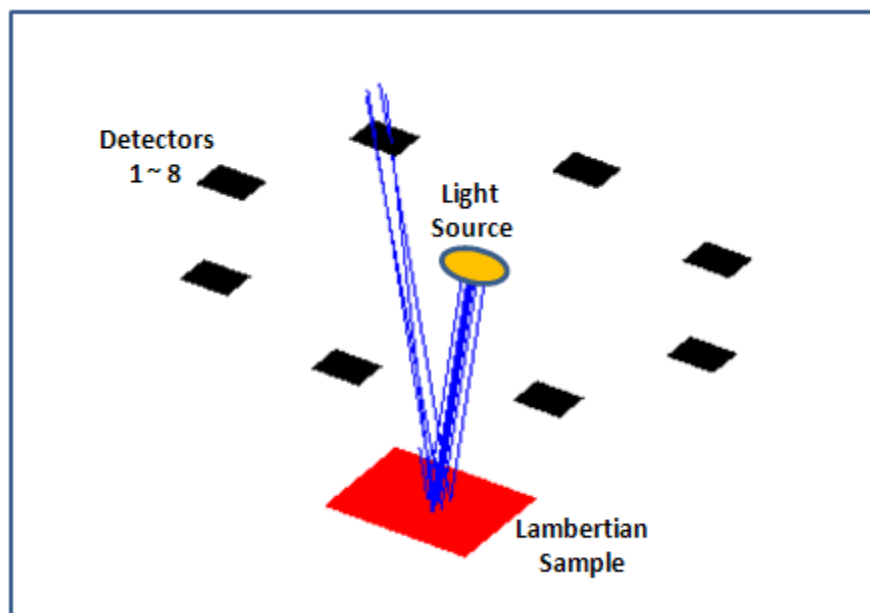


图 4.2-7: 程序 ex40203.ZPL 所涉及到的光学系统

```

1 ! ex40203
2 ! This program shows the application of importance sampling.
3 ! Assume the mode is total Non-Sequential Mode.
4
5 ! define constant and parameters
6 PI = 3.14159265
7 saveName$ = "D:\My Macros\ch4\ex40203.txt"
8 sourceRadius = 1           # the radius of a circular source
9 sampleHW = 3               # the half width of the sample
10 detectorHW = 1            # the half width of the detector
11 sourceY = 10              # the Y position of source
12 detectorY = 10            # the Y position of detector
13 rayNum = 1000000          # number of rays to be traced
14
15 ! define result as a 2-dimension array
16 detectorNum = 8           # assume 8 total detectors
17 tiltAngleNum = 10         # assume rotate 10 different angles, 1~10 degrees
18 DECLARE result, DOUBLE, 2, detectorNum, tiltAngleNum
19
20 ! clean and prepare the NSC editor
21 GOSUB prepareEditor
22
23 ! define the first object as Source Ellipse
24 GOSUB defineSource
25
26 ! define the second object as Rectangle sample
27 GOSUB defineSample
28
29 ! define the third to the tenth object as Detector Rectangle
30 GOSUB defineDetectors
31
32 ! analyzing
33 FOR theta = 0, tiltAngleNum-1, 1
34   GOSUB analyze
35 NEXT
36
37 ! print and output result
38 GOSUB reportResult
39
40 END
41
42

```

程序第 1~40 行为主程序，其中第 6~13 行定义了程序中需要用到的一些参数，第 16~18 行定义了一个二维数组用于存放结果，第 21 行调用子程序 `prepareEditor` 先将非顺序元件编辑器清空，并预置总共 10 个空元件。程序第 24 行调用子程序 `defineSource` 将第 1 个元件设为光源，第 27 行调用子程序 `defineSample` 将第 2 个元件设为 Lambertian 样品，第 30 行调用子程序 `defineDetectors` 将第 3~10 个元件设为探测器，第 33~35 行改变样品的倾角，并调用子程序 `analyze` 设置重点采样、进行光线追迹及分析，最后，程序第 38 行调用子程序 `reportResult` 输出结果。

```

42
43 SUB prepareEditor
44
45 totalObjNum = NOBJ(1)
46 FOR i, totalObjNum, 1, -1
47     DELETEOBJECT 1, i      # delete all the objects
48 NEXT                      # there is still a null object in the editor at last
49
50 ! add 9 more objects in the editor, so the total is 10
51 FOR i = 1, 9, 1
52     INSERTOBJECT 1, 1
53 NEXT
54
55 RETURN prepareEditor
56
57

```

程序第 43~55 行为子程序 prepareEditor，用于清空非顺序元件编辑器并预置总共 10 个空元件。

```

57
58 SUB defineSource
59
60 ! define the source type
61 SETNSCPROPERTY 1, 1, 0, 0, "NSC_SRCE"
62
63 ! define the position
64 SETNSCPOSITION 1, 1, 2, 10      # Y position
65 SETNSCPOSITION 1, 1, 4, 90      # rotate about X by 90 degrees
66
67 ! define the source parameters
68 SETNSCPARAMETER 1, 1, 1, 100      # Layout Rays
69 SETNSCPARAMETER 1, 1, 2, rayNum    # Analysis Rays
70 SETNSCPARAMETER 1, 1, 3, 1        # Power
71 SETNSCPARAMETER 1, 1, 6, sourceRadius # X Half Width
72 SETNSCPARAMETER 1, 1, 7, sourceRadius # Y Half Width
73 SETNSCPARAMETER 1, 1, 8, 0        # Source distance, assume collimated
74
75 RETURN defineSource
76
77

```

程序第 58~75 行为子程序 defineSource，用于将第 1 个元件设为圆形光源，其中第 73 行将输出光设为平行光束。

```

76
77
78 SUB defineSample
79
80 ! define the sample type
81 SETNSCPROPERTY 1, 2, 0, 0, "NSC_RECT"      # rectangle
82 SETNSCPROPERTY 1, 2, 4, 0, "MIRROR"        # reflective
83 SETNSCPROPERTY 1, 2, 7, 0, 1               # Lambertian
84 SETNSCPROPERTY 1, 2, 8, 0, 1               # Scatter fraction
85 SETNSCPROPERTY 1, 2, 9, 0, 1               # number of scatter rays
86
87 ! define the position
88 SETNSCPOSITION 1, 2, 2, 0                  # Y position
89 SETNSCPOSITION 1, 2, 4, 90                 # rotate about X by 90 degrees
90
91 ! define the sample size
92 SETNSCPARAMETER 1, 2, 1, sampleHW          # X Half Width
93 SETNSCPARAMETER 1, 2, 2, sampleHW          # Y Half Width
94
95 RETURN defineSample
96
97

```

程序第 78~95 行为子程序 defineSample，用于将第 2 个元件设为 Lambertian 散射样品，并将散射因子设为 1(第 84 行)，散射光线数设为 1(第 85 行)。

```

97
98 SUB defineDetectors
99
100 FOR i = 3, 10, 1
101   ! define the detector type as detector rectangle
102   SETNSCPROPERTY 1, i, 0, 0, "NSC_DETE"
103
104   ! define the position of the detector
105   theta = (i-3)*45*PI/180                # in radians
106   x = detectorY*COS(theta)                 # calculate X coordinate
107   z = detectorY*SIN(theta)                 # calculate Z coordinate
108   SETNSCPOSITION 1, i, 1, x                # X position
109   SETNSCPOSITION 1, i, 2, detectorY        # Y position
110   SETNSCPOSITION 1, i, 3, z                # Z position
111   SETNSCPOSITION 1, i, 4, 90               # rotate about X by 90 degrees
112
113   ! define the detector parameters
114   SETNSCPARAMETER 1, i, 1, detectorHW      # X Half Width
115   SETNSCPARAMETER 1, i, 2, detectorHW      # Y Half Width
116   SETNSCPARAMETER 1, i, 3, 1               # Number of X Pixels
117   SETNSCPARAMETER 1, i, 4, 1               # Number of Y Pixels
118 NEXT
119
120 RETURN defineDetectors
121
122

```

程序第 98~120 行为子程序 `defineDetectors`，用于将第 3~10 个元件设为探测器。由于我们只需要读取探测器上的总光强，所以选择将各个探测器像素数目设为 1(第 116、117 行)。

```

122
123 SUB analyze
124
125 ! tilt sample
126 SETNSCPOSITION 1, 2, 4, 90+theta # rotate about X by another theta degrees
127
128 FOR i = 3, 10, 1 # going through different detectors
129     ! set the importance sampling towards the detector
130     SETNSCPROPERTY 1, 2, 151, 0, 1 # sample at importance sampling mode
131     FORMAT 1 INT
132     targetObject$ = $STR(INTE(i))
133     FORMAT 4.2
134     targetSize$ = $STR(detectorHW*2)
135     targetData$ = "1 " + targetObject$ + " " + targetSize$ + " 0.6"
136     SETNSCPROPERTY 1, 2, 152, 0, targetData$ # importance sampling setting
137
138     ! ray tracing
139     temp = NSDD(1, 0, 0, 0) # clear the detector
140     NSTR 1, 1, 1, 1, 1, 1, 1, 0 # ray tracing
141     result(i-2, theta+1) = NSDD(1, 1, 0, 0) # record detector reading
142 NEXT
143
144 ! recover sample
145 SETNSCPOSITION 1, 2, 4, 90 # recover original rotation
146
147 RETURN analyze
148
149

```

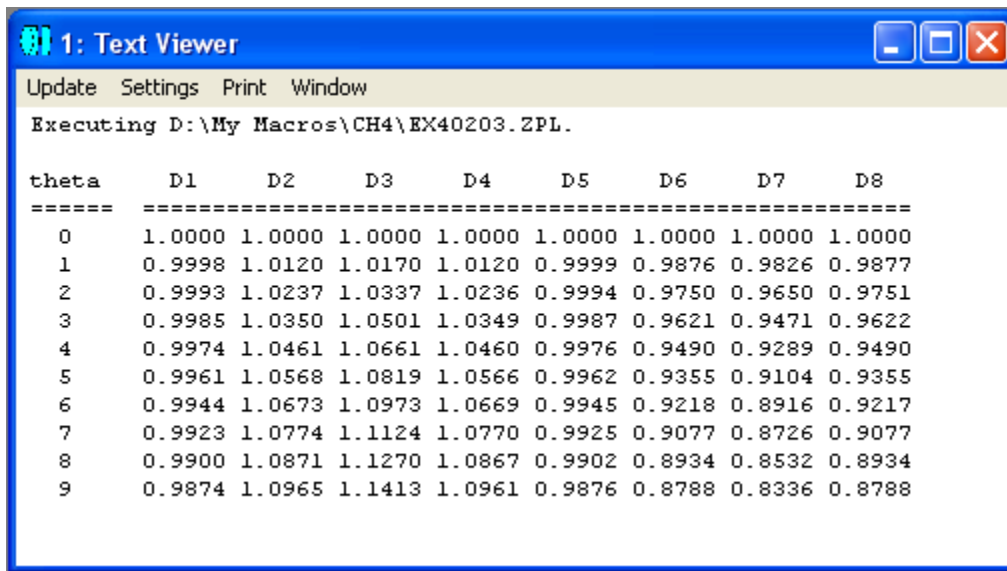
程序第 123~147 行为子程序 `analyze`，用于定义样品倾角(126 行)、重点采样目标，并进行光线追迹及分析。其中，第 130 行将散射模式设为重点采样，第 131~135 行用于根据不同的探测器目标生成目标数据字符串，第 136 行设置重点采样目标数据，第 139 行清空探测器，第 140 行光线追迹，第 141 行将探测器上的读数存入 `result` 数组。程序第 145 行用于恢复样品倾斜前的方向。

```

149
150 SUB reportResult
151
152 PRINT
153 PRINT "theta      D1      D2      D3      D4      D5      D6      D7      D8"
154 PRINT "===== "
155 FOR theta = 0, tiltAngleNum-1, 1
156     FORMAT 2 INT
157     PRINT " ", theta, " ",
158     FORMAT 5.4
159     FOR i = 1, detectorNum, 1
160         PRINT result(i, theta+1)/result(i, 1), " ",
161         IF i == detectorNum THEN PRINT " "
162     NEXT i
163 NEXT theta
164
165 ! save the result
166 SAVEWINDOW 1, saveName$           # assume there is no other windows open
167
168 RETURN reportResult

```

程序第 150 ~ 168 行为子程序 reportResult，用于输出结果至屏幕窗口，并将窗口内容保存到指定的文件中。注意在第 157 行及第 160 行的最后有一个逗号，表示打印不换行，第 161 行判断如果在某倾角下所有的探测器读数都打印完后，则打印一个空格并换行。另外请注意我们只关心探测器上的相对读数，因此第 160 行打印的内容为某探测器读数与 0 倾角下此探测器读数的比值。



theta	D1	D2	D3	D4	D5	D6	D7	D8
0	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	0.9998	1.0120	1.0170	1.0120	0.9999	0.9876	0.9826	0.9877
2	0.9993	1.0237	1.0337	1.0236	0.9994	0.9750	0.9650	0.9751
3	0.9985	1.0350	1.0501	1.0349	0.9987	0.9621	0.9471	0.9622
4	0.9974	1.0461	1.0661	1.0460	0.9976	0.9490	0.9289	0.9490
5	0.9961	1.0568	1.0819	1.0566	0.9962	0.9355	0.9104	0.9355
6	0.9944	1.0673	1.0973	1.0669	0.9945	0.9218	0.8916	0.9217
7	0.9923	1.0774	1.1124	1.0770	0.9925	0.9077	0.8726	0.9077
8	0.9900	1.0871	1.1270	1.0867	0.9902	0.8934	0.8532	0.8934
9	0.9874	1.0965	1.1413	1.0961	0.9876	0.8788	0.8336	0.8788

图 4.2-8: 程序 ex40203.ZPL 的运行结果

保存在指定文件中的内容可以用其它软件进行后处理，例如输入到 Excel 中，得到图 4.2-9 所示结果。我们可以看到，当样品倾斜后，各个探测器上的读数均相应改变，遵循余弦函数规律，这里就不再进行讨论。

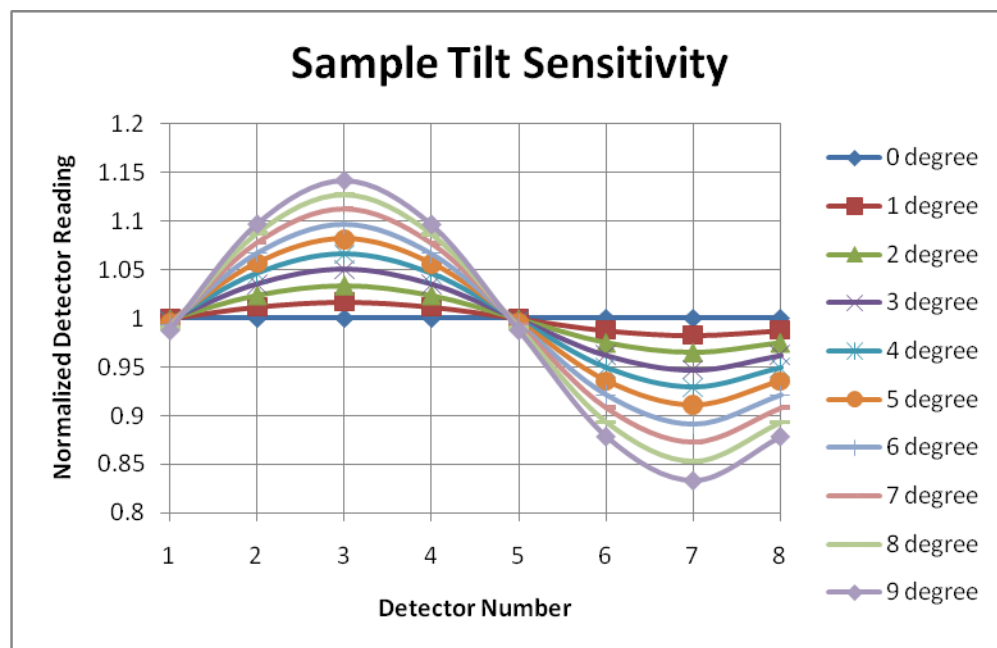


图 4.2-9: 程序 ex40203.ZPL 运行后得到的样品倾角与探测器读数关系

例4.2-4: 干涉条纹观测。

ZEMAX 的非顺序模式中提供的矩形探测器，是一个很有用的元件。除了可以用于探测能量外，还可以用于进行相干分析。在这个例子中，我们介绍一个最简单的干涉装置，并用矩形探测器观测干涉条纹。

如图 4.2-10 所示，假设有一激光光源，将准直光束照射到一玻璃板上，玻璃板上、下表面的反射光将会发生干涉。如果玻璃板的下表面为一标准平面，而上表面有一定弧度，则在探测器处可以看到干涉条纹。这里我们假设探测器尺寸小于光束横截面尺寸，而忽略光束边缘的影响。

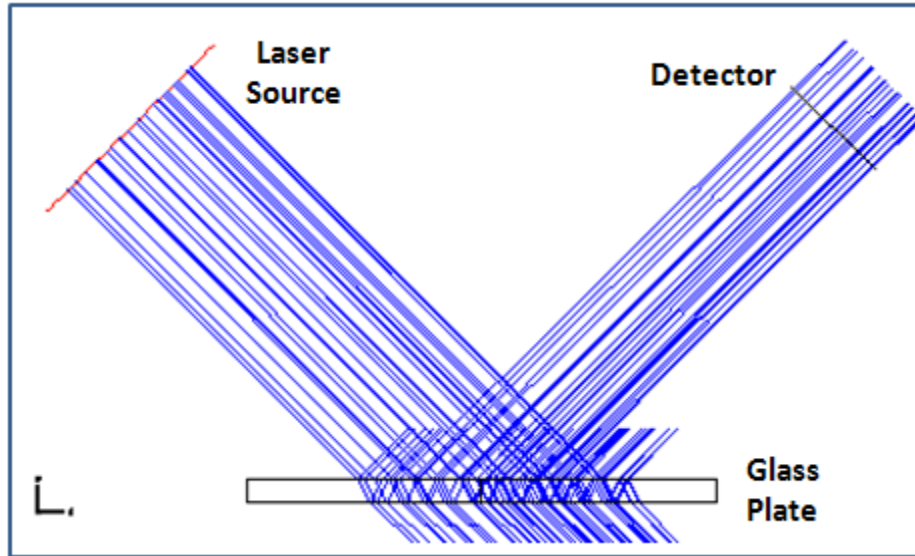


图4.2-10：玻璃板上、下表面反射光干涉示意图

程序 `ex40203.ZPL` 用于改变玻璃板上表面的曲率半径，观察探测器上干涉条纹的相应变化，并将干涉条纹输出到一系列文件中。

在程序运行之前，我们先要做一点准备工作。首先，在非顺序模式下新建一个镜头文件，这时非顺序元件编辑器中将有一个缺省的空元件，将其类型选择为“Detector Rect”，并打开一个探测器观察器，将其输出数据类型设置为“Coherent Irradiance”，如图 4.2-11 所示：

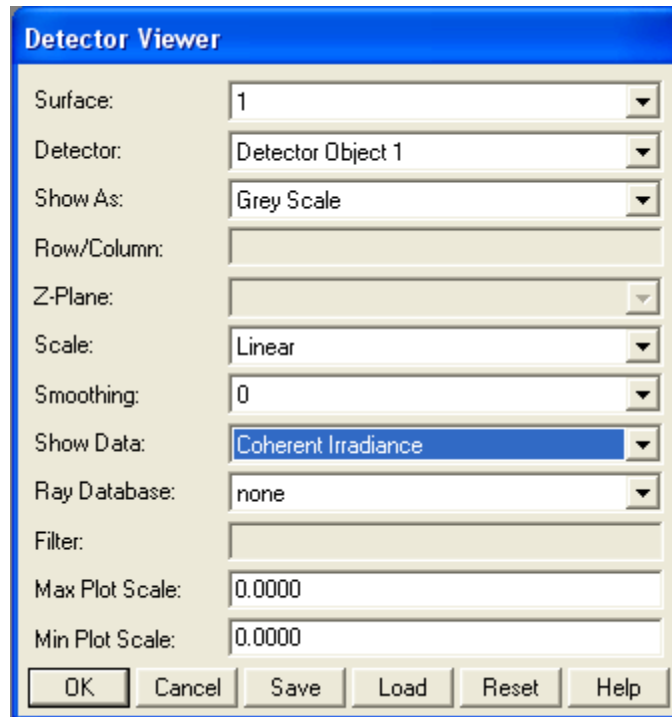


图4.2-11: 设置探测器观察器输出数据类型为相干光辐射照度

将探测器观察器设置好后，就可以运行以下程序了。程序第 1 ~ 34 行为主程序，其中第 6 ~ 17 行定义了一些程序中需要用到的参数，第 20 行调用子程序 `prepareEditor` 另行插入两个空元件，第 23 行调用子程序 `defineSource` 将第 1 个元件设为光源，第 26 行调用子程序 `defineGlassPlate` 将第 2 个元件设为玻璃板，第 29 行调用子程序 `defineDetector` 将第 3 个元件设为矩形探测器，第 32 行调用子程序 `analyze` 进行光线追迹并将探测器上的干涉图形输出到一系列指定文件。

```

1 ! ex40204
2 ! This program shows the observation of interference fringes.
3 ! Some pre-setting of Detector Viewer is needed.
4
5 ! define constant and parameters
6 sourceRadius = 1          # the radius of a circular source
7 detectorHW = 0.5          # the half width of the detector
8 sourceY = 3               # the Y position of source
9 sourceZ = -3              # the Z position of source
10 detectorY = 3             # the Y position of detector
11 detectorZ = 3             # the Z position of detector
12 plateThickness = 0.2     # lens unit, assume mm
13 r0 = 100                 # initial radius of top surface of glass plate
14 rayNum = 1000000         # number of rays to be traced
15
16 prefix$ = "D:\My Macros\ch4\ex40204 R"
17 ext$ = ".WHF"
18
19 ! prepare the NSC editor
20 GOSUB prepareEditor
21
22 ! define the first object as Source Ellipse
23 GOSUB defineSource
24
25 ! define the second object as glass plate
26 GOSUB defineGlassPlate
27
28 ! define the third object as detector
29 GOSUB defineDetector
30
31 ! analyze
32 GOSUB analyze
33
34 END
35
36

```

```

36
37 SUB prepareEditor
38
39 ! add 2 more objects in the editor, so the total is 3
40 FOR i = 1, 2, 1
41     INSERTOBJECT 1, 1
42 NEXT
43
44 RETURN prepareEditor
45
46

```

程序第 37 ~ 44 行为子程序 prepareEditor，用于插入两个空元件。

```

46
47 SUB defineSource
48
49 ! define the source type
50 SETNSCPROPERTY 1, 1, 0, 0, "NSC_SRCE"
51
52 ! define the position
53 SETNSCPOSITION 1, 1, 2, sourceY      # Y position
54 SETNSCPOSITION 1, 1, 3, sourceZ      # Z position
55 SETNSCPOSITION 1, 1, 4, 45           # rotate about X by 45 degrees
56
57 ! define the source parameters
58 SETNSCPARAMETER 1, 1, 1, 100         # Layout Rays
59 SETNSCPARAMETER 1, 1, 2, rayNum       # Analysis Rays
60 SETNSCPARAMETER 1, 1, 3, 1           # Power
61 SETNSCPARAMETER 1, 1, 6, sourceRadius # X Half Width
62 SETNSCPARAMETER 1, 1, 7, sourceRadius # Y Half Width
63 SETNSCPARAMETER 1, 1, 8, 0           # Source Distance, assume collimated
64
65 RETURN defineSource
66
67

```

程序第 47~65 行为子程序 defineSource，用于将第 1 个元件设置为准直光源。

```

67
68 SUB defineGlassPlate
69
70 ! define the object type and material
71 SETNSCPROPERTY 1, 2, 0, 0, "NSC_SLEN"
72 SETNSCPROPERTY 1, 2, 4, 0, "BK7"
73
74 ! define the position
75 SETNSCPOSITION 1, 2, 4, 90           # rotate about X by 90 degrees
76
77 ! define the glass plate parameters
78 SETNSCPARAMETER 1, 2, 1, 0           # Radius 1
79 SETNSCPARAMETER 1, 2, 3, 2           # Clear 1
80 SETNSCPARAMETER 1, 2, 4, 2           # Edge 1
81 SETNSCPARAMETER 1, 2, 5, plateThickness # Thickness
82 SETNSCPARAMETER 1, 2, 6, 0           # Radius 2
83 SETNSCPARAMETER 1, 2, 8, 2           # Clear 2
84 SETNSCPARAMETER 1, 2, 9, 2           # Edge 2
85
86 RETURN defineGlassPlate
87
88

```

程序第 68~86 行为子程序 defineGlassPlate，用于将第 2 个元件设置为玻璃板。我们将玻璃板的元件类型设为标准镜头，这样后面就可以很容易对其上表面曲率半径进行修改。

```

88
89 SUB defineDetector
90
91 ! define the detector type as detector rectangle
92 SETNSCPROPERTY 1, 3, 0, 0, "NSC_DETE"
93
94 ! define the position of the detector
95 SETNSCPOSITION 1, 3, 2, detectorY      # Y position
96 SETNSCPOSITION 1, 3, 3, detectorZ      # Z position
97 SETNSCPOSITION 1, 3, 4, -45           # rotate about X by -45 degrees
98
99 ! define the detector parameters
100 SETNSCPARAMETER 1, 3, 1, detectorHW    # X Half Width
101 SETNSCPARAMETER 1, 3, 2, detectorHW    # Y Half Width
102 SETNSCPARAMETER 1, 3, 3, 100          # Number of X Pixels
103 SETNSCPARAMETER 1, 3, 4, 100          # Number of Y Pixels
104
105 RETURN defineDetector
106
107

```

程序第 89~105 行为子程序 defineDetector，用于将第 3 个元件设置为矩形探测器。

```

107
108 SUB analyze
109
110 FOR i = 1, 10, 1
111     radius1 = r0 + POWR(2, i-1)
112     SETNSCPARAMETER 1, 2, 1, radius1      # change Radius 1
113     temp = NSDD(1, 0, 0, 0)              # clear the detector
114     NSTR 1, 1, 1, 1, 1, 1, 1, 0          # ray tracing
115     FORMAT 1 INT
116     r$ = $STR(radius1)
117     fileName$ = prefix$ + r$ + ext$
118     UPDATE 1                             # assume serial number of the graphic window is 1
119     EXPORTBMP 1, fileName$
120 NEXT
121
122 RETURN analyze
123

```

程序第 108~122 行为子程序 analyze，用于改变玻璃板上表面的曲率半径，进行光线追迹，并根据玻璃板上表面曲率半径生成输出文件名，将探测器浏览器窗口内容输出到指定文件。

程序运行结束后，我们可以看到指定文件夹中出现了下列文件：

ex40204 R101.BMP
 ex40204 R102.BMP
 ex40204 R104.BMP
 ex40204 R108.BMP
 ex40204 R116.BMP
 ex40204 R132.BMP
 ex40204 R164.BMP
 ex40204 R228.BMP
 ex40204 R356.BMP
 ex40204 R612.BMP

其中第 1、5、9 个文件的内容如图 4.2-12 所示：

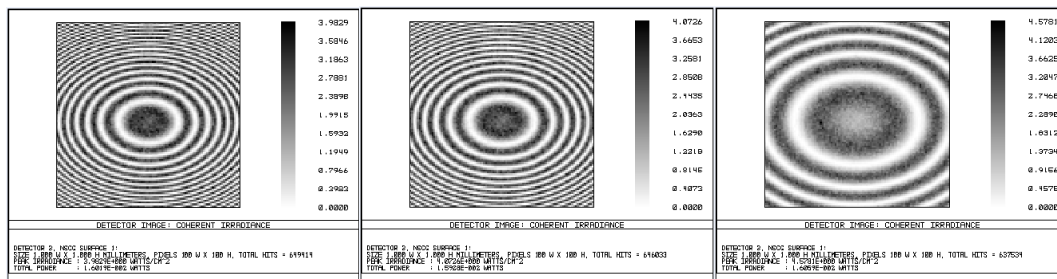


图4.2-12：程序 ex40204.ZPL 部分输出文件内容

例4.2-5：积分球入射孔尺寸与探测器尺寸对积分效率的影响。

积分球是光学测量中一种常用的仪器，其中一种结构是在两个相互垂直的方向上开有两个口，一个用于导入光源，另一个用于放置探测器，如图 4.2-13 所示。我们知道，光入射孔和探测器大小对积分效率均有影响，因为光会从这两个地方损耗掉。在本例中，我们就通过程序改变光源和探测器尺寸，研究其对积分效率的影响。

首先，我们将用 ZEMAX 中的球形元件来模拟积分球，并假设球的半径为1 (我们只关心相对值而不管其绝对大小)。然后，我们在球上 $Z = 1$ 附近放置一圆形光源， $Y = 1$ 附近放置一圆形探测器，光源和探测器的具体位置要做一下简单的计算，保证它们与球表面相切。另外，为了保证光线追迹时不会因物体重叠而出错，我们将光源和探测器向球心再移动一微小距离。我们可以将探测器的材料设为“ABSORB”，用于模拟光照射于其上的损耗。由于光源不能设为吸收材料，所以我们另将一与光源尺寸相同的圆形表面置于光源与球面顶点之间，紧贴光源，用于模拟当光线照射回入射孔时发生的损耗。这时的入射孔实际上是一个出射孔，光通过这个孔损耗掉。

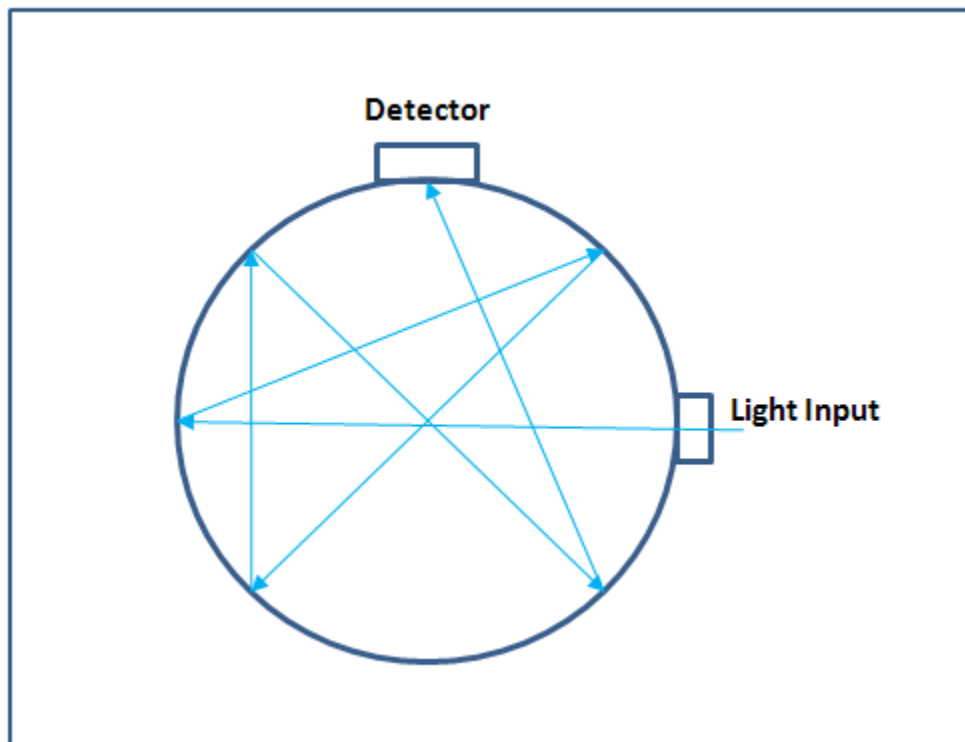


图4.2-13: 一种常用的积分球示意图

在 ZEMAX 中新建一个镜头文件，然后就可以运行我们的程序了。


```

1 ! ex40205
2 ! This program shows how to simulate an integrating sphere
3
4 ! define constant and parameters
5 sphereR = 1          # the radius of the integrating sphere
6 gap = 0.001          # a gap from the sphere
7 startSouR = 0        # start source radius, will skip the first one
8 stopSouR = 0.8       # stop source radius
9 stepSouNum = 20      # steps between start and stop source radius
10 startDetR = 0        # start detector radius, will skip the first one
11 stopDetR = 0.5       # stop detector radius
12 stepDetNum = 10      # steps between start and stop detector radius
13 rayNum = 100000      # number of rays to be traced
14
15 saveName$ = "D:\My Macros\ch4\ex40205.txt"
16
17 ! define result as a 2-dimension array
18 DECLARE result, DOUBLE, 2, stepSouNum, stepDetNum
19
20 ! prepare the NSC editor and define some system parameters
21 GOSUB prepareEditor
22
23 ! define the first object as Source Ellipse
24 sourceR = 0.01       # this number is just an innitial value, will change
25 GOSUB defineSource
26
27 ! define the second object as a sphere shell
28 GOSUB defineSphere
29
30 ! define the third object as detector
31 detectorR = 0.01     # this number is just an innitial value, will change
32 GOSUB defineDetector
33
34 ! define the fourth object as the output port
35 GOSUB defineOutputPort
36
37 ! analyze
38 GOSUB analyze
39
40 ! export result to a file
41 GOSUB exportToFile
42
43 END
44
45

```

程序第 1~43 行为主程序，其中第 5~15 行设置程序中要用到的参数，第 18 行用于定义一个二维数组以存放结果，接下来就和前面的例子一样，通过调用一系列子程序来设置系统、进行光线追迹和分析输出结果。在这个例子中，我们将用到子程序的嵌套，即在子程序中调用其它的子程序。

```

45
46 SUB prepareEditor
47
48 ! add 2 more objects in the editor, so the total is 3
49 FOR i = 1, 3, 1
50     INSERTOBJECT 1, 1
51 NEXT
52
53 SYSP 50, 4000          # maximum intersection per ray
54 SYSP 51, 50000        # maximum segments per ray
55 SYSP 53, 1E-6         # minimum relative energy
56
57 RETURN prepareEditor
58
59

```

程序第 46~57 行为子程序 prepareEditor，用于在非顺序元件编辑器中总共设置 3 个元件，并设置系统参数中的单根光线最大相交数目、单根光线最大节数及最小相对能量。

```

59
60 SUB defineSource
61
62 ! define the source type
63 SETNSCPROPERTY 1, 1, 0, 0, "NSC_SRCE"
64 SETNSCPROPERTY 1, 1, 4, 0, "ABSORB"
65
66 ! define the position
67 sourceZ = SQRT(sphereR*sphereR-sourceR*sourceR)
68 SETNSCPOSITION 1, 1, 3, sourceZ-gap      # Z position
69 SETNSCPOSITION 1, 1, 4, 180             # rotate about X by 180 degrees
70
71 ! define the source parameters
72 SETNSCPARAMETER 1, 1, 1, 100            # Layout Rays
73 SETNSCPARAMETER 1, 1, 2, rayNum        # Analysis Rays
74 SETNSCPARAMETER 1, 1, 3, 1            # Power
75 SETNSCPARAMETER 1, 1, 6, sourceR       # X Half Width
76 SETNSCPARAMETER 1, 1, 7, sourceR       # Y Half Width
77 SETNSCPARAMETER 1, 1, 8, 0            # Source Distance, assume collimated
78
79 RETURN defineSource
80
81

```

程序第 60~79 行为子程序 defineSource，用于设置光源，其中第 67 行计算光源的 Z 坐标位置，第 68 行设置光源位置并将其移动一微小距离。在这个子程序中，其光源半径参数 sourceR 未确定，需要在调用此子程序之前设定，这样就允许我们通过程序很方便地对不同的光源半径进行分析。

```

81
82 SUB defineSphere
83
84 ! define the object type as Lambertian reflective
85 SETNSCPROPERTY 1, 2, 0, 0, "NSC_SPHE"      # sphere
86 SETNSCPROPERTY 1, 2, 4, 0, "MIRROR"        # reflective
87 SETNSCPROPERTY 1, 2, 7, 0, 1               # Lambertian
88 SETNSCPROPERTY 1, 2, 8, 0, 1               # Scatter fraction
89 SETNSCPROPERTY 1, 2, 9, 0, 1               # number of scatter rays
90
91 ! define the sphere parameters
92 SETNSCPARAMETER 1, 2, 1, sphereR           # radius
93 SETNSCPARAMETER 1, 2, 2, 0                 # a shell, not a volume
94
95 RETURN defineSphere
96
97

```

程序第 82 ~ 95 行为子程序 `defineSphere`，用于设置积分球。我们将球形元件设为壳层(第 93 行)，并将其光学性质设为 Lambertian 散射(第 86 ~ 89 行)。

```

97
98 SUB defineDetector
99
100 ! define the detector type as detector surface
101 SETNSCPROPERTY 1, 3, 0, 0, "NSC_DETS"
102 SETNSCPROPERTY 1, 3, 4, 0, "ABSORB"
103
104 ! define the position of the detector
105 detectorY = SQRT(sphereR*sphereR-detectorR*detectorR)
106 SETNSCPOSITION 1, 3, 2, detectorY-gap      # Y position
107 SETNSCPOSITION 1, 3, 4, 90                 # rotate about X by 90 degrees
108
109 ! define the detector parameters
110 SETNSCPARAMETER 1, 3, 1, 0                  # Radius = 0, flat
111 SETNSCPARAMETER 1, 3, 3, detectorR          # max aperture
112 SETNSCPARAMETER 1, 3, 4, 0                  # min aperture
113
114 RETURN defineDetector
115
116

```

程序第 98 ~ 114 行为子程序 `defineDetector`，用于设置圆形探测器。和光源设置一样，探测器的尺寸也未确定，在调用此子程序之前设定，探测器位置也将根据其尺寸进行计算(第 105 行)。

```

116
117 SUB defineOutputPort
118
119 ! define the output port type as ellipse
120 SETNSCPROPERTY 1, 4, 0, 0, "NSC_ELLI"
121 SETNSCPROPERTY 1, 4, 4, 0, "ABSORB"
122
123 ! define the position
124 outputPortZ = SQRT(sphereR*sphereR-sourceR*sourceR)
125 SETNSCPOSITION 1, 4, 3, outputPortZ-gap/2 # Z position
126 SETNSCPOSITION 1, 4, 4, 90 # rotate about X by 90 degrees
127
128 ! define the output port parameters
129 SETNSCPARAMETER 1, 4, 1, sourceR # X Half Width, same as source
130 SETNSCPARAMETER 1, 4, 2, sourceR # Y Half Width, same as source
131
132 RETURN defineOutputPort
133
134

```

程序第 117 ~ 132 行为子程序 defineOutputPort，用于在光源后面放置一同样大小的吸收元件，以模拟光从此处损耗掉。同样，其尺寸要根据光源尺寸确定，其位置也需要相应计算。

```

134
135 SUB analyze
136
137 ! print header
138 GOSUB printHeader
139
140 ! ray tracing and print result
141 FOR i = 1, stepSouNum, 1
142   sourceR = startSouR+i*(stopSouR-startSouR)/stepSouNum
143   FORMAT 4.2
144   PRINT sourceR, " ",
145   FOR j = 1, stepDetNum, 1
146     detectorR = startDetR+j*(stopDetR-startDetR)/stepDetNum
147     GOSUB defineSource # modify source setting
148     GOSUB defineDetector # modify detector setting
149     GOSUB defineOutputPort # modify output port setting
150     temp = NSDD(1, 0, 0, 0) # clear the detector
151     NSTR 1, 1, 1, 1, 1, 1, 1, 0 # ray tracing
152     result(i, j) = NSDD(1, 3, 0, 0) # record detector reading
153     FORMAT 6.5
154     PRINT NSDD(1, 3, 0, 0), " ", # print result
155   NEXT j
156   PRINT
157 NEXT i
158
159 RETURN analyze
160
161

```

程序第 135 ~ 159 行为子程序 `analyze`，用于改变光源和探测器尺寸，进行光线追迹，并记录结果。在此子程序中，我们用列表的形式将每一次光线追迹的结果显示在屏幕上，并将结果存于数组 `result` 中。程序第 138 行调用另一子程序 `printHeader`，用于在屏幕上打印输出表头，第 141 ~ 157 行的外循环用于改变光源尺寸，第 145 ~ 155 行的内循环用于改变探测器尺寸。第 142、146 行分别计算光源和探测器尺寸，然后第 147、148、149 行调用不同的子程序分别设置光源、探测器和与光源相应的损耗孔，然后进行光线追迹和记录、打印数据。注意我们在主程序中和在此子程序中都可以调用相同的子程序。

```

161
162 SUB printHeader
163
164 PRINT
165 PRINT " S ",
166 FOR i = 1, stepDetNum, 1
167     FORMAT 1 INT
168     PRINT " D", i, " ",
169 NEXT
170 PRINT
171 PRINT " ",
172 FOR i = 1, stepDetNum, 1
173     detectorR = startDetR+i*(stopDetR-startDetR)/stepDetNum
174     FORMAT 4.2
175     PRINT " ", detectorR, " ",
176 NEXT
177 PRINT
178 PRINT "==== ",
179 FOR i = 1, stepDetNum, 1
180     PRINT "=====",
181 NEXT
182 PRINT " "
183
184 RETURN printHeader
185
186

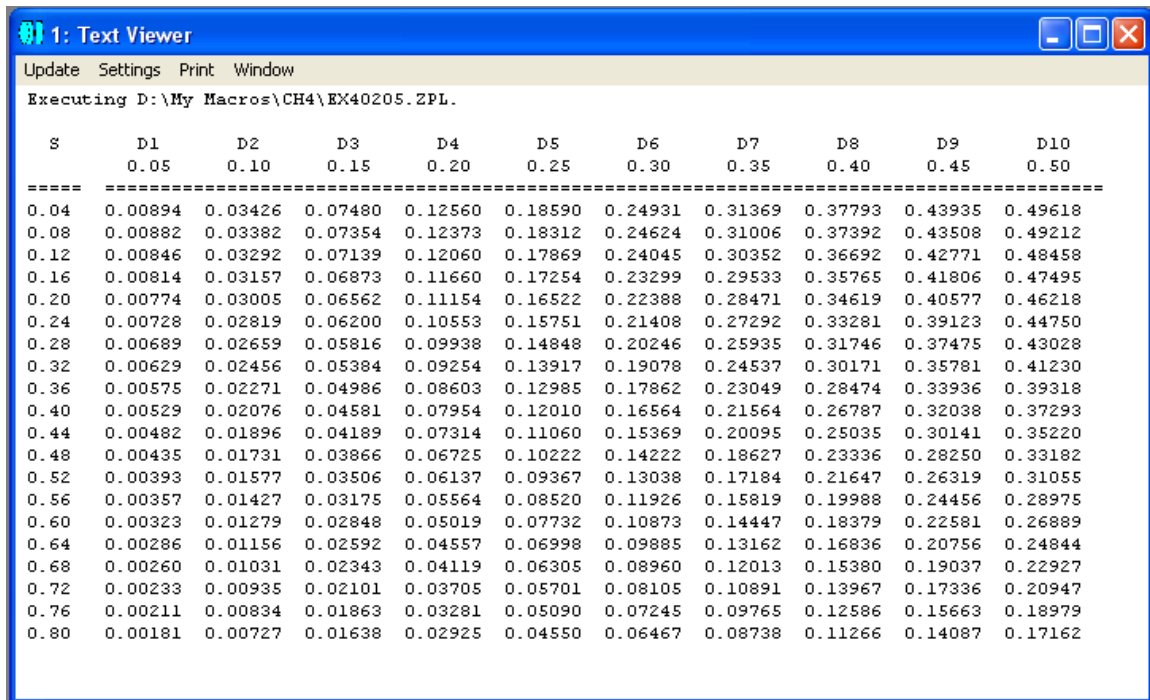
```

程序第 162 ~ 184 行为子程序 `printHeader`，用于打印表头。我们在这里使用缺省的输出目标，即屏幕。

```
186
187
188 SUB exportToFile
189
190 OUTPUT saveName$
191 FORMAT 6.5
192
193 PRINT "First row is detector radius, ",
194 PRINT "first column is source and output port radius,"
195 PRINT "and the rest data are corresponding detector readings."
196 PRINT
197
198 PRINT 0, " ",
199 FOR i = 1, stepDetNum, 1
200     detectorR = startDetR+i*(stopDetR-startDetR)/stepDetNum
201     PRINT detectorR, " ",
202 NEXT
203 PRINT
204 FOR i = 1, stepSouNum, 1
205     sourceR = startSouR+i*(stopSouR-startSouR)/stepSouNum
206     PRINT SourceR, " ",
207     FOR j = 1, stepDetNum, 1
208         PRINT result(i, j), " ",
209     NEXT j
210     PRINT
211 NEXT i
212
213 OUTPUT SCREEN
214
215 RETURN exportToFile
216
```

程序第 188~215 行为子程序 `exportToFile`，用于将保存于数组 `result` 中的结果以指定形式输出到文件中。当然我们可以和在例 4.2-3 中一样，直接将打印在屏幕上的列表保存到文件中，但这个子程序给出了另一种输出结果的方法。

程序运行结束后，在屏幕上可以看到如图 4.2-14 所示的结果：



1: Text Viewer

Update Settings Print Window

Executing D:\My Macros\CH4\EX40205.ZPL.

S	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50
0.04	0.00894	0.03426	0.07480	0.12560	0.18590	0.24931	0.31369	0.37793	0.43935	0.49618
0.08	0.00882	0.03382	0.07354	0.12373	0.18312	0.24624	0.31006	0.37392	0.43508	0.49212
0.12	0.00846	0.03292	0.07139	0.12060	0.17869	0.24045	0.30352	0.36692	0.42771	0.48458
0.16	0.00814	0.03157	0.06873	0.11660	0.17254	0.23299	0.29533	0.35765	0.41806	0.47495
0.20	0.00774	0.03005	0.06562	0.11154	0.16522	0.22388	0.28471	0.34619	0.40577	0.46218
0.24	0.00728	0.02819	0.06200	0.10553	0.15751	0.21408	0.27292	0.33281	0.39123	0.44750
0.28	0.00689	0.02659	0.05816	0.09938	0.14848	0.20246	0.25935	0.31746	0.37475	0.43028
0.32	0.00629	0.02456	0.05384	0.09254	0.13917	0.19078	0.24537	0.30171	0.35781	0.41230
0.36	0.00575	0.02271	0.04986	0.08603	0.12985	0.17862	0.23049	0.28474	0.33936	0.39318
0.40	0.00529	0.02076	0.04581	0.07954	0.12010	0.16564	0.21564	0.26787	0.32038	0.37293
0.44	0.00482	0.01896	0.04189	0.07314	0.11060	0.15369	0.20095	0.25035	0.30141	0.35220
0.48	0.00435	0.01731	0.03866	0.06725	0.10222	0.14222	0.18627	0.23336	0.28250	0.33182
0.52	0.00393	0.01577	0.03506	0.06137	0.09367	0.13038	0.17184	0.21647	0.26319	0.31055
0.56	0.00357	0.01427	0.03175	0.05564	0.08520	0.11926	0.15819	0.19988	0.24456	0.28975
0.60	0.00323	0.01279	0.02848	0.05019	0.07732	0.10873	0.14447	0.18379	0.22581	0.26889
0.64	0.00286	0.01156	0.02592	0.04557	0.06998	0.09885	0.13162	0.16836	0.20756	0.24844
0.68	0.00260	0.01031	0.02343	0.04119	0.06305	0.08960	0.12013	0.15380	0.19037	0.22927
0.72	0.00233	0.00935	0.02101	0.03705	0.05701	0.08105	0.10891	0.13967	0.17336	0.20947
0.76	0.00211	0.00834	0.01863	0.03281	0.05090	0.07245	0.09765	0.12586	0.15663	0.18979
0.80	0.00181	0.00727	0.01638	0.02925	0.04550	0.06467	0.08738	0.11266	0.14087	0.17162

图4.2-14: 程序ex40205.ZPL 运行后屏幕显示结果

同时，在程序中指定的文件夹中，可以看到出现了一个指定名字的文件，其内容如图 4.2-15 所示：

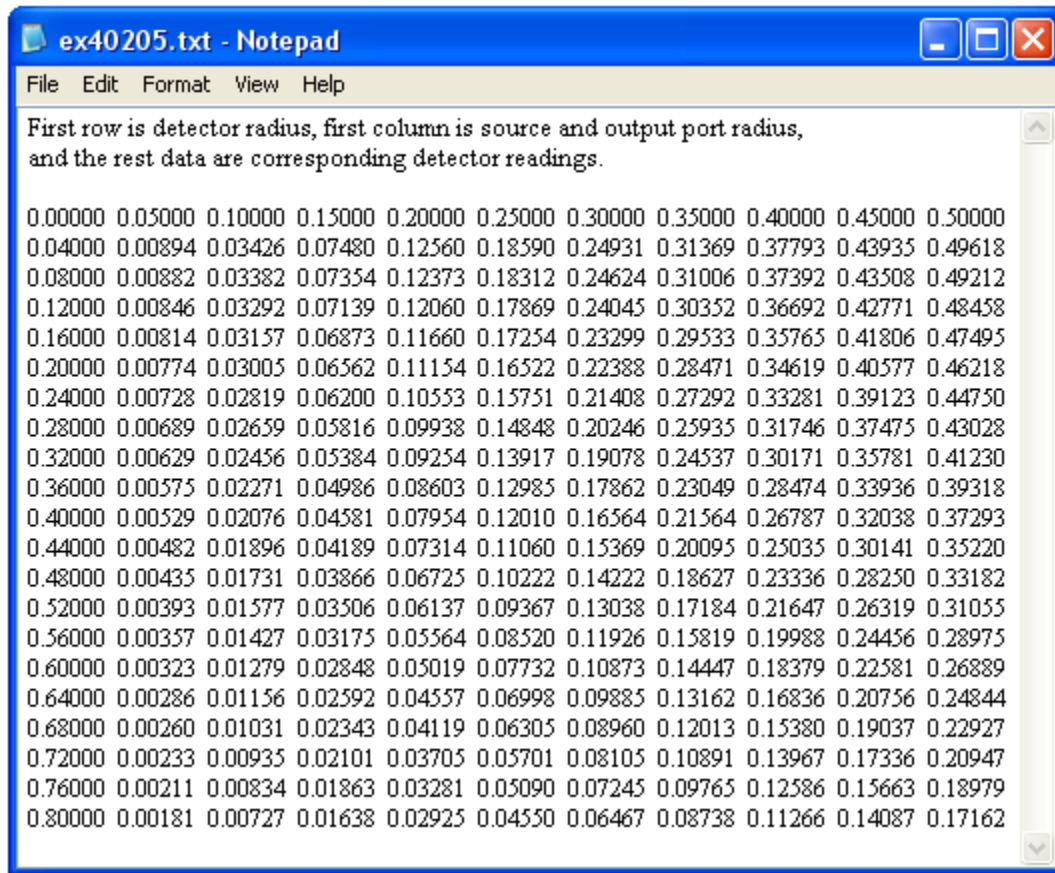


图4.2-15: 程序ex40205.ZPL 运行后保存文件内容

保存到文本文件中的结果可以很容易输入到其它软件中作进一步处理。图 4.2-16 所示为使用 Excel 做出的积分效率与光源、探测器尺寸关系曲线。

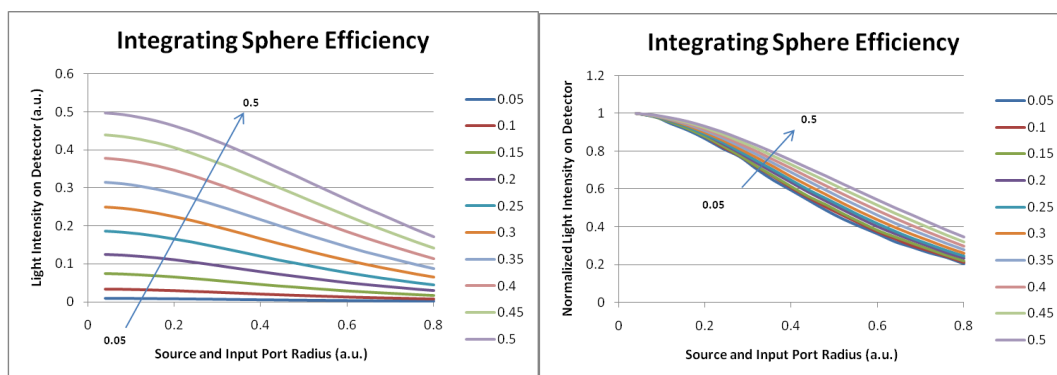


图4.2-16: 程序ex40205.ZPL 运行后得到的积分效率关系曲线

例4.2-6：用体探测器读取光强的三维分布。

体探测器是 ZEMAX 提供的一个很有效的分析元件，通过它可以读取三维空间的入射光强或吸收光强的分布。在下面的这个例子里，我们就将介绍如何利用体探测器来读取三维空间里的入射光强分布，并将此分布在三维图形中显示出来。我们使用的光学系统为ZEMAX提供的一个LED模型，这个模型可以从 ZEMAX 的安装目录下(缺省为“C:\Program Files\ZEMAX\Samples\Non-sequential\Sources\”)得到，文件名为“led_model.ZMX”，我们将其另存为“ex40206.ZMX”。在这个光学系统中，有一个简单的 LED 模型，还有一个平面探测器，如图 4.2-17 所示：

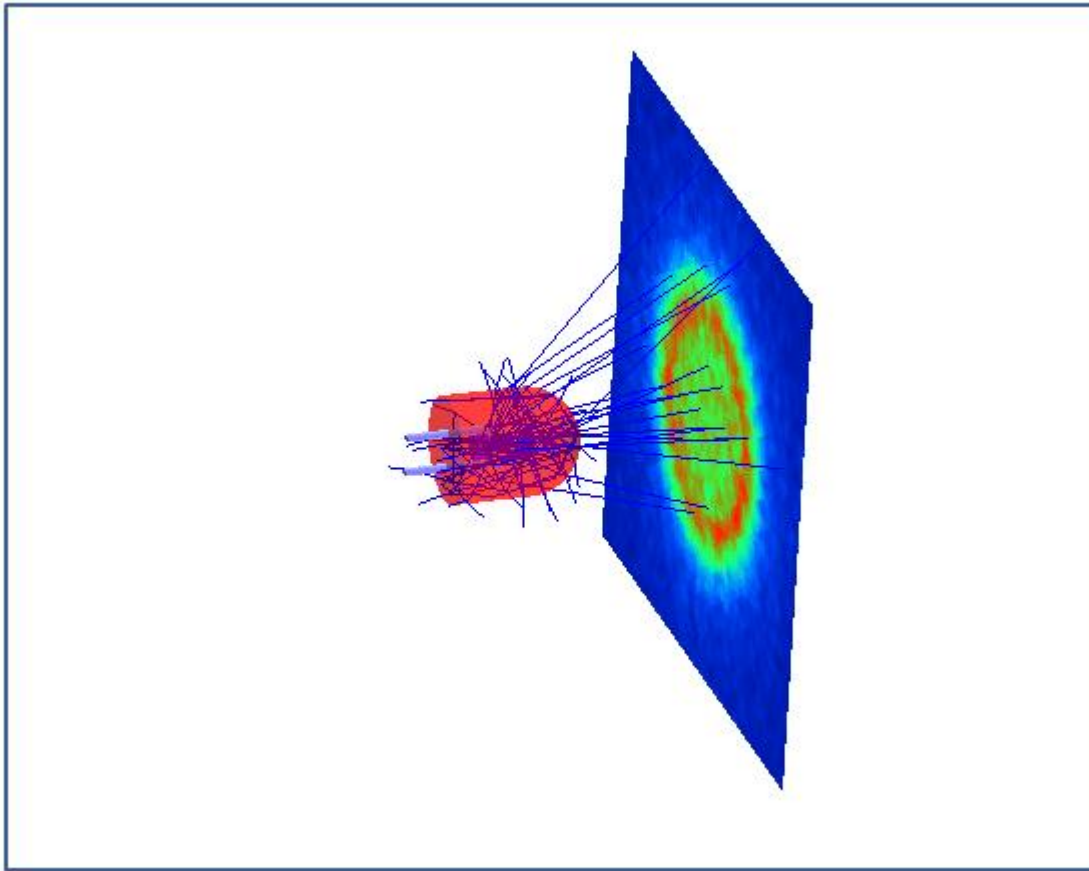


图4.2-17：光学系统 ex40206.ZMX 示意图

我们将对这个系统作一点简单修改，删掉平面探测器，再加上一个体探测器，如图 4.2-18 所示。这一步工作可以在非顺序元件编辑器中直接进行，也可以通过程序实现。在本例中我们通过程序来实现。

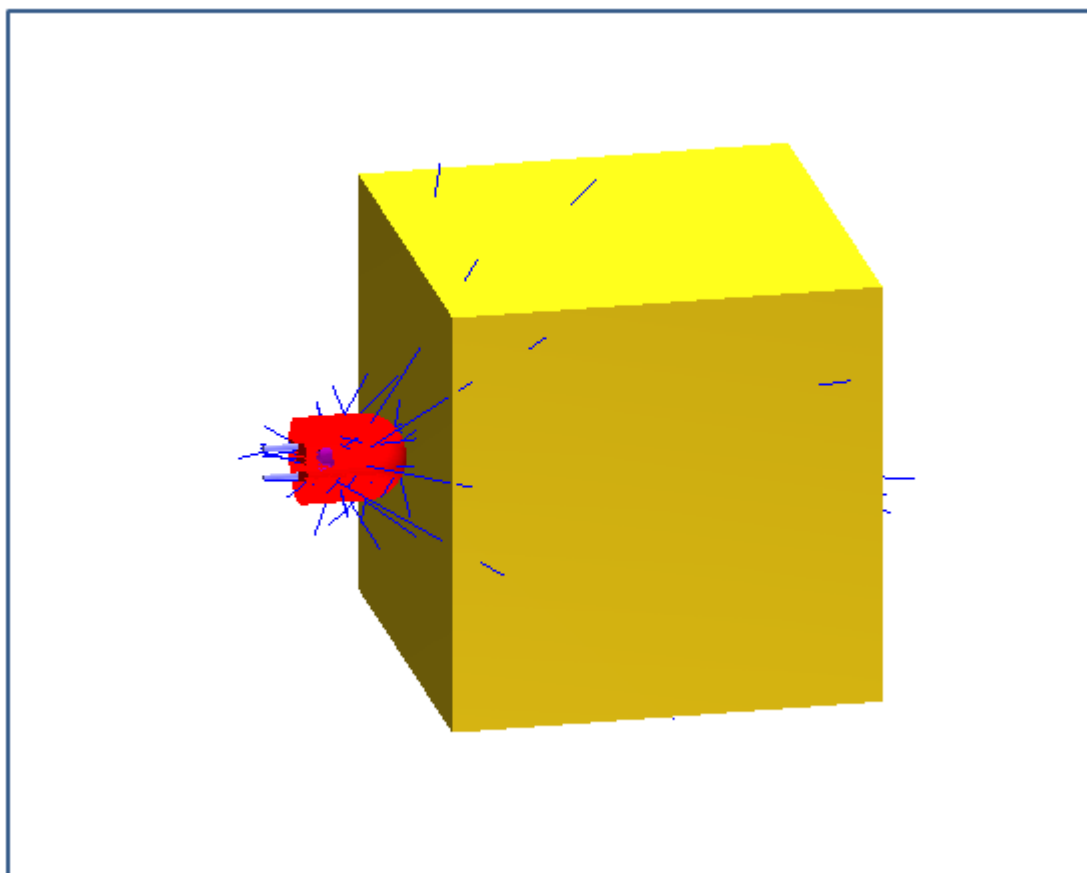


图4.2-18: 修改后的光学系统 ex40206.ZMX 示意图

```

1 ! ex40206
2 ! shows how to use detector volume to analyze light distribution
3 ! Assume the optical system is defined in ex40206.ZMX
4
5 ! initializing
6 hwx = 10          # x half width of voxel
7 hwy = 10          # y half width of voxel
8 hwz = 10          # z half width of voxel
9 nvx = 20          # pixel number in x
10 nvy = 20          # pixel number in y
11 nvz = 20          # pixel number in z
12
13 DECLARE intensity, DOUBLE, 1, nvx*nvy*nvz
14 DECLARE voxelColor, INTEGER, 1, nvx*nvy*nvz
15 DECLARE threshold, DOUBLE, 1, 4
16 threshold(1) = 0.5
17 threshold(2) = 0.2
18 threshold(3) = 0.05
19 threshold(4) = 0.01
20
21 ! revise model
22 GOSUB changeModel
23
24 ! record light intensity
25 GOSUB recordLight
26
27 ! assign color to each voxel
28 GOSUB setColor
29
30 ! create 3D model to visulize light intensity distribution
31 GOSUB visulize
32
33 END
34
35

```

程序第 1~33 行为主程序，其中，第 6~11 行定义了体探测器的大小及体像素元的数目，第 13 行定义了一个一维数组 `intensity` 用于存储空间入射光强度值，第 14 行定义了一个一维数组 `voxelColor` 用于存储各个体像素元所属的颜色组，由其相对光强度所决定，相对光强度的阈值由程序第 15~19 行定义。程序第 22 行调用子程序 `changeModel` 对光学系统进行简单修改，即我们前面所提到的删除平面探测器及增加体探测器，程序第 25 行调用子程序 `recordLight` 对系统进行光线追迹，并记录各个体像素元中的入射光强值。程序第 28 行调用子程序 `setColor`，将体探测器中各个体像素元的光强归一化，并根据其大小将其分入不同的颜色组。程序第 31 行调用子程序 `visulize`，按各个颜色组中的有效体像素元创建三维模型，用于代表满足阈值的光强分布。

```

35
36 SUB changeModel
37
38 objNum = NOBJ(1)           # find out total object number
39
40 FOR i = objNum, 1, -1
41   temp = NPRO(1, i, 0, 0)
42   A$ = $buffer()           # find out the object type
43   IF A$ $== "NSC_DETE" THEN DELETEOBJECT 1, i   # delete the detector
44 NEXT
45
46 INSERTOBJECT 1, 1
47 SETNSCPROPERTY 1, 1, 0, 0, "NSC_DETV"           # type as detector volume
48 SETNSCPARAMETER 1, 1, 1, hwx                     # x half width
49 SETNSCPARAMETER 1, 1, 2, hwy                     # y half width
50 SETNSCPARAMETER 1, 1, 3, hwz                     # z half width
51 SETNSCPARAMETER 1, 1, 4, nvx                     # number of x pixels
52 SETNSCPARAMETER 1, 1, 5, nvx                     # number of y pixels
53 SETNSCPARAMETER 1, 1, 6, nvz                     # number of z pixels
54 SETNSCPOSITION 1, 1, 3, hwz+5                     # put volume in front of LED
55 SETNSCPROPERTY 1, 1, 142, 0, 9                     # opacity as 10%
56
57 UPDATE ALL
58
59 RETURN
60
61

```

程序第 36~59 行为子程序 changeModel，用于修改原有的光学系统，即删除平面探测器，插入一个体探测器。程序第 40~44 行检查各个元件，如果其类型为矩形探测器(NSC_DETE)，则将其删除。程序第 46~55 行插入一个元件，然后定义其类型及相应参数，程序第 57 行将系统更新。

```
61
62 SUB recordLight
63
64 temp = NSDD(1, 0, 0, 0)           # clear detector
65
66 PRINT
67 PRINT "Ray tracing ..."
68 NSTR 1, 0, 1, 1, 1, 1, 0         # trace rays
69
70 IMax = 0
71 FOR ix = 1, nvx, 1
72   FOR iy = 1, nvx, 1
73     FOR iz = 1, nvz, 1
74       n = ix+(iy-1)*nvx+(iz-1)*nvx*nvy   # voxel number
75       intensity(n) = NSDD(1,1,n,0)        # read incident flux
76       IF IMax < intensity(n) THEN IMax = intensity(n) # update Imax
77     NEXT iz
78   NEXT iy
79 NEXT ix
80
81 RETURN
82
83
```

程序第 62~81 行为子程序 recordLight，用于进行光线追迹并记录入射到体探测器各体像素元上的光强，其中第 64 行清空探测器，第 68 行进行光线追迹，第 71~79 行的循环记录每一个体像素元的入射光强，并找出最大光强值，存入 Imax 中。

```

83
84 SUB setColor
85
86 FOR n = 1, nvx*nvy*nvz, 1
87   intensity(n) = intensity(n)/IMax      # normalize intensity of each voxel
88   IF intensity(n) > threshold(1)
89     voxelColor(n) = 1                  # red color
90   ELSE
91     IF intensity(n) > threshold(2)
92       voxelColor(n) = 2                # yellow color
93     ELSE
94       IF intensity(n) > threshold(3)
95         voxelColor(n) = 3              # green color
96       ELSE
97         IF intensity(n) > threshold(4)
98           voxelColor(n) = 4            # blue color
99         ELSE
100          voxelColor(n) = 0             # below the lowest threshold, then no color
101        ENDIF
102      ENDIF
103    ENDIF
104  ENDIF
105 NEXT n
106
107 RETURN
108
109

```

程序第 84 ~ 107 行为子程序 setColor，用于将各个体像素元的光强值归一化，并按照设定的阈值将各个体像素元分到不同的颜色组，基本想法是将入射光最强的体像素元设为红色，下面是黄色，然后是绿色，最后是蓝色。如果光强低于最小阈值，则忽略。

```

109
110 SUB visualize
111
112 objPath$ = $OBJECTPATH()              # get current object path name
113 lensFileName$ = $FILENAME()           # get current lens file name
114
115 FOR colorNum = 1, 4, 1
116   GOSUB createObj
117 NEXT
118
119 FOR colorNum = 1, 4, 1
120   GOSUB import
121 NEXT
122
123 RETURN
124
125

```

程序第 110~123 行为子程序 visulize，用于对不同颜色组中的有效元素创建与其相应的三维模型元件，即用一个立方体代表每一个有效的体像素元，最后将各个颜色组相应的三维元件导入到当前光学系统中。程序第 112、113 行分别读取当前元件路径名及当前镜头文件名，第 115~117 行的循环针对每个颜色组调用子程序 createObj 创建相应的三维元件，第 119~121 行的循环将各个颜色组相应的三维元件导入到当前光学系统中。

```

125
126 SUB createObj
127
128 objTotal = NOBJ(1)           # current total object number
129 objNum = objTotal
130
131 FOR n = 1, nvx*nvy*nvz, 1
132   IF voxelColor(n) == colorNum
133     objNum = objNum+1
134     REWIND
135     FORMAT 1 INT
136     PRINT "Inserting voxel ", n, " for group ", colorNum, " ..."
137
138     INSERTOBJECT 1, objNum    # insert one object at the end of NSC editor
139     SETNSCPROPERTY 1, objNum, 0, 0, "NSC_RBLK"    # type as rect block
140     SETNSCPARAMETER 1, objNum, 1, hwx/nvx        # x1 half width
141     SETNSCPARAMETER 1, objNum, 2, hwy/nvy        # y1 half width
142     SETNSCPARAMETER 1, objNum, 3, 2*hwz/nvz      # z length
143     SETNSCPARAMETER 1, objNum, 4, hwx/nvx        # x2 half width
144     SETNSCPARAMETER 1, objNum, 5, hwy/nvy        # y2 half width
145     SETNSCPROPERTY 1, objNum, 16, 0, 1           # rays ignore this object
146
147     nz = INTE((n-1)/(nvx*nvy))+1
148     ny = INTE((n-1-(nz-1)*nvx*nvy)/nvx)+1
149     nx = n-(nz-1)*nvx*nvy-(ny-1)*nvx
150     x = (nx-(nvx+1)/2)*2*hwx/nvx                # calculate x position
151     y = (ny-(nvy+1)/2)*2*hwy/nvy                # calculate y position
152     z = (nz-1-nvz/2)*2*hwz/nvz+hwz+5            # calculate z position
153     SETNSCPOSITION 1, objNum, 1, x
154     SETNSCPOSITION 1, objNum, 2, y
155     SETNSCPOSITION 1, objNum, 3, z
156   ENDIF
157 NEXT n
158
159 REWIND
160 PRINT "Updating object group ", colorNum, " ..."
161 UPDATE ALL
162
163 exportStartNum = objTotal+1
164 exportStopNum = objNum
165 GOSUB export
166
167 RETURN
168
169

```

程序第 126 ~ 167 行为子程序 `createObj`，用于创建与各颜色组有效体像素元相应的三维模型，其中第 128 行读取当前系统中的总元件数目，记录为 `objTotal`，第 129 行将此总元件数目赋给变量 `objNum`。程序第 131 ~ 157 行的循环检查每一个体像素元，如果它属于指定的颜色组(第 132 行)，则在非顺序元件编辑器的最后(由变量 `objNum` 的值加1确定)插入一个新元件(第 138 行)，并设定其相应参数(第 139 ~ 145 行)。程序第 147 ~ 152 行计算体像素元相应的坐标位置，第 153 ~ 155 行按此位置设定新创建的立方体的坐标。程序第 161 行在指定颜色组相应的循环结束后，更新当前光学系统，第 163 ~ 165 行指明所有新创建的元件在非顺序元件编辑器中的起始位置及终止位置，调用子程序 `export` 将这些元件输出到指定文件中。


```

169
170 SUB export
171
172 ! setting for export
173 VEC1(1) = 3           # CAD file type, choose STL
174 VEC2(2) = 8           # spline segments
175 VEC1(3) = exportStartNum # start object
176 VEC1(4) = exportStopNum # end object
177 VEC1(5) = 1           # ray data layer
178 VEC1(6) = 0           # lens data layer
179 VEC1(7) = 0           # do not export dummy surfaces
180 VEC1(8) = 1           # export surfaces as solids
181 VEC1(9) = 5           # ray pattern NONE
182 VEC1(10) = 0          # number of rays = 0
183 VEC1(11) = 0          # wavenumber
184 VEC1(12) = 0          # field number
185 VEC1(13) = 0          # do not 'delete vignetted'
186 VEC1(14) = 1          # dummy thickness
187 VEC1(15) = 0          # do not split rays
188 VEC1(16) = 0          # do not scatter rays
189 VEC1(17) = 0          # do not use polarization
190 VEC1(18) = 0          # use the current configuration
191
192 FORMAT 1 INT
193 temp$ = $STR(colorNum)
194 comment$ = lensFileName$+"Color"+temp$+".STL"
195 exportFileName$ = objPath$+"\ "+comment$
196
197 REWIND
198 PRINT "Exporting object group ", temp$, " ..."
199 EXPORTCAD exportFileName$
200
201 FOR i = exportStopNum, exportStartNum, -1
202     REWIND
203     PRINT "Deleting object ", i, " ..."
204     DELETEOBJECT 1, i
205 NEXT
206
207 RETURN
208
209

```

程序第 170 ~ 207 行为子程序 `export`，用于将指定元件输出到 CAD 文件中。其中，第 173 ~ 190 行定义了与输出文件有关的相应参数(有一些值为缺省值，可以不必在程序中指明)，我们选择输出文件类型为 STL 文件(第 173 行)，输出元件的范围由 `exportStartNum` 和 `exportStopNum` 确定(第 175、176 行)，并将元件以实体形式输出(第 180 行)。程序第 192 ~ 195 行根据颜色组的值生成输出文件名，第 199 行将元件输出到指定文件中，第 201 ~ 205 行将输出完毕的元件从非顺序元件编辑器中清除。

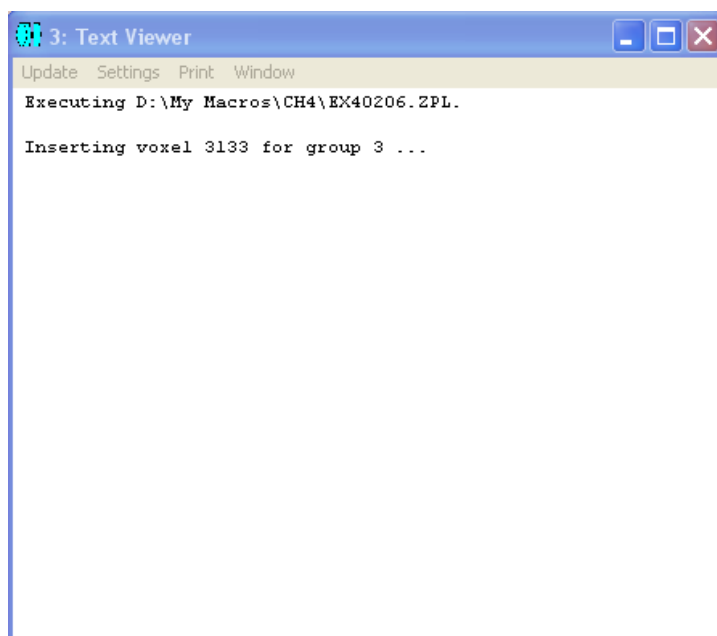
```

209
210 SUB import
211
212 FORMAT 1 INT
213 temp$ = $STR(colorNum)
214 comment$ = lensFileName$+"Color"+temp$+".STL"
215
216 newObjNum = NOBJ{1}+1
217
218 REWIND
219 PRINT "Importing object group ", temp$, " ..."
220
221 INSERTOBJECT 1, newObjNum
222 SETNSCPROPERTY 1, newObjNum, 0, 0, "NSC_STLO"           # object type
223 SETNSCPROPERTY 1, newObjNum, 1, 0, comment$           # comment
224 SETNSCPROPERTY 1, newObjNum, 2, 0, 0                   # reference object
225 SETNSCPROPERTY 1, newObjNum, 16, 0, 1                   # rays ignore this object
226 SETNSCPROPERTY 1, newObjNum, 142, 0, 9                 # opacity as 10%
227 SETNSCPARAMETER 1, newObjNum, 1, 1                     # Scale be 1
228 SETNSCPARAMETER 1, newObjNum, 2, 1                     # is volume
229
230 UPDATE ALL
231
232 REWIND
233 PRINT ""
234
235 RETURN
236

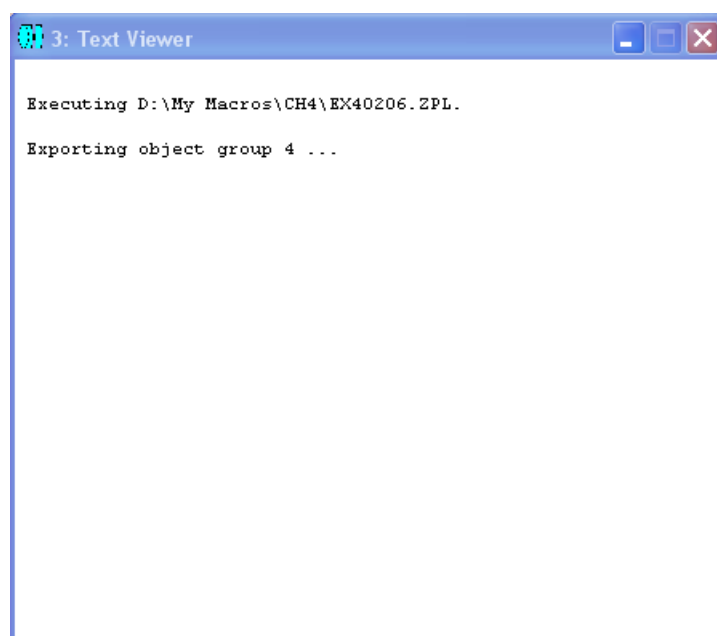
```

程序第 210~235 行为子程序 `import`，用于将生成的各颜色组实体模型导入当前光学系统中，以供观察。其中，第 212~214 行根据颜色组的值确定文件名，注意此处没有路径名，指定文件必须存于缺省元件路径中。程序第 221 行在非顺序元件编辑器最后插入一个空元件，第 222 行指定元件类型，第 223 行插入注释内容，代表输入元件的文件名，第 224~228 行设定输入元件相关参数，第 230 行更新当前光学系统。

由于程序运行的时间较长，我们在屏幕输出窗口中加入了一些提示行，用于显示程序的运行情况，如第 67 行、第 134~136 行、第 159~160 行、第 197~198 行、第 202~203 行。图 4.2-19 所示为程序运行过程中屏幕窗口出现的提示信息。



(a)



(b)

图4.2-19: 程序ex40206.ZPL 运行中的提示信息

在程序运行结束后，用户可以对输入到当前光学系统中的代表空间光强分布的固体模型自行设置透明度、颜色等参数，以方便观察。当然，由于固体模型已存于相应文件中，也可以将文件导入其它CAD软件中作进一步观察及处理。图 4.2-20 所示为在 CAD 软件中看到的不同颜色组相应的光强空间分布。由于印刷原因，这里只显示灰度图，如果加上颜色，会更为清楚直观。

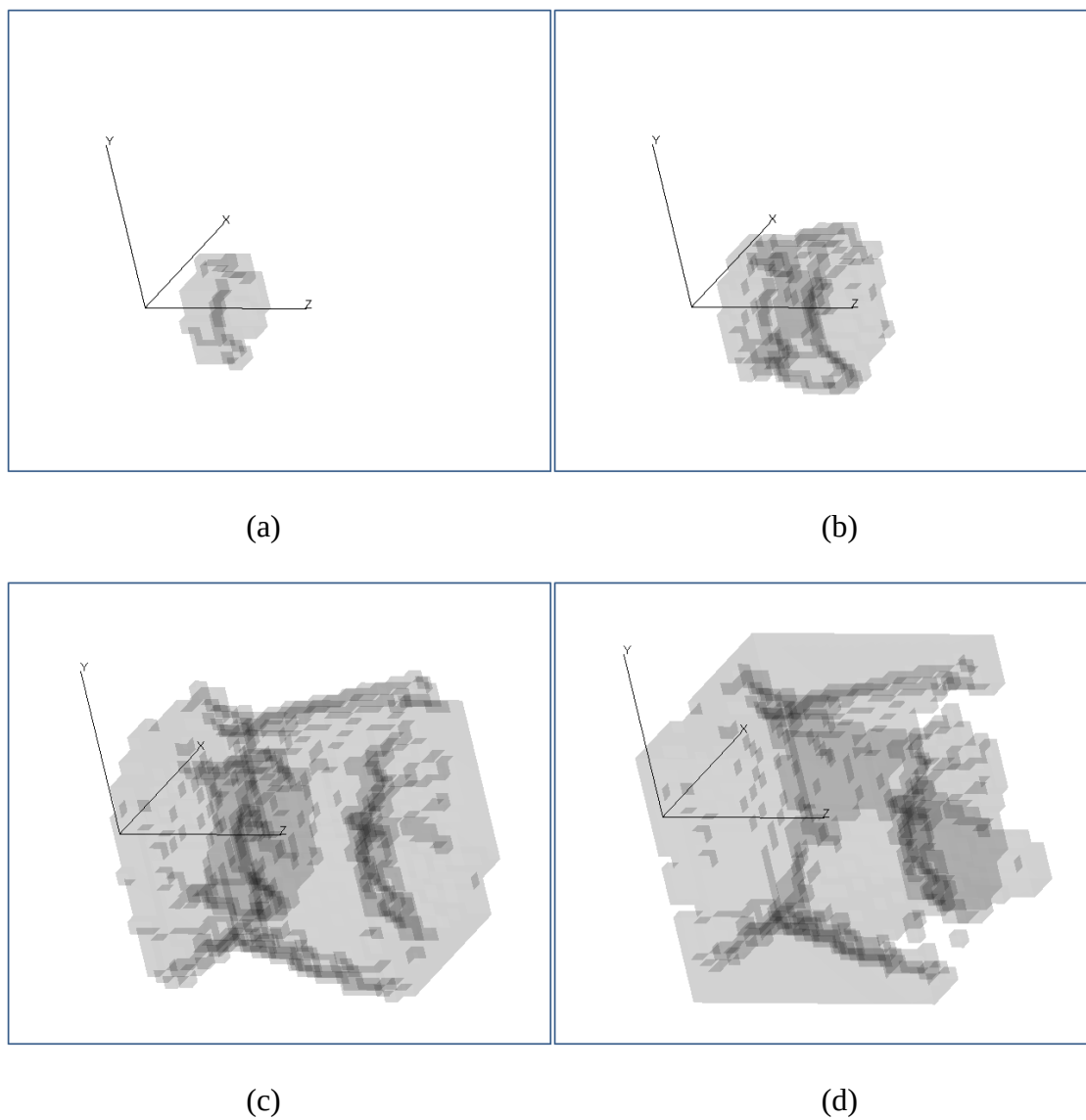


图4.2-20：程序ex40206.ZPL 运行后输出的光强分布三维模型。
(a) ~ (d) 光强由强到弱。